LAPORAN TUGAS MATA KULIAH PEMROGRAMAN APLIKASI MOBILE

PENGEMBANGAN APLIKASI

"*Planify: Smart Daily Planner & Productivity Assistant*"



Disusun Oleh:

Dean Anggara Putra	124240001
Radityas Shifa Zuleika	124240002
Sabilla Isjayanti	124240020

PROGRAM STUDI SISTEM INFORMASI

JURUSAN INFORMATIKA

FAKULTAS TEKNIK INDUSTRI

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"

YOGYAKARTA

2026

HALAMAN PENGESAHAN

LAPORAN TUGAS

Disusun Oleh:

Dean Anggara Putra	124240001
Radityas Shifa Zuleika	124240002
Sabilla Isjayanti	124240020

Telah Diperiksa dan Disetujui oleh Dosen

Pemrograman Aplikasi Mobile

Pada Tanggal: ……

Menyetujui,

Dosen Pemrograman Aplikasi Mobile

Bagus Muhammad A. S.ST., M.Kom.

DAFTAR ISI

HALAMAN PENGESAHAN **2**
DAFTAR ISI **3**
DAFTAR TABEL **7**
DAFTAR GAMBAR **8**
DAFTAR KODE PROGRAM **9**
BAB I PENDAHULUAN **10**
1.1 Latar Belakang 10
1.2 Rumusan Masalah 10
1.3 Tujuan 11
1.4 Manfaat 11
BAB II LANDASAN TEORI DAN TEMA APLIKASI **12**
2.1 Landasan Teori 12
2.1.1 Aplikasi Mobile 12
2.1.2 Flutter 12
2.1.3 Dart 12
2.1.4 Database 13
2.1.5 Web Service/API 13
2.1.6 Location Based Service (LBS) 13
2.1.7 Accelerometer dan Gyroscope 13
2.1.8 Artificial Intelligence/Large Language Model 14
2.1.9 Biometric Authentication 14
2.1.10 Notifikasi 14
2.1.11 Pomodoro dan Habit Tracking 14
2.1.12 Blockchain 15
2.2 Tema Aplikasi 15
2.2.1 Deskripsi Planify 15
2.2.2 Latar Belakang Pemilihan Tema 15
2.2.3 Tujuan Aplikasi 15
2.2.4 Target Pengguna 15
2.2.5 Konsep Utama Aplikasi 16
2.2.6 Fitur Utama Planify 16
2.3 Pemetaan Fitur terhadap Ketentuan Tugas 16
BAB III PERANCANGAN REKAYASA KEBUTUHAN PERANGKAT LUNAK **20**
3.1 Gambaran Umum Sistem 20
3.1.1 Alur Umum Penggunaan Aplikasi 20
3.1.2 Diagram Arsitektur 21
3.2 Analisis Kebutuhan Sistem 21
3.2.1 Identifikasi Pengguna 21
3.2.2 Kebutuhan Fungsional 22

3.2.3 Kebutuhan Non-Fungsional..... 24
3.3 Use Case Diagram..... 26
3.4 Deskripsi Use Case..... 27
3.5 Activity Diagram..... 29
3.6 Class Diagram..... 30
3.7 Perancangan Database..... 32
3.7.1 Strategi Penyimpanan Data..... 32
3.7.2 Rancangan Entitas..... 32
3.8 Perancangan Arsitektur Aplikasi..... 33
3.9 Perancangan Fitur Pendukung..... 34
3.9.1 Perancangan Location Based Service (LBS)..... 34
3.9.2 Perancangan Accelerometer..... 34
3.9.3 Perancangan Gyroscope..... 35
3.9.4 Perancangan AI Assistant..... 35
3.9.5 Perancangan Biometric Authentication..... 36
3.9.6 Perancangan Notifikasi dan Pengingat..... 36
3.9.7 Perancangan Budget Helper..... 37
3.9.8 Perancangan Time Zone Planner..... 37
3.9.9 Perancangan Habit Record dan Hash Chain..... 38
3.9.10 Perancangan Sinkronisasi dan Penyimpanan Data..... 38
3.10 Perancangan Keamanan..... 39
BAB IV IMPLEMENTASI DAN PENJELASAN SOURCE CODE.....40
4.1 Implementasi Lingkungan Pengembangan..... 40
4.1.1 Perangkat Keras..... 40
4.1.2 Perangkat Lunak..... 41
4.1.3 Teknologi dan Library yang Digunakan..... 42
4.2 Implementasi Struktur Project..... 43
4.2.1 Struktur Folder dan File..... 43
4.2.2 Pembagian Fungsi File..... 44
4.3 Implementasi Autentikasi Pengguna..... 45
4.3.1 Implementasi Registrasi..... 45
4.3.2 Implementasi Login..... 48
4.3.3 Implementasi Login Biometrik..... 54
4.3.4 Implementasi Session dan Logout..... 65
4.4 Implementasi Halaman Home..... 73
4.4.1 Tampilan dan Komponen Home..... 73
4.4.2 Implementasi Informasi dan Ringkasan Produktivitas..... 74
4.5 Implementasi Planner..... 79
4.5.1 Tampilan Halaman Planner..... 79
4.5.2 Implementasi Pengelolaan Agenda..... 80
4.5.3 Implementasi Penyimpanan dan Pengambilan Data Agenda..... 84

4.6 Implementasi Focus Mode dan Pomodoro....89
4.6.1 Tampilan Halaman Focus....90
4.6.2 Implementasi Timer Pomodoro....91
4.6.3 Implementasi Sesi Fokus dan Istirahat....100
4.7 Implementasi AI Assistant....100
4.7.1 Tampilan Halaman AI Assistant....100
4.7.2 Implementasi Pengolahan Permintaan Pengguna....102
4.7.3 Implementasi Rekomendasi Jadwal....106
4.8 Implementasi Habit Tracker....109
4.8.1 Tampilan Halaman Habit Tracker....110
4.8.2 Implementasi Pengelolaan Kebiasaan....110
4.8.3 Implementasi Pengelolaan Kebiasaan....110
4.9 Implementasi Habit Consistency Record....113
4.9.1 Tampilan Fitur Habit Consistency Record....114
4.9.2 Implementasi Pencatatan Riwayat Kebiasaan....114
4.9.3 Implementasi Hash SHA-256....114
4.10 Fitur Budget Helper....120
4.10.1 Tampilan Antarmuka dan Alur Penggunaan....121
4.10.2 Konversi Mata Uang dan Integrasi Nilai Tukar....122
4.10.3 Manajemen Anggaran Aktivitas dan Kategori Pengeluaran....126
4.11 Fitur Time Zone Planner....130
4.11.1 Tampilan Antarmuka dan Alur Penggunaan....130
4.11.2 Konversi Waktu Antar Zona....131
4.11.3 Penanganan Daylight Saving Time (DST) dan Integrasi dengan Agenda....136
4.12 Fitur Notifikasi dan Pengingat....137
4.12.1 Konfigurasi dan Inisialisasi Notifikasi Lokal....137
4.12.3 Penjadwalan Pengingat Berdasarkan Deadline Tugas....144
4.12.3 Penanganan Interaksi Pengguna terhadap Notifikasi....145
4.13 Fitur Location-Based Service (LBS)....145
4.13.1 Tampilan Antarmuka Peta dan Pencarian Tempat....146
4.13.2 Pengambilan Lokasi Pengguna dan Penentuan Radius....147
4.13.3 Integrasi API Peta dan Penentuan Rute Lokasi Produktif....152
4.14 Fitur Otomatisasi Focus Mode dengan Sensor Accelerometer....153
4.14.1 Deteksi Orientasi Perangkat Menggunakan Accelerometer....153
4.14.2 Pengaktifan Focus Mode Otomatis Saat Perangkat Ditengkurapkan....157
4.15 Fitur Mini Game Keseimbangan Fokus dengan Sensor Gyroscope....158
4.15.1 Tampilan dan Mekanisme Permainan....158
4.15.2 Pembacaan Sensor Gyroscope dan Pengendalian Posisi Bola....159
4.15.3 Perhitungan Skor dan Penyimpanan Riwayat Permainan....164
4.16 Database dan Integrasi API....164
4.16.1 Tampilan dan Mekanisme Permainan....165

4.16.2 Penyimpanan Awan dan Sinkronisasi Data Supabase....168
4.16.3 Integrasi API Eksternal.... 173
4.17 Halaman Profil dan Pengaturan Pengguna....174
4.17.1 Tampilan Antarmuka dan Komponen Profil.... 174
4.17.2 Pengelolaan Akun dan Preferensi Tampilan.... 175
4.17.3 Pengaturan Keamanan dan Fitur Biometrik....176
4.18 Navigasi dan Integrasi Antarmuka Aplikasi.... 179
4.18.1 Struktur Navigasi Utama Aplikasi.... 179
4.18.2 Penanganan Rute dan Redirection Berdasarkan Status Autentikasi.... 183
4.18.3 Integrasi Keseluruhan Fitur dalam Arsitektur Aplikasi.... 186
BAB V LEMBAR EVALUASI PENGUJIAN PERANGKAT....188
5.1 Tujuan Pengujian....188
5.2 Lingkungan Pengujian....189
5.3 Metode Pengujian....190
5.4 Skenario dan Hasil Pengujian Fungsional Aplikasi Planify....191
5.4.1 Autentikasi Pengguna dan Manajemen Sesi.... 191
5.4.2 Halaman Home dan Dasbor Produktivitas....194
5.4.4 Focus Mode dan Pomodoro Timer....197
5.4.5 AI Assistant dan Penjadwalan Cerdas....199
5.4.6 Habit Tracker dan Pelacakan Rutinitas.... 201
5.4.7 Habit Consistency Record dan Hash SHA-256....203
5.4.8 Budget Helper dan Pengelolaan Anggaran.... 204
5.4.9 Time Zone Planner dan Penjadwalan Lintas Zona....206
5.4.10 Notifikasi Lokal dan Pengingat....208
5.4.11 Location-Based Service (LBS) Navigator....210
5.4.12 Otomatisasi Focus Mode Sensor Accelerometer.... 212
5.4.13 Mini Game Keseimbangan Sensor Gyroscope....214
5.4.15 Basis Data Lokal Hive dan Sinkronisasi Awan....215
5.4.15 Profil Pengguna dan Pengaturan Keamanan.... 217
5.4.16 Navigasi Antarmuka dan Penjaga Rute....218
5.5 Rekapitulasi Hasil Pengujian.... 220
BAB VI PENUTUP.... 223
6.1 Evaluasi Fitur yang Memenuhi Harapan....223
6.2 Keterbatasan dan Kendala yang Ditemukan.... 224
6.3 Saran Perbaikan Lanjutan....224
6.4 Kesimpulan Akhir.... 225

DAFTAR TABEL

Tabel 2.2.6.1 Fitur Utama Planify....16
Tabel 2.3.1 Pemetaan Fitur....16
Tabel 3.2.1.1 Identifikasi Pengguna....21
Tabel 3.2.2.1 Kebutuhan Fungsional....22
Tabel 3.2.3.1 Kebutuhan Non-Fungsional....24
Tabel 3.4.1 Deskripsi Use Case....27
Tabel 3.7.1.1 Strategi Penyimpanan Data....32
Tabel 3.7.2.1 Rancangan Entitas....32
Tabel 3.8.1 Perancangan Arsitektur Aplikasi....33
Tabel 3.9.1.1 Perancangan Location Based Service (LBS)....34
Tabel 3.9.2.1 Perancangan Accelerometer....34
Tabel 3.9.3.1 Perancangan Gyroscope....35
Tabel 3.9.4.1 Perancangan AI Assistant....35
Tabel 3.9.5.1 Perancangan Biometric Authentication....36
Tabel 3.9.6.1 Perancangan Notifikasi dan Pengingat....36
Tabel 3.9.7.1 Perancangan Budget Helper....37
Tabel 3.9.8.1 Perancangan Time Zone Planner....37
Tabel 3.9.9.1 Perancangan Habit Record dan Hash Chain....38
Tabel 3.9.10.1 Perancangan Sinkronisasi dan Penyimpanan Data....38
Tabel 3.10.1 Perancangan Keamanan....39
Tabel 4.1.2.1 Perangkat Lunak....41
Tabel 4.2.1.1 Struktur Folder dan File....43
Tabel 5.4.1.1 Skenario dan Hasil Pengujian Fungsional Aplikasi Planify....191
Tabel 5.4.2.1 Halaman Home dan Dasbor Produktivitas....194
Tabel 5.4.3.1 Bukti Pengujian Autentikasi Pengguna....195
Tabel 5.4.4.1 Focus Mode dan Podomoro Timer....197
Tabel 5.4.5.1 AI Assistant dan Penjadwalan Cerdas....199
Tabel 5.4.6.1 Habit Tracker dan Pelacakan Rutinitas....201
Tabel 5.4.7.1 Habit Consistency Record dan Hash SHA-256....203
Tabel 5.4.8.1 Bukti PengujianBudget Helper dan Pengelolaan Anggaran....204
Tabel 5.4.9.1 Time Zone Planner dan Penjadwalan Lintas Zona....206
Tabel 5.4.10.1 Bukti Pengujian Notifikasi Lokal dan Pengingat....208
Tabel 5.4.11.1 Location-Based Service (LBS) Navigator....210
Tabel 5.4.12.1 Otomatisasi Focus Mode Sensor Accelerometer....212
Tabel 5.4.14.1 Basis Data Lokal Hive dan Sinkronisasi Awan....215
Tabel 5.4.15.1 Profil Pengguna dan Pengaturan Keamanan....217
Tabel 5.4.16.1 Navigasi Antarmuka dan Penjaga Rute....218
Tabel 5.5.1 Rekapitulasi Hasil Pengujian Kotak Hitam Aplikasi Planify....220

DAFTAR GAMBAR

Gambar 3.3.1 Use Case Diagram Planify..... 26
Gambar 3.5.1 Activity Diagram Login..... 29
Gambar 3.6.1 Class Diagram..... 31
Gambar 4.3.1.1 Tampilan Halaman Registrasi Akun..... 48
Gambar 4.3.2.1 Halaman Login..... 54
Gambar 4.3.3.1 Tampilan Dialog Autentikasi Biometrik di Halaman Login..... 65
Gambar 4.4.1.1 Tampilan Halaman Utama (Home)..... 74
Gambar 4.5.1.1 Tampilan Halaman Planner..... 79
Gambar 4.5.1.2 Formulir Tambah dan Edit Tugas Planner..... 80
Gambar 4.6.1.1 Tampilan Antarmuka Focus Mode..... 90
Gambar 4.6.1.2 Pengaturan Durasi Sesi Pomodoro..... 91
Gambar 4.7.1.1 Tampilan Formulir AI Assistant..... 101
Gambar 4.7.1.2 Tampilan Rekomendasi Jadwal AI Review..... 101
Gambar 4.8.1.1 Tampilan Daftar Kebiasaan (Habit Tracker)..... 110
Gambar 4.9.1.1 Tampilan Riwayat Konsistensi Kebiasaan..... 114
Gambar 4.10.1.1 Tampilan Halaman Budget Helper..... 121
Gambar 4.11.1.1 Tampilan Halaman Time Zone Planner..... 131
Gambar 4.13.1.1 Tampilan Peta Interaktif dan Rekomendasi Tempat Produktif..... 147
Gambar 4.14.2.1 Tampilan Dialog Tawaran Focus Mode Otomatis..... 157
Gambar 4.15.1.1 Tampilan Permainan Keseimbangan Fokus (Balance Focus Game)..... 158
Gambar 4.17.1.1 Tampilan Halaman Profil Pengguna..... 174
Gambar 4.17.3.1 Tampilan Pengaturan Keamanan dan Preferensi..... 176
Gambar 4.18.1.1 Tampilan Navigasi Utama (Bottom Navigation Bar)..... 180
Gambar 4.18.3.1 Planify Arsitektur..... 186
Gambar 5.4.1.1 Bukti Pengujian Autentikasi Pengguna..... 194
Gambar 5.4.2.1 Bukti Pengujian Autentikasi Pengguna..... 195
Gambar 5.4.3.1 Bukti Pengujian Manajemen Agenda Planner..... 197
Gambar 5.4.4.1 Bukti Pengujian Focus Mode dan Podomoro Timer..... 199
Gambar 5.4.6.1 Bukti Pengujian Habit Tracker dan Pelacakan Rutinitas..... 203
Gambar 5.4.7.1 Bukti Pengujian Habit Consistency Record dan Hash SHA-256..... 204
Gambar 5.4.8.1 Bukti Pengujian Budget Helper dan Pengelolaan Anggaran..... 206
Gambar 5.4.9.1 Bukti Pengujian Time Zone Planner dan Penjadwalan Lintas Zona..... 208
Gambar 5.4.10.1 Bukti Pengujian Notifikasi Lokal dan Pengingat..... 210
Gambar 5.4.11.1 Bukti Pengujian Location-Based Service (LBS) Navigator..... 212
Gambar 5.4.12.1 Bukti Pengujian Otomatisasi Focus Mode Sensor Accelerometer..... 214
Gambar 5.4.13.1 Bukti Pengujian Mini Game Keseimbangan Sensor Gyroscope..... 215
Gambar 5.4.14.1 Bukti Pengujian Basis Data Lokal Hive dan Sinkronisasi Awan..... 217
Gambar 5.4.15.1 Bukti Pengujian Profil Pengguna dan Pengaturan Keamanan..... 218
Gambar 5.4.16.1 Bukti Pengujian Navigasi Antarmuka dan Penjaga Rute..... 220

DAFTAR KODE PROGRAM

Kode Program 4.3.1.1 Implementasi Registrasi..... 46
Kode Program 4.3.2.1 Implementasi Login.....49
Kode Program 4.3.3.1 Verifikator Biometrik Lokal Perangkat Keras.....55
Kode Program 4.3.3.2 Pengendali Aksi Autentikasi dan Login Biometrik.....58
Kode Program 4.3.4.1 Pengendali Status Sesi dan Siklus Hidup Aplikasi.....66
Kode Program 4.4.2.1 Model Filter dan Ringkasan Produktivitas Harian..... 75
Kode Program 4.5.2.1 Pengendali Tampilan dan Interaksi Agenda Planner.....81
Kode Program 4.5.3.1 Entitas Tugas dan Metadata Sinkronisasi.....85
Kode Program 4.6.2.1 Mesin Waktu dan Siklus Pomodoro Berbasis Jam Dinding.....92
Kode Program 4.7.2.1 Model Data Permintaan dan Usulan Jadwal AI.....103
Kode Program 4.7.3.1 Mesin Penjadwal Berbasis Aturan Deterministik.....107
Kode Program 4.8.3.1 Algoritma Perhitungan Rentetan Hari Berturut-turut (Streak)..... 111
Kode Program 4.9.3.1 Rantai Hash Kriptografis SHA-256 Lokal.....115
Kode Program 4.10.2.1 Model Mata Uang dan Konversi..... 123
Kode Program 4.10.3.1 Model Item Anggaran Aktivitas.....128
Kode Program 4.11.2.1 Logika Konversi Zona Waktu IANA.....132
Kode Program 4.12.1.1 Layanan Notifikasi Lokal.....138
Kode Program 4.13.2.1 Model dan Parser Lokasi Tempat Produktif.....148
Kode Program 4.14.1.1 Algoritma Deteksi Orientasi Perangkat Telungkup.....154
Kode Program 4.15.2.1 Simulasi Fisika dan Kalibrasi Gyroscope.....160
Kode Program 4.16.1.1 Konfigurasi dan Manajemen Penyimpanan Box Hive..... 166
Kode Program 4.16.2.1 Metadata dan Logika Sinkronisasi Data Offline-First.....169
Kode Program 4.17.3.1 Penyimpanan Kunci Terenkripsi.....177
Kode Program 4.18.1.1 Kerangka Shell Navigasi Utama.....181
Kode Program 4.18.2.1 Logika Pengalihan Rute Berbasis Status Sesi..... 185

BAB I
PENDAHULUAN

1.1 Latar Belakang

Kegiatan harian mahasiswa maupun pekerja mencakup banyak hal sekaligus, seperti tugas dengan tenggat waktu berbeda, jadwal pertemuan, kebiasaan yang ingin dibangun, dan kebutuhan untuk menjaga fokus. Apabila kegiatan tersebut tidak dikelola secara terstruktur, pengguna berisiko melewatkan tugas, kesulitan menentukan prioritas, dan kurang optimal dalam membagi waktu.

Dalam praktiknya, pengelolaan tersebut sering terpecah ke beberapa alat yang berbeda, misalnya aplikasi catatan, alarm, kalender, pengatur waktu, dan pencatat kebiasaan. Perpindahan antar aplikasi membuat informasi tersebar dan tidak terhubung. Pengguna tidak memperoleh gambaran utuh tentang apa yang harus dikerjakan, kapan, dan seberapa banyak waktu yang tersedia.

Perangkat mobile saat ini selalu dibawa oleh pengguna dan memiliki kemampuan yang relevan untuk perencanaan harian, antara lain notifikasi, sensor gerak, lokasi, autentikasi biometrik, dan koneksi internet. Di sisi lain, large language model (LLM) telah berkembang sebagai model yang memiliki kemampuan kuat pada berbagai tugas pemrosesan bahasa alami (Zhao et al., 2023), sehingga dapat dimanfaatkan sebagai asisten penyusun jadwal. Pengembangan aplikasi lintas platform dengan Flutter juga memungkinkan satu basis kode digunakan untuk membangun aplikasi mobile (Flutter, t.th.-a).

Berdasarkan hal tersebut, kami merancang Planify, aplikasi berbasis mobile yang menggaungkan pengelolaan kegiatan, pengingat, mode fokus, dan pelacakan kebiasaan, asisten AI untuk perencanaan harian, serta fitur pendukung dalam satu aplikasi.

1.2 Rumusan Masalah

1. Bagaimana merancang aplikasi mobile yang dapat mengelola kegiatan, tugas, kebiasaan, waktu, dan fokus pengguna dalam satu aplikasi?
2. Bagaimana merancang pengingat dan notifikasi agar pengguna tidak melewatkan kegiatan terjadwal?
3. Bagaimana merancang asisten AI yang berfokus pada penyusunan jadwal dan rekomendasi pembagian waktu, bukan chatbot umum?
4. Bagaimana merancang pemanfaatan lokasi dan sensor perangkat untuk mendukung produktivitas pengguna?

5. Bagaimana merancang penyimpanan data dan keamanan akses pengguna pada aplikasi?
6. Bagaimana merancang pengujian untuk menilai kesesuaian fungsi aplikasi dengan kebutuhan yang ditetapkan?

1.3 Tujuan

Tujuan pembuatan Planify adalah:

1. Merancang dan membangun aplikasi mobile perencana harian yang terintegrasi dengan fitur pengelolaan task, pengingat, fokus, dan kebiasaan.
2. Merancang asisten AI yang membantu menyusun jadwal berdasarkan kegiatan pengguna.
3. Menerapkan fitur berbasis lokasi dan sensor perangkat yang relevan dengan produktivitas.
4. Menerapkan penyimpanan data lokal/remote, autentikasi, dan keamanan dasar pada aplikasi.
5. Menyusun dokumentasi perancangan, implementasi, dan pengujian sesuai ketentuan mata kuliah.

1.4 Manfaat

1. Bagi pengguna, Planify diharapkan membantu melihat seluruh kegiatan, prioritas, kebiasaan, dan waktu fokus dalam satu tempat, serta mendapat bantuan penyusunan jadwal harian.
2. Bagi pembelajaran dan pengembangan aplikasi mobile, pengerjaan Planify memberi pengalaman menerapkan Flutter dan Dart, perancangan perangkat lunak (use case, activity diagram, class diagram, perancangan database), integrasi web service, pemanfaatan sensor dan lokasi, penerapan autentikasi biometrik, serta pengujian perangkat lunak.

BAB II
LANDASAN TEORI DAN TEMA APLIKASI

2.1 Landasan Teori

Sub Bab ini membahas beberapa teori yang relevan dengan aplikasi Planify. Setiap sub bab terdiri dari landasan teori yang bersumber dari referensi dan disertai sitasi, serta keterkaitan dengan aplikasi Planify, yaitu rancangan atau keputusan desain kelompok.

2.1.1 Aplikasi Mobile

Aplikasi mobile adalah perangkat lunak yang dijalankan pada perangkat bergerak seperti ponsel. Pada platform Android, setiap aplikasi berjalan dalam security sandbox-nya sendiri, dibangun dari komponen aplikasi yang dideklarasikan pada berkas manifest, dan sistem menerapkan prinsip least privilege; izin untuk mengakses fitur tertentu harus diberikan secara eksplisit oleh pengguna (Android Developer, t.th.).

2.1.2 Flutter

Flutter adalah framework sumber terbuka yang dikelola Google untuk membangun aplikasi natively compiled bagi perangkat mobile, web, desktop, dan perangkat tertanam dari satu basis kode (Flutter, t.th.-a). Flutter tidak bergantung pada tampilan web maupun widget bawaan sistem, melainkan merender antarmuka dengan mesin renderingnya sendiri, dan menyediakan widget Material Design serta Cupertino. Fitur perangkat seperti sensor dan penyimpanan lokal diakses melalui plugin di pub.dev atau platform channel (Flutter, t.th.-a)

2.1.3 Dart

Dart adalah bahasa pemrograman yang dirancang optimal untuk aplikasi klien; Dart Native mencakup Dart VM dengan kompilasi JIT serta kompilator AOT yang menghasilkan kode mesin (Dart, t.th.). Dalam konteks Flutter, JIT mendukung stateful hot reload saat pengembangan dan AOT dipakai untuk menghasilkan kode native pada rilis. Dart menangani pekerjaan asinkron dengan `Future` dan `Stream`, serta komputasi berat dengan isolate (Flutter, t.th.-a).

2.1.4 *Database*

Dokumentasi Flutter menyarankan penggunaan database, bukan berkas lokal atau penyimpanan *key-value*, apabila aplikasi perlu menyimpan dan meng query data dalam jumlah besar pada perangkat; aplikasi Flutter dapat memakai SQLite melalui plugin `sqflite` untuk operasi menyimpan, membaca. mengubah, dan menghapus data (Flutter, t.th.-c).

2.1.5 Web Service/API

W3C mendefinisikan web service sebagai sistem perangkat lunak yang dirancang untuk mendukung interaksi mesin ke mesin yang interoperable melalui jaringan (W3C Web Services Architecture Working Group, 2004). Definisi tersebut dirumuskan dengan SOAP dan WSDL sebagai acuan; pada aplikasi Flutter, pengambilan data dari internet dilakukan melalui permintaan HTTP dengan paket `http`, hasilnya dikonversi menggunakan `dart:convert`, dan aplikasi Android perlu mendeklarasikan izin internet (Flutter, t.th.-b).

2.1.6 *Location Based Service* (LBS)

Layanan berbasis lokasi (*location*-based services) mengintegrasikan lokasi geografis pengguna dengan konsep layanan, misalnya navigasi kendaraan atau panggilan darurat, dan memadukan komunikasi bergerak dengan data spasial (Schiller & Voisard, 2004). Pada Android, aplikasi yang memakai layanan lokasi wajib meminta izin lokasi; pengguna dapat memberikan lokasi perkiraan saja, sehingga aplikasi sebaiknya tetap berfungsi dalam kondisi tertentu (Android Developers, t.t.-d).

2.1.7 Accelerometer dan Gyroscope

Platform Android menyediakan sensor gerak untuk memantau pergerakan perangkat. Accelerometer dan gyroscope selalu berbasis perangkat keras; accelerometer memberikan data gaya percepatan pada tiga sumbu, sedangkan gyroscope memberikan data laju rotasi pada tiga sumbu. Sensor gerak berguna untuk mendeteksi gerakan seperti kemiringan, guncangan, dan rotasi. Hampir semua perangkat Android memiliki accelerometer, tetapi hanya sebagian yang dilengkapi gyroscope (Android Developers, t.t.-b). Dalam Flutter sensor diakses melalui plugin (Flutter, t.th.-a).

2.1.8 *Artificial Intelligence/Large Language Model*

Large language model (LLM) adalah istilah yang digunakan komunitas riset untuk model bahasa pra-latih berbasis Transformer yang berukuran besar (puluhan hingga ratusan miliar parameter) dan dilatih pada korpus berskala besar. Model tersebut menunjukkan kemampuan kuat pada berbagai tugas pemrosesan bahasa alami serta kemampuan yang tidak terlihat pada model kecil, seperti *in-context learning* (Zhao et al., 2023).

2.1.9 Biometric Authentication

Pengenalan biometrik adalah pengenalan otomatis individu berdasarkan karakteristik fisiologis dan/atau perilaku. Sistem biometrik pada dasarnya adalah sistem pengenalan pola yang beroperasi dalam mode verifikasi atau identifikasi (Jain et al., 2004). Pada Android, dialog autentikasi biometrik ditampilkan oleh sistem; aplikasi menentukan tingkat autentikator yang diterima (misalnya biometrik Kelas 3 atau Kelas 2, atau kombinasi dengan kredensial kunci layar) dan memeriksa ketersediaan biometrik sebelum menampilkan dialog (Android Developers, t.t.-f).

2.1.10 Notifikasi

Android 13 (API 33) ke atas memiliki izin runtime `POST_NOTIFICATIONS` untuk mengirim notifikasi, dan aplikasi yang menargetkan Android 13 atau lebih baru mengendalikan kapan dialog izin ditampilkan (Android Developers, t.t.-c). Untuk penjadwalan, alarm presisi hanya sebaiknya dipakai bila fungsi yang terlihat pengguna memerlukan ketepatan waktu; izin `SCHEDULE_EXACT_ALARM` tidak diberikan otomatis pada instalasi baru yang menargetkan Android 13 ke atas, dan sebagian besar aplikasi dapat memakai alarm tidak presisi. Alarm berulang pada API 19 ke atas selalu bersifat tidak presisi (Android Developers, t.t.-e).

2.1.11 Pomodoro dan Habit Tracking

Pomodoro. Biwer et al. (2023) membandingkan istirahat sistematis (istirahat 6 menit setelah blok belajar 24 menit, disebut teknik Pomodoro) dengan istirahat yang diatur sendiri pada 87 mahasiswa dalam satu sesi belajar. Mahasiswa yang mengatur sendiri istirahat belajar dan beristirahat lebih lama, sedangkan istirahat sistematis memberi manfaat suasana hati dan tampak lebih

efisien karena penyelesaian tugas serupa dalam waktu lebih singkat (Biwer et al., 2023).

Habit tracking. Lally et al (2010) meneliti 96 relawan selama 12 minggu dan menemukan bahwa pengulangan perilaku dalam konteks yang konsisten meningkatkan otomatisitas mengikuti kurva asimtotik. Waktu mencapai 95% asimtot berkisar 18 sampai 254 hari, dan melewatkan satu kesempatan tidak berpengaruh material pada proses pembentukan kebiasaan (Lally et al., 2010).

2.1.12 Blockchain

Menurut NIST, *blockchain* adalah buku besar digital yang *tamper evident* dan *tamper resistant*, diimplementasikan secara terdistribusi tanpa repository pusat dan biasanya tanpa otoritas pusat. Pada tingkat dasar, *blockchain* memungkinkan suatu komunitas mencatat transaksi pada buku besar sehingga, pada operasi normal, transaksi yang telah dipublikasi tidak dapat diubah (Yaga et al., 2018).

2.2 Tema Aplikasi

2.2.1 Deskripsi Planify

Planify adalah aplikasi mobile produktivitas serbaguna yang mengkombinasikan pencatatan tugas, pelacak kebiasaan, manajemen waktu Pomodoro, pembuat jadwal otomatis berbantuan AI, pencarian lokasi tempat belajar, serta berbagai fitur utilitas pendukung dalam satu aplikasi.

2.2.2 Latar Belakang Pemilihan Tema

Tema perencana harian dipilih karena kebutuhan mengelola kegiatan bersifat umum dan berulang, serta mencakup banyak fungsi yang dapat memanfaatkan kemampuan perangkat mobile: penyimpanan data, notifikasi, sensor, lokasi, layanan eksternal, dan autentikasi. Dengan demikian satu tema dapat mencakup seluruh cakupan teknis mata kuliah secara bermakna.

2.2.3 Tujuan Aplikasi

Membantu pengguna merencanakan kegiatan, mengingat jadwal, menjaga fokus, membangun kebiasaan, dan membagi waktu dengan bantuan asisten AI, dalam satu aplikasi yang terintegrasi.

2.2.4 Target Pengguna

Target pengguna adalah mahasiswa dan pelajar, serta pekerja yang membutuhkan alat sederhana untuk merencanakan kegiatan harian.

2.2.5 Konsep Utama Aplikasi

Planify berpusat pada lima area navigasi bawah: Home | Planner | Focus | AI | Profile. Home merangkum aktivitas hari ini; Planner mengelola task dan kalender; Focus berisi pengatur waktu Pomodoro; AI membantu menyusun jadwal; Profile berisi data pengguna dan pengaturan. Fitur pendukung (Nearby Productivity, Currency Converter, Timezone Converter, Mini Game) direncanakan diakses melalui pintasan pada Home atau Profile.

2.2.6 Fitur Utama Planify

Tabel 2.2.6.1 Fitur Utama Planify

Modul	Ringkasan Fitur
Home	Ringkasan aktivitas hari ini, progres, Priority Task, habit, kegiatan berikutnya
Planner	Tambah, ubah, hapus, selesaikan task; tanggal dan waktu; kategori; prioritas; kalender; pencarian; filter
Smart Reminder	Pengingat berdasarkan tanggal dan waktu, tugas berulang, notifikasi
Focus Mode	Pomodoro, sesi fokus dan istirahat, progres sesi
Habit Tracker	Membuat habit, menandai selesai, progres dan streak
AI Daily Assistant	Penyusunan jadwal dan rekomendasi pembagian waktu berbasis LLM
Pendukung	LBS, sensor (Shake to Add, gyroscope), mini game, Currency Converter, Timezone Converter
Akun dan keamanan	Login, hashing password, session, biometrik, profil, kesan dan saran, logout
Data dan verifikasi	Database, web service/API, blockchain achievement

2.3 Pemetaan Fitur terhadap Ketentuan Tugas

Tabel 2.3.1 Pemetaan Fitur

NO	Ketentuan	Implementasi pada PLANIFY	Keterangan
1	Aplikasi mobile	Aplikasi Flutter/Dart	Rancangan 3.1, 3.8; Implementasi 4.1–4.2
2	Login dan session	Registrasi, login, logout, dan session pengguna melalui Supabase Auth	Rancangan 3.2.2 (KF-01, KF-02), 3.4 (UC-01), 3.5, 3.10; Implementasi 4.3; Uji 5.5
3	Hashing/enkripsi password	Password dikelola Supabase Auth dan disimpan dalam bentuk hash.	Rancangan 3.10; Implementasi 4.3
4	Biometric authentication	Akses sesi dengan biometrik pada perangkat yang kompatibel	Rancangan 3.2.2 (KF-03), 3.9.5, 3.10; Implementasi 4.16; Uji 5.6
5	Database	Hive (lokal) dan Supabase PostgreSQL: data pengguna/profil, agenda (task), habit, log habit, sesi fokus, Habit Record (achievement); reminder tersimpan sebagai bagian agenda.	Rancangan 3.7; Implementasi 4.4
6	Web service/API	LLM (AI Assistant), kurs (Budget Helper), pencarian tempat (LBS)	Rancangan 3.8, 3.9.1, 3.9.4, 3.9.7; Implementasi 4.10, 4.11, 4.14
7	LBS	Nearby Productivity	Rancangan 3.2.2 (KF-14), 3.9.1; Implementasi 4.11; Uji 5.6

8	Sensor accelerometer	Deteksi perangkat menghadap ke bawah untuk tawaran sesi fokus.	Rancangan 3.2.2 (KF-15), 3.9.2; Implementasi 4.12; Uji 5.6
9	Sensor gyroscope	Kontrol kemiringan pada mini game	Rancangan 3.2.2 (KF-16), 3.9.3; Implementasi 4.12–4.13
10	Mini game	Mini game bertema produktivitas	Rancangan 3.4 (UC-12), 3.9.3; Implementasi 4.13; Uji 5.6
11	Currency Converter (minimal 3 mata uang)	IDR, USD, MYR, SGD	Rancangan 3.2.2 (KF-17, KF-18), 3.9.7; Implementasi 4.14; Uji 5.6
12	Timezone Converter	WIB, WITA, WIT, London	Rancangan 3.2.2 (KF-20, KF-21), 3.9.8; Implementasi 4.15; Uji 5.6
13	Profile (foto, data, pengaturan, kesan dan saran, logout)	Halaman Profile	Rancangan 3.2.2 (KF-02, KF-12), 3.4 (UC-08), 3.8; Implementasi 4.16; Uji 5.6
14	Blockchain sederhana	Habit Record	Rancangan 3.2.2 (KF-22, KF-23), 3.9.9, 3.10; Implementasi 4.17; Uji 5.6
15	Reminder dan notifikasi	Smart Reminder	Rancangan 3.2.2 (KF-13), 3.9.6; Implementasi 4.7; Uji 5.5
16	Fitur tema planner	Home, Planner, Focus, Habit, AI Assistant	Rancangan 3.2.2 (KF-04

			s.d. KF-11), 3.3–3.5, 3.9.4; Implementasi 4.5–4.10; Uji 5.5

BAB III
PERANCANGAN REKAYASA KEBUTUHAN PERANGKAT LUNAK

3.1 Gambaran Umum Sistem

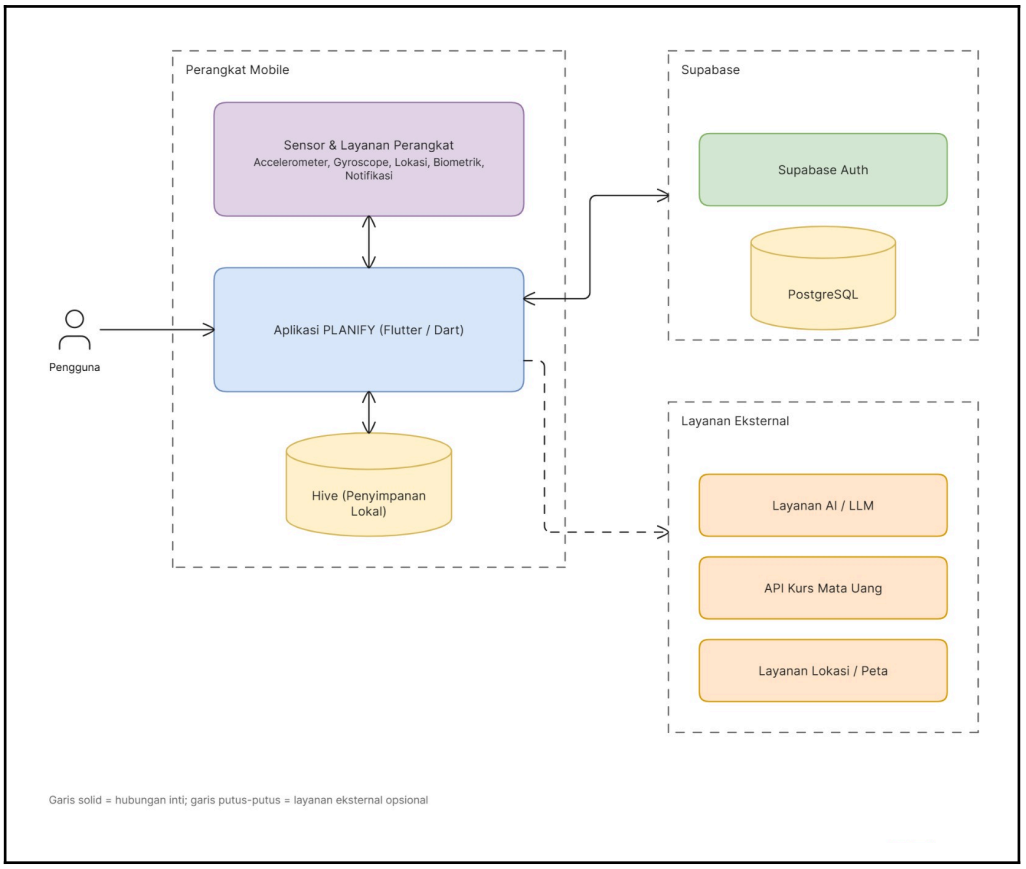
PLANIFY (Smart Daily Planner & Productivity Assistant) adalah aplikasi mobile yang membantu pengguna mengelola aktivitas harian dan meningkatkan produktivitas. Aplikasi ini mengumpulkan beberapa kebutuhan produktivitas dalam satu aplikasi: pengelolaan agenda, sesi fokus berbasis Pomodoro, pemantauan kebiasaan (habit), rekomendasi jadwal dari AI, serta fitur pendukung seperti pengingat, layanan berbasis lokasi, sensor perangkat, perencanaan anggaran, dan penjadwalan lintas zona waktu.

3.1.1 Alur Umum Penggunaan Aplikasi

Berikut adalah alur umum penggunaan aplikasi Planify.

1. Pengguna membuka aplikasi dan melakukan autentikasi (registrasi atau login melalui Supabase Auth). Pada perangkat yang kompatibel, pengguna dapat memakai autentikasi biometrik. Setelah autentikasi berhasil, pengguna masuk ke halaman Home yang menampilkan ringkasan aktivitas.
2. Pengguna mengelola agenda pada Planner, menjalankan sesi fokus pada Focus, dan mencatat habit.
3. Pengguna dapat meminta rekomendasi pada AI Assistant.
4. Pengguna dapat mengelola data akun pada Profile dan memakai fitur pendukung sesuai kebutuhan.

3.1.2 Diagram Arsitektur



Gambar 3.1.2.1 Diagram Arsitektur

3.2 Analisis Kebutuhan Sistem

Analisis kebutuhan dilakukan untuk menentukan layanan yang harus disediakan PLANIFY (kebutuhan fungsional) dan mutu yang diharapkan dari layanan tersebut (kebutuhan non-fungsional).

3.2.1 Identifikasi Pengguna

PLANIFY dirancang untuk satu aktor utama, yaitu Pengguna, yaitu individu yang memakai aplikasi untuk mengatur aktivitas dan meningkatkan produktivitas pribadinya.

Tabel 3.2.1.1 Identifikasi Pengguna

Pihak	Peran	Tujuan	Interaksi dengan Sistem
Pengguna	Mengelola kegiatan, habit, fokus, dan menggunakan seluruh	Mengelola agenda, menjaga fokus,	Melakukan autentikasi, mengelola agenda

	fitur setelah login	membangun kebiasaan, dan memperoleh rekomendasi jadwal	dan habit, menjalankan sesi fokus, berinteraksi dengan AI Assistant, memakai fitur pendukung, dan mengelola profil

Pada rancangan ini tidak ada aktor administrator pada sisi aplikasi. Layanan seperti Supabase, layanan AI, API kurs, dan layanan lokasi dipandang sebagai komponen sistem pendukung, bukan aktor yang berinteraksi langsung dengan sistem sebagai pengguna.

3.2.2 Kebutuhan Fungsional

Tabel 3.2.2.1 Kebutuhan Fungsional

ID	Nama Kebutuhan	Deskripsi	Prioritas / Keterangan
KF-01	Registrasi Akun	Pengguna dapat membuat akun melalui Supabase Auth.	Tinggi
KF-02	Login dan Logout	Pengguna dapat masuk dan keluar dari aplikasi, dan sesi dikelola oleh Supabase Auth.	Tinggi
KF-03	Autentikasi Biometrik	Pengguna dapat memakai biometrik pada perangkat yang kompatibel.	Tinggi
KF-04	Menampilkan Ringkasan Beranda	Halaman Home menampilkan ringkasan agenda, fokus, dan habit.	Tinggi
KF-05	Mengelola Agenda	Pengguna dapat menambah, melihat, mengubah, dan menghapus agenda.	Tinggi
KF-06	Menandai Status Agenda	Pengguna dapat menandai agenda sebagai selesai atau belum selesai.	Tinggi
KF-07	Menjalankan Sesi Fokus (Pomodoro)	Pengguna dapat memulai, menjeda, dan mengakhiri sesi fokus beserta jeda istirahat.	Tinggi

KF-08	Mengelola Habit	Pengguna dapat menambah, mengubah, dan menghapus habit yang dipantau.	Tinggi
KF-09	Mencatat Pencapaian Habit	Pengguna dapat mencatat pencapaian habit pada tanggal tertentu.	Tinggi
KF-10	Memantau Progres Habit	Aplikasi menampilkan riwayat dan perkembangan habit.	Sedang
KF-11	Memperoleh Rekomendasi AI	Pengguna dapat meminta rekomendasi jadwal atau saran produktivitas dari AI Assistant.	Tinggi
KF-12	Mengelola Profil	Pengguna dapat melihat dan memperbarui data profil.	Sedang
KF-13	Mengelola Notifikasi dan Pengingat	Aplikasi mengirim pengingat aktivitas dan pengguna dapat mengatur pengingat.	Tinggi
KF-14	Memanfaatkan Layanan Berbasis Lokasi	Aplikasi memakai lokasi pengguna untuk fitur terkait lokasi. Bentuk fiturnya.	Sedang
KF-15	Mendeteksi Posisi Perangkat (Accelerometer)	Aplikasi mendeteksi perangkat menghadap ke bawah dan menawarkan sesi fokus. Pengguna yang memutuskan memulai atau tidak.	Sedang
KF-16	Memainkan Permainan Keseimbangan (Gyroscope)	Pengguna memainkan permainan sederhana yang dikendalikan rotasi atau orientasi perangkat.	Rendah
KF-17	Mengkonversi Mata Uang (Budget Helper)	Pengguna dapat mengkonversi nilai antar mata uang.	Sedang
KF-18	Memperkirakan Anggaran Aktivitas	Pengguna dapat membuat estimasi anggaran aktivitas berdasarkan hasil konversi.	Sedang

KF-19	Mencatat Pengeluaran	Pengguna mencatat pengeluaran aktivitas. Fungsi ini hanya jika tersedia.	Rendah
KF-20	Mengonversi Waktu Antarzona (Time Zone Planner)	Pengguna memilih zona asal dan target untuk mengkonversi tanggal dan jam.	Sedang
KF-21	Menjadwalkan Agenda Berdasarkan Zona Waktu	Hasil konversi dipakai untuk membantu penjadwalan agenda. Dukungan implementasi.	Tinggi
KF-22	Mencatat Habit Record Berantai Hash	Pencapaian habit dicatat sebagai rantai hash SHA-256.	Sedang
KF-23	Memeriksa Integritas Habit Record	Aplikasi memeriksa apakah rantai catatan konsisten dan menampilkan hasilnya.	Sedang
KF-24	Menyimpan dan Menyinkronkan Data	Data disimpan di Hive dan/atau Supabase. Mekanisme sinkronisasi.	Tinggi

3.2.3 Kebutuhan Non-Fungsional

Kriteria berikut dirumuskan secara kualitatif dan dapat diuji lewat pengujian fungsional atau inspeksi. Tidak ada angka target performa yang ditetapkan, karena target tersebut belum diputuskan. Angka dapat ditambahkan setelah ada kesepakatan.

Tabel 3.2.3.1 Kebutuhan Non-Fungsional

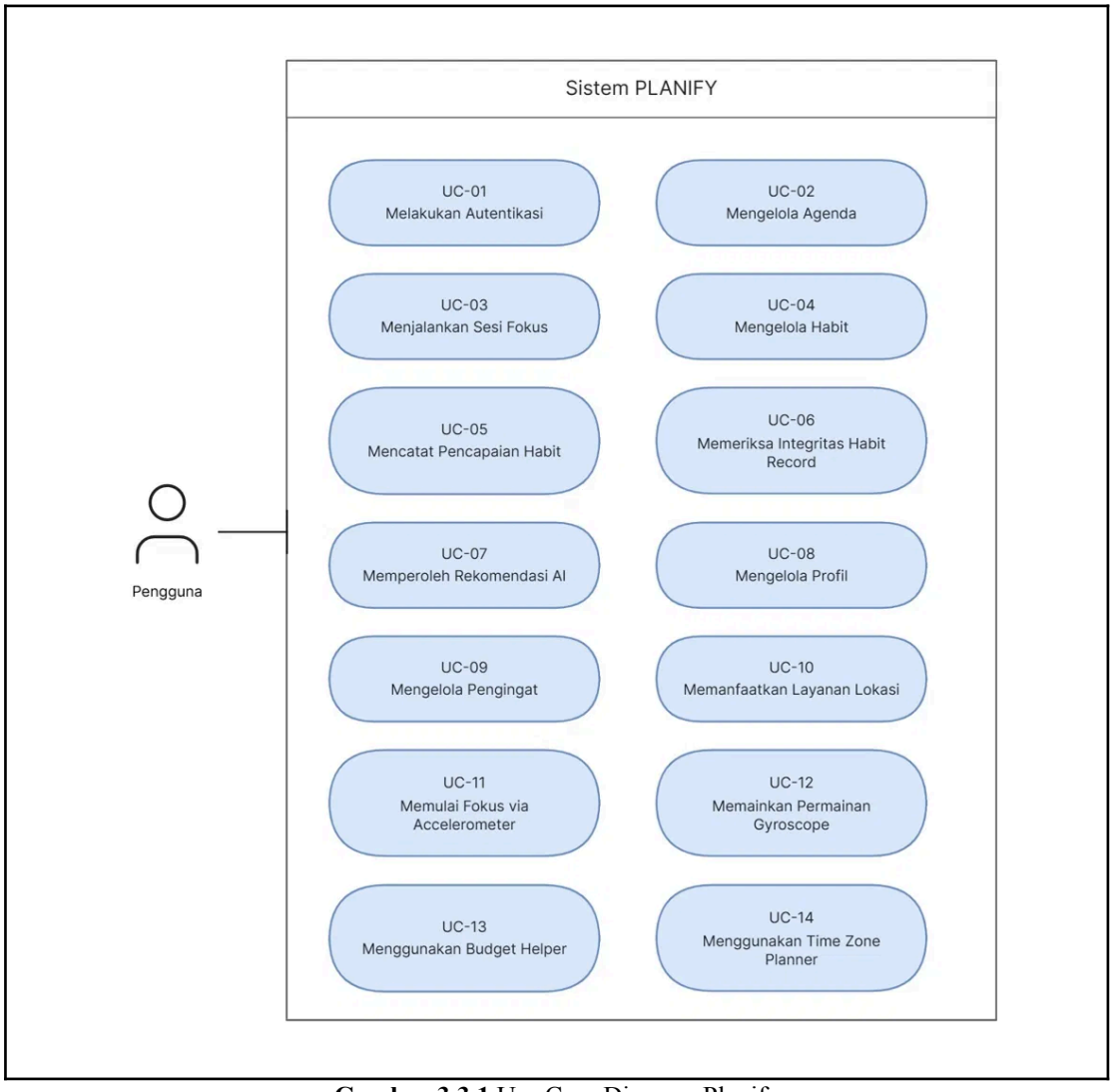
ID	Kategori	Deskripsi Kebutuhan	Kriteria / Indikator Pemenuhan
KNF-01	Usability	Antarmuka mudah dipahami dengan navigasi utama yang konsisten.	Lima menu utama dapat diakses dari navigasi, dan fitur utama dapat diselesaikan pengguna tanpa panduan tambahan pada uji coba pengguna.
KNF-02	Performance	Aplikasi tetap	Tidak ada hambatan

		responsif pada penggunaan normal.	yang berarti saat berpindah halaman dan menampilkan daftar agenda pada pengujian di perangkat uji. Target angka.
KNF-03	Security	Akses data dibatasi untuk pengguna terautentikasi.	Fitur utama tidak dapat diakses tanpa login, dan data pengguna tidak dapat dibaca akun lain. Validasi dilakukan lewat pengujian akses.
KNF-04	Security	Kredensial rahasia tidak disimpan di aplikasi klien.	Inspeksi kode dan konfigurasi tidak menemukan API key rahasia pada sisi klien.
KNF-05	Reliability	Aplikasi menangani kegagalan jaringan atau layanan eksternal dengan baik.	Saat koneksi gagal, aplikasi menampilkan pesan yang jelas dan tidak berhenti mendadak.
KNF-06	Reliability	Fitur lokal tetap berjalan tanpa internet sejauh bergantung pada Hive.	Fitur pada Hive tetap dapat dibuka saat perangkat luring. Cakupan fitur.
KNF-07	Compatibility	Aplikasi berjalan pada perangkat target.	Aplikasi dapat dijalankan pada versi Android uji. Versi minimum.
KNF-08	Compatibility	Fitur sensor dan biometrik bersifat opsional.	Pada perangkat tanpa sensor atau biometrik, fitur terkait menampilkan informasi ketidaktersediaan dan fitur lain tetap berjalan.
KNF-09	Maintainability	Kode terorganisasi per lapisan.	Struktur proyek memisahkan UI, logika, dan akses data.
KNF-10	Privacy	Data lokasi dan sensor dipakai secara terbatas dan dengan izin pengguna.	Aplikasi meminta izin saat dibutuhkan, dan fitur tetap menangani kondisi izin ditolak.

KNF-11	Data Integrity	Perubahan pada Habit Record dapat terdeteksi.	Pemeriksaan hash chain menandai ketidakkonsistenan jika catatan diubah.

3.3 *Use Case Diagram*

Use case diagram menggambarkan interaksi aktor dengan fungsi-fungsi sistem dari sudut pandang pengguna. Pada PLANIFY terdapat satu aktor, yaitu Pengguna, dan seluruh use case di bawah ini diturunkan dari kebutuhan fungsional pada subbab 3.2.2. Mermaid tidak mendukung notasi UML use case secara langsung, sehingga diagram dibuat dengan flowchart (aktor di luar kotak sistem, use case berbentuk oval). Tidak digunakan relasi include atau extend, karena belum ada dasar yang jelas.



Gambar 3.3.1 Use Case Diagram Planify

3.4 Deskripsi *Use Case*

Tabel 3.4.1 Deskripsi *Use Case*

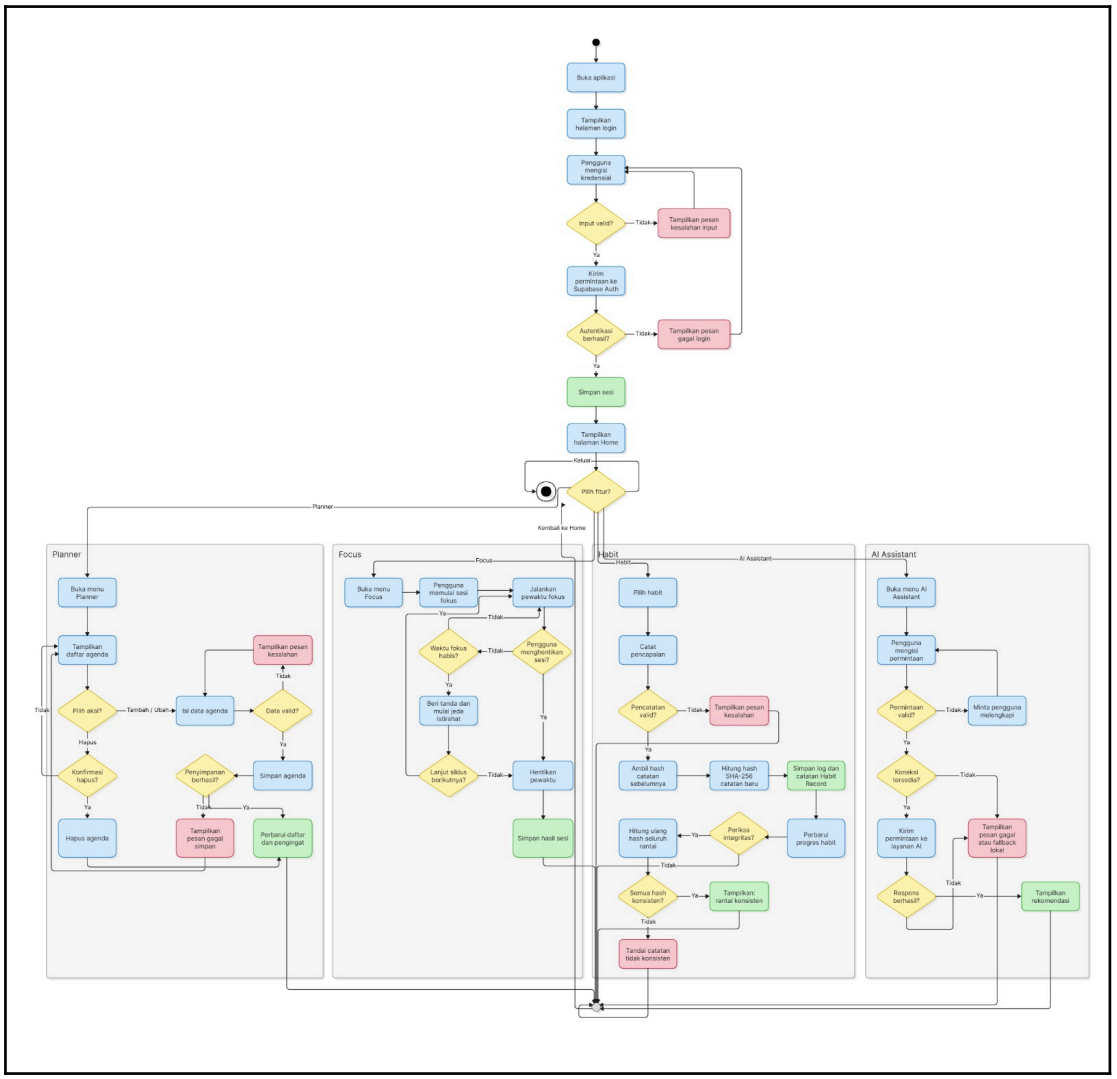
ID	Nama Use Case	Tujuan	Prasyarat	Kondisi Akhir
UC-01	Melakukan Autentikasi	Masuk ke aplikasi secara sah untuk mengakses data pribadi.	Aplikasi terpasang; perangkat terhubung internet untuk registrasi atau login.	Berhasil: pengguna masuk ke Home. Gagal: pengguna tetap di halaman autentikasi.
UC-02	Mengelola Agenda	Menambah, melihat, mengubah, menghapus, dan menandai status agenda.	Pengguna sudah login.	Data agenda tersimpan dan daftar agenda diperbarui.
UC-03	Menjalankan Sesi Fokus	Menjaga fokus kerja dengan teknik Pomodoro.	Pengguna sudah login dan berada di menu Focus, atau menerima tawaran dari UC-11.	Sesi selesai atau dihentikan dan hasilnya tercatat.
UC-04	Mengelola Haibit	Membuat, mengubah, menghapus, dan memantau habit.	Pengguna sudah login.	Data habit dan progres tersimpan serta ditampilkan.
UC-05	Mencatat Pencapaian Habit	Mencatat pencapaian habit pada tanggal tertentu dan membentuk Habit Record.	Pengguna sudah login dan sudah memiliki habit.	Log pencapaian dan catatan Habit Record tersimpan.
UC-06	Memeriksa Integritas Habit Record	Mengetahui apakah rantai catatan habit masih konsisten.	Pengguna sudah login dan memiliki Habit Record.	Hasil pemeriksaan (konsisten atau tidak) ditampilkan.
UC-07	Memperoleh Rekomendasi AI	Mendapatkan rekomendasi	Pengguna sudah login;	Rekomendasi ditampilkan sebagai

		jadwal atau saran produktivitas.	internet tersedia untuk pemrosesan AI.	saran; pengguna menentukan pemakaiannya.
UC-08	Mengelola Profil	Melihat dan memperbarui data profil.	Pengguna sudah login.	Data profil terbaru tersimpan.
UC-09	Mengelola Pengingat	Mengatur pengingat aktivitas dan menerima notifikasi.	Pengguna sudah login; izin notifikasi diberikan.	Pengingat terjadwal dan notifikasi tampil pada waktunya.
UC-10	Memanfaatkan Layanan Lokasi	Memakai lokasi pengguna untuk fitur berbasis lokasi [PERLU DIKONFIRMASI: bentuk fitur].	Pengguna sudah login; izin lokasi diberikan; layanan lokasi aktif.	Informasi berbasis lokasi ditampilkan.
UC-11	Memulai Fokus via Accelerometer	Menerima tawaran sesi fokus saat perangkat diletakkan menghadap ke bawah.	Perangkat memiliki accelerometer dan fitur aktif.	Sesi fokus dimulai hanya jika pengguna menyetujui.
UC-12	Memainkan Permainan Gyroscope	Memainkan permainan keseimbangan sederhana berbasis rotasi perangkat.	Perangkat memiliki gyroscope.	Permainan berjalan atau diakhiri pengguna.
UC-13	Menggunakan Budget Helper	Mengkonversi mata uang dan memperkirakan anggaran aktivitas.	Pengguna sudah login; internet tersedia bila kurs diambil dari layanan eksternal.	Hasil konversi atau estimasi anggaran ditampilkan.
UC-14	Menggunakan Time Zone Planner	Mengkonversi tanggal dan jam antar zona	Pengguna sudah login.	Hasil konversi waktu ditampilkan.

		waktu untuk membantu penjadwalan.		

3.5 *Activity Diagram*

Activity diagram menggambarkan alur aktivitas, percabangan keputusan, dan kondisi berhasil maupun gagal pada suatu proses. Pada PLANIFY, lima alur utama aplikasi digabungkan menjadi satu diagram agar keterkaitan antara proses terlihat jelas. Kelima alur tersebut adalah autentikasi pengguna, pengelolaan agenda, sesi fokus (Pomodoro), pencatatan habit serta pemeriksaan integritas Habit Record, dan penggunaan AI Assistant. Alur autentikasi berada di bagian awal diagram. Keempat alur lainnya merupakan percabangan fitur yang dapat dipilih pengguna setelah masuk ke halaman Home.



Gambar 3.5.1 *Activity Diagram* Login

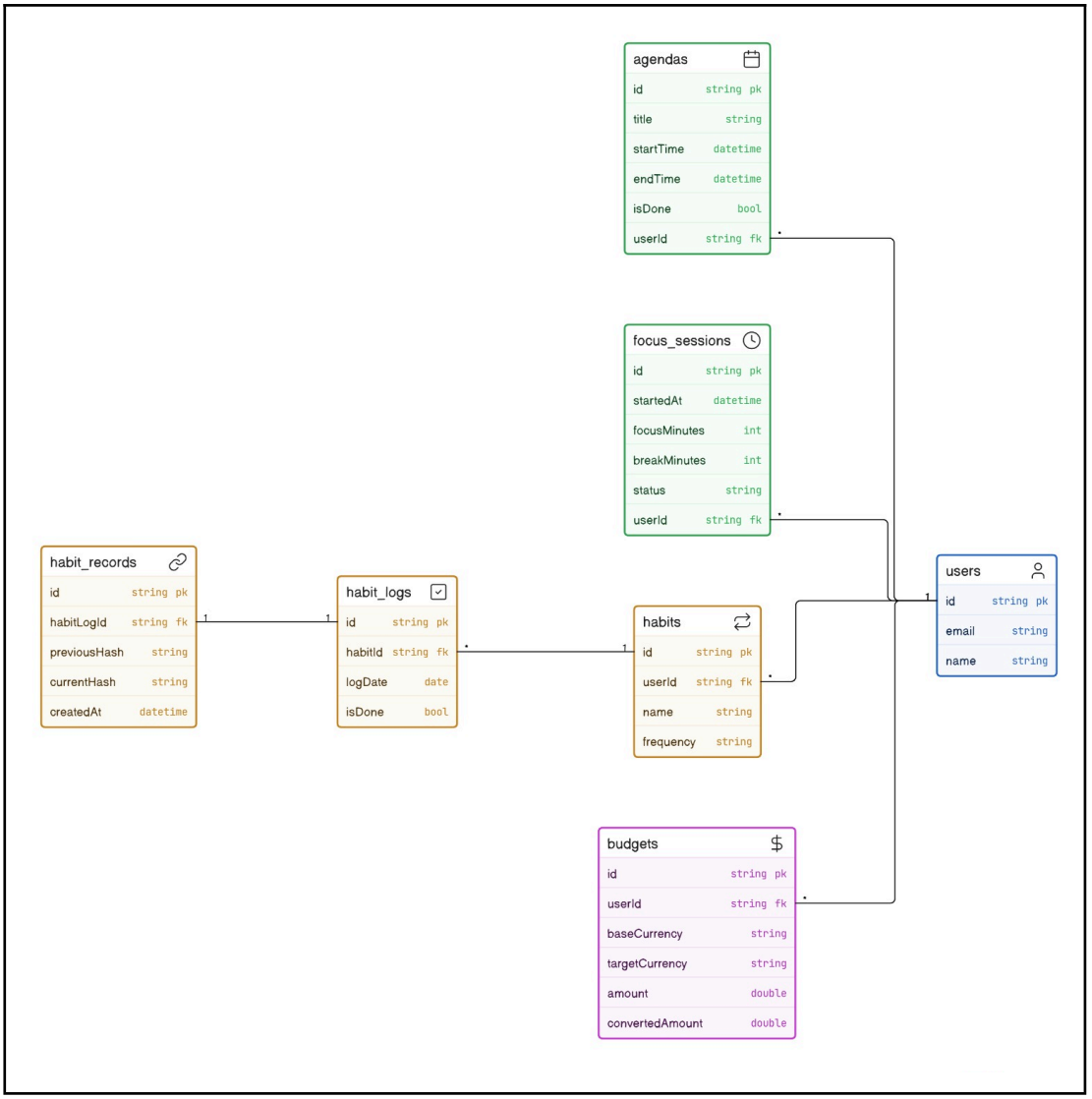
Penjelasan Diagram

1. Alur Autentikasi

   Proses dimulai ketika pengguna membuka aplikasi dan sistem menampilkan halaman login. Pengguna mengisi kredensial, kemudian sistem memvalidasi input. Jika input tidak valid, sistem menampilkan pesan kesalahan dan pengguna kembali mengisi kredensial. Jika input valid, sistem mengirim permintaan ke Supabase Auth. Apabila autentikasi gagal, sistem menampilkan pesan gagal login dan pengguna kembali ke tahap pengisian kredensial. Apabila berhasil, sistem menyimpan sesi dan menampilkan halaman Home. Peran autentikasi biometrik dalam alur ini belum digambarkan karena implementasinya masih perlu dikonfirmasi.

3.6 *Class Diagram*

Class diagram bersifat konseptual. Kelas ditentukan dari kebutuhan sistem dan bukan dari nama kelas aktual pada *source code*. Diagram memuat kelas domain dan dua service penting. Komponen UI tidak dimasukkan agar diagram tetap terbaca.



Gambar 3.6.1 *Class Diagram*

Penjelasan:

- Model domain: User, Agenda, FocusSession, Habit, HabitLog, HabitRecord, dan Budget.
- Service: HabitRecordService membuat dan memverifikasi rantai hash, sedangkan AIService mengelola permintaan rekomendasi. Penggunaan konteks agenda oleh AIService masih perlu dikonfirmasi.
- Relasi: satu User memiliki banyak Agenda, FocusSession, Habit, dan Budget; satu Habit memiliki banyak HabitLog; satu HabitLog membentuk paling banyak satu HabitRecord.
- Model data Time Zone Planner tidak dijadikan kelas tersendiri karena fungsinya berupa konversi dan hasilnya belum tentu disimpan. Repository tidak ditampilkan karena belum ada acuan arsitektur aktual.

3.7 Perancangan Database

3.7.1 Strategi Penyimpanan Data

Tabel 3.7.1.1 Strategi Penyimpanan Data

No	Penyimpanan	Peran
1	Hive (lokal)	Penyimpanan data di perangkat pengguna, misalnya data agenda, habit, dan pengaturan.
2	Supabase PostgreSQL	Database online untuk data yang disimpan di sisi server.
3	Supabase Auth	Pengelolaan akun, kredensial, dan sesi autentikasi. Data kredensial tidak dirancang sebagai tabel aplikasi.
4	Cache / data pendukung	Penyimpanan sementara, misalnya hasil kurs terakhir.

3.7.2 Rancangan Entitas

Tabel 3.7.2.1 Rancangan Entitas

Entitas	Atribut Utama	Tipe Data	PK	FK / Relasi	Fungsi
profiles	id, nama, email	uuid, text, text	id	id merujuk akun Supabase Auth	Menyimpan data profil pengguna.
agendas	id, user_id, judul, waktu_mulai, waktu_selesai, status_selesai	uuid, uuid, text, timestamp, timestamp, boolean	id	user_id → profiles.id	Menyimpan agenda.
focus_sessions	id, user_id, waktu_mulai, durasi_fokus, durasi_istirahat, status	uuid, uuid, timestamp, integer, integer, text	id	user_id → profiles.id	Menyimpan riwayat sesi fokus.
habits	id, user_id, nama, frekuensi	uuid, uuid, text, text	id	user_id → profiles.id	Menyimpan daftar habit.
habit_logs	id, habit_id, tanggal, selesai	uuid, uuid, date, boolean	id	habit_id → habits.id	Menyimpan pencapaian habit.

habit_records	id, habit_log_id, previous_hash, current_hash, created_at	uuid, uuid, text, text, timestamp	id	habit_log_id → habit_logs.id	Menyimpan catatan rantai hash.
budgets	id, user_id, mata_uang_asal, mata_uang_target, nominal, hasil_konversi	uuid, uuid, text, text, numeric, numeric	id	user_id → profiles.id	Menyimpan estimasi anggaran **[PERLU DIKONFIRMASI: apakah disimpan]**.

3.8 Perancangan Arsitektur Aplikasi

Arsitektur dirancang berlapis agar tanggung jawab terpisah. Pola yang digunakan (misalnya Provider, Riverpod, Bloc, atau Clean Architecture) dan struktur folder aktual belum diketahui, sehingga pembagian berikut bersifat konseptual.

Tabel 3.8.1 Perancangan Arsitektur Aplikasi

Lapisan	Tanggung Jawab
Presentation / UI	Halaman dan widget (Home, Planner, Focus, AI Assistant, Profile, dan halaman pendukung).
State Management	Mengelola status tampilan dan menghubungkan UI dengan logika. Pendekatan.
Repository / Service	Logika akses data, aturan bisnis, hash chain, dan pemanggilan layanan eksternal.
Local Data Source	Akses ke Hive.
Remote Data Source	Akses ke Supabase (Auth dan PostgreSQL).
Layanan Eksternal / Perangkat	Layanan AI, API kurs, lokasi, sensor, biometrik, dan notifikasi.

Penjelasan aliran data: Aksi pengguna pada UI diteruskan ke state management, lalu ke lapisan repository/service yang memilih sumber data: lokal (Hive), remote (Supabase), perangkat, atau layanan eksternal. Hasilnya dikembalikan melalui lapisan yang sama hingga UI diperbarui.

Fitur yang bekerja secara lokal (tanpa internet, selama datanya berada di Hive atau perangkat): pewaktu fokus, deteksi accelerometer, permainan gyroscope,

notifikasi lokal, dan konversi waktu antar zona (bila dihitung di perangkat). Cakupan pastinya.

Fitur yang membutuhkan internet: autentikasi (Supabase Auth), akses data PostgreSQL, rekomendasi AI, pengambilan kurs mata uang (bila dari API), dan layanan lokasi/peta tertentu.

3.9 Perancangan Fitur Pendukung

3.9.1 Perancangan Location Based Service (LBS)

Tabel 3.9.1.1 Perancangan *Location Based Service* (LBS)

Aspek	Penjelasan
Tujuan	Mendukung fitur yang memanfaatkan lokasi pengguna dalam perencanaan aktivitas. Bentuk fitur.
Input	Izin lokasi dan koordinat dari perangkat.
Proses	Aplikasi meminta izin, mengambil lokasi, dan memakainya untuk fitur terkait (misalnya menampilkan lokasi agenda).
Output	Informasi atau tampilan berbasis lokasi.
Komponen	Layanan lokasi perangkat, aplikasi Flutter, serta layanan peta/geokode bila digunakan.
Kegagalan / Batasan	Izin ditolak, GPS nonaktif, atau sinyal lemah. Fitur menampilkan pesan dan fitur lain tetap berjalan.

3.9.2 Perancangan Accelerometer

Tabel 3.9.2.1 Perancangan Accelerometer

Aspek	Penjelasan
Tujuan	Mendeteksi pola posisi perangkat (misalnya menghadap ke bawah) sebagai pemicu tawaran memulai sesi fokus.
Input	Data percepatan tiga sumbu dari accelerometer.
Proses	Aplikasi membaca data sensor, mengenali pola posisi, lalu menampilkan tawaran sesi fokus.
Output	Tawaran memulai sesi. Sesi baru dimulai bila pengguna menyetujui.

Komponen	Sensor accelerometer, aplikasi Flutter, modul Focus.
Kegagalan / Batasan	Sensor tidak tersedia, pola salah terdeteksi, atau pengguna menolak. Sensor ini berbeda dengan GPS dan kompas.

3.9.3 Perancangan Gyroscope

Tabel 3.9.3.1 Perancangan Gyroscope

Aspek	Penjelasan
Tujuan	Mendukung permainan keseimbangan sederhana berbasis rotasi atau orientasi perangkat.
Input	Data rotasi dari gyroscope.
Proses	Aplikasi membaca data rotasi dan menggerakkan elemen permainan sesuai orientasi perangkat.
Output	Tampilan permainan yang bergerak mengikuti rotasi perangkat.
Komponen	Sensor gyroscope dan aplikasi Flutter.
Kegagalan / Batasan	Sensor tidak tersedia pada sebagian perangkat. Sensor ini tidak berhubungan dengan GPS atau kompas.

3.9.4 Perancangan AI Assistant

Tabel 3.9.4.1 Perancangan AI Assistant

Aspek	Penjelasan
Tujuan	Memberikan rekomendasi jadwal atau saran produktivitas.
Input	Permintaan teks pengguna dan, bila dirancang, konteks agenda.
Proses	Aplikasi menyusun permintaan dan mengirimkannya ke layanan AI/LLM. Layanan memproses dan mengembalikan jawaban, kemudian aplikasi menyajikannya. Penyedia layanan.
Output	Rekomendasi atau saran yang ditampilkan di menu AI Assistant.
Komponen	Menu AI Assistant, layanan AI, dan jalur pemanggilan API.

Kegagalan / Batasan	Koneksi atau layanan gagal; jawaban dapat kurang akurat. Bila ada fallback aturan lokal, pemrosesan AI aktual dan fallback harus dibedakan. API key rahasia tidak boleh disimpan di aplikasi klien. Pemanggilan sebaiknya melalui perantara sisi server, misalnya Supabase Edge Function.

3.9.5 Perancangan Biometric Authentication

Tabel 3.9.5.1 Perancangan Biometric Authentication

Aspek	Penjelasan
Tujuan	Mendukung autentikasi biometrik pada perangkat yang kompatibel.
Input	Biometrik pengguna yang diproses oleh sistem operasi.
Proses	Aplikasi meminta verifikasi biometrik pada sistem operasi dan menerima hasil berhasil atau gagal. Aplikasi tidak menyimpan data biometrik.
Output	Akses diberikan atau ditolak.
Komponen	API biometrik perangkat dan aplikasi Flutter. Perannya terhadap Supabase Auth.
Kegagalan / Batasan	Perangkat tidak mendukung atau belum terdaftar biometrik; verifikasi gagal berulang. Pengguna perlu jalur autentikasi alternatif.

3.9.6 Perancangan Notifikasi dan Pengingat

Tabel 3.9.6.1 Perancangan Notifikasi dan Pengingat

Aspek	Penjelasan
Tujuan	Mengingatkan pengguna pada agenda atau aktivitas.
Input	Waktu dan keterangan pengingat dari agenda, serta izin notifikasi.
Proses	Aplikasi menjadwalkan notifikasi lokal pada waktu yang ditentukan dan menampilkannya saat waktunya tiba.
Output	Notifikasi pada perangkat.
Komponen	Layanan notifikasi perangkat dan modul Planner.

Kegagalan / Batasan	Izin ditolak, mode hemat daya yang membatasi aplikasi, atau agenda diubah/dihapus sehingga pengingat perlu diperbarui.

3.9.7 Perancangan Budget Helper

Tabel 3.9.7.1 Perancangan Budget Helper

Aspek	Penjelasan
Tujuan	Membantu perencanaan anggaran aktivitas melalui konversi mata uang.
Input	Mata uang asal dan target, serta nominal.
Proses	Aplikasi memperoleh kurs (dari API eksternal), menghitung nilai konversi, lalu menampilkannya sebagai dasar estimasi anggaran. Bila tersedia, nilai estimasi dapat dibandingkan dengan pencatatan pengeluaran.
Output	Nilai konversi dan estimasi anggaran.
Komponen	Modul Budget Helper dan API kurs.
Kegagalan / Batasan	Kurs tidak tersedia saat luring; kurs bersifat perkiraan dan dapat berubah. Pencatatan pengeluaran hanya dijelaskan bila fungsinya tersedia.

3.9.8 Perancangan Time Zone Planner

Tabel 3.9.8.1 Perancangan Time Zone Planner

Aspek	Penjelasan
Tujuan	Membantu penjadwalan aktivitas pada zona waktu berbeda.
Input	Zona asal, zona target, tanggal, dan jam.
Proses	Aplikasi mengkonversi tanggal dan jam dari zona asal ke zona target, lalu menampilkan hasilnya. Hasil dapat dipakai untuk menjadwalkan agenda bila didukung.
Output	Tanggal dan jam pada zona target.
Komponen	Modul Time Zone Planner dan data zona waktu (pustaka atau layanan).
Kegagalan / Batasan	Perubahan tanggal akibat selisih zona, aturan waktu musim panas, atau data zona waktu yang

	tidak mutakhir.

3.9.9 Perancangan Habit Record dan Hash Chain

Tabel 3.9.9.1 Perancangan Habit Record dan Hash Chain

Aspek	Penjelasan
Tujuan	Menghubungkan riwayat pencapaian habit dalam rantai hash dan membantu mendeteksi perubahan pada catatan.
Input	Data pencapaian habit (habit, tanggal, status) dan hash catatan sebelumnya.
Proses	Saat pencapaian dicatat, aplikasi menggabungkan data catatan dengan hash catatan sebelumnya dan menghitung hash SHA-256 sebagai hash catatan baru. Pemeriksaan integritas menghitung ulang hash berurutan dan membandingkannya dengan hash tersimpan.
Output	Catatan berantai dan hasil pemeriksaan (konsisten atau tidak).
Komponen	`HabitRecordService`, penyimpanan lokal, dan halaman Habit Record.

3.9.10 Perancangan Sinkronisasi dan Penyimpanan Data

Tabel 3.9.10.1 Perancangan Sinkronisasi dan Penyimpanan Data

Aspek	Penjelasan
Tujuan	Mengatur penyimpanan data di perangkat (Hive) dan hubungannya dengan Supabase.
Input	Data aplikasi (agenda, habit, sesi fokus, dll.) dan status koneksi.
Proses	Data disimpan di Hive dan/atau Supabase sesuai implementasi. Arah sinkronisasi, pemicu, dan penanganan konflik belum terkonfirmasi.
Output	Data tersedia sesuai penyimpanan yang dipakai.
Komponen	Hive, Supabase PostgreSQL, dan repository/service.
Kegagalan / Batasan	Koneksi terputus, perbedaan data lokal dan

	online, serta kegagalan tulis. Laporan ini tidak mengklaim sinkronisasi dua arah, offline-first, atau otomatis sebelum dikonfirmasi.

3.10 Perancangan Keamanan

Tabel 3.10.1 Perancangan Keamanan

Aspek	Rancangan	Status
Autentikasi dan sesi	Menggunakan Supabase Auth, dengan sesi dikelola Supabase.	Rancangan
Biometric Authentication	Verifikasi lewat API sistem operasi, tanpa menyimpan data biometrik.	Rancangan
Validasi input	Validasi pada form (kolom wajib, format, rentang nilai) sebelum data disimpan atau dikirim.	Rancangan
Perlindungan akses data	Data dikaitkan dengan pengguna pemilik. Pembatasan akses di sisi server direkomendasikan memakai Row Level Security.	Rekomendasi
Komunikasi dengan backend	Menggunakan koneksi terenkripsi (HTTPS).	Rancangan
Kredensial dan API key	Kunci rahasia (misalnya layanan AI) tidak disimpan di klien dan sebaiknya dipanggil lewat perantara sisi server. Kunci publik Supabase (anon key) tetap bergantung pada kebijakan akses.	Rekomendasi
Kebijakan akses Supabase dan RLS	Kebijakan RLS membatasi baca/tulis tiap tabel pada baris milik pengguna.	Rekomendasi
Privasi lokasi dan sensor	Izin diminta saat dibutuhkan, data hanya dipakai untuk fitur terkait, dan penolakan izin ditangani.	Rancangan
Penanganan error	Pesan kesalahan ditampilkan tanpa membocorkan detail teknis internal.	Rancangan
Integritas Habit Record	Hash chain SHA-256 untuk mendeteksi perubahan.	Rancangan

BAB IV
IMPLEMENTASI DAN PENJELASAN SOURCE CODE

4.1 Implementasi Lingkungan Pengembangan

Pengembangan aplikasi Planify memanfaatkan kerangka kerja *multi-platform* Flutter dengan bahasa pemrograman Dart. Bagian ini merinci spesifikasi perangkat keras, perangkat lunak, serta pustaka (*library*) eksternal yang menjadi fondasi pembangunan aplikasi.

4.1.1 Perangkat Keras

Pengembangan, kompilasi, dan pengujian aplikasi Planify dilakukan pada lingkungan perangkat keras komputer pengembang (*workstation*) dan perangkat uji eksekusi berbasis Android:

1. Perangkat Komputer Pengembang (Development Workstation):
    - Prosesor (CPU): Arsitektur x86_64 multi-core dengan clock speed tinggi (AMD Ryzen 5/7 atau Intel Core i5/i7) yang mendukung instruksi virtualisasi perangkat keras (AMD-V / Intel VT-x) untuk menjalankan emulator Android secara responsif.
    - Memori Utama (RAM): Kapasitas 16 GB DDR4/DDR5 guna mendukung eksekusi terdistribusi antara editor kode, proses daemon Gradle, Dart Development Service (DDS), dan emulator sistem operasi.
    - Media Penyimpanan: Solid State Drive (SSD) NVMe dengan ruang kosong minimal 50 GB untuk penampungan Android SDK, Gradle cache, file build artefak, dan dependensi lokal.
    - Periferal Masukan/Keluaran: Monitor Full HD (1920×1080) untuk inspeksi tata letak visual antarmuka pengguna.
2. Perangkat Pengujian (*Testing Devices*):
    - *Smartphone* Fisik Android: Perangkat smartphone fisik dengan sistem operasi Android 10 (API level 29) hingga Android 14/15 yang dibekali sensor akselerometer 3-sumbu, giroskop, modul GPS fisik, serta pemindai biometrik sidik jari (fingerprint scanner). Keberadaan perangkat fisik krusial untuk memverifikasi interaksi sensor gerak dan biometrik nyata.

4.1.2 Perangkat Lunak

Lingkungan perangkat lunak mencakup sistem operasi, perangkat *toolchain* kompilasi, kerangka kerja aplikasi, hingga penyedia layanan *backend-as-a-service* (BaaS) dan *Application Programming Interface* (API) eksternal. Rincian perangkat lunak disajikan pada Tabel 4.1.

Tabel 4.1.2.1 Perangkat Lunak

Komponen Perangkat Lunak	Versi / Rilis	Peran dan Fungsi Utama
Sistem Operasi Pengembang	Microsoft Windows 11 Pro 64-bit	Lingkungan kerja pengembangan dan eksekusi kompilasi utama.
Kerangka Kerja (Framework)	Flutter SDK 3.47.7 (syarat minimum ≥ 3.38.1)	Framework utama untuk pembangunan antarmuka UI reaktif multiplatform.
Bahasa Pemrograman	Dart SDK 3.13.5 (syarat minimum ≥ 3.9.0)	Bahasa pemrograman berorientasi objek modern dengan dukungan null safety statis ketat.
Java Development Kit (JDK)	OpenJDK 17 (Java 17 LTS)	Kompiler Java untuk menjalankan Gradle dan toolchain kompilasi Android.
Android Build Environment	Android SDK 36, Gradle 8.x, AGP 8.x	Target kompilasi Android dengan minSdk 24, compileSdk 36, dan targetSdk 36.
Integrated Development Environment (IDE)	Visual Studio Code dan Android Studio Ladybug	Editor kode utama dengan ekstensi Flutter, Dart, dan GitLens.
Backend Cloud Platform	Supabase Cloud Platform (Region Seoul)	Menyediakan PostgreSQL 15+, GoTrue Authentication, dan Row Level Security (RLS).
Serverless Runtime	Supabase Edge	Menjalankan fungsi

Engine	Functions (Deno 2.x)	serverless berbasis Deno untuk logika penyusunan jadwal cerdas.
Layanan Model Bahasa (LLM)	Google Gemini 2.5 Flash-Lite	Model AI untuk penyusunan jadwal otomatis melalui Supabase Secrets di sisi server.
Layanan Geospasial / LBS	Geoapify Places API v2 dan OpenStreetMap	Mengambil data POI (Point of Interest) tempat produktif dan menyediakan tile peta digital.
Layanan Kurs Mata Uang	Frankfurter API (European Central Bank)	Menyediakan data nilai tukar mata uang global harian tanpa memerlukan kredensial API.

4.1.3 Teknologi dan Library yang Digunakan

Aplikasi Planify mengintegrasikan sejumlah pustaka (*package*) pihak ketiga yang tercatat pada berkas konfigurasi `pubspec.yaml`. Pustaka-pustaka ini dipilih untuk mendukung arsitektur *offline-first*, efisiensi memori, serta keandalan fungsional:

1. *State Management*: `flutter_riverpod` (^3.4.3), kerangka manajemen state deklaratif dan reaktif tanpa generator kode (*non-code-generation*), memisahkan lapisan antarmuka dari logika bisnis.
2. Navigasi dan Penjaluran: `go_router` (^17.0.0), routing deklaratif berbasis URI dengan dukungan percabangan stateful shell (*StatefulNavigationShell*), *deep linking*, dan *redirect guard*.
3. Autentikasi dan *Backend Cloud*: `supabase_flutter` (^2.18.3), klien resmi Supabase untuk menangani pendaftaran, sesi GoTrue, pertukaran token, dan pemanggilan Edge Functions.
4. Keamanan dan Biometrik: `local_auth` (^3.0.2) untuk verifikasi sidik jari/wajah lokal, serta `flutter_secure_storage` (^11.2.0) untuk menyimpan token sesi dan kredensial Keystore terenkripsi.
5. Penyimpanan Lokal NoSQL: `hive_ce` (^2.22.0) dan `hive_ce_flutter` (^2.5.0), basis data lokal berbasis dokumen biner berkecepatan tinggi

yang menyimpan rekaman data dalam bentuk string JSON per kotak (*box*).

6. Notifikasi dan Pewaktuan: `flutter_local_notifications` (^22.3.2) untuk penjadwalan alarm presisi Android, dipadukan dengan pustaka zona waktu `timezone` (^0.11.2) dan `flutter_timezone` (^5.1.1).
7. Sensor Perangkat Keras: `sensors_plus` (^7.1.1) untuk membaca aliran data sensor akselerometer dan giroskop perangkat secara real-time.
8. Layanan Geospasial: `geolocator` (14.0.1) untuk mendeteksi koordinat GPS pengguna, serta `flutter_map` (^8.3.2) dan `latlong2` (^0.9.1) untuk merender peta interaktif OpenStreetMap.
9. Kriptografi dan Jaringan: `crypto` (^3.0.7) untuk kalkulasi intisari hash SHA-256 pada Habit Consistency Record, `http` (^1.6.0) untuk komunikasi data HTTP REST, `uuid` (^4.6.0) untuk pembuatan pengenal unik entitas, serta `connectivity_plus` (^7.3.2) untuk memantau konektivitas jaringan.

4.2 Implementasi Struktur Project

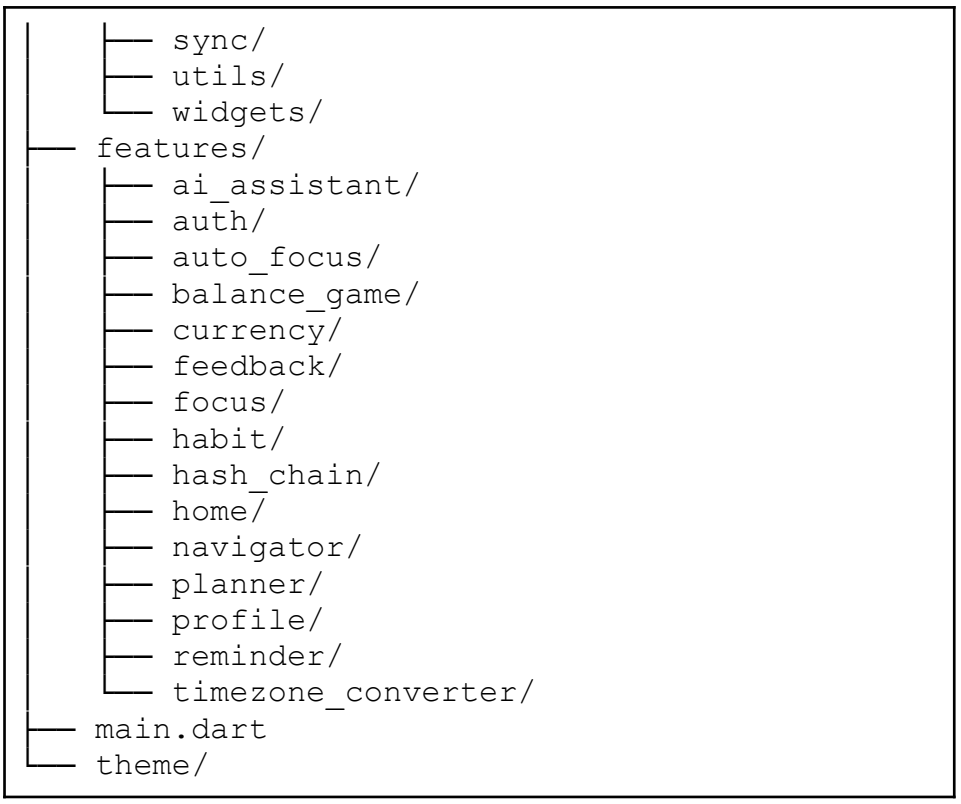
Arsitektur kode Planify dirancang menggunakan pendekatan feature-first layered architecture, dimana setiap fitur dipartisi ke dalam direktori modular yang mandiri.

4.2.1 Struktur Folder dan File

Struktur direktori utama di dalam folder `lib/` disusun sebagai berikut:

Tabel 4.2.1.1 Struktur Folder dan File

```
lib/
├── app/
│   ├── app.dart
│   ├── launch_animation_controller.dart
│   ├── theme_mode_controller.dart
│   ├── dev/
│   ├── router/
│   ├── shell/
│   └── sync/
├── bootstrap.dart
├── core/
│   ├── config/
│   ├── error/
│   ├── network/
│   ├── notifications/
│   ├── sensors/
│   ├── storage/
```



4.2.2 Pembagian Fungsi File

Pemisahan tanggung jawab kode mengikuti prinsip Separation of Concerns (SoC) dan Single Responsibility Principle (SRP), yang dibagi menjadi empat lapisan berikut:

1. Lapisan Presentasi (presentation/): Menangani antarmuka visual menggunakan ConsumerWidget atau ConsumerStatefulWidget. Lapisan ini mengamati state dari Riverpod notifier dan meneruskan interaksi pengguna ke controller terkait.
2. Lapisan Aplikasi (application/): Mengelola state dan alur kerja aplikasi, termasuk koordinasi sesi, autentikasi biometrik, serta siklus hidup aplikasi.
3. Lapisan Domain (domain/): Berisi entitas data (entity), aturan validasi input, dan logika murni (pure logic) tanpa ketergantungan pada UI. Contohnya meliputi algoritma Pomodoro, deteksi orientasi menggunakan akselerometer, dan rantai hash SHA-256.
4. Lapisan Data (data/): Mengelola komunikasi dengan penyedia data, meliputi basis data lokal NoSQL Hive, penyimpanan aman Android Keystore, API eksternal, dan Supabase sebagai layanan backend (Backend-as-a-Service atau BaaS).

4.3 Implementasi Autentikasi Pengguna

Modul autentikasi bertanggung jawab atas siklus hidup identitas pengguna, mencakup registrasi akun baru, proses masuk berbasis kata sandi, autentikasi cepat menggunakan biometrik perangkat keras, pengelolaan status sesi, dan proses keluar sistem (*logout*).

4.3.1 Implementasi Registrasi

Proses pendaftaran akun memvalidasi kelayakan nama lengkap, alamat email, serta kekuatan kata sandi sebelum mengirimkan permintaan ke Supabase Auth. Seluruh aturan validasi masukan diisolasi pada berkas domain `auth_validators.dart`.

Kode Program 4.3.1.1 Implementasi Registrasi

```
abstract final class AuthValidators {
  static const int minPasswordLength = 8;
  static const int minNameLength = 2;
  /// Sama dengan check `profiles.display_name` (1–50) di
Tahap 2 §7.3.
  static const int maxNameLength = 50;

  static final RegExp _email =
      RegExp(r'^[^\s@]+@[^\s@]+\.[^\s@]{2,}$');
  static final RegExp _hasLetter = RegExp(r'[A-Za-z]');
  static final RegExp _hasDigit = RegExp(r'\d');

  static String? email(String? value) {
    final String text = value?.trim() ?? '';
    if (text.isEmpty) {
      return 'Email wajib diisi.';
    }
    if (!_email.hasMatch(text)) {
      return 'Format email belum benar. Contoh:
nama@contoh.id';
    }
    return null;
  }

  /// Untuk login: cukup tidak kosong (aturan kekuatan hanya
saat membuat).
  static String? passwordRequired(String? value) {
    if (value == null || value.isEmpty) {
      return 'Password wajib diisi.';
    }
    return null;
  }

  /// Untuk membuat/mengganti password.
  static String? newPassword(String? value) {
    if (value == null || value.isEmpty) {
      return 'Password wajib diisi.';
    }
    if (value.length < minPasswordLength ||
        !_hasLetter.hasMatch(value) ||
```

```
        !_hasDigit.hasMatch(value)) {
      return 'Minimal $minPasswordLength karakter, berisi
huruf dan angka.';
    }
    return null;
  }

  /// Mengembalikan validator yang membandingkan dengan
password asli.
  static FormFieldValidatorString confirmPassword(
    String Function() original,
  ) {
    return (String? value) {
      if (value == null || value.isEmpty) {
        return 'Ulangi password untuk memastikan tidak salah
ketik.';
      }
      if (value != original()) {
        return 'Password belum sama. Periksa kembali.';
      }
      return null;
    };
  }

  static String? fullName(String? value) {
    final String text = value?.trim() ?? '';
    if (text.isEmpty) {
      return 'Nama wajib diisi.';
    }
    if (text.length < minNameLength) {
      return 'Nama minimal $minNameLength karakter.';
    }
    if (text.length > maxNameLength) {
      return 'Nama maksimal $maxNameLength karakter.';
    }
    return null;
  }
}
```

Penjelasan Source Code auth_validators.dart

1. Identitas File

   Berkas auth_validators.dart terletak pada direktori lib/features/auth/domain/auth_validators.dart. File ini berfungsi menyediakan fungsi validasi untuk formulir autentikasi, meliputi pendaftaran akun, login, dan reset kata sandi.

2. Konstanta dan Ekspresi Reguler (Regex)

   - Baris 7–10 mendefinisikan batasan panjang input, yaitu minPasswordLength = 8, minNameLength = 2, dan maxNameLength = 50. Batasan nama disesuaikan dengan constraint basis data PostgreSQL pada kolom profiles.display_name.

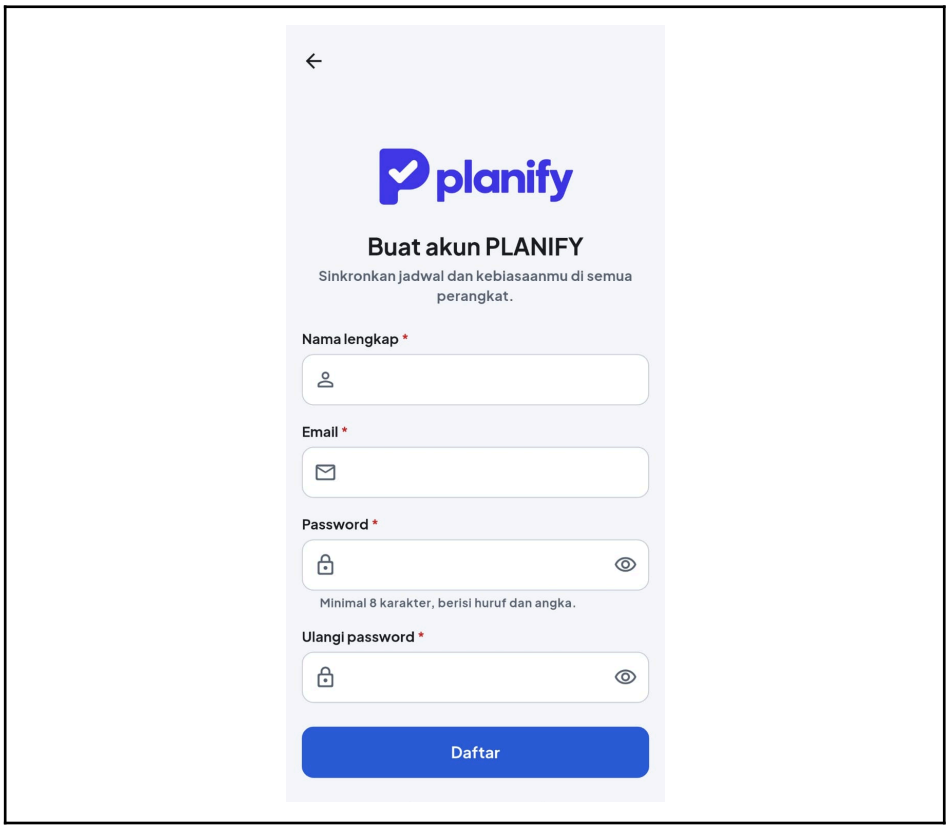
- Baris 12–15 mendefinisikan ekspresi reguler _email untuk memvalidasi struktur alamat email, _hasLetter untuk memastikan adanya huruf alfabet, dan _hasDigit untuk memastikan adanya angka numerik.

3. Metode Validasi
   - email(String? value) (baris 17–26): Menghapus spasi di awal dan akhir input, menolak nilai kosong dengan pesan "Email wajib diisi.", serta memeriksa format email menggunakan regex.
   - passwordRequired(String? value) (baris 29–34): Digunakan pada formulir login untuk memastikan kata sandi tidak kosong tanpa menerapkan aturan kompleksitas.
   - newPassword(String? value) (baris 37–47): Digunakan saat registrasi atau pembaruan kata sandi untuk memastikan panjang minimal delapan karakter serta mengandung kombinasi huruf dan angka.
   - confirmPassword(String Function() original) (baris 50–62): Merupakan higher-order function yang menghasilkan fungsi validator untuk mencocokkan kata sandi konfirmasi dengan kata sandi asli.
   - fullName(String? value) (baris 64–76): Memvalidasi nama pengguna agar memiliki panjang antara 2 hingga 50 karakter.
   - FormFieldValidatorString (baris 80): Mendefinisikan alias tipe fungsi yang kompatibel dengan signature FormFieldValidator milik Flutter tanpa perlu mengimpor paket widget UI.
4. Alur Kerja

   Saat pengguna mengisi formulir registrasi melalui RegisterPage atau mengirimkan formulir, setiap kolom teks menjalankan fungsi validator yang sesuai. Jika input tidak memenuhi aturan yang ditentukan, pesan kesalahan ditampilkan di bawah kolom input sehingga pengguna dapat memperbaikinya sebelum melanjutkan.
5. Hubungan dengan File Lain

   File auth_validators.dart digunakan oleh beberapa halaman autentikasi, yaitu lib/features/auth/presentation/register_page.dart, lib/features/auth/presentation/login_page.dart, dan

lib/features/auth/presentation/reset_password_page.dart. Fungsi validasi pada file ini membantu menjaga konsistensi aturan input di seluruh formulir autentikasi.



Gambar 4.3.1.1 Tampilan Halaman Registrasi Akun

4.3.2 Implementasi Login

Implementasi komunikasi autentikasi dengan server Supabase dikelola oleh repositori `SupabaseAuthRepository` yang menerapkan antarmuka `AuthRepository`.

Kode Program 4.3.2.1 Implementasi Login

```
import 'package:supabase_flutter/supabase_flutter.dart' as sb;

import '../../../core/error/app_failure.dart';
import '../domain/auth_event.dart';
import '../domain/auth_repository.dart';
import '../domain/auth_user.dart';
import 'supabase_auth_error_mapper.dart';

/// [AuthRepository] berbasis Supabase Auth (email + password,
alur PKCE).
class SupabaseAuthRepository implements AuthRepository {
  SupabaseAuthRepository(this._auth, {sb.SupabaseClient?
client})
      : _client = client;

  final sb.GoTrueClient _auth;
```

```
  /// Untuk memperbarui tabel `profiles` (opsional di test).
  final sb.SupabaseClient? _client;

  /// URL yang dibuka dari email (konfirmasi akun dan atur
ulang password).
  /// WAJIB didaftarkan di Supabase Dashboard → Authentication
→ URL
  /// Configuration → Redirect URLs, dan di AndroidManifest
(docs/AUTH_SETUP.md).
  static const String redirectUrl =
'id.planify.planify://auth-callback';

  @override
  bool get isConfigured => true;

  @override
  AuthUser? get currentUser => _toUser(_auth.currentUser);

  @override
  Stream<AuthEvent> get events =>
      _auth.onAuthStateChange.map((sb.AuthState state) {
        return AuthEvent(
          _mapEvent(state.event),
          user: _toUser(state.session?.user),
        );
      });

  @override
  Future<AuthUser> signInWithPassword({
    required String email,
    required String password,
  }) async {
    try {
      final sb.AuthResponse response = await
_auth.signInWithPassword(
        email: email.trim(),
        password: password,
      );
      final AuthUser? user = _toUser(response.user ??
response.session?.user);
      if (user == null) {
        throw const AppFailure(AppFailureKind.unknown);
      }
      return user;
    } catch (error) {
      throw mapSupabaseAuthError(error);
    }
  }

  @override
  Future<SignUpOutcome> signUp({
    required String fullName,
    required String email,
    required String password,
  }) async {
    try {
      final sb.AuthResponse response = await _auth.signUp(
        email: email.trim(),
        password: password,
```

```
        // `display_name` dibaca trigger handle_new_user
(Tahap 2 §7.3) untuk
        // mengisi tabel profiles. `full_name` disimpan juga
untuk kompatibilitas.
        data: <String, dynamic>{
          'display_name': fullName.trim(),
          'full_name': fullName.trim(),
        },
        emailRedirectTo: redirectUrl,
      );
      // Tanpa sesi = konfirmasi email aktif. Untuk email yang
sudah terdaftar,
      // Supabase (proteksi enumerasi) juga mengembalikan
tanpa sesi sehingga
      // UI sengaja menampilkan pesan netral yang sama.
      return response.session != null
          ? SignUpOutcome.signedIn
          : SignUpOutcome.confirmationRequired;
    } catch (error) {
      throw mapSupabaseAuthError(error);
    }
  }

  @override
  Future<void> sendPasswordResetEmail(String email) async {
    try {
      await _auth.resetPasswordForEmail(email.trim(),
redirectTo: redirectUrl);
    } catch (error) {
      throw mapSupabaseAuthError(error);
    }
  }

  @override
  Future<void> updatePassword(String newPassword) async {
    try {
      await _auth.updateUser(sb.UserAttributes(password:
newPassword));
    } catch (error) {
      throw mapSupabaseAuthError(error);
    }
  }

  @override
  Future<AuthUser> updateDisplayName(String name) async {
    final String clean = name.trim();
    try {
      final sb.UserResponse res = await _auth.updateUser(
        sb.UserAttributes(
          data: <String, dynamic>{'display_name': clean,
'full_name': clean},
        ),
      );
      final AuthUser? user = _toUser(res.user);
      if (user == null) {
        throw const AppFailure(AppFailureKind.unknown);
      }
      final sb.SupabaseClient? client = _client;
      if (client != null) {
        // Trigger tg_touch_row menaikkan version; RLS: hanya
```

```
baris sendiri.
        await client
            .from('profiles')
            .update(<String, dynamic>{'display_name': clean})
            .eq('id', user.id);
      }
      return user;
    } catch (error) {
      throw mapSupabaseAuthError(error);
    }
  }

  @override
  Future<void> signOut() async {
    try {
      await _auth.signOut(scope: sb.SignOutScope.local);
    } on sb.AuthRetryableFetchException {
      // Offline: sesi lokal sudah dibersihkan klien; token di
server akan
      // kedaluwarsa sendiri. Tidak perlu menampilkan error ke
pengguna.
    } catch (error) {
      throw mapSupabaseAuthError(error);
    }
  }

  @override
  String? get currentRefreshToken =>
_auth.currentSession?.refreshToken;

  @override
  Future<AuthUser> signInWithRefreshToken(String refreshToken)
async {
    try {
      final sb.AuthResponse response = await
_auth.setSession(refreshToken);
      final AuthUser? user = _toUser(response.user ??
response.session?.user);
      if (user == null) {
        throw const AppFailure(AppFailureKind.unauthorized);
      }
      return user;
    } catch (error) {
      throw mapSupabaseAuthError(error);
    }
  }

  static AuthEventType _mapEvent(sb.AuthChangeEvent event) {
    switch (event) {
      case sb.AuthChangeEvent.initialSession:
        return AuthEventType.initialSession;
      case sb.AuthChangeEvent.signedIn:
        return AuthEventType.signedIn;
      case sb.AuthChangeEvent.signedOut:
        return AuthEventType.signedOut;
      case sb.AuthChangeEvent.passwordRecovery:
        return AuthEventType.passwordRecovery;
      case sb.AuthChangeEvent.userUpdated:
        return AuthEventType.userUpdated;
      case sb.AuthChangeEvent.tokenRefreshed:
```

```
        return AuthEventType.tokenRefreshed;
      default:
        return AuthEventType.tokenRefreshed;
    }
  }

  static AuthUser? _toUser(sb.User? user) {
    if (user == null) {
      return null;
    }
    final Object? name =
        user.userMetadata?['display_name'] ??
user.userMetadata?['full_name'];
    return AuthUser(
      id: user.id,
      email: user.email ?? '',
      displayName: name is String ? name : null,
    );
  }
}
```

Penjelasan Source Code supabase_auth_repository.dart

1. Identitas File

   Berkas supabase_auth_repository.dart terletak pada direktori lib/features/auth/data/supabase_auth_repository.dart. File ini mengimplementasikan kontrak AuthRepository untuk menangani operasi autentikasi yang terhubung dengan backend Supabase.

2. Inisialisasi dan Konfigurasi
   - Baris 11–17: Konstruktor menerima instance GoTrueClient melalui variabel _auth dan SupabaseClient opsional melalui _client.
   - Baris 22: Mendefinisikan skema deep link id.planify.planify://auth-callback untuk menangani pengalihan kembali ke aplikasi. Skema ini perlu didaftarkan pada konfigurasi Redirect URL Supabase dan AndroidManifest.xml.

3. Metode Autentikasi Masuk dan Keluar
   - signInWithPassword (baris 40–57): Memanggil API GoTrue signInWithPassword untuk memverifikasi kredensial pengguna. Jika berhasil, objek pengguna Supabase dipetakan menjadi entitas domain AuthUser. Kesalahan ditangani melalui mapSupabaseAuthError(error).
   - signUp (baris 60–87): Mendaftarkan akun baru melalui API signUp dengan menyertakan metadata display_name dan full_name. Jika konfirmasi email diaktifkan pada Supabase,

metode ini mengembalikan SignUpOutcome.confirmationRequired.

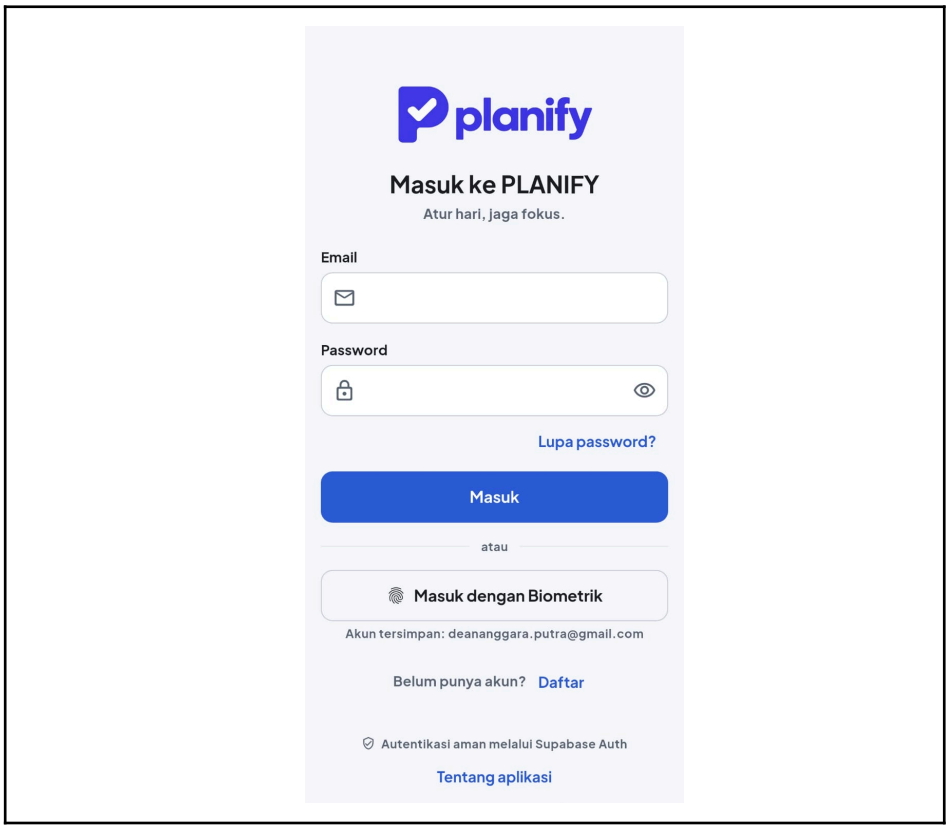
- sendPasswordResetEmail (baris 89–95): Mengirimkan tautan untuk mengatur ulang kata sandi ke alamat email pengguna.
- updatePassword (baris 98–104): Memperbarui kata sandi akun yang sedang terautentikasi.
- updateDisplayName (baris 107–131): Memperbarui metadata pengguna pada GoTrue sekaligus memperbarui data profil pada tabel PostgreSQL profiles.
- signOut (baris 134–143): Memanggil _auth.signOut(scope: SignOutScope.local) untuk mengakhiri sesi autentikasi lokal pada perangkat.
- signInWithRefreshToken (baris 149–160): Memulihkan sesi pengguna menggunakan refresh token melalui metode setSession.

4. Pemetaan Event dan Entitas
   - events (baris 31–37) dan _mapEvent (baris 162–179): Mengubah aliran onAuthStateChange dari Supabase menjadi aliran event domain AuthEvent, seperti signedIn, signedOut, dan passwordRecovery.
   - _toUser (baris 181–192): Fungsi pembantu untuk mengubah objek sb.User menjadi entitas domain AuthUser, sehingga data pengguna mengikuti struktur yang digunakan oleh aplikasi.
5. Alur Kerja

   Ketika pengguna memasukkan kredensial melalui LoginPage, controller memanggil metode signInWithPassword pada repositori. Permintaan diteruskan ke backend Supabase melalui HTTPS. Jika autentikasi berhasil, Supabase memberikan token JWT dan aliran events memancarkan event signedIn. Event tersebut kemudian digunakan untuk memperbarui status autentikasi dan mengarahkan pengguna ke halaman Home melalui router.
6. Hubungan dengan File Lain

   File ini mengimplementasikan AuthRepository, menggunakan supabase_auth_error_mapper.dart untuk memetakan kesalahan autentikasi, menghasilkan entitas AuthUser, serta diakses oleh

AuthActionController dan SessionController dalam pengelolaan autentikasi dan sesi pengguna.



Gambar 4.3.2.1 Halaman Login

4.3.3 Implementasi Login Biometrik

Autentikasi biometrik memadukan verifikasi sensor perangkat keras lokal dengan pembuatan sesi aktif Supabase via kredensial terenkripsi di Keystore. Implementasi ini didukung oleh `biometric_authenticator.dart` dan `auth_action_controller.dart`.

Kode Program 4.3.3.1 Verifikator Biometrik Lokal Perangkat Keras

```
import 'package:flutter/foundation.dart';
import 'package:local_auth/local_auth.dart';

/// Apakah perangkat dapat memverifikasi pengguna secara
lokal.
enum BiometricAvailability {
  available,

  /// Perangkat tidak punya biometrik maupun kunci layar, atau
tidak didukung
  /// (mis. beberapa emulator, Web).
  unavailable,
}

enum BiometricOutcome {
  success,
```

```
  /// Pengguna membatalkan atau sidik jari/wajah tidak cocok.
  cancelled,

  /// Terlalu banyak percobaan gagal; coba lagi nanti.
  lockedOut,

  /// Biometrik/kunci layar tidak tersedia atau belum diatur.
  unavailable,

  /// Kesalahan tak terduga dari perangkat.
  failed,
}

/// Verifikasi identitas LOKAL. Hasilnya hanya membuka kunci
aplikasi; sesi
/// server tetap dikelola Supabase Auth (Tahap 1/2: biometrik
bukan pengganti
/// sesi server).
abstract interface class BiometricAuthenticator {
  Future<BiometricAvailability> availability();

  Future<BiometricOutcome> authenticate({required String
reason});
}

/// Implementasi `local_auth`.
///
/// Memakai `biometricOnly: false` sehingga PIN/pola/sandi
perangkat dapat
/// menjadi cadangan bila biometrik gagal. Label UI karena itu
menyebut
/// "biometrik atau kunci layar".
class LocalAuthBiometricAuthenticator implements
BiometricAuthenticator {
  LocalAuthBiometricAuthenticator([LocalAuthentication? auth])
      : _auth = auth ?? LocalAuthentication();

  final LocalAuthentication _auth;

  @override
  Future<BiometricAvailability> availability() async {
    try {
      final bool supported = await _auth.isDeviceSupported();
      return supported
          ? BiometricAvailability.available
          : BiometricAvailability.unavailable;
    } catch (error) {
      debugPrint('Biometric availability check gagal:
$error');
      return BiometricAvailability.unavailable;
    }
  }

  @override
  Future<BiometricOutcome> authenticate({required String
reason}) async {
    try {
      final bool ok = await _auth.authenticate(
        localizedReason: reason,
```

```
        persistAcrossBackgrounding: true,
      );
      return ok ? BiometricOutcome.success :
BiometricOutcome.cancelled;
    } on LocalAuthException catch (error) {
      return _mapCode(error.code.name);
    } catch (error) {
      debugPrint('Biometric authenticate gagal: $error');
      return BiometricOutcome.failed;
    }
  }

  /// Dipetakan lewat nama kode (bukan enum) agar tahan
perubahan nama anggota
  /// `LocalAuthExceptionCode` antarversi.
  @visibleForTesting
  static BiometricOutcome mapCode(String codeName) =>
_mapCode(codeName);

  static BiometricOutcome _mapCode(String codeName) {
    final String name = codeName.toLowerCase();
    if (name.contains('cancel') || name.contains('timeout')) {
      return BiometricOutcome.cancelled;
    }
    if (name.contains('lockout')) {
      return BiometricOutcome.lockedOut;
    }
    if (name.contains('nobiometric') ||
        name.contains('nocredential') ||
        name.contains('notavailable') ||
        name.contains('notenrolled') ||
        name.contains('passcodenotset')) {
      return BiometricOutcome.unavailable;
    }
    return BiometricOutcome.failed;
  }
}
```

Penjelasan Source Code biometric_authenticator.dart

1. Identitas File

   Berkas biometric_authenticator.dart terletak pada direktori lib/features/auth/data/biometric_authenticator.dart. File ini bertanggung jawab membungkus interaksi dengan pustaka local_auth untuk mengakses mekanisme autentikasi biometrik pada sistem operasi Android melalui BiometricPrompt.

2. Struktur Enum

   - BiometricAvailability (baris 5–11): Menentukan status ketersediaan autentikasi lokal pada perangkat, yaitu available atau unavailable.

- BiometricOutcome (baris 13–27): Merepresentasikan hasil proses autentikasi biometrik, meliputi success, cancelled, lockedOut, unavailable, dan failed.

3. Kelas LocalAuthBiometricAuthenticator
   - availability() (baris 50–60): Memeriksa apakah perangkat mendukung autentikasi lokal melalui _auth.isDeviceSupported().
   - authenticate({required String reason}) (baris 62–76): Memunculkan dialog autentikasi biometrik bawaan sistem Android. Parameter persistAcrossBackgrounding: true digunakan agar proses autentikasi dapat dipertahankan ketika aplikasi sementara berpindah ke latar belakang, sesuai dukungan pustaka dan platform.
   - _mapCode(String codeName) (baris 83–99): Memetakan kode kesalahan dari LocalAuthException menjadi status yang sesuai secara defensif, misalnya dengan mendeteksi kata cancel, lockout, atau notenrolled.
4. Alur Kerja

   Ketika pengguna menjalankan fitur autentikasi biometrik, sistem memanggil metode authenticate(). Android kemudian menampilkan dialog autentikasi bawaan sistem, seperti pemindaian sidik jari. Hasil proses tersebut diterjemahkan menjadi nilai BiometricOutcome agar aplikasi dapat menentukan tindakan selanjutnya, seperti melanjutkan autentikasi atau menangani pembatalan dan kegagalan.
5. Hubungan dengan File Lain

   Berkas ini diinjeksikan melalui provider Riverpod biometricAuthenticatorProvider dan digunakan oleh AuthActionController untuk proses login biometrik serta SessionController untuk verifikasi terkait pengaturan sesi dan keamanan akun.

Kode Program 4.3.3.2 Pengendali Aksi Autentikasi dan Login Biometrik

```
import 'dart:async';

import 'package:flutter/foundation.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../core/error/app_failure.dart';
```

```
import
'../../../core/storage/secure_key_value_store.dart';
import '../data/biometric_authenticator.dart';
import '../domain/auth_event.dart';
import '../domain/auth_repository.dart';
import 'auth_providers.dart';
import 'session_controller.dart';

@immutable
class AuthActionState {
  const AuthActionState({this.isLoading = false,
this.failure});

  final bool isLoading;
  final AppFailure? failure;
}

/// Aksi form autentikasi (masuk, daftar, lupa/atur ulang
password).
///
/// Dibuat `autoDispose` sehingga status loading/error
tidak terbawa antar
/// halaman. Setiap aksi mengembalikan hasil agar halaman
dapat menampilkan
/// state sukses; perpindahan halaman akibat sesi
dilakukan router.
class AuthActionController extends
Notifier<AuthActionState> {
  late final AuthRepository _repository =
ref.read(authRepositoryProvider);
  late final SecureKeyValueStore _store =
ref.read(secureStoreProvider);
  late final BiometricAuthenticator _biometric =
      ref.read(biometricAuthenticatorProvider);

  @override
  AuthActionState build() => const AuthActionState();

  void clearError() {
    if (state.failure != null) {
      state = const AuthActionState();
    }
  }

  /// Menjalankan [action]; mencegah eksekusi ganda dan
mengubah error menjadi
  /// [AppFailure]. Mengembalikan `null` bila gagal atau
sedang berjalan.
  Future<T?> _run<T>(Future<T> Function() action) async {
    if (state.isLoading) {
      return null;
    }
    state = const AuthActionState(isLoading: true);
    try {
      final T result = await action();
      if (ref.mounted) {
        state = const AuthActionState();
      }
      return result;
    } catch (error) {
```

```
      final AppFailure failure = AppFailure.from(error);
      if (kDebugMode && failure.debugDetail != null) {
        debugPrint('Auth gagal (${failure.kind}):
${failure.debugDetail}');
      }
      if (ref.mounted) {
        state = AuthActionState(failure: failure);
      }
      return null;
    }
  }

  /// `true` bila berhasil. Router memindahkan pengguna
lewat event sesi.
  Future<bool> signIn({required String email, required
String password}) async {
    final String cleanEmail = email.trim();
    final Object? user = await _run<Object>(
      () => _repository.signInWithPassword(
        email: cleanEmail,
        password: password,
      ),
    );
    if (user != null) {

ref.read(sessionControllerProvider.notifier).setCachedSec
ret(password);
    }
    return user != null;
  }

  /// Masuk menggunakan autentikasi biometrik lokal dan
memulihkan sesi autentikasi.
  Future<bool> signInWithBiometric({
    String reason = 'Konfirmasi biometrik untuk masuk ke
PLANIFY',
  }) async {
    if (state.isLoading) {
      return false;
    }
    state = const AuthActionState(isLoading: true);
    try {
      final String? owner = await
_store.read(SecureKeys.biometricOwner);
      final String? email = await
_store.read(SecureKeys.biometricEmail);
      final String? secret = await
_store.read(SecureKeys.biometricSecret);
      final String? token = await
_store.read(SecureKeys.biometricToken);

      final bool hasCredential = (secret != null &&
secret.isNotEmpty) ||
          (token != null && token.isNotEmpty);

      if (owner == null || owner.isEmpty ||
!hasCredential) {
        if (ref.mounted) {
          state = const AuthActionState(
            failure: AppFailure(
```

```
              AppFailureKind.unauthorized,
              customTitle: 'Biometrik belum siap',
              message:
                  'Kredensial biometrik tidak ditemukan.
Silakan masuk menggunakan email dan password.',
            ),
          );
        }
        return false;
      }

      final BiometricOutcome outcome =
          await _biometric.authenticate(reason: reason);

      switch (outcome) {
        case BiometricOutcome.success:
          break; // Lanjutkan ke autentikasi Supabase
        case BiometricOutcome.cancelled:
          if (ref.mounted) {
            state = const AuthActionState();
          }
          return false;
        case BiometricOutcome.lockedOut:
          if (ref.mounted) {
            state = const AuthActionState(
              failure: AppFailure(
                AppFailureKind.external,
                customTitle: 'Terlalu banyak percobaan',
                message:
                    'Sensor biometrik terkunci sementara.
Gunakan email dan password atau coba lagi nanti.',
              ),
            );
          }
          return false;
        case BiometricOutcome.unavailable:
          if (ref.mounted) {
            state = const AuthActionState(
              failure: AppFailure(
                AppFailureKind.external,
                customTitle: 'Biometrik tidak tersedia',
                message:
                    'Biometrik atau kunci layar belum
tersedia di perangkat ini.',
              ),
            );
          }
          return false;
        case BiometricOutcome.failed:
          if (ref.mounted) {
            state = const AuthActionState(
              failure: AppFailure(
                AppFailureKind.external,
                customTitle: 'Verifikasi biometrik
gagal',
                message:
                    'Sidik jari tidak dikenali. Coba lagi
atau gunakan email dan password.',
              ),
            );
```

```
          }
          return false;
        }

        try {
          if (secret != null &&
              secret.isNotEmpty &&
              email != null &&
              email.isNotEmpty) {
            await _repository.signInWithPassword(
              email: email,
              password: secret,
            );

ref.read(sessionControllerProvider.notifier).setCachedSec
ret(secret);
          } else if (token != null && token.isNotEmpty) {
            await
_repository.signInWithRefreshToken(token);
          }
          if (ref.mounted) {
            state = const AuthActionState();
          }
          return true;
        } catch (authError) {
          final AppFailure failure =
AppFailure.from(authError);
          // HANYA hapus kredensial jika error adalah
unauthorized (mis. password telah diubah).
          // Jangan hapus jika kesalahan jaringan atau
kendala sementara.
          if (failure.kind == AppFailureKind.unauthorized)
{
            await
_store.delete(SecureKeys.biometricSecret);
            await _store.delete(SecureKeys.biometricToken);
            ref.invalidate(biometricLoginInfoProvider);
          }
          rethrow;
        }
      } catch (error) {
        final AppFailure failure = AppFailure.from(error);
        if (kDebugMode && failure.debugDetail != null) {
          debugPrint('Biometric sign in gagal:
${failure.debugDetail}');
        }
        if (ref.mounted) {
          state = AuthActionState(failure: failure);
        }
        return false;
      }
    }

    Future<SignUpOutcome?> signUp({
      required String fullName,
      required String email,
      required String password,
    }) {
      return _run<SignUpOutcome>(
        () => _repository.signUp(
```

```
        fullName: fullName,
        email: email,
        password: password,
      ),
    );
  }

  Future<bool> sendPasswordReset(String email) async {
    final bool? ok = await _run<bool>(() async {
      await _repository.sendPasswordResetEmail(email);
      return true;
    });
    return ok ?? false;
  }

  /// Menyimpan password baru lalu menyelesaikan mode
pemulihan.
  Future<bool> updatePassword(String newPassword) async {
    final bool? ok = await _run<bool>(() async {
      await _repository.updatePassword(newPassword);
      return true;
    });
    if (ok == true) {
      final SessionController sessionCtrl =
          ref.read(sessionControllerProvider.notifier);
      sessionCtrl.setCachedSecret(newPassword);
      if
(ref.read(sessionControllerProvider).biometricEnabled) {
        unawaited(ref
            .read(secureStoreProvider)
            .write(SecureKeys.biometricSecret,
newPassword));
      }
      sessionCtrl.completeRecovery();
    }
    return ok ?? false;
  }
}

final NotifierProvider<AuthActionController,
AuthActionState>
    authActionProvider =
    NotifierProvider.autoDispose<AuthActionController,
AuthActionState>(
  AuthActionController.new,
);
```

Penjelasan Source Code auth_action_controller.dart

1. Identitas File

   Berkas auth_action_controller.dart terletak pada direktori lib/features/auth/application/auth_action_controller.dart. File ini bertanggung jawab mengelola state dan proses interaksi formulir autentikasi menggunakan Riverpod Notifier dengan mekanisme autoDispose.

2. Struktur State AuthActionState

   AuthActionState menyimpan informasi yang diperlukan untuk mengontrol proses autentikasi, yang terdiri atas:

   - isLoading: Nilai boolean yang menunjukkan apakah proses autentikasi sedang berlangsung dan indikator progres perlu ditampilkan.
   - failure: Objek yang menyimpan informasi kesalahan terstruktur dalam bentuk AppFailure apabila proses autentikasi mengalami kegagalan.

3. Logika Eksekusi signInWithBiometric
   - Pengambilan kredensial (baris 91–95): Membaca data dari penyimpanan aman perangkat, meliputi biometricOwner, biometricEmail, biometricSecret, dan biometricToken.
   - Verifikasi biometrik (baris 113): Memanggil _biometric.authenticate() untuk memverifikasi identitas pengguna melalui sensor biometrik perangkat.
   - Autentikasi ulang ke server (baris 163–174): Setelah verifikasi biometrik berhasil, aplikasi menggunakan _repository.signInWithPassword dengan email dan kata sandi yang tersimpan secara aman. Proses ini memungkinkan Supabase menerbitkan token sesi JWT dan refresh token baru.
   - Penanganan kesalahan (baris 181–189): Kredensial biometrik lokal hanya dihapus ketika server menolak autentikasi secara permanen dengan jenis kesalahan AppFailureKind.unauthorized, misalnya ketika kata sandi telah diubah. Sementara itu, jika terjadi gangguan jaringan atau timeout, kredensial tetap dipertahankan agar pengguna dapat mencoba autentikasi biometrik kembali.

4. Metode Autentikasi Lainnya

   Controller ini juga menyediakan metode signIn, signUp, sendPasswordReset, dan updatePassword. Metode tersebut menangani proses login, pendaftaran akun, pengiriman permintaan reset kata sandi, serta pembaruan kata sandi. Ketika kata sandi

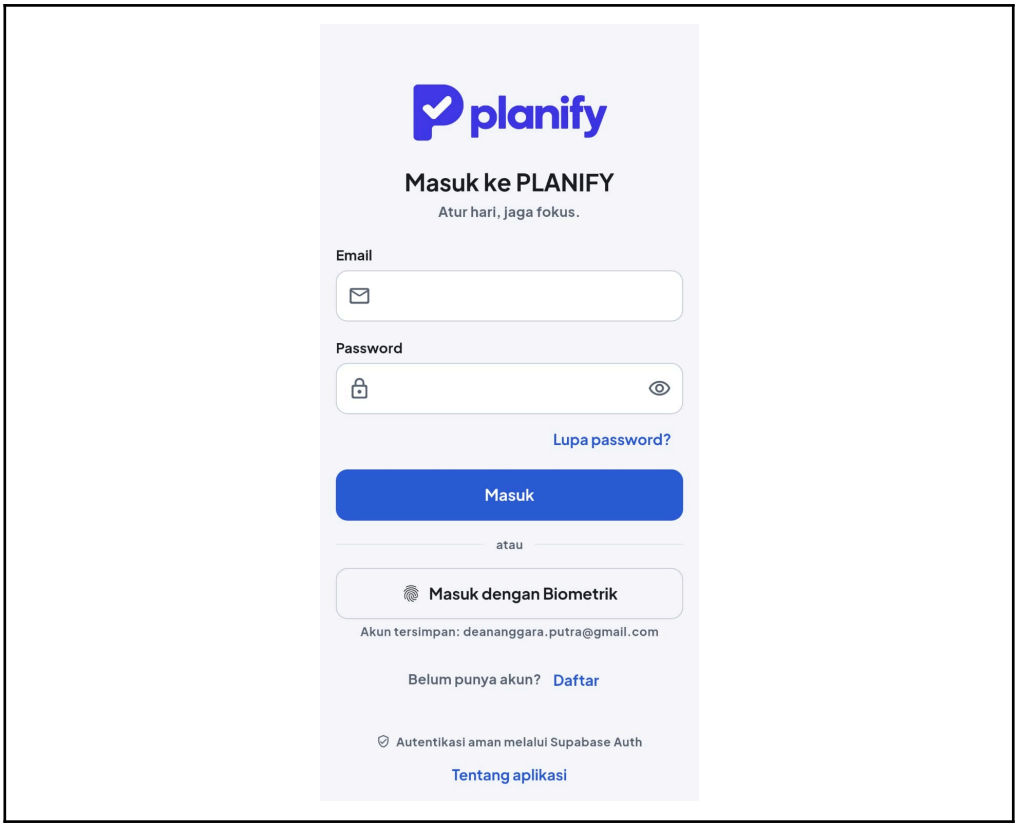
diperbarui, kredensial yang tersimpan pada penyimpanan aman juga diperbarui sesuai alur implementasi aplikasi.

5. Alur Kerja

   Pada halaman LoginPage, pengguna menekan tombol autentikasi biometrik untuk menjalankan metode signInWithBiometric(). Aplikasi membaca kredensial yang tersimpan, kemudian meminta verifikasi biometrik melalui sensor perangkat. Jika verifikasi berhasil, aplikasi melakukan autentikasi ke Supabase menggunakan kredensial tersebut. Setelah proses autentikasi berhasil, sesi pengguna diperbarui dan aplikasi mengarahkan pengguna ke halaman Home.

6. Hubungan dengan File Lain

   auth_action_controller.dart berinteraksi dengan SecureKeyValueStore untuk mengakses penyimpanan kredensial yang aman, BiometricAuthenticator untuk melakukan verifikasi biometrik, dan AuthRepository untuk menjalankan operasi autentikasi ke backend. Controller ini juga berkoordinasi dengan SessionController dalam pengelolaan sesi pengguna.



Gambar 4.3.3.1 Tampilan Dialog Autentikasi Biometrik di Halaman Login

4.3.4 Implementasi Session dan Logout

Pengelolaan siklus hidup sesi aplikasi dikontrol secara terpusat oleh `SessionController`. Berkas ini juga menegakkan prinsip bahwa autentikasi biometrik pada Planify hanya digunakan untuk login kembali setelah logout manual.

Kode Program 4.3.4.1 Pengendali Status Sesi dan Siklus Hidup Aplikasi

```
import 'dart:async';

import 'package:flutter/foundation.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../core/storage/secure_key_value_store.dart';
import '../data/biometric_authenticator.dart';
import '../domain/auth_event.dart';
import '../domain/auth_repository.dart';
import '../domain/auth_user.dart';
import 'auth_providers.dart';
import 'session_state.dart';

/// Hasil mengaktifkan kunci biometrik.
enum BiometricEnableResult { enabled, cancelled, unavailable,
lockedOut, failed }

/// Sumber kebenaran status sesi. Dibaca router (guard) dan
seluruh UI.
///
```

```
/// Prinsip keamanan:
/// - Biometrik hanya membuka kunci APLIKASI di perangkat ini.
Sesi server
///   (Supabase Auth) tetap sumber kebenaran; tanpa sesi,
biometrik tidak bisa
///   dipakai untuk masuk.
/// - Penanda biometrik terikat pada ID pengguna (bukan
sekadar boolean), jadi
///   tidak berlaku untuk akun lain yang masuk di perangkat
yang sama.
class SessionController extends Notifier<SessionState> {
  late final AuthRepository _repository =
ref.read(authRepositoryProvider);
  late final SecureKeyValueStore _store =
ref.read(secureStoreProvider);
  late final BiometricAuthenticator _biometric =
      ref.read(biometricAuthenticatorProvider);

  StreamSubscription<AuthEvent>? _subscription;
  bool _initialized = false;
  bool _userInitiatedSignOut = false;
  String? _cachedSecret;

  bool get hasCachedSecret => _cachedSecret != null &&
_cachedSecret!.isNotEmpty;

  void setCachedSecret(String secret) {
    _cachedSecret = secret;
  }

  @override
  SessionState build() {
    _subscription = _repository.events.listen(_onEvent);
    ref.onDispose(() => _subscription?.cancel());
    // Ditunda satu microtask: `state` belum boleh diubah
selagi build berjalan.
    unawaited(Future<void>.microtask(_initialize));
    return const SessionState();
  }

  Future<void> _initialize() async {
    final AuthUser? user = _repository.currentUser;
    if (user == null) {
      _initialized = true;
      if (ref.mounted && state.phase ==
SessionPhase.unauthenticated) {
        state = state.copyWith(phase:
SessionPhase.unauthenticated);
      }
      return;
    }
    final bool enabled = await
_isBiometricEnabledFor(user.id);
    _initialized = true;
    if (!ref.mounted) {
      return;
    }
    // Peristiwa passwordRecovery yang tiba selama pembacaan
tidak boleh
    // ditimpa kembali menjadi "authenticated".
```

```
    if (state.phase == SessionPhase.passwordRecovery) {
      state = state.copyWith(biometricEnabled: enabled);
      return;
    }
    // Pembukaan dingin (cold start) dengan sesi tersimpan:
langsung ke Home.
    // Biometrik di PLANIFY hanya untuk login kembali setelah
logout, bukan App Lock.
    state = SessionState(
      phase: SessionPhase.authenticated,
      user: user,
      biometricEnabled: enabled,
      isLocked: false,
    );
  }

  Future<void> _onEvent(AuthEvent event) async {
    if (!ref.mounted) {
      return;
    }
    switch (event.type) {
      case AuthEventType.initialSession:
        // Keadaan awal ditentukan _initialize() dari
currentUser.
        break;
      case AuthEventType.signedIn:
        await _handleSignedIn(event.user);
      case AuthEventType.signedOut:
        _handleSignedOut();
      case AuthEventType.passwordRecovery:
        final AuthUser? recoveryUser = event.user ??
_repository.currentUser;
        state = state.copyWith(
          phase: SessionPhase.passwordRecovery,
          user: recoveryUser,
          isLocked: false,
        );
      case AuthEventType.userUpdated:
      case AuthEventType.tokenRefreshed:
        final AuthUser? updated = event.user;
        if (updated != null && state.isAuthenticated) {
          state = state.copyWith(user: updated);
        }
        if (state.biometricEnabled) {
          final String? token =
_repository.currentRefreshToken;
          if (token != null && token.isNotEmpty) {
            unawaited(_store.write(SecureKeys.biometricToken,
token));
          }
        }
    }
  }

  Future<void> _handleSignedIn(AuthUser? user) async {
    if (user == null || !_initialized) {
      return;
    }
    _userInitiatedSignOut = false;
    // Supabase dapat memancarkan signedIn lagi saat aplikasi
```

```
kembali aktif;
    // jangan mereset kunci/halaman untuk pengguna yang sama.
    final bool sameUser = state.user?.id == user.id;
    if (sameUser &&
        (state.phase == SessionPhase.authenticated ||
            state.phase == SessionPhase.passwordRecovery)) {
      if (state.phase == SessionPhase.authenticated) {
        state = state.copyWith(user: user);
      }
      return;
    }
    final String? owner = await
_store.read(SecureKeys.biometricOwner);
    if (owner != null && owner != user.id) {
      await _store.delete(SecureKeys.biometricOwner);
      await _store.delete(SecureKeys.biometricEmail);
      await _store.delete(SecureKeys.biometricSecret);
      await _store.delete(SecureKeys.biometricToken);
    } else if (owner == user.id) {
      if (_cachedSecret != null && _cachedSecret!.isNotEmpty)
{
        unawaited(_store.write(SecureKeys.biometricSecret,
_cachedSecret!));
      }
      if (_repository.currentRefreshToken != null) {
        unawaited(_store.write(
          SecureKeys.biometricToken,
          _repository.currentRefreshToken!,
        ));
      }
    }
    final bool enabled = await
_isBiometricEnabledFor(user.id);
    if (!ref.mounted) {
      return;
    }
    // Baru saja memasukkan password → tidak perlu membuka
kunci lagi.
    state = SessionState(
      phase: SessionPhase.authenticated,
      user: user,
      biometricEnabled: enabled,
    );
  }

  void _handleSignedOut() {
    final bool expired =
        !_userInitiatedSignOut && state.phase ==
SessionPhase.authenticated;
    _userInitiatedSignOut = false;
    state = SessionState(
      phase: SessionPhase.unauthenticated,
      sessionExpired: expired,
    );
  }

  Future<bool> _isBiometricEnabledFor(String userId) async {
    try {
      return await _store.read(SecureKeys.biometricOwner) ==
userId;
```

```
    } catch (error) {
      debugPrint('Gagal membaca penanda biometrik: $error');
      return false;
    }
  }

  // ---- Aksi dari UI
----------------------------------------------------

  /// Keluar manual. Sesi lokal dibersihkan. Penanda dan token
biometrik
  /// tetap dipertahankan jika pengguna mengaktifkannya,
sehingga dapat masuk
  /// kembali lewat biometrik di perangkat ini.
  Future<void> signOut() async {
    _userInitiatedSignOut = true;
    _cachedSecret = null;
    try {
      await _repository.signOut();
    } catch (_) {
      _userInitiatedSignOut = false;
      rethrow;
    } finally {
      if (ref.mounted && _repository.currentUser == null) {
        state = const SessionState(phase:
SessionPhase.unauthenticated);
      }
    }
  }

  /// Dipanggil setelah password baru tersimpan pada alur atur
ulang.
  void completeRecovery() {
    final AuthUser? user = state.user ??
_repository.currentUser;
    if (user == null) {
      state = const SessionState(phase:
SessionPhase.unauthenticated);
      return;
    }
    state = SessionState(phase: SessionPhase.authenticated,
user: user);
    unawaited(_refreshBiometricFlag(user.id));
  }

  Future<void> _refreshBiometricFlag(String userId) async {
    final bool enabled = await _isBiometricEnabledFor(userId);
    if (ref.mounted && state.user?.id == userId) {
      state = state.copyWith(biometricEnabled: enabled);
    }
  }

  /// Pengguna membatalkan alur atur ulang password.
  Future<void> cancelRecovery() => signOut();

  /// Membuka kunci aplikasi dengan biometrik/kunci layar.
  Future<BiometricOutcome> unlock({required String reason})
async {
    final BiometricOutcome outcome =
        await _biometric.authenticate(reason: reason);
```

```
    if (outcome == BiometricOutcome.success && ref.mounted) {
      state = state.copyWith(isLocked: false);
    }
    return outcome;
  }

  /// Mengaktifkan login biometrik untuk pengguna saat ini.
Pengguna harus
  /// lolos verifikasi sekali agar pengaturan terbukti
berfungsi.
  Future<BiometricEnableResult> enableBiometric({
    required String reason,
    String? password,
  }) async {
    final AuthUser? user = state.user;
    if (user == null || !state.isAuthenticated) {
      return BiometricEnableResult.failed;
    }
    if (await _biometric.availability() !=
BiometricAvailability.available) {
      return BiometricEnableResult.unavailable;
    }
    final String? effectiveSecret = password ?? _cachedSecret;
    final BiometricOutcome outcome =
        await _biometric.authenticate(reason: reason);
    switch (outcome) {
      case BiometricOutcome.success:
        await _store.write(SecureKeys.biometricOwner,
user.id);
        if (user.email.isNotEmpty) {
          await _store.write(SecureKeys.biometricEmail,
user.email);
        }
        if (effectiveSecret != null &&
effectiveSecret.isNotEmpty) {
          _cachedSecret = effectiveSecret;
          await _store.write(SecureKeys.biometricSecret,
effectiveSecret);
        }
        final String? token = _repository.currentRefreshToken;
        if (token != null && token.isNotEmpty) {
          await _store.write(SecureKeys.biometricToken,
token);
        }
        if (ref.mounted) {
          state = state.copyWith(biometricEnabled: true);
        }
        return BiometricEnableResult.enabled;
      case BiometricOutcome.cancelled:
        return BiometricEnableResult.cancelled;
      case BiometricOutcome.lockedOut:
        return BiometricEnableResult.lockedOut;
      case BiometricOutcome.unavailable:
        return BiometricEnableResult.unavailable;
      case BiometricOutcome.failed:
        return BiometricEnableResult.failed;
    }
  }

  Future<void> disableBiometric() async {
```

```
    _cachedSecret = null;
    await _store.delete(SecureKeys.biometricOwner);
    await _store.delete(SecureKeys.biometricEmail);
    await _store.delete(SecureKeys.biometricSecret);
    await _store.delete(SecureKeys.biometricToken);
    if (ref.mounted) {
      state = state.copyWith(biometricEnabled: false,
isLocked: false);
    }
  }

  // ---- Siklus hidup aplikasi
-------------------------------------------

  /// Aplikasi masuk latar belakang.
  void onBackgrounded() {}

  /// Aplikasi kembali aktif. Sesuai arsitektur Planify,
biometrik tidak
  /// digunakan sebagai App Lock saat kembali dari latar
belakang.
  void onForegrounded() {}
}

final NotifierProvider<SessionController, SessionState>
    sessionControllerProvider =
    NotifierProvider<SessionController,
SessionState>(SessionController.new);
```

Penjelasan Source Code session_controller.dart

1. Identitas File

   Berkas session_controller.dart terletak pada direktori lib/features/auth/application/session_controller.dart. File ini berfungsi sebagai single source of truth untuk status sesi autentikasi yang diamati oleh GoRouter dan antarmuka aplikasi.

2. Inisialisasi Sesi Tanpa App Lock
   - Metode _initialize() (baris 51–79): Memeriksa sesi yang tersimpan melalui _repository.currentUser. Jika ditemukan sesi aktif, state diinisialisasi dengan phase: SessionPhase.authenticated dan isLocked: false. Dengan demikian, pengguna dapat langsung masuk ke halaman utama (Home) tanpa harus melewati dialog biometrik ketika aplikasi pertama kali dibuka (cold start).
   - Metode onBackgrounded() dan onForegrounded() (baris 288–293): Tidak mengunci aplikasi ketika berpindah ke latar belakang atau kembali digunakan (background resume), sehingga pengguna dapat melanjutkan aktivitas tanpa verifikasi biometrik tambahan.

3. Isolasi Akun dan Pembersihan Data
    - Metode _handleSignedIn(AuthUser? user) (baris 115–158): Memeriksa identitas pemilik biometrik yang tersimpan pada Keystore melalui biometricOwner. Jika akun yang masuk berbeda dari akun sebelumnya, data biometrik lama dibersihkan untuk menjaga privasi dan mencegah penggunaan kredensial lintas akun.
    - Metode signOut() (baris 184–197): Mengatur _userInitiatedSignOut = true dan memanggil _repository.signOut(). Proses ini mengakhiri sesi lokal, tetapi tetap mempertahankan penanda biometrik dan kredensial terenkripsi pada Keystore agar pengguna dapat masuk kembali menggunakan sidik jari melalui halaman login.
4. Manajemen Fitur Biometrik
    - Metode enableBiometric() (baris 232–273): Meminta verifikasi biometrik terlebih dahulu. Jika berhasil, kredensial pengguna berupa owner, email, secret, dan token disimpan secara aman ke Keystore melalui _store.write.
    - Metode disableBiometric() (baris 275–284): Menonaktifkan fitur biometrik dengan menghapus seluruh entri kredensial dan penanda biometrik dari penyimpanan aman.
5. Alur Kerja

    Selama aplikasi berjalan, SessionController mengamati aliran perubahan status autentikasi dari Supabase. Ketika sesi berakhir atau pengguna keluar dari akun, state diperbarui menjadi unauthenticated. Perubahan tersebut kemudian diamati oleh router untuk mengarahkan pengguna ke halaman Login. Sebaliknya, ketika autentikasi berhasil, status sesi diperbarui agar pengguna dapat mengakses halaman yang sesuai.
6. Hubungan dengan File Lain

    Berkas ini menyediakan sessionControllerProvider dan berkoordinasi dengan beberapa komponen aplikasi, yaitu auth_redirect.dart sebagai pengatur pengalihan rute, SettingsPage untuk pengaturan keamanan biometrik, serta ProfilePage untuk menjalankan proses keluar akun.

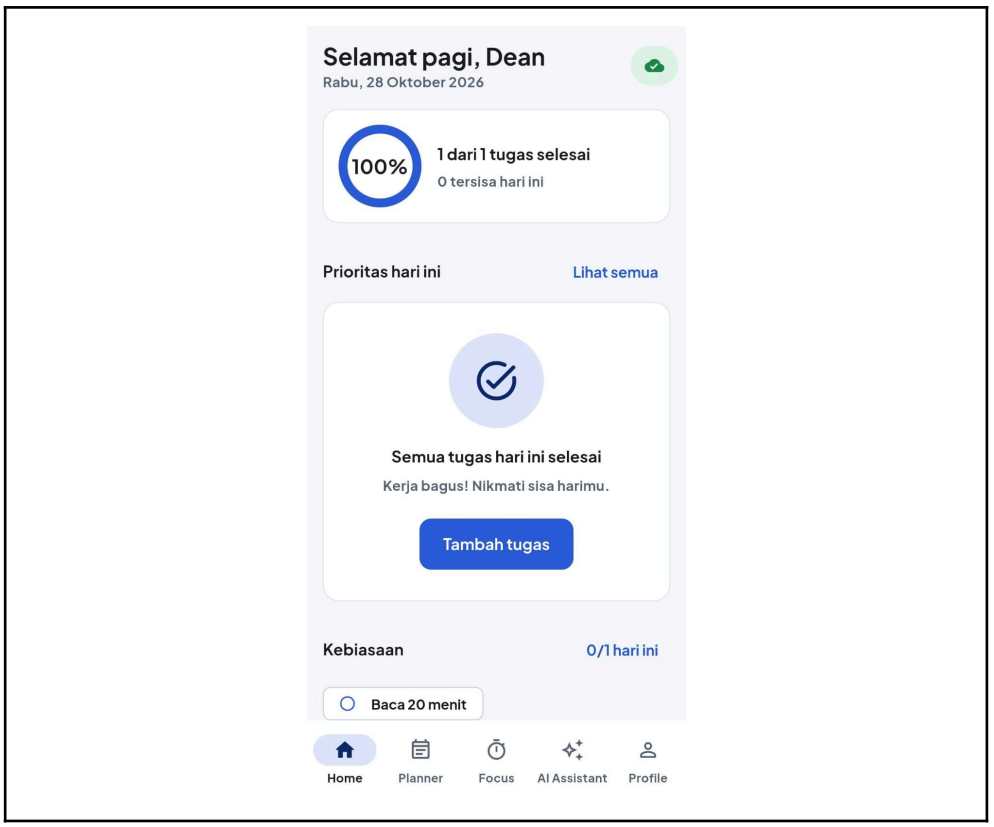
4.4 Implementasi Halaman Home

Halaman Home (`HomePage`) merupakan gerbang utama antarmuka pengguna setelah sesi autentikasi berhasil divalidasi. Halaman ini berfungsi sebagai dasbor sentral yang merangkum status produktivitas harian pengguna secara terpadu.

4.4.1 Tampilan dan Komponen Home

Tampilan antarmuka Home dirancang secara bersih dan hierarkis menggunakan tata letak SingleChildScrollView dengan penyegaran usap (pull-to-refresh). Struktur visual terdiri atas:

1. Bilah Aplikasi (AppBar): Menampilkan sapaan kontekstual berbasis waktu (Pagi, Siang, Sore, Malam), nama panggilan pengguna, tanggal lengkap hari ini, serta ikon indikator sinkronisasi awan (AppSyncIndicator).
2. Kartu Progres Hari Ini (_ProgressCard): Visualisasi lingkaran progres (progress ring) yang menampilkan persentase penyelesaian tugas hari ini beserta jumlah tugas selesai, tugas tersisa, dan indikator tugas terlambat (overdue).
3. Kartu Agenda Terdekat (_NextTaskCard): Kartu sorotan dengan warna kontras yang menampilkan tugas berikutnya yang memiliki batas waktu paling mendesak dengan indikator waktu relatif ("SEBENTAR LAGI", "X MENIT LAGI").
4. Daftar Prioritas Hari Ini: Daftar ringkas maksimal 3 tugas prioritas teratas yang belum selesai pada hari berjalan dengan kemampuan mencentang selesai langsung dari kartu tugas.
5. Ringkasan Kebiasaan (_HabitSummary): Menampilkan jumlah kebiasaan harian yang telah dipenuhi dan status streak aktif.
6. Ringkasan Fokus (_FocusSummary): Menampilkan akumulasi menit fokus Pomodoro yang dicapai hari ini.
7. Bilah Akses Cepat (_QuickActions): Pintasan cepat menuju modul Focus Mode, AI Assistant, Habit Tracker, Tempat Produktif, Budget Helper, Time Zone Planner, dan Habit Consistency Record.



Gambar 4.4.1.1 Tampilan Halaman Utama (Home)

4.4.2 Implementasi Informasi dan Ringkasan Produktivitas

Informasi ringkasan produktivitas harian dihitung secara reaktif dari daftar entitas tugas lokal melalui model DaySummary dan TaskFilter. Lokasi file: lib/features/planner/domain/task_filter.dart

Kode Program 4.4.2.1 Model Filter dan Ringkasan Produktivitas Harian

```
import 'package:flutter/foundation.dart';

import '../../../core/utils/date_only.dart';
import 'task.dart';
import 'task_priority.dart';

/// Status penyelesaian untuk filter.
enum TaskStatusFilter {
  all('Semua'),
  open('Belum selesai'),
  overdue('Terlambat'),
  done('Selesai');

  const TaskStatusFilter(this.label);

  final String label;
}

enum TaskSort {
  due('Tenggat terdekat'),
  priority('Prioritas tertinggi'),
  newest('Terbaru dibuat');
```

```
  const TaskSort(this.label);

  final String label;
}

/// Filter Planner (FR-016). State UI sementara, tidak
disimpan.
@immutable
class TaskFilter {
  const TaskFilter({
    this.query = '',
    this.status = TaskStatusFilter.all,
    this.categories = const <TaskCategory>{},
    this.priorities = const <TaskPriority>{},
    this.sort = TaskSort.due,
  });

  final String query;
  final TaskStatusFilter status;
  final Set<TaskCategory> categories;
  final Set<TaskPriority> priorities;
  final TaskSort sort;

  /// Jumlah filter aktif (untuk badge tombol Filter).
  int get activeCount =>
      (status == TaskStatusFilter.all ? 0 : 1) +
      categories.length +
      priorities.length;

  bool get isEmpty => activeCount == 0 &&
query.trim().isEmpty;

  TaskFilter copyWith({
    String? query,
    TaskStatusFilter? status,
    Set<TaskCategory>? categories,
    Set<TaskPriority>? priorities,
    TaskSort? sort,
  }) {
    return TaskFilter(
      query: query ?? this.query,
      status: status ?? this.status,
      categories: categories ?? this.categories,
      priorities: priorities ?? this.priorities,
      sort: sort ?? this.sort,
    );
  }

  bool matches(Task task, DateTime now) {
    if (task.isDeleted) {
      return false;
    }
    final String q = query.trim().toLowerCase();
    if (q.isNotEmpty &&
        !task.title.toLowerCase().contains(q) &&
        !(task.description?.toLowerCase().contains(q) ??
false)) {
      return false;
    }
    switch (status) {
```

```
      case TaskStatusFilter.all:
        break;
      case TaskStatusFilter.open:
        if (task.isCompleted) {
          return false;
        }
      case TaskStatusFilter.overdue:
        if (!task.isOverdue(now)) {
          return false;
        }
      case TaskStatusFilter.done:
        if (!task.isCompleted) {
          return false;
        }
    }
    if (categories.isNotEmpty &&
!categories.contains(task.category)) {
      return false;
    }
    if (priorities.isNotEmpty &&
!priorities.contains(task.priority)) {
      return false;
    }
    return true;
  }

  List<Task> apply(Iterable<Task> tasks, DateTime now) {
    final List<Task> out =
        tasks.where((Task t) => matches(t,
now)).toList(growable: false);
    return sortTasks(out, sort);
  }
}

/// Urutan stabil: kriteria utama lalu tenggat, lalu judul.
List<Task> sortTasks(List<Task> tasks, TaskSort sort) {
  final List<Task> copy = List<Task>.of(tasks);
  int byDue(Task a, Task b) =>
a.dueMoment.compareTo(b.dueMoment);
  int byTitle(Task a, Task b) =>
      a.title.toLowerCase().compareTo(b.title.toLowerCase());
  copy.sort((Task a, Task b) {
    final int primary = switch (sort) {
      TaskSort.due => byDue(a, b),
      TaskSort.priority =>
b.priority.value.compareTo(a.priority.value),
      TaskSort.newest => b.createdAt.compareTo(a.createdAt),
    };
    if (primary != 0) {
      return primary;
    }
    final int due = byDue(a, b);
    return due != 0 ? due : byTitle(a, b);
  });
  return copy;
}

/// Ringkasan hari ini untuk Home (FR-008/009).
@immutable
class DaySummary {
```

```
  const DaySummary({
    required this.total,
    required this.completed,
    required this.overdue,
    required this.todayOpen,
    required this.next,
  });

  /// Tugas bertenggat hari ini (selesai maupun belum).
  final int total;
  final int completed;

  /// Tugas belum selesai yang tenggatnya sudah lewat (semua
tanggal).
  final int overdue;

  /// Tugas hari ini yang belum selesai, urut prioritas lalu
jam.
  final List<Task> todayOpen;

  /// Tugas belum selesai berikutnya yang punya jam dan belum
lewat.
  final Task? next;

  double get progress => total == 0 ? 0 : completed / total;

  static DaySummary of(Iterable<Task> tasks, DateTime now) {
    final List<Task> active =
        tasks.where((Task t) => !t.isDeleted).toList(growable:
false);
    final List<Task> today = active
        .where((Task t) => DateOnly.isSameDay(t.dueDate, now))
        .toList(growable: false);
    final List<Task> open = sortTasks(
      today.where((Task t) => !t.isCompleted).toList(),
      TaskSort.priority,
    );
    final List<Task> upcoming = sortTasks(
      active
          .where((Task t) =>
              !t.isCompleted &&
              t.dueTimeMinutes != null &&
              t.dueMoment.isAfter(now))
          .toList(),
      TaskSort.due,
    );
    return DaySummary(
      total: today.length,
      completed: today.where((Task t) =>
t.isCompleted).length,
      overdue: active.where((Task t) =>
t.isOverdue(now)).length,
      todayOpen: open,
      next: upcoming.isEmpty ? null : upcoming.first,
    );
  }
}
```

Penjelasan Source Code task_filter.dart

1. Identitas File

   Berkas ini berada pada lib/features/planner/domain/task_filter.dart dan mendefinisikan model penyaringan tugas, aturan pengurutan, serta pabrik pembentukan ringkasan harian DaySummary untuk halaman Home.

2. Enum dan Opsi Filter
   - TaskStatusFilter (baris 8–17): Opsi penyaringan status tugas (all, open, overdue, done).
   - TaskSort (baris 19–27): Opsi pengurutan daftar tugas (due, priority, newest).

3. Kelas TaskFilter dan Metode sortTasks
   - matches(Task task, DateTime now) (baris 70–103): Mengevaluasi apakah suatu tugas memenuhi kriteria pencarian query teks, status penyelesaian, kategori tugas, dan tingkat prioritas.
   - sortTasks (baris 113–131): Mengurutkan koleksi tugas secara stabil. Pengurutan pertama didasarkan pada opsi sort, disusul waktu tenggat (dueMoment), dan judul tugas secara alfabetis.

4. Kelas DaySummary
   - DaySummary.of(Iterable<Task> tasks, DateTime now) (baris 159–185): Mengisolasi tugas aktif hari ini, menghitung total tugas bertenggat hari ini, jumlah tugas selesai, jumlah tugas yang terlambat dari seluruh tanggal, mengurutkan tugas aktif hari ini berdasarkan prioritas (todayOpen), serta mencari agenda mendesak berikutnya (next) yang tenggat waktunya belum terlewati.
   - Properti progress (baris 157): Menghitung rasio desimal penyelesaian tugas harian ($0.0 - 1.0$) untuk menggerakkan indikator visual AppProgressRing.

5. Alur Kerja

   Pada HomePage, state tasksProvider dipantau. Setiap kali ada perubahan entitas tugas lokal di Hive, DaySummary.of` otomatis dipanggil ulang untuk memperbarui persentase cincin progres dan kartu agenda terdekat.

6. Hubungan dengan File Lain

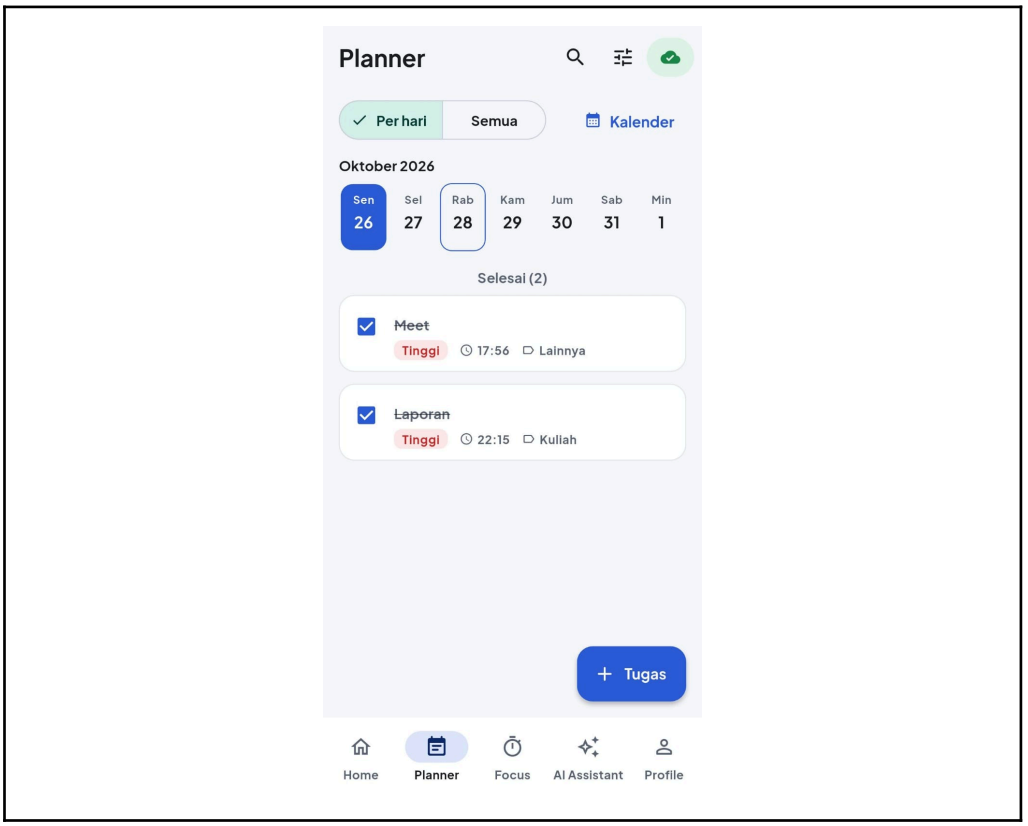
Berkas ini digunakan oleh lib/features/home/presentation/home_page.dart` untuk ringkasan dasbor, serta oleh lib/features/planner/presentation/planner_page.dart dan planner_controller.dart untuk penyaringan tugas.

4.5 Implementasi Planner

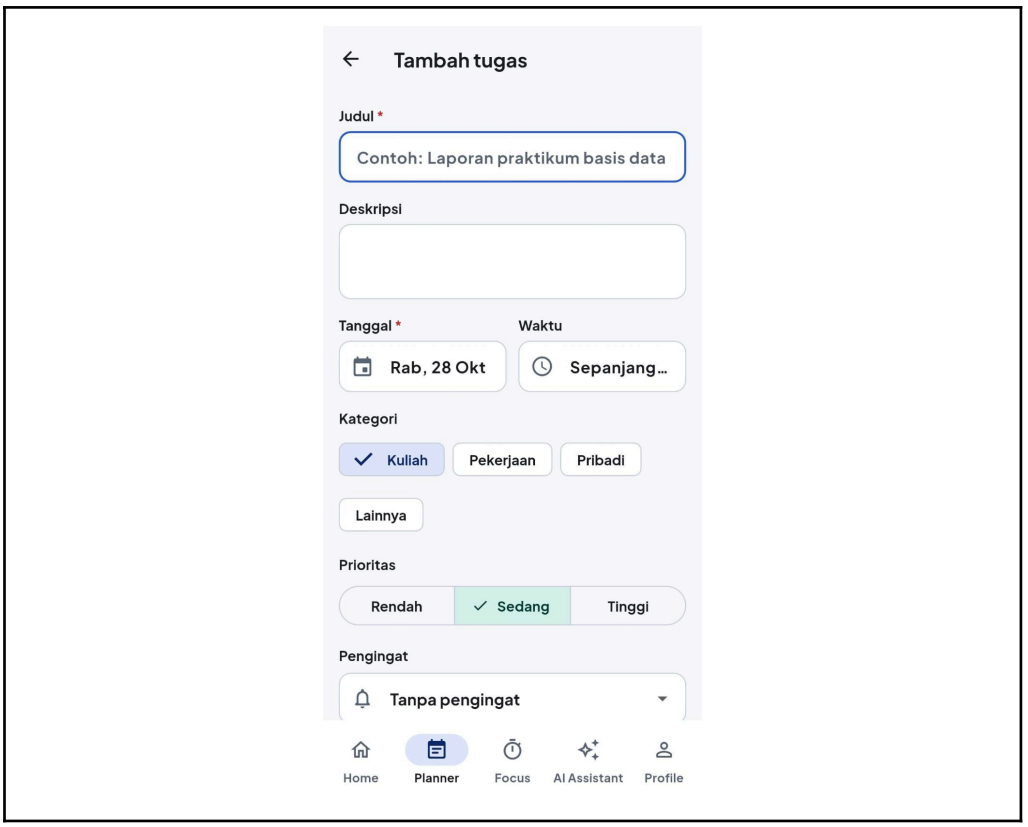
Modul Planner mengelola seluruh siklus hidup agenda kegiatan dan tugas terencana pengguna, mendukung perpindahan mode harian (Day Mode) dan komprehensif (All Tasks Mode).

4.5.1 Tampilan Halaman Planner

Antarmuka Planner menyediakan bilah atas pencarian tugas, kalender strip mingguan, pemilih kalender bulanan penuh, kartu pengelompokan tugas berdasarkan status atau tanggal, serta tombol tambah agenda (Floating Action Button).



Gambar 4.5.1.1 Tampilan Halaman Planner



Gambar 4.5.1.2 Formulir Tambah dan Edit Tugas Planner

4.5.2 Implementasi Pengelolaan Agenda

Pengelolaan status tampilan dan interaksi pengguna diatur oleh PlannerViewController dan fungsi pembantu buildSections serta toggleTaskWithUndo.

Kode Program 4.5.2.1 Pengendali Tampilan dan Interaksi Agenda Planner

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../../../core/error/app_failure.dart';
import '../../../core/utils/date_only.dart';
import '../data/task_repository.dart';
import '../domain/task.dart';
import '../domain/task_filter.dart';

enum PlannerMode { day, all }

@immutable
class PlannerViewState {
  const PlannerViewState({
    required this.selectedDay,
    this.mode = PlannerMode.day,
    this.filter = const TaskFilter(),
  });

  final DateTime selectedDay;
  final PlannerMode mode;
  final TaskFilter filter;
```

```
  PlannerViewState copyWith({
    DateTime? selectedDay,
    PlannerMode? mode,
    TaskFilter? filter,
  }) {
    return PlannerViewState(
      selectedDay: selectedDay ?? this.selectedDay,
      mode: mode ?? this.mode,
      filter: filter ?? this.filter,
    );
  }
}

/// State UI Planner (filter, mode, tanggal terpilih). Tidak
disimpan.
class PlannerViewController extends Notifier<PlannerViewState>
{
  @override
  PlannerViewState build() =>
      PlannerViewState(selectedDay:
DateOnly.of(DateTime.now()));

  void selectDay(DateTime day) => state = state.copyWith(
        selectedDay: DateOnly.of(day),
        mode: PlannerMode.day,
      );

  void setMode(PlannerMode mode) => state =
state.copyWith(mode: mode);

  void setFilter(TaskFilter filter) => state =
state.copyWith(filter: filter);

  void setQuery(String query) =>
      state = state.copyWith(filter:
state.filter.copyWith(query: query));

  void clearFilter() => state = state.copyWith(
        filter: TaskFilter(sort: state.filter.sort),
      );
}

final NotifierProvider<PlannerViewController,
PlannerViewState>
    plannerViewProvider =
    NotifierProvider<PlannerViewController, PlannerViewState>(
  PlannerViewController.new,
);

/// Kelompok daftar Planner.
@immutable
class TaskSection {
  const TaskSection(this.title, this.tasks);

  final String title;
  final List<Task> tasks;
}

/// Mengelompokkan tugas untuk tampilan (mockup S-20: "Selasa,
20 Okt · 4
```

```
/// tugas", "Selesai (1)").
List<TaskSection> buildSections(
  List<Task> tasks,
  PlannerViewState view,
  DateTime now,
  String Function(DateTime) dayLabel,
) {
  final List<Task> filtered = view.filter.apply(tasks, now);
  if (view.mode == PlannerMode.day) {
    final List<Task> day = filtered
        .where((Task t) => DateOnly.isSameDay(t.dueDate,
view.selectedDay))
        .toList();
    final List<Task> open = day.where((Task t) =>
!t.isCompleted).toList();
    final List<Task> done = day.where((Task t) =>
t.isCompleted).toList();
    return <TaskSection>[
      if (open.isNotEmpty)
        TaskSection(
          '${dayLabel(view.selectedDay)} · ${open.length}
tugas',
          open,
        ),
      if (done.isNotEmpty) TaskSection('Selesai
(${done.length})', done),
    ];
  }
  final DateTime today = DateOnly.of(now);
  final List<Task> overdue = <Task>[];
  final List<Task> todayList = <Task>[];
  final List<Task> upcoming = <Task>[];
  final List<Task> earlier = <Task>[];
  final List<Task> done = <Task>[];
  for (final Task t in filtered) {
    if (t.isCompleted) {
      done.add(t);
    } else if (t.isOverdue(now)) {
      overdue.add(t);
    } else if (DateOnly.isSameDay(t.dueDate, today)) {
      todayList.add(t);
    } else if (t.dueDate.isAfter(today)) {
      upcoming.add(t);
    } else {
      earlier.add(t);
    }
  }
  return <TaskSection>[
    if (overdue.isNotEmpty) TaskSection('Terlambat
(${overdue.length})', overdue),
    if (todayList.isNotEmpty)
      TaskSection('Hari ini (${todayList.length})',
todayList),
    if (upcoming.isNotEmpty)
      TaskSection('Mendatang (${upcoming.length})', upcoming),
    if (earlier.isNotEmpty) TaskSection('Sebelumnya',
earlier),
    if (done.isNotEmpty) TaskSection('Selesai
(${done.length})', done),
  ];
```

```
}

/// Mencentang tugas dengan snackbar "Urungkan" (Tahap 3:
centang punya undo).
Future<void> toggleTaskWithUndo(
  BuildContext context,
  WidgetRef ref,
  Task task,
  bool completed,
) async {
  final ScaffoldMessengerState messenger =
ScaffoldMessenger.of(context);
  final TaskRepository repo =
ref.read(taskRepositoryProvider);
  try {
    final Task saved = await repo.setCompleted(task,
completed);
    messenger
      ..hideCurrentSnackBar()
      ..showSnackBar(
        SnackBar(
          content: Text(
            completed ? 'Tugas ditandai selesai.' : 'Tugas
dibuka lagi.',
          ),
          action: SnackBarAction(
            label: 'Urungkan',
            onPressed: () => repo.setCompleted(saved,
!completed),
          ),
        ),
      );
  } catch (error) {
    final AppFailure f = AppFailure.from(error);
    messenger.showSnackBar(
      SnackBar(content: Text('${f.title}. ${f.userMessage}')),
    );
  }
}
```

Penjelasan Source Code planner_controller.dart

1. Identitas File

   Berkas ini berada pada lib/features/planner/presentation/planner_controller.dart dan mengelola state interaktif antarmuka Planner (tanggal aktif, mode tampilan, kata kunci pencarian, dan filter).

2. State `PlannerViewState` dan Notifier
   - PlannerMode: Mode tampilan harian (day) atau seluruh tugas (all).

- selectDay, setMode, setFilter, setQuery, dan clearFilter memfasilitasi manipulasi state tanpa mengubah data persisten di database.

3. Pengelompokan Antarmuka buildSections

   Baris 77–127: Mengelompokkan tugas menjadi objek TaskSection. Pada mode harian, tugas dipecah menjadi bagian tugas terbuka dan tugas selesai. Pada mode semua tugas, tugas dikelompokkan secara cerdas menjadi "Terlambat", "Hari ini", "Mendatang", "Sebelumnya", dan "Selesai".

4. Fitur Pembatalan toggleTaskWithUndo

   Baris 130–159: Menyediakan pengalaman pengguna yang aman saat menandai tugas selesai. Status tugas diperbarui di repositori lokal dan snackbar visual ditampilkan dengan aksi "Urungkan" (Undo) selama beberapa detik. Jika pengguna menekan urungkan, status tugas dikembalikan ke semula.

5. Alur Kerja

   Di PlannerPage, pemilih kalender atau strip tanggal memanggil selectDay(). PlannerViewController memperbarui PlannerViewState. Widget antarmuka secara reaktif memanggil buildSections untuk me-render daftar kartu tugas yang relevan.

6. Hubungan dengan File Lain

   Berkas ini mengimpor TaskRepository dan TaskFilte, serta digunakan oleh PlannerPage dan widget pendukungnya (task_card.dart).

4.5.3 Implementasi Penyimpanan dan Pengambilan Data Agenda

Entitas agenda diwujudkan dalam kelas domain Task yang mengimplementasikan antarmuka SyncEntity untuk sinkronisasi data offline-first.

Kode Program 4.5.3.1 Entitas Tugas dan Metadata Sinkronisasi

```
import 'package:flutter/foundation.dart';

import '../../../core/sync/sync_entity.dart';
import '../../../core/utils/date_only.dart';
import 'task_priority.dart';
```

```
/// Kategori tugas (check `tasks.category`, Tahap 2 §7.3).
enum TaskCategory {
  kuliah('Kuliah'),
  pekerjaan('Pekerjaan'),
  pribadi('Pribadi'),
  lainnya('Lainnya');

  const TaskCategory(this.label);

  final String label;

  static TaskCategory parse(Object? value) =>
TaskCategory.values.firstWhere(
        (TaskCategory c) => c.name == value,
        orElse: () => TaskCategory.lainnya,
      );
}

/// Asal tugas (check `tasks.source`).
enum TaskSource {
  manual('manual'),
  aiLlm('ai_llm'),
  ruleBased('rule_based');

  const TaskSource(this.sql);

  final String sql;

  static TaskSource parse(Object? value) =>
TaskSource.values.firstWhere(
        (TaskSource s) => s.sql == value,
        orElse: () => TaskSource.manual,
      );
}

/// Tugas Planner. Bentuk JSON = kolom tabel `public.tasks` +
metadata sinkron.
@immutable
class Task implements SyncEntity<Task> {
  const Task({
    required this.id,
    required this.userId,
    required this.title,
    required this.dueDate,
    required this.createdAt,
    required this.updatedAt,
    this.description,
    this.dueTimeMinutes,
    this.category = TaskCategory.lainnya,
    this.priority = TaskPriority.medium,
    this.isCompleted = false,
    this.completedAt,
    this.reminderAt,
    this.source = TaskSource.manual,
    this.deletedAt,
    this.syncInfo = const SyncInfo(),
  });

  static const SyncTableSpec spec = SyncTableSpec(
    table: 'tasks',
```

```
    boxName: 'tasks',
    pkColumns: <String>['id'],
  );

  final String id;
  final String userId;
  final String title;
  final String? description;

  /// Tanggal lokal tanpa jam.
  final DateTime dueDate;

  /// Menit sejak 00:00; `null` = sepanjang hari.
  final int? dueTimeMinutes;
  final TaskCategory category;
  final TaskPriority priority;
  final bool isCompleted;
  final DateTime? completedAt;

  /// Waktu pengingat (UTC). `null` = tanpa pengingat.
  final DateTime? reminderAt;
  final TaskSource source;
  final DateTime createdAt;
  @override
  final DateTime updatedAt;
  @override
  final DateTime? deletedAt;
  @override
  final SyncInfo syncInfo;

  @override
  String get localKey => id;

  bool get isDeleted => deletedAt != null;

  /// Saat tenggat dalam waktu lokal. Tugas tanpa jam dianggap
berakhir
  /// pukul 23:59 pada tanggalnya (untuk status "terlambat").
  DateTime get dueMoment {
    final int minutes = dueTimeMinutes ?? (23 * 60 + 59);
    return DateTime(
      dueDate.year,
      dueDate.month,
      dueDate.day,
      minutes ~/ 60,
      minutes % 60,
    );
  }

  bool isOverdue(DateTime now) => !isCompleted &&
dueMoment.isBefore(now);

  static Task fromJson(Map<String, dynamic> json) {
    final Object? priority = json['priority'];
    return Task(
      id: json['id'] as String,
      userId: json['user_id'] as String,
      title: json['title'] as String,
      description: json['description'] as String?,
      dueDate: DateOnly.parse(json['due_date'] as String),
```

```
      dueTimeMinutes:
TimeOfDayMinutes.fromSql(json['due_time']),
      category: TaskCategory.parse(json['category']),
      priority: TaskPriority.fromValue(priority is num ?
priority.toInt() : 2),
      isCompleted: json['is_completed'] == true,
      completedAt:
parseTimestamp(json['completed_at'])?.toUtc(),
      reminderAt:
parseTimestamp(json['reminder_at'])?.toUtc(),
      source: TaskSource.parse(json['source']),
      createdAt: parseTimestamp(json['created_at'])?.toUtc()
??
          readUpdatedAt(json),
      updatedAt: readUpdatedAt(json),
      deletedAt: parseTimestamp(json['deleted_at'])?.toUtc(),
      syncInfo: SyncInfo.fromJson(json),
    );
  }

  @override
  Map<String, dynamic> toRow() => <String, dynamic>{
        'id': id,
        'user_id': userId,
        'title': title,
        'description': description,
        'due_date': DateOnly.format(dueDate),
        'due_time': dueTimeMinutes == null
            ? null
            : TimeOfDayMinutes.toSql(dueTimeMinutes!),
        'category': category.name,
        'priority': priority.value,
        'is_completed': isCompleted,
        'completed_at':
completedAt?.toUtc().toIso8601String(),
        'reminder_at': reminderAt?.toUtc().toIso8601String(),
        'source': source.sql,
        'created_at': createdAt.toUtc().toIso8601String(),
        'deleted_at': deletedAt?.toUtc().toIso8601String(),
      };

  @override
  Task withSync(SyncInfo info, {DateTime? updatedAt}) =>
      copyWith(syncInfo: info, updatedAt: updatedAt);

  Task copyWith({
    String? title,
    String? description,
    bool clearDescription = false,
    DateTime? dueDate,
    int? dueTimeMinutes,
    bool clearDueTime = false,
    TaskCategory? category,
    TaskPriority? priority,
    bool? isCompleted,
    DateTime? completedAt,
    bool clearCompletedAt = false,
    DateTime? reminderAt,
    bool clearReminder = false,
    TaskSource? source,
```

```
    DateTime? updatedAt,
    DateTime? deletedAt,
    SyncInfo? syncInfo,
  }) {
    return Task(
      id: id,
      userId: userId,
      title: title ?? this.title,
      description: clearDescription ? null : (description ??
this.description),
      dueDate: dueDate ?? this.dueDate,
      dueTimeMinutes:
          clearDueTime ? null : (dueTimeMinutes ??
this.dueTimeMinutes),
      category: category ?? this.category,
      priority: priority ?? this.priority,
      isCompleted: isCompleted ?? this.isCompleted,
      completedAt:
          clearCompletedAt ? null : (completedAt ??
this.completedAt),
      reminderAt: clearReminder ? null : (reminderAt ??
this.reminderAt),
      source: source ?? this.source,
      createdAt: createdAt,
      updatedAt: updatedAt ?? this.updatedAt,
      deletedAt: deletedAt ?? this.deletedAt,
      syncInfo: syncInfo ?? this.syncInfo,
    );
  }

  @override
  bool operator ==(Object other) =>
      other is Task &&
      other.id == id &&
      other.updatedAt == updatedAt &&
      other.syncInfo == syncInfo &&
      other.isCompleted == isCompleted &&
      other.deletedAt == deletedAt;

  @override
  int get hashCode => Object.hash(id, updatedAt, syncInfo,
isCompleted);
}
```

Penjelasan Source Code `task.dart`

1. Identitas File

   Berkas ini berada pada lib/features/planner/domain/task.dart dan merepresentasikan entitas tugas pada basis data lokal Hive dan tabel PostgreSQL public.tasks.

2. Metadata dan Struktur Data

   - TaskCategory (baris 8–22): Kategori tugas (kuliah, pekerjaan, pribadi, lainnya).

- TaskSource (baris 25–38): Asal pembentukan tugas (manual, ai_llm, rule_based).
- spec (baris 62–66): Spesifikasi sinkronisasi yang memetakan entitas ke tabel tasks, box Hive tasks, dengan primary key id.

3. Properti Waktu dan Status
   - dueDate menyimpan tanggal tenggat, sedangkan dueTimeMinutes menyimpan menit sejak pukul 00:00 (misalnya 540 = 09:00).
   - dueMoment (baris 101–110): Properti getter yang menggabungkan tanggal dan jam menjadi objek DateTime lokal yang utuh. Jika jam tidak diatur, waktu diasumsikan berakhir pukul 23:59 pada tanggal tersebut.
   - isOverdue(DateTime now) (baris 112): Mengevaluasi apakah tugas belum selesai dan waktu tenggatnya telah terlewati.
4. Serialisasi Data
   - fromJson (baris 114–135): Mengurai data JSON dari box lokal Hive menjadi instance Task.
   - toRow() (baris 138–155): Menghasilkan Map representasi baris SQL yang siap dikirimkan ke server Supabase melalui mesin sinkronisasi.
5. Alur Kerja

   Setiap kali tugas dibuat atau disunting di antarmuka Planner, objek Task baru dibentuk via copyWith, disimpan di Hive box tasks, dan memicu penjadwal pengingat notifikasi ReminderScheduler.
6. Hubungan dengan File Lain

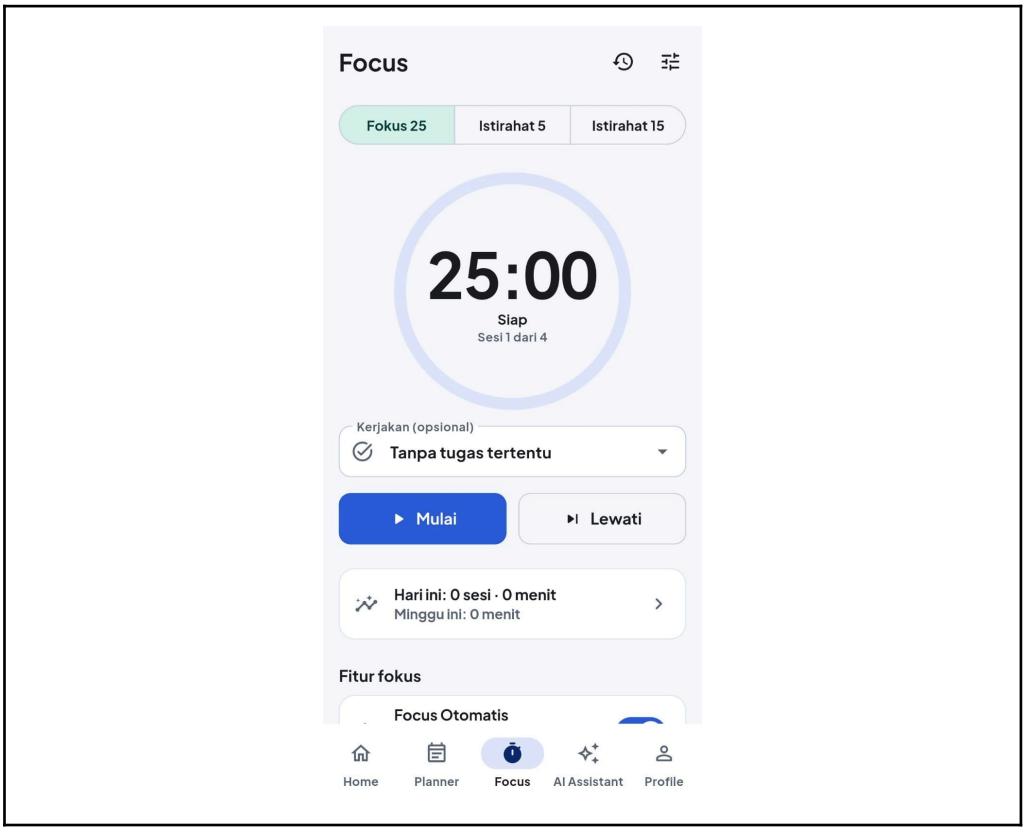
   Berkas ini mengimplementasikan SyncEntity, disimpan oleh TaskRepository, diamati oleh HomePage dan PlannerPage, serta ditautkan pada fitur FocusMode dan BudgetHelper.

4.6 Implementasi Focus Mode dan Pomodoro

Focus Mode memfasilitasi pengguna untuk berkonsentrasi pada aktivitas terarah menggunakan teknik Pomodoro. Durasi fokus dan istirahat dapat dikustomisasi, serta status waktu dihitung menggunakan jam dinding agar tahan terhadap interupsi sistem operasi.

4.6.1 Tampilan Halaman Focus

Halaman Focus menyajikan lingkaran waktu mundur (countdown circle) berdimensi besar dengan kode warna yang berubah sesuai fase (Fokus, Istirahat Pendek, Istirahat Panjang), tombol kontrol utama (Mulai, Jeda, Selesai), pemilih tugas dari Planner, serta lembar pengaturan durasi waktu.



Gambar 4.6.1.1 Tampilan Antarmuka Focus Mode



Gambar 4.6.1.2 Pengaturan Durasi Sesi Pomodoro

4.6.2 Implementasi Timer Pomodoro

Ketahanan timer Pomodoro terhadap kondisi aplikasi diminimalkan atau ditutup paksa diwujudkan oleh kelas domain PomodoroEngine, TimerState, dan PomodoroSettings.

Kode Program 4.6.2.1 Mesin Waktu dan Siklus Pomodoro Berbasis Jam Dinding

```
import 'package:flutter/foundation.dart';

import '../../../core/utils/date_only.dart';
import 'focus_session.dart';

enum TimerStatus { idle, running, paused }

/// Durasi Pomodoro yang dapat diatur (prompt implementasi
final §D; Tahap 1
/// default 25/5/15 dengan istirahat panjang setiap 4 sesi
fokus).
@immutable
class PomodoroSettings {
  const PomodoroSettings({
    this.focusMinutes = 25,
    this.shortBreakMinutes = 5,
    this.longBreakMinutes = 15,
    this.sessionsBeforeLongBreak = 4,
  });

  static const int minMinutes = 1;
  static const int maxMinutes = 120;
```

```
  final int focusMinutes;
  final int shortBreakMinutes;
  final int longBreakMinutes;
  final int sessionsBeforeLongBreak;

  int secondsFor(PomodoroMode mode) => switch (mode) {
        PomodoroMode.focus => focusMinutes * 60,
        PomodoroMode.shortBreak => shortBreakMinutes * 60,
        PomodoroMode.longBreak => longBreakMinutes * 60,
      };

  int minutesFor(PomodoroMode mode) => secondsFor(mode) ~/ 60;

  PomodoroSettings copyWith({
    int? focusMinutes,
    int? shortBreakMinutes,
    int? longBreakMinutes,
    int? sessionsBeforeLongBreak,
  }) {
    int clamp(int v) => v.clamp(minMinutes, maxMinutes);
    return PomodoroSettings(
      focusMinutes: clamp(focusMinutes ?? this.focusMinutes),
      shortBreakMinutes: clamp(shortBreakMinutes ??
this.shortBreakMinutes),
      longBreakMinutes: clamp(longBreakMinutes ??
this.longBreakMinutes),
      sessionsBeforeLongBreak:
          (sessionsBeforeLongBreak ??
this.sessionsBeforeLongBreak).clamp(2, 8),
    );
  }

  Map<String, dynamic> toJson() => <String, dynamic>{
        'focus': focusMinutes,
        'short': shortBreakMinutes,
        'long': longBreakMinutes,
        'cycle': sessionsBeforeLongBreak,
      };

  static PomodoroSettings fromJson(Map<String, dynamic>? json)
{
    if (json == null) {
      return const PomodoroSettings();
    }
    int read(String k, int d) {
      final Object? v = json[k];
      return v is num ? v.toInt() : d;
    }

    return const PomodoroSettings().copyWith(
      focusMinutes: read('focus', 25),
      shortBreakMinutes: read('short', 5),
      longBreakMinutes: read('long', 15),
      sessionsBeforeLongBreak: read('cycle', 4),
    );
  }

  @override
  bool operator ==(Object other) =>
```

```
      other is PomodoroSettings &&
      other.focusMinutes == focusMinutes &&
      other.shortBreakMinutes == shortBreakMinutes &&
      other.longBreakMinutes == longBreakMinutes &&
      other.sessionsBeforeLongBreak ==
sessionsBeforeLongBreak;

  @override
  int get hashCode => Object.hash(
        focusMinutes,
        shortBreakMinutes,
        longBreakMinutes,
        sessionsBeforeLongBreak,
      );
}

/// Status timer yang disimpan di box `timer_state` (Tahap 2
§6.2). Waktu
/// dihitung dari jam dinding, bukan dari jumlah tick,
sehingga tetap akurat
/// setelah aplikasi di background atau ditutup paksa (FR-023,
NFR-012).
@immutable
class TimerState {
  const TimerState({
    this.mode = PomodoroMode.focus,
    this.status = TimerStatus.idle,
    this.plannedSeconds = 25 * 60,
    this.startedAt,
    this.pausedAt,
    this.accumulatedPausedSeconds = 0,
    this.completedFocusCount = 0,
    this.taskId,
    this.cycleDate,
  });

  final PomodoroMode mode;
  final TimerStatus status;
  final int plannedSeconds;

  /// Saat sesi pertama kali dimulai (UTC).
  final DateTime? startedAt;
  final DateTime? pausedAt;
  final int accumulatedPausedSeconds;

  /// Jumlah sesi fokus selesai dalam siklus saat ini (reset
tiap hari).
  final int completedFocusCount;

  /// Tugas yang sedang dikerjakan (opsional,
`focus_sessions.task_id`).
  final String? taskId;

  /// Tanggal siklus (yyyy-MM-dd) untuk mereset hitungan
setiap hari.
  final String? cycleDate;

  bool get isRunning => status == TimerStatus.running;
  bool get isIdle => status == TimerStatus.idle;
```

```
  int elapsedSeconds(DateTime now) {
    final DateTime? start = startedAt;
    if (start == null) {
      return 0;
    }
    final DateTime until =
        status == TimerStatus.paused ? (pausedAt ?? now) :
now;
    final int elapsed =
        until.difference(start).inSeconds -
accumulatedPausedSeconds;
    return elapsed.clamp(0, plannedSeconds);
  }

  int remainingSeconds(DateTime now) =>
      isIdle ? plannedSeconds : plannedSeconds -
elapsedSeconds(now);

  /// Saat timer berakhir bila terus berjalan.
  DateTime? get endsAt {
    final DateTime? start = startedAt;
    if (start == null || status != TimerStatus.running) {
      return null;
    }
    return start.add(
      Duration(seconds: plannedSeconds +
accumulatedPausedSeconds),
    );
  }

  TimerState copyWith({
    PomodoroMode? mode,
    TimerStatus? status,
    int? plannedSeconds,
    DateTime? startedAt,
    bool clearStartedAt = false,
    DateTime? pausedAt,
    bool clearPausedAt = false,
    int? accumulatedPausedSeconds,
    int? completedFocusCount,
    String? taskId,
    bool clearTaskId = false,
    String? cycleDate,
  }) {
    return TimerState(
      mode: mode ?? this.mode,
      status: status ?? this.status,
      plannedSeconds: plannedSeconds ?? this.plannedSeconds,
      startedAt: clearStartedAt ? null : (startedAt ??
this.startedAt),
      pausedAt: clearPausedAt ? null : (pausedAt ??
this.pausedAt),
      accumulatedPausedSeconds:
          accumulatedPausedSeconds ??
this.accumulatedPausedSeconds,
      completedFocusCount: completedFocusCount ??
this.completedFocusCount,
      taskId: clearTaskId ? null : (taskId ?? this.taskId),
      cycleDate: cycleDate ?? this.cycleDate,
    );
```

```
  }

  Map<String, dynamic> toJson() => <String, dynamic>{
        'mode': mode.sql,
        'status': status.name,
        'planned_seconds': plannedSeconds,
        'started_at': startedAt?.toUtc().toIso8601String(),
        'paused_at': pausedAt?.toUtc().toIso8601String(),
        'paused_seconds': accumulatedPausedSeconds,
        'completed_focus': completedFocusCount,
        'task_id': taskId,
        'cycle_date': cycleDate,
      };

  static TimerState fromJson(Map<String, dynamic>? json) {
    if (json == null) {
      return const TimerState();
    }
    int read(String k, int d) {
      final Object? v = json[k];
      return v is num ? v.toInt() : d;
    }

    return TimerState(
      mode: PomodoroMode.parse(json['mode']),
      status: TimerStatus.values.firstWhere(
        (TimerStatus s) => s.name == json['status'],
        orElse: () => TimerStatus.idle,
      ),
      plannedSeconds: read('planned_seconds', 25 * 60),
      startedAt: parseTimestamp(json['started_at'])?.toUtc(),
      pausedAt: parseTimestamp(json['paused_at'])?.toUtc(),
      accumulatedPausedSeconds: read('paused_seconds', 0),
      completedFocusCount: read('completed_focus', 0),
      taskId: json['task_id'] as String?,
      cycleDate: json['cycle_date'] as String?,
    );
  }

  @override
  bool operator ==(Object other) =>
      other is TimerState && _jsonEq(other.toJson(),
toJson());

  @override
  int get hashCode => Object.hashAll(toJson().values);
}

bool _jsonEq(Map<String, dynamic> a, Map<String, dynamic> b) {
  if (a.length != b.length) {
    return false;
  }
  for (final String k in a.keys) {
    if (a[k] != b[k]) {
      return false;
    }
  }
  return true;
}
```

```
/// Hasil penyelesaian sesi.
@immutable
class CompletedRun {
  const CompletedRun({
    required this.mode,
    required this.plannedSeconds,
    required this.actualSeconds,
    required this.startedAt,
    required this.endedAt,
    required this.status,
    this.taskId,
  });

  final PomodoroMode mode;
  final int plannedSeconds;
  final int actualSeconds;
  final DateTime startedAt;
  final DateTime endedAt;
  final FocusSessionStatus status;
  final String? taskId;
}

/// Logika Pomodoro murni (Tahap 2 §4:
`pomodoro_engine.dart`).
abstract final class PomodoroEngine {
  /// Sesi fokus yang dibatalkan baru dicatat bila sudah
berjalan selama ini.
  static const int minCancelledSecondsToRecord = 60;

  static TimerState _rollCycle(TimerState s, DateTime now) {
    final String today = DateOnly.format(now.toLocal());
    if (s.cycleDate == today) {
      return s;
    }
    return s.copyWith(completedFocusCount: 0, cycleDate:
today);
  }

  /// Memilih mode saat idle; durasi mengikuti [settings].
  static TimerState selectMode(
    TimerState s,
    PomodoroMode mode,
    PomodoroSettings settings,
  ) {
    if (!s.isIdle) {
      return s;
    }
    return s.copyWith(mode: mode, plannedSeconds:
settings.secondsFor(mode));
  }

  static TimerState start(TimerState s, DateTime now, {String?
taskId}) {
    if (!s.isIdle) {
      return s;
    }
    return _rollCycle(s, now).copyWith(
      status: TimerStatus.running,
      startedAt: now.toUtc(),
      clearPausedAt: true,
```

```
      accumulatedPausedSeconds: 0,
      taskId: taskId,
    );
  }

  static TimerState pause(TimerState s, DateTime now) {
    if (s.status != TimerStatus.running) {
      return s;
    }
    return s.copyWith(status: TimerStatus.paused, pausedAt:
now.toUtc());
  }

  static TimerState resume(TimerState s, DateTime now) {
    final DateTime? pausedAt = s.pausedAt;
    if (s.status != TimerStatus.paused || pausedAt == null) {
      return s;
    }
    final int paused =
now.toUtc().difference(pausedAt).inSeconds;
    return s.copyWith(
      status: TimerStatus.running,
      clearPausedAt: true,
      accumulatedPausedSeconds: s.accumulatedPausedSeconds +
paused,
    );
  }

  static bool isFinished(TimerState s, DateTime now) =>
      s.isRunning && s.remainingSeconds(now) <= 0;

  /// Reset ke idle. Mengembalikan catatan "dibatalkan" bila
sesi fokus sudah
  /// berjalan cukup lama, agar tetap tercatat jujur (bukan
"selesai").
  static (TimerState, CompletedRun?) reset(
    TimerState s,
    DateTime now,
    PomodoroSettings settings,
  ) {
    CompletedRun? cancelled;
    final DateTime? start = s.startedAt;
    final int elapsed = s.elapsedSeconds(now);
    if (!s.isIdle &&
        start != null &&
        s.mode == PomodoroMode.focus &&
        elapsed >= minCancelledSecondsToRecord) {
      cancelled = CompletedRun(
        mode: s.mode,
        plannedSeconds: s.plannedSeconds,
        actualSeconds: elapsed,
        startedAt: start,
        endedAt: now.toUtc(),
        status: FocusSessionStatus.cancelled,
        taskId: s.taskId,
      );
    }
    final TimerState idle = s.copyWith(
      status: TimerStatus.idle,
      plannedSeconds: settings.secondsFor(s.mode),
```

```
      clearStartedAt: true,
      clearPausedAt: true,
      accumulatedPausedSeconds: 0,
    );
    return (idle, cancelled);
  }

  /// Mode berikutnya: setiap N sesi fokus → istirahat
panjang.
  static PomodoroMode nextMode(
    PomodoroMode finished,
    int completedFocusCount,
    PomodoroSettings settings,
  ) {
    if (finished != PomodoroMode.focus) {
      return PomodoroMode.focus;
    }
    return completedFocusCount %
settings.sessionsBeforeLongBreak == 0
        ? PomodoroMode.longBreak
        : PomodoroMode.shortBreak;
  }

  /// Menyelesaikan sesi yang waktunya habis. `endedAt` =
waktu berakhir
  /// teoretis (bukan saat aplikasi dibuka lagi) agar riwayat
akurat.
  static (TimerState, CompletedRun) complete(
    TimerState s,
    DateTime now,
    PomodoroSettings settings,
  ) {
    final DateTime start = s.startedAt ?? now.toUtc();
    final DateTime ended = s.endsAt ?? now.toUtc();
    final int count = s.mode == PomodoroMode.focus
        ? s.completedFocusCount + 1
        : s.completedFocusCount;
    final PomodoroMode next = nextMode(s.mode, count,
settings);
    final CompletedRun run = CompletedRun(
      mode: s.mode,
      plannedSeconds: s.plannedSeconds,
      actualSeconds: s.plannedSeconds,
      startedAt: start,
      endedAt: ended,
      status: FocusSessionStatus.completed,
      taskId: s.taskId,
    );
    final TimerState idle = TimerState(
      mode: next,
      plannedSeconds: settings.secondsFor(next),
      completedFocusCount:
          next == PomodoroMode.focus && s.mode ==
PomodoroMode.longBreak
              ? 0
              : count,
      taskId: s.taskId,
      cycleDate: s.cycleDate,
    );
    return (idle, run);
```

```
  }
}
```

Penjelasan Source Code pomodoro_engine.dart

1. Identitas File

   Berkas ini berada pada lib/features/focus/domain/pomodoro_engine.dart dan mengimplementasikan seluruh aturan mesin waktu Pomodoro murni (pure logic tanpa dependensi UI).

2. Kelas PomodoroSettings
   - Menyimpan durasi waktu fokus (focusMinutes), istirahat pendek (shortBreakMinutes), istirahat panjang (longBreakMinutes), serta siklus menuju istirahat panjang (sessionsBeforeLongBreak).
   - Mendukung serialisasi JSON untuk disimpan di box Hive `device_settings`.
3. Kelas TimerState (Prinsip Jam Dinding)
   - Baris 130–140: Metode elapsedSeconds menghitung durasi yang telah berlalu melalui selisih matematis antara waktu saat ini (now) dengan waktu sesi dimulai (startedAt), dikurangi akumulasi jeda (accumulatedPausedSeconds).
   - Baris 146–155: Properti endsAt menghitung proyeksi waktu berakhirnya sesi dalam stempel waktu UTC absolut.
   - Ketahanan ini menjamin bahwa jika aplikasi ditutup atau diminimalkan saat timer berjalan 25 menit, ketika pengguna membuka kembali aplikasi 20 menit kemudian, sisa waktu akan secara akurat menampilkan 5 menit.
4. Logika Transisi Sesi PomodoroEngine
   - start, pause, dan resume: Mengatur transisi status timer.
   - nextMode (baris 359–370): Menghitung mode berikutnya. Jika pengguna menyelesaikan sesi fokus ke-4, mode berikutnya secara otomatis dialihkan ke PomodoroMode.longBreak, sedangkan pada sesi lainnya beralih ke PomodoroMode.shortBrea`.

- complete (baris 374–405): Menghasilkan rekaman CompletedRun berstatus FocusSessionStatus.completed untuk dicatat ke dalam riwayat sesi.

5. Alur Kerja

   Saat tombol Mulai ditekan di FocusPage, PomodoroController memanggil PomodoroEngine.start(). Waktu akhir dijadwalkan ke NotificationService sebagai alarm sistem. Setiap detik, antarmuka memperbarui visual sisa detik via TimerState.remainingSeconds().

6. Hubungan dengan File Lain

   Berkas ini digunakan oleh PomodoroController, disimpan ke box Hive timer_state, memicu notifikasi selesai via NotificationService, dan menghasilkan entitas FocusSession untuk disimpan di FocusRepository.

4.6.3 Implementasi Sesi Fokus dan Istirahat

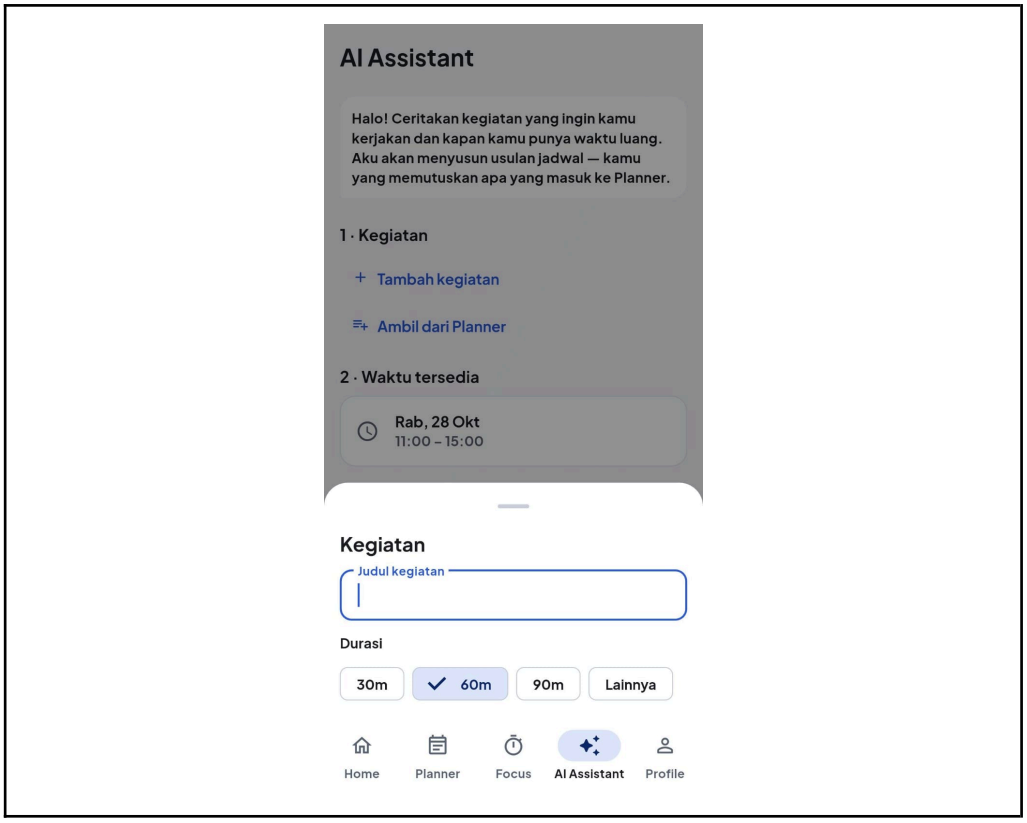
Sesi yang berhasil diselesaikan dicatat secara permanen ke dalam tabel lokal dan disinkronkan ke Supabase public.focus_sessions. Riwayat ini memuat durasi aktual, stempel waktu mulai dan selesai, serta ID tugas Planner yang sedang dikerjakan. Jika sesi fokus dibatalkan di tengah jalan tetapi telah berlangsung lebih dari 60 detik (minCancelledSecondsToRecord), sesi tersebut tetap dicatat secara transparan dengan status cancelled untuk menjaga kejujuran statistik produktivitas pengguna.

4.7 Implementasi AI Assistant

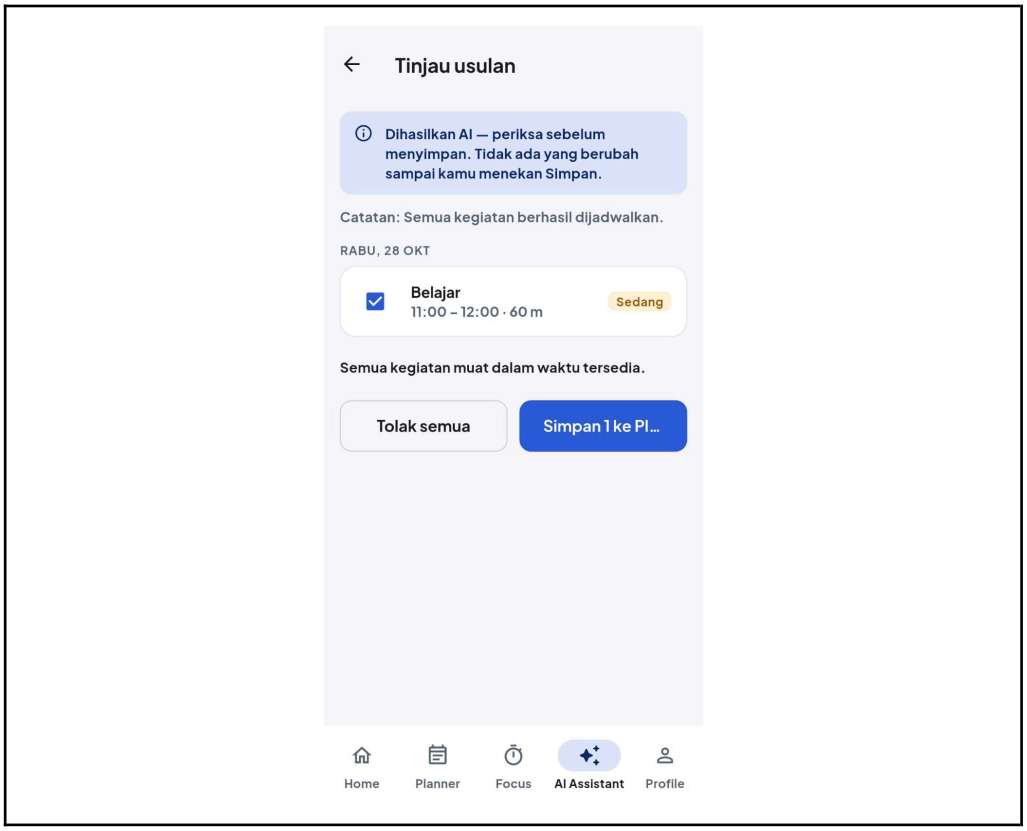
Fitur AI Assistant menganalisis sekumpulan agenda kegiatan pengguna yang belum terjadwal (unscheduled tasks) dan merekomendasikan penempatan slot waktu optimal berdasarkan preferensi jendela kerja harian dan batas waktu tenggat.

4.7.1 Tampilan Halaman AI Assistant

Antarmuka AI Assistant menyediakan formulir input parameter penjadwalan (rentang tanggal, jendela jam kerja, dan durasi istirahat), pemilih agenda dari Planner, indikator proses generasi berbasis model bahasa besar (Large Language Model), serta halaman tinjauan jadwal (AI Review Page) sebelum jadwal diterapkan secara permanen ke Planner.



Gambar 4.7.1.1 Tampilan Formulir AI Assistant



Gambar 4.7.1.2 Tampilan Rekomendasi Jadwal AI Review

4.7.2 Implementasi Pengolahan Permintaan Pengguna

Struktur data kontrak komunikasi penjadwalan didefinisikan secara ketat pada berkas domain schedule_models.dart.

Kode Program 4.7.2.1 Model Data Permintaan dan Usulan Jadwal AI

```
import 'package:flutter/foundation.dart';

import '../../planner/domain/task_priority.dart';

/// Kegiatan yang ingin dijadwalkan (kontrak §11.2 Tahap 2).
@immutable
class ScheduleActivity {
  const ScheduleActivity({
    required this.ref,
    required this.title,
    required this.durationMinutes,
    this.priority = TaskPriority.medium,
    this.deadline,
    this.taskId,
  });

  /// ID lokal kegiatan di permintaan ini (bukan ID tugas,
bukan ID pengguna).
  final String ref;
  final String title;
  final int durationMinutes;
  final TaskPriority priority;

  /// Tenggat (waktu lokal). `null` = tanpa tenggat.
  final DateTime? deadline;

  /// Diisi bila kegiatan diambil dari tugas Planner yang
sudah ada. TIDAK
  /// dikirim ke server.
  final String? taskId;

  ScheduleActivity copyWith({
    String? title,
    int? durationMinutes,
    TaskPriority? priority,
    DateTime? deadline,
    bool clearDeadline = false,
  }) {
    return ScheduleActivity(
      ref: ref,
      title: title ?? this.title,
      durationMinutes: durationMinutes ??
this.durationMinutes,
      priority: priority ?? this.priority,
      deadline: clearDeadline ? null : (deadline ??
this.deadline),
      taskId: taskId,
    );
  }
}

/// Rentang waktu kosong (waktu lokal).
@immutable
```

```
class TimeWindow {
  const TimeWindow({required this.start, required this.end});

  final DateTime start;
  final DateTime end;

  int get minutes => end.difference(start).inMinutes;

  bool contains(DateTime s, DateTime e) => !s.isBefore(start)
&& !e.isAfter(end);
}

@immutable
class ScheduleRequest {
  const ScheduleRequest({
    required this.timezone,
    required this.activities,
    required this.windows,
  });

  /// Nama zona IANA perangkat, mis. `Asia/Jakarta`.
  final String timezone;
  final List<ScheduleActivity> activities;
  final List<TimeWindow> windows;

  /// Body untuk Edge Function. Hanya judul, durasi,
prioritas, tenggat, dan
  /// jendela waktu — tanpa email/ID pengguna (Tahap 2 §15,
NFR-007).
  Map<String, dynamic> toJson() => <String, dynamic>{
        'timezone': timezone,
        'activities': <Map<String, dynamic>>[
          for (final ScheduleActivity a in activities)
            <String, dynamic>{
              'ref': a.ref,
              'title': a.title,
              'duration_minutes': a.durationMinutes,
              'priority': a.priority.value,
              'deadline': a.deadline == null ? null :
isoWithOffset(a.deadline!),
            },
        ],
        'available_windows': <Map<String, dynamic>>[
          for (final TimeWindow w in windows)
            <String, dynamic>{
              'start': isoWithOffset(w.start),
              'end': isoWithOffset(w.end),
            },
        ],
      };
}

enum SuggestionSource {
  /// Dari Edge Function + LLM.
  llm,

  /// `RuleBasedScheduler` di perangkat — BUKAN AI.
  ruleBased;

  String get label => switch (this) {
```

```
        SuggestionSource.llm => 'Dihasilkan AI',
        SuggestionSource.ruleBased => 'Berbasis aturan (bukan
AI)',
      };
}

@immutable
class ScheduledItem {
  const ScheduledItem({
    required this.ref,
    required this.title,
    required this.start,
    required this.end,
  });

  final String ref;
  final String title;
  final DateTime start;
  final DateTime end;

  int get minutes => end.difference(start).inMinutes;
}

@immutable
class ScheduleSuggestion {
  const ScheduleSuggestion({
    required this.source,
    required this.items,
    this.unscheduled = const <String>[],
    this.note,
  });

  final SuggestionSource source;
  final List<ScheduledItem> items;

  /// `ref` kegiatan yang tidak muat.
  final List<String> unscheduled;
  final String? note;

  /// Membaca respons sukses Edge Function. Melempar
[FormatException] bila
  /// bentuknya salah.
  static ScheduleSuggestion fromJson(Map<String, dynamic>
json) {
    final Object? items = json['items'];
    if (items is! List) {
      throw const FormatException('items tidak ada');
    }
    return ScheduleSuggestion(
      source: SuggestionSource.llm,
      items: <ScheduledItem>[
        for (final Object? raw in items)
          if (raw is Map)
            ScheduledItem(
              ref: raw['ref'] as String,
              title: raw['title'] as String,
              start: DateTime.parse(raw['start'] as
String).toLocal(),
              end: DateTime.parse(raw['end'] as
String).toLocal(),
```

```
            ),
      ],
      unscheduled: <String>[
        for (final Object? r in (json['unscheduled'] as List?)
?? <Object?>[])
          if (r is String) r else if (r is Map && r['ref'] is
String) r['ref'] as String,
      ],
      note: json['note'] as String?,
    );
  }
}

/// ISO-8601 dengan offset lokal, mis.
`2026-10-21T13:00:00+07:00`.
String isoWithOffset(DateTime local) {
  final DateTime t = local.toLocal();
  String two(int n) => n.toString().padLeft(2, '0');
  final Duration off = t.timeZoneOffset;
  final String sign = off.isNegative ? '-' : '+';
  final int mins = off.inMinutes.abs();
  return '${t.year.toString().padLeft(4,
'0')}-${two(t.month)}-${two(t.day)}'
      'T${two(t.hour)}:${two(t.minute)}:${two(t.second)}'
      '$sign${two(mins ~/ 60)}:${two(mins % 60)}';
}
```

Penjelasan Source Code schedule_models.dart

1. Identitas File

   Berkas ini berada pada lib/features/ai_assistant/domain/schedule_models.dart dan mendefinisikan seluruh model data masukan dan luaran penjadwalan aktivitas.

2. Model Masukan
   - ScheduleActivity (baris 6–46): Mewakili satu aktivitas yang akan dijadwalkan, memuat pengenal sementara ref`, judul, estimasi durasi dalam menit, prioritas, dan tenggat waktu.
   - TimeWindow (baris 49–59): Mewakili rentang waktu luang pengguna yang dapat diisi oleh kegiatan.
   - ScheduleRequest (baris 61–96): Membungkus daftar kegiatan, jendela waktu, dan zona waktu IANA perangkat (timezone). Metode toJson() hanya memuat data esensial penjadwalan tanpa menyertakan identitas pribadi (surel atau ID akun) demi privasi pengguna.
3. Model Luaran dan Asal Rekomendasi

- SuggestionSource (baris 98–109): Menandai asal usulan jadwal secara transparan, yaitu llm ("Dihasilkan AI") atau ruleBased ("Berbasis aturan (bukan AI)").
- ScheduledItem (baris 111–127): Mewakili aktivitas yang berhasil dialokasikan slot waktunya dengan waktu mulai (start) dan selesai (end).
- ScheduleSuggestion (baris 129–170): Objek hasil penyusunan yang memuat kumpulan items, daftar unscheduled (kegiatan yang secara matematis tidak muat dalam jendela waktu), serta catatan note.
- isoWithOffset (baris 173–182): Memformat stempel waktu menjadi string ISO-8601 lengkap dengan offset zona waktu lokal (misalnya +07:00 untuk WIB).

4. Alur Kerja

   Saat pengguna menekan tombol "Susun Jadwal", antarmuka membentuk instance ScheduleRequest. Objek ini dikirimkan ke AiRepository untuk diteruskan ke fungsi serverless atau mesin aturan lokal.

5. Hubungan dengan File Lain

   Berkas ini digunakan oleh AiRepository, AiAssistantPage, AiReviewPage, dan RuleBasedScheduler

4.7.3 Implementasi Rekomendasi Jadwal

Penjadwalan cerdas Planify menerapkan prinsip pertahanan berlapis. Ketika pemanggilan serverless Edge Function ke model Google Gemini 2.5 Flash-Lite gagal (akibat kendala jaringan, batasan kuota, atau perangkat offline), sistem secara otomatis beralih ke mesin aturan lokal deterministik RuleBasedScheduler.

Kode Program 4.7.3.1 Mesin Penjadwal Berbasis Aturan Deterministik

```
import 'schedule_models.dart';

/// Penyusun jadwal cadangan berbasis ATURAN (FR-031). **Bukan
LLM/AI.**
///
/// 1. Urutkan: tenggat terdekat → prioritas tertinggi →
durasi terpendek.
/// 2. Isi slot paling awal yang muat (greedy earliest-fit) di
```

```
jendela waktu,
///    tanpa melewati tenggat dan tanpa tumpang tindih.
/// 3. Yang tidak muat → `unscheduled`.
abstract final class RuleBasedScheduler {
  /// Jeda antar-kegiatan (menit) supaya jadwal tidak terlalu
rapat.
  static const int gapMinutes = 0;

  static ScheduleSuggestion schedule(
    ScheduleRequest request, {
    required DateTime now,
  }) {
    final List<ScheduleActivity> ordered =
        List<ScheduleActivity>.of(request.activities)
          ..sort((ScheduleActivity a, ScheduleActivity b) {
            final DateTime far = DateTime(9999);
            final int byDeadline =
                (a.deadline ?? far).compareTo(b.deadline ??
far);
            if (byDeadline != 0) {
              return byDeadline;
            }
            final int byPriority =
b.priority.value.compareTo(a.priority.value);
            if (byPriority != 0) {
              return byPriority;
            }
            return
a.durationMinutes.compareTo(b.durationMinutes);
          });

    // Slot kosong, dimulai tidak lebih awal dari sekarang
(dibulatkan ke
    // 5 menit berikutnya).
    final DateTime earliest = _roundUp(now);
    final List<TimeWindow> free = <TimeWindow>[
      for (final TimeWindow w in
List<TimeWindow>.of(request.windows)
        ..sort((TimeWindow a, TimeWindow b) =>
a.start.compareTo(b.start)))
        if (w.end.isAfter(earliest))
          TimeWindow(
            start: w.start.isBefore(earliest) ? earliest :
w.start,
            end: w.end,
          ),
    ];

    final List<ScheduledItem> items = <ScheduledItem>[];
    final List<String> unscheduled = <String>[];

    for (final ScheduleActivity a in ordered) {
      final Duration need = Duration(minutes:
a.durationMinutes);
      int? slotIndex;
      for (int i = 0; i < free.length; i++) {
        final DateTime start = free[i].start;
        final DateTime end = start.add(need);
        final bool fits = !end.isAfter(free[i].end);
        final bool beforeDeadline =
```

```
            a.deadline == null || !end.isAfter(a.deadline!);
        if (fits && beforeDeadline) {
          slotIndex = i;
          break;
        }
      }
      if (slotIndex == null) {
        unscheduled.add(a.ref);
        continue;
      }
      final TimeWindow slot = free[slotIndex];
      final DateTime end = slot.start.add(need);
      items.add(ScheduledItem(
        ref: a.ref,
        title: a.title.trim(),
        start: slot.start,
        end: end,
      ));
      final DateTime nextStart = end.add(const
Duration(minutes: gapMinutes));
      if (nextStart.isBefore(slot.end)) {
        free[slotIndex] = TimeWindow(start: nextStart, end:
slot.end);
      } else {
        free.removeAt(slotIndex);
      }
    }

    items.sort((ScheduledItem a, ScheduledItem b) =>
a.start.compareTo(b.start));
    return ScheduleSuggestion(
      source: SuggestionSource.ruleBased,
      items: items,
      unscheduled: unscheduled,
      note: 'Disusun dengan aturan sederhana: tenggat terdekat
dan prioritas '
          'tertinggi ditempatkan lebih awal.',
    );
  }

  static DateTime _roundUp(DateTime t) {
    final int extra = (5 - t.minute % 5) % 5;
    final DateTime base = DateTime(t.year, t.month, t.day,
t.hour, t.minute);
    return (extra == 0 && t.second == 0 && t.millisecond == 0)
        ? base
        : base.add(Duration(minutes: extra == 0 ? 5 : extra));
  }
}
```

Penjelasan Source Code rule_based_scheduler.dart

1. Identitas File

   Berkas ini berada pada lib/features/ai_assistant/domain/rule_based_scheduler.dart dan mengimplementasikan algoritma penjadwalan heuristik deterministik murni di sisi perangkat klien.

2. Tahapan Algoritma Penjadwalan
    - Pengurutan Prioritas (Prioritization): Baris 17–31 mengurutkan seluruh kegiatan berdasarkan tiga kriteria utama: tenggat waktu paling mendesak (deadline), tingkat prioritas tugas tertinggi (priority.value), dan durasi kegiatan terpendek (durationMinutes).
    - Penyesuaian Waktu Mulai Awal: Baris 35 memanggil fungsi _roundUp(now) untuk membulatkan waktu awal ke kelipatan 5 menit berikutnya, mencegah penjadwalan kegiatan di masa lampau.
    - Pencarian Slot (Greedy Earliest-Fit): Baris 49–82 menelusuri setiap jendela waktu bebas. Kegiatan dialokasikan pada slot paling awal yang mencukupi durasi kegiatan serta tidak melewati tenggat waktu. Jendela waktu bebas kemudian dipangkas sebesar durasi kegiatan. Kegiatan yang tidak menemukan ruang dialihkan ke daftar unscheduled.
3. Alur Kerja

    Ketika AiRepository mendeteksi bahwa fungsi AI server tidak dapat diakses, metode RuleBasedScheduler.schedule() dieksekusi secara instan tanpa sambungan internet. Hasil jadwal diserahkan ke AiReviewPage dengan label transparan source: SuggestionSource.ruleBased.

4. Hubungan dengan File Lain

    Berkas ini mengonsumsi model ScheduleRequest dan menghasilkan objek ScheduleSuggestion, serta dipanggil oleh AiRepository.

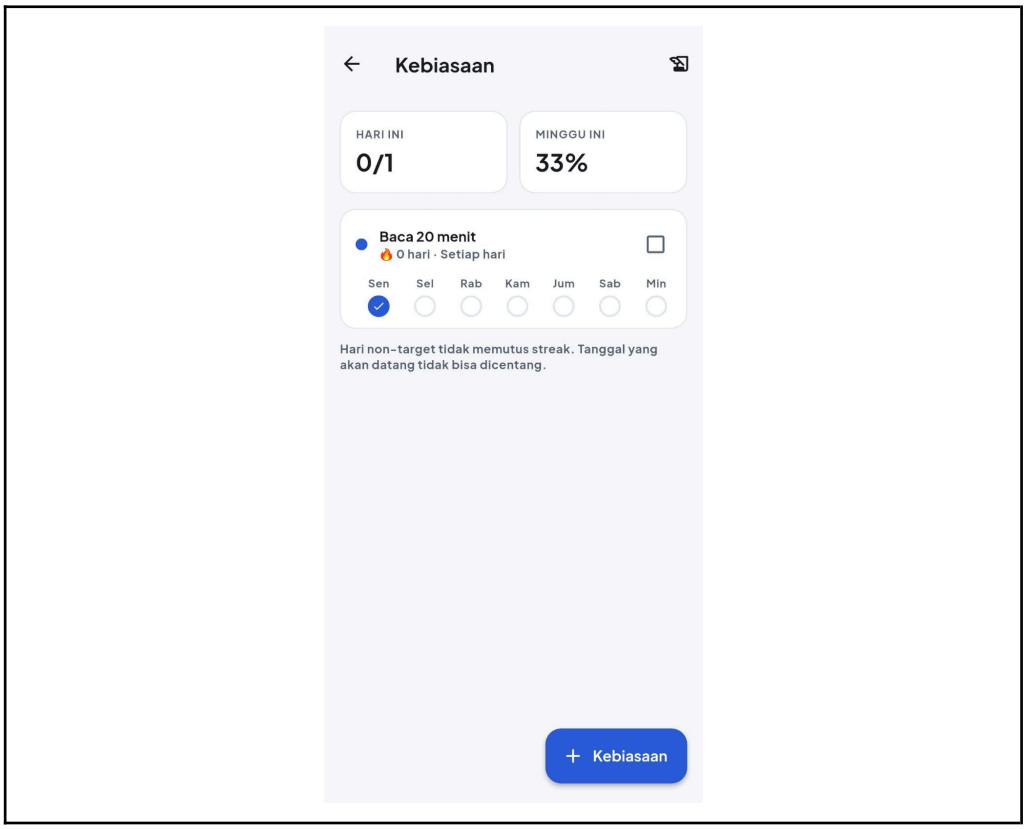
4.8 Implementasi Habit Tracker

Modul Habit Tracker memfasilitasi pembentukan kebiasaan positif berulang melalui pemantauan target harian, visualisasi konsistensi mingguan, dan kalkulasi rekor rentetan hari berturut-turut (streak).

4.8.1 Tampilan Halaman Habit Tracker

Antarmuka Habit Tracker menyajikan daftar kartu kebiasaan aktif yang dilengkapi pemilih target hari (Senin hingga Minggu), indikator warna

pembeda, tombol centang penyelesaian instan untuk hari ini, serta akses menuju halaman detail kebiasaan (HabitDetailPage).



Gambar 4.8.1.1 Tampilan Daftar Kebiasaan (Habit Tracker)

4.8.2 Implementasi Pengelolaan Kebiasaan

Pengguna dapat membuat, menyunting, dan menonaktifkan kebiasaan melalui `HabitFormPage`. Setiap entitas `Habit` menyimpan target hari aktif mingguan (`targetWeekdays`), representasi warna hex 32-bit (`colorValue`), serta kunci ikon visual (`iconKey`). Kebiasaan yang dihapus menerapkan mekanisme *soft delete* (`deletedAt`) untuk menjaga keutuhan riwayat capaian di masa lalu.

4.8.3 Implementasi Pengelolaan Kebiasaan

Setiap kali pengguna menandai kebiasaan selesai, sebuah entri log baru dibuat pada entitas `HabitLog` dan disimpan ke dalam box Hive `habit_logs`. Perhitungan rentetan keberhasilan dihitung secara deterministik oleh `StreakCalculator`.

Kode Program 4.8.3.1 Algoritma Perhitungan Rentetan Hari Berturut-turut (Streak)

```
import '../../../core/utils/date_only.dart';

/// Perhitungan streak dan progres habit (FR-027/028). Logika
```

```
murni.
///
/// Aturan (Tahap 1 §10.2 / Tahap 2 §16):
/// - Hanya hari target yang dihitung; hari non-target tidak
memutus streak.
/// - Satu hari target terlewat = streak kembali 0.
/// - Hari ini belum dicatat tidak memutus streak (dihitung
dari kemarin).
abstract final class StreakCalculator {
  static int current({
    required Set<DateTime> doneDates,
    required List<int> targetWeekdays,
    required DateTime today,
    DateTime? startedOn,
  }) {
    if (targetWeekdays.isEmpty) {
      return 0;
    }
    final Set<String> done =
doneDates.map(DateOnly.format).toSet();
    final DateTime start =
        startedOn == null ? DateTime(1970) :
DateOnly.of(startedOn);
    DateTime day = DateOnly.of(today);

    // Hari ini boleh belum dicatat.
    if (targetWeekdays.contains(day.weekday) &&
        !done.contains(DateOnly.format(day))) {
      day = DateOnly.addDays(day, -1);
    }

    int streak = 0;
    // Batas aman: maksimal ~3 tahun ke belakang.
    for (int i = 0; i < 1100 && !day.isBefore(start); i++) {
      if (targetWeekdays.contains(day.weekday)) {
        if (done.contains(DateOnly.format(day))) {
          streak++;
        } else {
          break;
        }
      }
      day = DateOnly.addDays(day, -1);
    }
    return streak;
  }

  static int best({
    required Set<DateTime> doneDates,
    required List<int> targetWeekdays,
    required DateTime today,
  }) {
    if (doneDates.isEmpty || targetWeekdays.isEmpty) {
      return 0;
    }
    final List<DateTime> sorted =
        doneDates.map(DateOnly.of).toSet().toList()..sort();
    final Set<String> done =
sorted.map(DateOnly.format).toSet();
    int best = 0;
    int run = 0;
```

```
    DateTime day = sorted.first;
    final DateTime end = DateOnly.of(today);
    while (!day.isAfter(end)) {
      if (targetWeekdays.contains(day.weekday)) {
        if (done.contains(DateOnly.format(day))) {
          run++;
          if (run > best) {
            best = run;
          }
        } else if (!DateOnly.isSameDay(day, end)) {
          run = 0;
        }
      }
      day = DateOnly.addDays(day, 1);
    }
    return best;
  }

  /// Persentase hari target yang tercatat dalam [days] hari
terakhir
  /// (termasuk hari ini), 0–1. Hari sebelum [startedOn] tidak
dihitung.
  static double completionRate({
    required Set<DateTime> doneDates,
    required List<int> targetWeekdays,
    required DateTime today,
    int days = 7,
    DateTime? startedOn,
  }) {
    final Set<String> done =
doneDates.map(DateOnly.format).toSet();
    final DateTime start =
        startedOn == null ? DateTime(1970) :
DateOnly.of(startedOn);
    int target = 0;
    int hit = 0;
    for (int i = 0; i < days; i++) {
      final DateTime day =
DateOnly.addDays(DateOnly.of(today), -i);
      if (day.isBefore(start) ||
!targetWeekdays.contains(day.weekday)) {
        continue;
      }
      target++;
      if (done.contains(DateOnly.format(day))) {
        hit++;
      }
    }
    return target == 0 ? 0 : hit / target;
  }
}
```

Penjelasan Source Code streak_calculator.dart

1. Identitas File

   Berkas ini berada pada lib/features/habit/domain/streak_calculator.dart dan mengimplementasikan algoritma penghitungan streak kebiasaan murni.

2. Aturan Perhitungan
    - current(...) (baris 10–43): Menghitung rentetan hari berturut-turut saat ini. Jika hari ini merupakan hari target tetapi pengguna belum mencentangnya, streak tidak langsung diputus, melainkan dihitung mundur dari hari target kemarin. Perhitungan hanya mengevaluasi hari-hari yang termasuk dalam targetWeekdays, sehingga hari non-target tidak memutus rentetan.
    - best(...) (baris 45–74): Melakukan penelusuran maju (forward traversal) dari log pencapaian pertama hingga hari ini untuk mencari rekor rentetan terpanjang (longest streak) yang pernah diraih.
    - completionRate(...) (baris 78–101): Menghitung persentase pemenuhan kebiasaan dalam rentang N hari terakhir (default 7 hari) sebagai rasio desimal $0.0 - 1.0$.
3. Alur Kerja

    Saat pengguna mencentang kebiasaan di antarmuka HabitsPage, log baru disimpan. Notifier memanggil StreakCalculator.current() dan best(), lalu hasilnya langsung di-render pada kartu kebiasaan dan kartu ringkasan Home.
4. Hubungan dengan File Lain

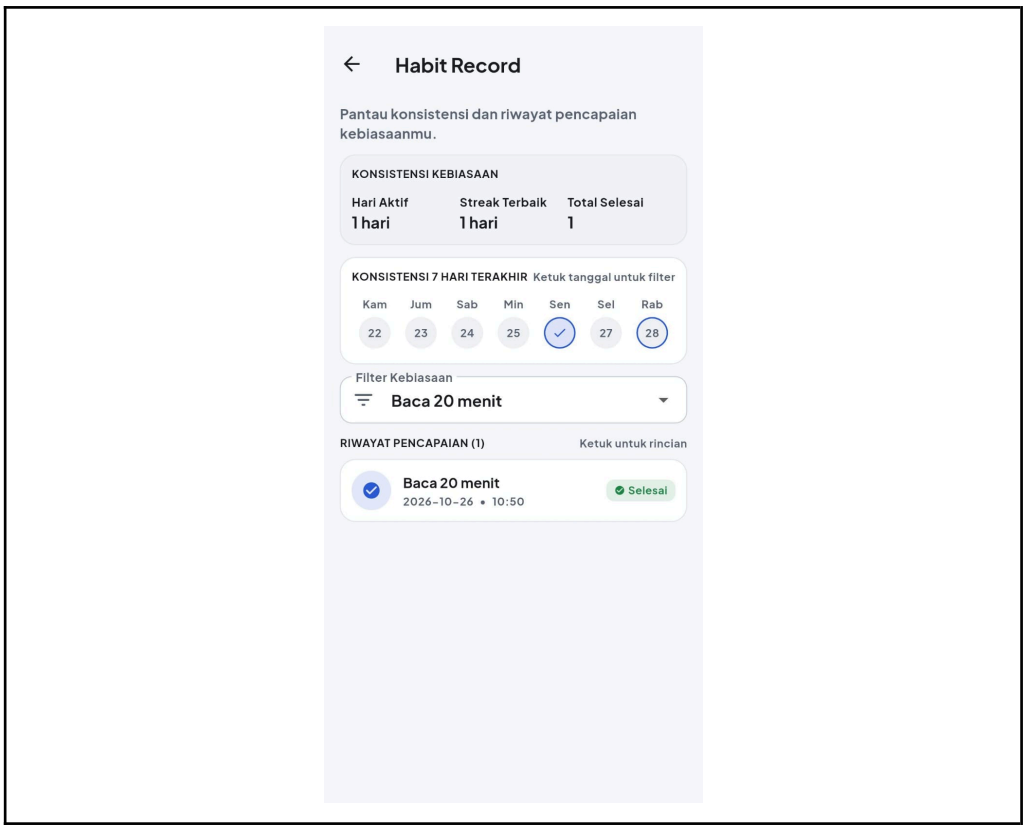
    Berkas ini digunakan oleh HabitsPage, HabitDetailPage, serta HomePage (_HabitSummary).

4.9 Implementasi Habit Consistency Record

Fitur Habit Consistency Record mengintegrasikan simulasi rantai hash kriptografis lokal (hash chain) untuk mencatat setiap pencapaian habit pengguna secara berurutan. Fitur ini berfungsi sebagai mekanisme audit internal yang membuktikan bahwa riwayat kebiasaan konsisten, terhubung, dan tidak mengalami manipulasi data lokal.

4.9.1 Tampilan Fitur Habit Consistency Record

Antarmuka Habit Consistency Record menampilkan kartu status integritas yang ramah pengguna, indikator pemeriksaan otomatis, serta daftar riwayat pencapaian kebiasaan yang disajikan secara kronologis.



Gambar 4.9.1.1 Tampilan Riwayat Konsistensi Kebiasaan

4.9.2 Implementasi Pencatatan Riwayat Kebiasaan

Setiap kali sebuah kebiasaan ditandai selesai pada Habit Tracker, sistem secara otomatis meneruskan peristiwa pencapaian tersebut ke AchievementRepository. Repositori membentuk blok baru yang merekam pengenal kebiasaan (habitId), nama kebiasaan (habitName), tanggal pencatatan (logDate), dan menghubungkannya dengan hash blok sebelumnya.

4.9.3 Implementasi Hash SHA-256

Pembangunan dan pembuktian integritas rantai dikelola secara matematis oleh kelas HashChain.

Kode Program 4.9.3.1 Rantai Hash Kriptografis SHA-256 Lokal

```
import 'dart:convert';

import 'package:crypto/crypto.dart';
import 'package:flutter/foundation.dart';

/// Satu blok pencapaian (Tahap 2 §12.4). SIMULASI hash chain lokal —
/// bukan blockchain publik, tanpa konsensus/desentralisasi.
@immutable
class Block {
  const Block({
    required this.index,
```

```
    required this.timestamp,
    required this.data,
    required this.prevHash,
    required this.hash,
  });

  final int index;

  /// ISO-8601 UTC (string apa adanya agar hash
deterministik).
  final String timestamp;
  final Map<String, dynamic> data;
  final String prevHash;
  final String hash;

  Map<String, dynamic> toJson() => <String, dynamic>{
        'index': index,
        'timestamp': timestamp,
        'data': data,
        'prev_hash': prevHash,
        'hash': hash,
      };

  static Block fromJson(Map<String, dynamic> json) => Block(
        index: (json['index'] as num).toInt(),
        timestamp: json['timestamp'] as String,
        data: Map<String, dynamic>.from(json['data'] as Map),
        prevHash: json['prev_hash'] as String,
        hash: json['hash'] as String,
      );

  Block copyWith({Map<String, dynamic>? data, String? hash})
=> Block(
        index: index,
        timestamp: timestamp,
        data: data ?? this.data,
        prevHash: prevHash,
        hash: hash ?? this.hash,
      );

  String get type => (data['type'] as String?) ?? 'unknown';

  /// Properti pendukung Habit Consistency Record
  String? get habitId => data['habitId'] as String?;
  String? get habitName => (data['habitName'] ?? data['name']
?? data['title']) as String?;
  String? get logDate => data['logDate'] as String?;
  String? get status => (data['status'] as String?) ?? (isDone
? 'completed' : null);
  String? get eventRef => data['eventRef'] as String?;
  bool get isHabit => type == 'habit_completed' || type ==
'habit_streak';
  bool get isDone => type == 'habit_completed' || type ==
'task_completed';
}

/// Hasil validasi.
@immutable
class ChainValidation {
  const ChainValidation.valid()
```

```
      : isValid = true,
        brokenAt = null,
        reason = null;

  const ChainValidation.invalid(int this.brokenAt, String
this.reason)
      : isValid = false;

  final bool isValid;
  final int? brokenAt;
  final String? reason;
}

/// Status integritas hash chain untuk tampilan antarmuka
Habit Record.
enum HabitRecordIntegrityStatus {
  notReady, // Status belum tersedia
  checking, // Sedang memeriksa
  valid, // Riwayat tercatat dengan baik
  invalid, // Riwayat perlu diperiksa
  failed, // Pemeriksaan gagal
}

/// Representasi status pemeriksaan integritas yang ramah
pengguna.
@immutable
class HabitRecordIntegrityState {
  const HabitRecordIntegrityState({
    required this.status,
    this.brokenAt,
    this.reason,
    this.errorMessage,
  });

  const HabitRecordIntegrityState.notReady()
      : status = HabitRecordIntegrityStatus.notReady,
        brokenAt = null,
        reason = null,
        errorMessage = null;

  const HabitRecordIntegrityState.checking()
      : status = HabitRecordIntegrityStatus.checking,
        brokenAt = null,
        reason = null,
        errorMessage = null;

  const HabitRecordIntegrityState.valid()
      : status = HabitRecordIntegrityStatus.valid,
        brokenAt = null,
        reason = null,
        errorMessage = null;

  const HabitRecordIntegrityState.invalid(int this.brokenAt,
String this.reason)
      : status = HabitRecordIntegrityStatus.invalid,
        errorMessage = null;

  const HabitRecordIntegrityState.failed(String
this.errorMessage)
      : status = HabitRecordIntegrityStatus.failed,
```

```
        brokenAt = null,
        reason = null;

  final HabitRecordIntegrityStatus status;
  final int? brokenAt;
  final String? reason;
  final String? errorMessage;

  bool get isValid => status ==
HabitRecordIntegrityStatus.valid;
  bool get isChecking => status ==
HabitRecordIntegrityStatus.checking;
  bool get isInvalid => status ==
HabitRecordIntegrityStatus.invalid;
  bool get isFailed => status ==
HabitRecordIntegrityStatus.failed;
  bool get isNotReady => status ==
HabitRecordIntegrityStatus.notReady;

  String get title => switch (status) {
        HabitRecordIntegrityStatus.checking => 'Sedang
memeriksa',
        HabitRecordIntegrityStatus.valid => 'Riwayat tercatat
dengan baik',
        HabitRecordIntegrityStatus.invalid => 'Riwayat perlu
diperiksa',
        HabitRecordIntegrityStatus.notReady => 'Status belum
tersedia',
        HabitRecordIntegrityStatus.failed => 'Pemeriksaan
gagal',
      };

  String get description => switch (status) {
        HabitRecordIntegrityStatus.checking =>
          'Memeriksa konsistensi dan integritas riwayat
kebiasaan...',
        HabitRecordIntegrityStatus.valid =>
          'Seluruh catatan kebiasaan utuh, berurutan, dan
konsisten.',
        HabitRecordIntegrityStatus.invalid =>
          'Ditemukan ketidaksesuaian data pada riwayat
kebiasaan. Catatan perlu ditinjau.',
        HabitRecordIntegrityStatus.notReady =>
          'Menyiapkan data riwayat kebiasaan dari
penyimpanan...',
        HabitRecordIntegrityStatus.failed =>
          errorMessage ?? 'Terjadi kendala teknis saat membaca
data riwayat.',
      };
}

abstract final class HashChain {
  static final String zeroHash = '0' * 64;

  /// JSON dengan key diurutkan secara rekursif (hash
deterministik).
  static String canonicalJson(Object? value) {
    if (value is Map) {
      final List<String> keys =
          value.keys.map((Object? k) =>
```

```
k.toString()).toList()..sort();
      return '{${keys.map((String k) =>
'${jsonEncode(k)}:${canonicalJson(value[k])}').join(',')}}';
    }
    if (value is List) {
      return '[${value.map(canonicalJson).join(',')}]';
    }
    return jsonEncode(value);
  }

  /// SHA-256 dari
`"$index|$timestamp|canonicalJson(data)|$prevHash"`.
  static String computeHash(
    int index,
    String timestamp,
    Map<String, dynamic> data,
    String prevHash,
  ) {
    final String input =
        '$index|$timestamp|${canonicalJson(data)}|$prevHash';
    return sha256.convert(utf8.encode(input)).toString();
  }

  static Block genesis(DateTime now) {
    final String ts = now.toUtc().toIso8601String();
    const Map<String, dynamic> data = <String,
dynamic>{'type': 'genesis'};
    return Block(
      index: 0,
      timestamp: ts,
      data: data,
      prevHash: zeroHash,
      hash: computeHash(0, ts, data, zeroHash),
    );
  }

  static Block next(Block prev, Map<String, dynamic> data,
DateTime now) {
    final int index = prev.index + 1;
    final String ts = now.toUtc().toIso8601String();
    return Block(
      index: index,
      timestamp: ts,
      data: data,
      prevHash: prev.hash,
      hash: computeHash(index, ts, data, prev.hash),
    );
  }

  /// Hash hitung ulang dari isi blok (untuk ditampilkan saat
demo).
  static String recompute(Block b) =>
      computeHash(b.index, b.timestamp, b.data, b.prevHash);

  /// Validasi: hash cocok dengan isi, `prevHash` menunjuk
blok sebelumnya,
  /// dan indeks berurutan.
  static ChainValidation validate(List<Block> chain) {
    for (int i = 0; i < chain.length; i++) {
      final Block b = chain[i];
```

```
      if (b.index != i) {
        return ChainValidation.invalid(i, 'Indeks tidak
berurutan.');
      }
      if (recompute(b) != b.hash) {
        return ChainValidation.invalid(
          i,
          'Hash tidak cocok setelah data diubah.',
        );
      }
      final String expectedPrev = i == 0 ? zeroHash : chain[i
- 1].hash;
      if (b.prevHash != expectedPrev) {
        return ChainValidation.invalid(
          i,
          'prevHash tidak menunjuk hash blok sebelumnya.',
        );
      }
    }
    return const ChainValidation.valid();
  }
}
```

Penjelasan Source Code hash_chain.dart

1. Identitas File

   Berkas ini berada pada lib/features/hash_chain/domain/hash_chain.dart dan mengimplementasikan struktur data blok dan algoritma verifikasi integritas rantai hash kriptografis SHA-256.

2. Kelas Block
   - Merepresentasikan satu entri blok pencapaian yang memuat nomor urut index, stempel waktu ISO-8601 UTC timestamp, muatan data (seperti habitId, habitName, logDate), hash blok sebelumnya prevHash, dan intisari hash blok ini hash.

3. Fungsi Kriptografis HashChain
   - canonicalJson (baris 161–171): Mengurutkan kunci map secara alfabetis sebelum diserialisasi menjadi string JSON. Langkah ini menjamin bahwa representasi string bersifat deterministik di semua platform.
   - computeHash (baris 174–183): Menggabungkan string '$index|$timestamp|${canonicalJson(data)}|$prevHash' dan menghitung intisari SHA-256 menggunakan pustaka crypto.

- genesis (baris 185–195): Membangun blok pertama (indeks 0) dengan prevHash berupa 64 karakter angka nol (zeroHash).
- next (baris 197–207): Menghubungkan blok baru ke blok sebelumnya dengan menyalin hash sebelumnya sebagai prevHash.
- validate (baris 215–236): Memverifikasi seluruh rantai dari awal hingga akhir, memastikan nomor indeks berurutan, isi data menghasilkan hash yang sama dengan yang tercatat (recompute(b) == b.hash), serta prevHash mengarah persis pada blok sebelumnya.

4. Alur Kerja

   Setiap kali habit diselesaikan, AchievementRepository memanggil HashChain.next() dan menyimpannya ke box Hive achievements. Ketika HashChainPage dibuka, fungsi validate() dieksekusi secara otomatis untuk memeriksa integritas data riwayat.

5. Hubungan dengan File Lain

   Berkas ini digunakan oleh AchievementRepository dan HashChainPage.

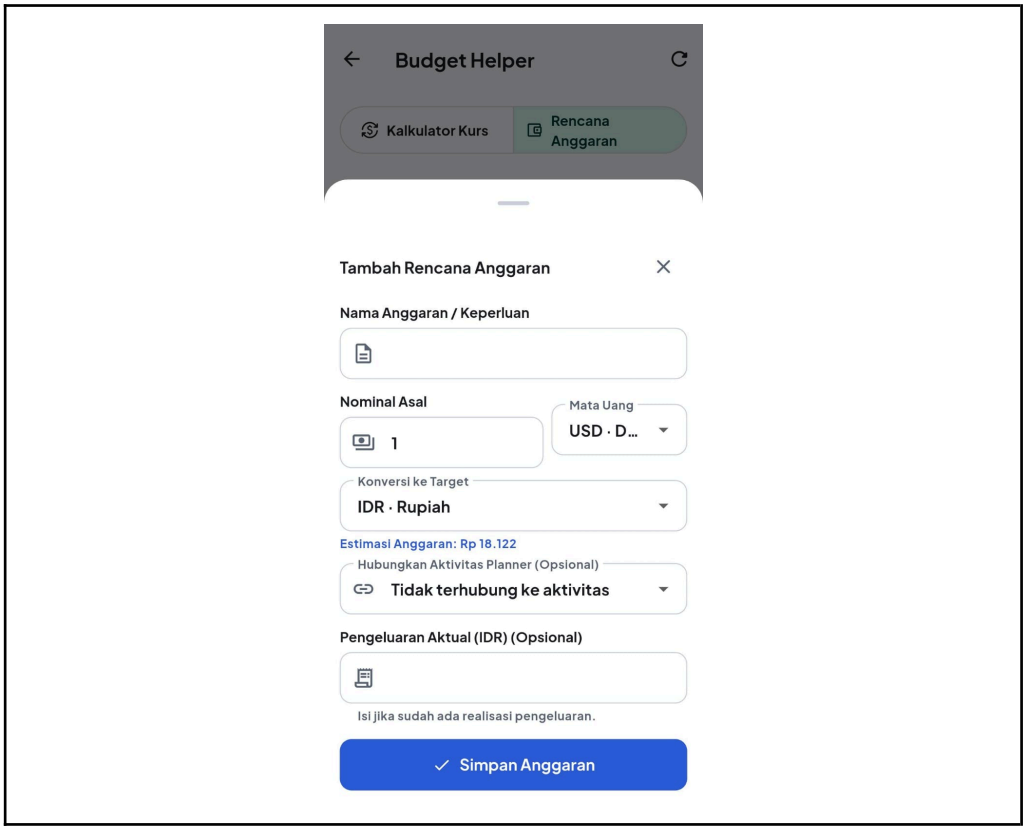
4.10 Fitur Budget Helper

Fitur Budget Helper pada aplikasi Planify merupakan modul pengelolaan estimasi anggaran operasional dan biaya aktivitas yang dirancang untuk mendukung perencanaan kegiatan pengguna. Berbeda dengan aplikasi pencatat keuangan murni, Budget Helper secara khusus dihubungkan dengan agenda kegiatan pada Planner, sehingga pengguna dapat mengalokasikan perkiraan biaya (seperti biaya registrasi seminar, akomodasi perjalanan dinas, pembelian bahan studi, maupun konsumsi rapat) secara terukur. Fitur ini dilengkapi dengan sistem konversi mata uang multi-kurs terintegrasi yang mendukung 4 mata uang utama, yaitu Rupiah (IDR), Dolar Amerika Serikat (USD), Ringgit Malaysia (MYR), dan Dolar Singapura (SGD), serta didukung mekanisme penyimpanan cache nilai tukar untuk penggunaan secara luring (offline-first).

4.10.1 Tampilan Antarmuka dan Alur Penggunaan

Antarmuka Budget Helper dirancang mengutamakan kejelasan alokasi dana dan kemudahan konversi nilai mata uang. Pada bagian atas halaman,

pengguna disajikan kalkulator konversi cepat yang menampilkan pilihan mata uang asal (from), mata uang target (to), kotak input nominal dengan penanganan format desimal lokal, tombol pertukaran mata uang (swap), serta status pembaruan kurs terkini. Di bawah panel konversi, terdapat daftar kartu anggaran (budget items) yang mencantumkan nama aktivitas, keterkaitan dengan tugas di Planner, target batas anggaran, realisasi pengeluaran aktual (actual spent), serta indikator persentase pemakaian anggaran (budget consumption progress bar). Pengguna dapat menambah, menyunting, maupun menghapus pos anggaran dengan umpan balik visual instan.



Gambar 4.10.1.1 Tampilan Halaman Budget Helper

4.10.2 Konversi Mata Uang dan Integrasi Nilai Tukar

Logika domain konversi nilai tukar, snapshot kurs, pemformatan angka moneter lokal, serta validasi ketikan pengguna dienkapsulasi secara terpusat pada berkas currency.dart.

Kode Program 4.10.2.1 Model Mata Uang dan Konversi

```
import 'package:flutter/foundation.dart';
import 'package:intl/intl.dart';

/// Mata uang yang didukung (FR-039).
enum Currency {
```

```
  idr('IDR', 'Rupiah', 'Rp', 0),
  usd('USD', 'Dolar AS', r'US$', 2),
  myr('MYR', 'Ringgit Malaysia', 'RM', 2),
  sgd('SGD', 'Dolar Singapura', r'S$', 2);

  const Currency(this.code, this.name, this.symbol,
this.decimals);

  final String code;
  final String name;
  final String symbol;

  /// Desimal saat ditampilkan (IDR 0, lainnya 2) — `[PERLU
DIKONFIRMASI]`.
  final int decimals;

  static Currency? parse(String code) {
    for (final Currency c in Currency.values) {
      if (c.code == code.toUpperCase()) {
        return c;
      }
    }
    return null;
  }
}

/// Snapshot kurs (Tahap 2 §11.3). `rates[x]` = berapa x untuk
1 [base].
@immutable
class RateSnapshot {
  const RateSnapshot({
    required this.base,
    required this.rates,
    required this.fetchedAt,
    required this.source,
    this.providerDate,
  });

  final Currency base;
  final Map<Currency, double> rates;

  /// Saat aplikasi mengambil data (UTC).
  final DateTime fetchedAt;

  /// Tanggal kurs menurut penyedia (mis. kurs referensi
harian).
  final String? providerDate;
  final String source;

  double? rateOf(Currency c) => c == base ? 1 : rates[c];

  Map<String, dynamic> toJson() => <String, dynamic>{
        'base': base.code,
        'rates': <String, double>{
          for (final MapEntry<Currency, double> e in
rates.entries)
            e.key.code: e.value,
        },
        'fetched_at': fetchedAt.toUtc().toIso8601String(),
        'provider_date': providerDate,
```

```
        'source': source,
      };

  static RateSnapshot? fromJson(Map<String, dynamic>? json) {
    if (json == null) {
      return null;
    }
    final Currency? base = Currency.parse(json['base'] as
String? ?? '');
    final Object? raw = json['rates'];
    final DateTime? fetched =
        DateTime.tryParse(json['fetched_at'] as String? ??
'');
    if (base == null || raw is! Map || fetched == null) {
      return null;
    }
    final Map<Currency, double> rates = <Currency, double>{};
    raw.forEach((Object? k, Object? v) {
      final Currency? c = Currency.parse(k.toString());
      if (c != null && v is num) {
        rates[c] = v.toDouble();
      }
    });
    return RateSnapshot(
      base: base,
      rates: rates,
      fetchedAt: fetched,
      providerDate: json['provider_date'] as String?,
      source: json['source'] as String? ?? '-',
    );
  }
}

abstract final class CurrencyConverter {
  static const double maxAmount = 1e12;

  /// Konversi silang via base: `amount × rate[to] /
rate[from]`.
  /// `null` bila kurs salah satu mata uang tidak tersedia.
  static double? convert(
    double amount,
    Currency from,
    Currency to,
    RateSnapshot snapshot,
  ) {
    final double? rf = snapshot.rateOf(from);
    final double? rt = snapshot.rateOf(to);
    if (rf == null || rt == null || rf == 0) {
      return null;
    }
    return amount * rt / rf;
  }

  /// Mengurai nominal ketikan pengguna. Menerima format
Indonesia
  /// (`1.250.000,50`) maupun titik desimal (`1250000.50`).
  static double? parseAmount(String input) {
    String s = input.trim().replaceAll(' ', '');
    if (s.isEmpty) {
      return null;
```

```
    }
    if (!RegExp(r'^[0-9.,]+$').hasMatch(s)) {
      return null;
    }
    final int lastComma = s.lastIndexOf(',');
    final int lastDot = s.lastIndexOf('.');
    if (lastComma >= 0 && lastDot >= 0) {
      // Pemisah yang muncul terakhir = desimal.
      if (lastComma > lastDot) {
        s = s.replaceAll('.', '').replaceAll(',', '.');
      } else {
        s = s.replaceAll(',', '');
      }
    } else if (lastComma >= 0) {
      // Hanya koma: desimal Indonesia, kecuali pola ribuan
(1,000,000).
      s = RegExp(r'^\d{1,3}(,\d{3})+$').hasMatch(s)
          ? s.replaceAll(',', '')
          : s.replaceAll(',', '.');
    } else if (lastDot >= 0) {
      // Hanya titik: ribuan Indonesia bila pola 1.000 /
1.000.000.
      if (RegExp(r'^\d{1,3}(\.\d{3})+$').hasMatch(s)) {
        s = s.replaceAll('.', '');
      }
    }
    return double.tryParse(s);
  }

  /// Pesan validasi nominal atau `null` bila valid.
  static String? validateAmount(String input) {
    if (input.trim().isEmpty) {
      return 'Nominal wajib diisi.';
    }
    final double? v = parseAmount(input);
    if (v == null) {
      return 'Nominal hanya boleh berisi angka.';
    }
    if (v <= 0) {
      return 'Nominal harus lebih dari 0.';
    }
    if (v > maxAmount) {
      return 'Nominal terlalu besar.';
    }
    return null;
  }

  static String format(double value, Currency c) {
    final NumberFormat f = NumberFormat.currency(
      locale: 'id_ID',
      symbol: '${c.symbol} ',
      decimalDigits: c.decimals,
    );
    return f.format(value);
  }

  static String formatRate(double rate) {
    final int digits = rate >= 100 ? 2 : 4;
    return NumberFormat.decimalPatternDigits(
      locale: 'id_ID',
```

```
      decimalDigits: digits,
    ).format(rate);
  }
}
```

Penjelasan Source Code currency.dart

1. Identitas File

   Berkas ini berada pada lib/features/currency/domain/currency.dart dan mendefinisikan tipe data mata uang yang didukung, struktur penyimpanan snapshot kurs, dan pustaka fungsi murni untuk kalkulasi konversi silang antar mata uang.

2. Enum Currency` (baris 5–28)
   - Mendefinisikan empat mata uang: `idr` (Rupiah, simbol 'Rp', 0 desimal), `usd` (Dolar AS, simbol 'US$', 2 desimal), `myr` (Ringgit Malaysia, simbol 'RM', 2 desimal), dan `sgd` (Dolar Singapura, simbol 'S$', 2 desimal).
   - Menyediakan metode statis `parse` untuk memetakan representasi kode string ISO-4217 menjadi instance enum terkait secara aman.
3. Kelas RateSnapshot (baris 32–90)
   - Struktur data immutable yang menyimpan kurs dasar (base), peta nilai tukar (rates), stempel waktu pengambilan data (fetchedAt), serta informasi penyedia kurs (source dan providerDate).
   - Menyediakan metode rateOf(Currency c) yang mengembalikan nilai 1.0 jika mata uang yang diminta sama dengan mata uang dasar, atau mengambil nilai rasio dari map rates.
   - Dilengkapi serialisasi JSON dua arah (toJson dan fromJson) untuk memfasilitasi persistensi nilai kurs ke dalam penyimpanan lokal Hive box currency_cache.
4. Kelas Abstrak CurrencyConverter (baris 92–178)
   - convert (baris 97–109): Mengimplementasikan rumus konversi silang via mata uang dasar: $$\text{Hasil} = \text{Nominal} \times \frac{\text{Kurs Target}}{\text{Kurs Asal}}$$
   - parseAmount (baris 113–142): Algoritma pengurai string angka cerdas yang mampu mengenali format angka baku Indonesia (pemisah ribuan titik dan desimal koma, misal 1.250.000,50)

maupun format baku internasional (pemisah ribuan koma dan desimal titik).

- validateAmount (baris 145–160): Memastikan input nominal tidak kosong, berupa angka positif lebih dari nol, dan tidak melebihi batas rasional 10^{12} (maxAmount).
- format dan formatRate (baris 162–177): Memformat nominal angka dan nilai tukar menggunakan pustaka `intl` dengan lokal id_ID.

5. Alur Kerja

   Saat halaman Budget Helper dibuka, sistem memuat RateSnapshot dari box Hive currency_cache atau melakukan permintaan HTTP ke API Frankfurter. Setiap kali pengguna mengetik nominal atau mengubah pilihan mata uang, CurrencyConverter.parseAmount() dan CurrencyConverter.convert() dieksekusi secara reaktif untuk menyajikan hasil konversi instan.

6. Hubungan dengan File Lain

   Berkas ini digunakan oleh CurrencyRepository untuk caching kurs, BudgetHelperPage untuk rendering UI kalkulator konversi, dan BudgetItem untuk kalkulasi nilai konversi anggaran aktivitas.

4.10.3 Manajemen Anggaran Aktivitas dan Kategori Pengeluaran

Untuk merepresentasikan setiap alokasi dana kegiatan, aplikasi Planify menggunakan model data `BudgetItem` yang terintegrasi secara modular dengan tugas Planner.

Kode Program 4.10.3.1 Model Item Anggaran Aktivitas

```
import 'package:flutter/foundation.dart';
import 'currency.dart';

/// Model anggaran aktivitas untuk Budget Helper.
@immutable
class BudgetItem {
  const BudgetItem({
    required this.id,
    required this.title,
    required this.amount,
    required this.currency,
    required this.targetCurrency,
    required this.convertedAmount,
    this.taskId,
    this.taskTitle,
    this.actualSpent = 0.0,
```

```
    required this.createdAt,
    this.updatedAt,
  });

  final String id;
  final String title;
  final double amount;
  final Currency currency;
  final Currency targetCurrency;
  final double convertedAmount;

  /// ID tugas terkait di Planner (opsional).
  final String? taskId;

  /// Judul tugas terkait di Planner (opsional, untuk tampilan
cepat).
  final String? taskTitle;

  /// Pengeluaran aktual yang dicatat pengguna (dalam
targetCurrency).
  final double actualSpent;

  final DateTime createdAt;
  final DateTime? updatedAt;

  /// Selisih anggaran dengan realisasi: positif = hemat,
negatif = over-budget.
  double get remainingBudget => convertedAmount - actualSpent;

  /// Persentase pemakaian anggaran (0.0 – 1.0+).
  double get spentRatio =>
      convertedAmount > 0 ? (actualSpent / convertedAmount) :
0.0;

  bool get hasLinkedTask => taskId != null &&
taskId!.isNotEmpty;

  BudgetItem copyWith({
    String? id,
    String? title,
    double? amount,
    Currency? currency,
    Currency? targetCurrency,
    double? convertedAmount,
    String? taskId,
    String? taskTitle,
    bool clearTask = false,
    double? actualSpent,
    DateTime? createdAt,
    DateTime? updatedAt,
  }) {
    return BudgetItem(
      id: id ?? this.id,
      title: title ?? this.title,
      amount: amount ?? this.amount,
      currency: currency ?? this.currency,
      targetCurrency: targetCurrency ?? this.targetCurrency,
      convertedAmount: convertedAmount ??
this.convertedAmount,
      taskId: clearTask ? null : (taskId ?? this.taskId),
```

```
      taskTitle: clearTask ? null : (taskTitle ??
this.taskTitle),
      actualSpent: actualSpent ?? this.actualSpent,
      createdAt: createdAt ?? this.createdAt,
      updatedAt: updatedAt ?? this.updatedAt,
    );
  }

  Map<String, dynamic> toJson() => <String, dynamic>{
        'id': id,
        'title': title,
        'amount': amount,
        'currency': currency.code,
        'target_currency': targetCurrency.code,
        'converted_amount': convertedAmount,
        'task_id': taskId,
        'task_title': taskTitle,
        'actual_spent': actualSpent,
        'created_at': createdAt.toUtc().toIso8601String(),
        'updated_at': updatedAt?.toUtc().toIso8601String(),
      };

  static BudgetItem? fromJson(Map<String, dynamic>? json) {
    if (json == null) return null;
    final String? id = json['id'] as String?;
    final String? title = json['title'] as String?;
    final num? amount = json['amount'] as num?;
    final Currency? currency =
        Currency.parse(json['currency'] as String? ?? '');
    final Currency? targetCurrency =
        Currency.parse(json['target_currency'] as String? ??
'');
    final num? convertedAmount = json['converted_amount'] as
num?;
    final DateTime? createdAt =
        DateTime.tryParse(json['created_at'] as String? ??
'');

    if (id == null ||
        title == null ||
        amount == null ||
        currency == null ||
        targetCurrency == null ||
        convertedAmount == null ||
        createdAt == null) {
      return null;
    }

    return BudgetItem(
      id: id,
      title: title,
      amount: amount.toDouble(),
      currency: currency,
      targetCurrency: targetCurrency,
      convertedAmount: convertedAmount.toDouble(),
      taskId: json['task_id'] as String?,
      taskTitle: json['task_title'] as String?,
      actualSpent: (json['actual_spent'] as num?)?.toDouble()
?? 0.0,
      createdAt: createdAt.toLocal(),
```

```
      updatedAt: DateTime.tryParse(json['updated_at'] as
String? ?? '')?.toLocal(),
    );
  }
}
```

Penjelasan Source Code budget_item.dart

1. Identitas File

   Berkas ini terletak pada lib/features/currency/domain/budget_item.dart dan mendefinisikan entitas data BudgetItem untuk mencatat alokasi biaya kegiatan, mata uang acuan, nilai konversi, realisasi pemakaian dana, dan relasi opsional terhadap tugas di Planner.

2. Atribut Entitas (baris 21–38)
   - id: Pengenal unik item anggaran berbasis UUID v4.
   - title: Judul atau pos alokasi anggaran (misalnya "Tiket Pesawat Konferensi" atau "Biaya Modul Belajar").
   - amount & currency: Nilai nominal awal dan mata uang yang digunakan pada transaksi asal.
   - targetCurrency & convertedAmount: Mata uang dasar pengguna (umumnya IDR) dan hasil konversi nilai nominal berdasarkan kurs terkini.
   - taskId & taskTitle: Kunci relasi opsional terhadap tugas yang terdaftar pada modul Planner.
   - actualSpent: Total realisasi dana yang telah dikeluarkan secara faktual oleh pengguna.
3. Getter Komputasi Finansial (baris 40–47)
   - remainingBudget: Menghitung sisa anggaran yang belum terpakai (convertedAmount - actualSpent). Nilai negatif menandakan kondisi over-budget.
   - spentRatio: Menghitung rasio penyerapan anggaran dalam rentang 0.0 hingga ≥ 1.0 untuk merender indikator progres batang (progress bar) dengan warna adaptif (hijau untuk aman, oranye untuk mendekati batas, dan merah saat melampaui anggaran).
4. Alur Kerja

Saat pengguna membuat item anggaran baru di halaman Budget Helper, sistem membuat instance BudgetItem baru, menghitung `convertedAmount` menggunakan kurs terkini, dan menyimpannya ke penyimpanan lokal Hive box preferences. Saat pengguna memasukkan pengeluaran aktual, actualSpent diperbarui menggunakan metode copyWith dan disimpan kembali.

5. Hubungan dengan File Lain

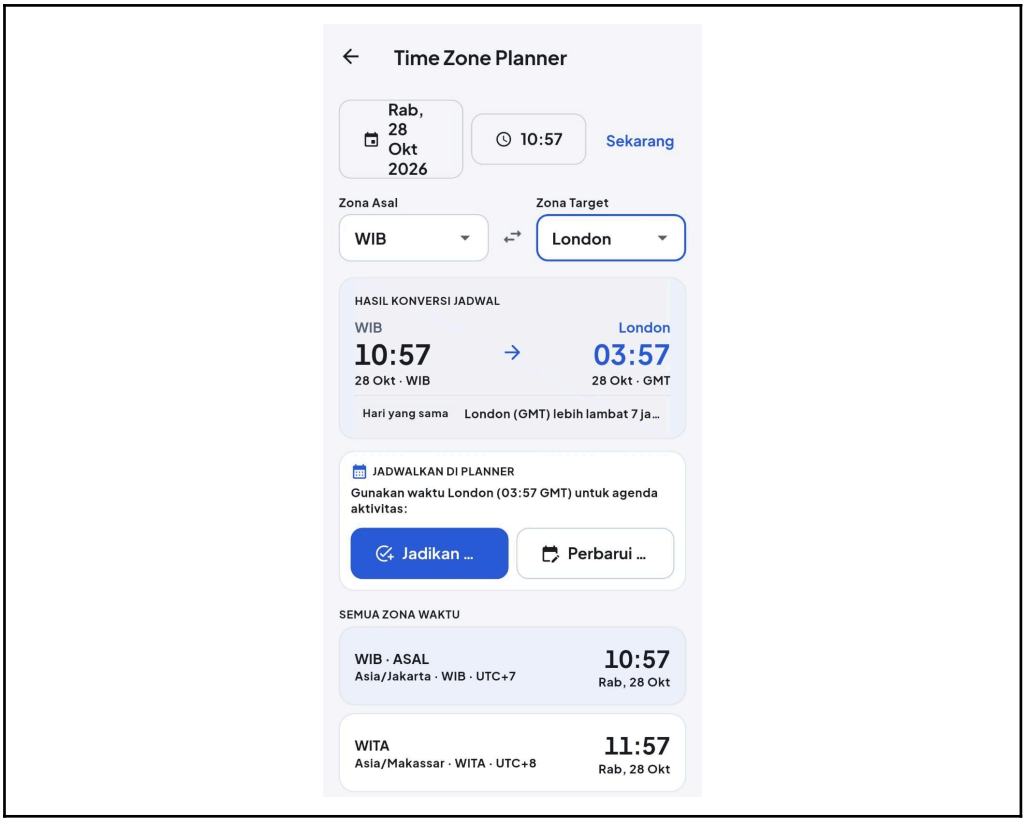
   Berkas ini mengimpor currency.dart untuk manipulasi enum Currency, digunakan oleh BudgetRepository untuk operasi CRUD lokal, dan berinteraksi secara relasional dengan model Task pada modul Planner.

4.11 Fitur Time Zone Planner

Fitur Time Zone Planner pada aplikasi Planify dirancang untuk membantu pengguna mengelola jadwal kegiatan dan kolaborasi lintas zona waktu secara akurat. Modul ini mendukung konversi waktu antara tiga zona waktu utama Indonesia, yaitu Waktu Indonesia Barat (WIB / Asia/Jakarta, UTC+7), Waktu Indonesia Tengah (WITA / Asia/Makassar, UTC+8), dan Waktu Indonesia Timur (WIT / Asia/Jayapura, UTC+9), serta zona waktu internasional London (Europe/London, UTC+0 / UTC+1). Fitur ini secara khusus menangani fenomena pergantian waktu musiman (Daylight Saving Time / DST) dan terintegrasi langsung dengan agenda tugas pada Planner.

4.11.1 Tampilan Antarmuka dan Alur Penggunaan

Halaman Time Zone Planner menyajikan antarmuka visual yang intuitif untuk simulasi dan penjadwalan waktu lintas wilayah. Pengguna dapat memilih tanggal dan jam acuan menggunakan time picker, memilih zona waktu asal pertemuan, serta mengaitkan sesi tersebut dengan salah satu tugas yang ada di Planner melalui menu dropdown. Antarmuka secara simultan menampilkan empat kartu waktu yang merepresentasikan jam dinding di wilayah WIB, WITA, WIT, dan London. Setiap kartu menampilkan singkatan zona (misal WIB, WITA, WIT, GMT, atau BST), offset UTC terformat (misal UTC+7 atau UTC+1), waktu lokal lengkap dengan indikator pergeseran hari (day shift indicator, seperti Hari yang sama, +1 hari besok, atau −1 hari kemarin), serta lencana peringatan (warning badge) apabila waktu yang dipilih bertepatan dengan jam peralihan DST di wilayah London.



Gambar 4.11.1.1 Tampilan Halaman Time Zone Planner

4.11.2 Konversi Waktu Antar Zona

Seluruh kalkulasi penyesuaian waktu, offset zona waktu IANA, deteksi pergeseran tanggal, dan validasi jam ambigu akibat DST diimplementasikan pada berkas domain murni tz_converter.dart.

Kode Program 4.11.2.1 Logika Konversi Zona Waktu IANA

```
import 'package:flutter/foundation.dart';
import 'package:timezone/timezone.dart' as tz;

/// Zona yang didukung (Tahap 2 §11.5).
enum AppZone {
  wib('WIB', 'Asia/Jakarta'),
  wita('WITA', 'Asia/Makassar'),
  wit('WIT', 'Asia/Jayapura'),
  london('London', 'Europe/London');

  const AppZone(this.label, this.iana);

  final String label;
  final String iana;
}

enum DstIssue {
  none,

  /// Jam tidak ada (London memajukan jam pada akhir Maret).
  nonexistent,
```

```
  /// Jam terjadi dua kali (London memundurkan jam pada akhir
Oktober).
  ambiguous,
}

@immutable
class ZoneTime {
  const ZoneTime({
    required this.zone,
    required this.local,
    required this.abbreviation,
    required this.offset,
    required this.dayShift,
  });

  final AppZone zone;

  /// Komponen tanggal/jam di zona ini.
  final DateTime local;

  /// WIB/WITA/WIT, atau GMT/BST untuk London.
  final String abbreviation;
  final Duration offset;

  /// −1 / 0 / +1 dibanding tanggal asal.
  final int dayShift;

  String get offsetLabel {
    final int minutes = offset.inMinutes;
    final String sign = minutes < 0 ? '-' : '+';
    final int h = minutes.abs() ~/ 60;
    final int m = minutes.abs() % 60;
    return 'UTC$sign$h${m == 0 ? '' :
':${m.toString().padLeft(2, '0')}'}';
  }
}

@immutable
class TzResult {
  const TzResult({
    required this.utc,
    required this.times,
    required this.issue,
  });

  final DateTime utc;
  final List<ZoneTime> times;
  final DstIssue issue;
}

/// Konversi zona waktu murni berbasis basis data IANA
(`timezone`).
/// Bekerja offline. Pemanggil wajib
`tz.initializeTimeZones()` lebih dulu.
abstract final class TzConverter {
  static tz.Location location(AppZone z) =>
tz.getLocation(z.iana);

  static String _abbr(AppZone z, tz.TZDateTime t) {
    if (z != AppZone.london) {
```

```
      return z.label;
    }
    return t.timeZone.isDst ? 'BST' : 'GMT';
  }

  /// Mendeteksi jam yang tidak ada/ambigu di zona [from]
(relevan untuk
  /// London). Round-trip check + perbandingan offset ±1 jam
(Tahap 2 §11.5).
  static DstIssue checkDst(AppZone from, DateTime wall) {
    final tz.Location loc = location(from);
    final tz.TZDateTime t = tz.TZDateTime(
      loc,
      wall.year,
      wall.month,
      wall.day,
      wall.hour,
      wall.minute,
    );
    if (t.hour != wall.hour || t.minute != wall.minute ||
t.day != wall.day) {
      return DstIssue.nonexistent;
    }
    final Duration before =
tz.TZDateTime.fromMillisecondsSinceEpoch(
      loc,
      t.millisecondsSinceEpoch - const Duration(hours:
1).inMilliseconds,
    ).timeZoneOffset;
    final Duration after =
tz.TZDateTime.fromMillisecondsSinceEpoch(
      loc,
      t.millisecondsSinceEpoch + const Duration(hours:
1).inMilliseconds,
    ).timeZoneOffset;
    if (before != after) {
      // Offset berubah dalam ±1 jam. Jam ambigu bila komponen
yang sama
      // muncul lagi satu jam kemudian dengan offset lain.
      final tz.TZDateTime later =
tz.TZDateTime.fromMillisecondsSinceEpoch(
        loc,
        t.millisecondsSinceEpoch + const Duration(hours:
1).inMilliseconds,
      );
      final tz.TZDateTime earlier =
tz.TZDateTime.fromMillisecondsSinceEpoch(
        loc,
        t.millisecondsSinceEpoch - const Duration(hours:
1).inMilliseconds,
      );
      if ((later.hour == t.hour && later.minute == t.minute)
||
          (earlier.hour == t.hour && earlier.minute ==
t.minute)) {
        return DstIssue.ambiguous;
      }
    }
    return DstIssue.none;
  }
```

```
  /// Mengonversi [wall] (tanggal+jam dinding di zona [from])
ke semua zona.
  static TzResult convert(AppZone from, DateTime wall) {
    final tz.TZDateTime source = tz.TZDateTime(
      location(from),
      wall.year,
      wall.month,
      wall.day,
      wall.hour,
      wall.minute,
    );
    final DateTime utc = source.toUtc();
    final DateTime originDate = DateTime.utc(wall.year,
wall.month, wall.day);
    final List<ZoneTime> times = <ZoneTime>[
      for (final AppZone z in AppZone.values)
        () {
          final tz.TZDateTime t = tz.TZDateTime.from(utc,
location(z));
          final DateTime date = DateTime.utc(t.year, t.month,
t.day);
          return ZoneTime(
            zone: z,
            local: DateTime(t.year, t.month, t.day, t.hour,
t.minute),
            abbreviation: _abbr(z, t),
            offset: t.timeZoneOffset,
            dayShift: date.difference(originDate).inDays,
          );
        }(),
    ];
    return TzResult(utc: utc, times: times, issue:
checkDst(from, wall));
  }

  /// Zona aplikasi yang paling cocok dengan nama IANA
perangkat.
  static AppZone? fromIana(String name) {
    for (final AppZone z in AppZone.values) {
      if (z.iana == name) {
        return z;
      }
    }
    return null;
  }
}
```

Penjelasan Source Code `tz_converter.dart`

1. Identitas File

   Berkas ini berada pada lib/features/timezone_converter/domain/tz_converter.dart dan mengimplementasikan model domain zona waktu, struktur hasil kalkulasi, serta pustaka fungsi konversi waktu berbasis pangkalan data IANA (tz database).

2. Enum AppZone dan DstIssue (baris 5–25)
    - AppZone: Mendefinisikan empat zona target yaitu wib (Asia/Jakarta), wita (Asia/Makassar), wit (Asia/Jayapura), dan london (Europe/London).
    - DstIssue: Menandai status keabnormalan jam dinding akibat perpindahan DST, mencakup none (normal), nonexistent (jam hilang saat musim semi ketika jam dimajukan), dan ambiguous (jam berulang dua kali saat musim gugur ketika jam dimundurkan).
3. Kelas ZoneTime dan TzResult (baris 28–69)
    - ZoneTime: Menyimpan detail waktu pada satu zona tertentu, mencakup jam dinding lokal, singkatan zona (misal WIB atau BST), durasi offset dari UTC, serta dayShift (selisih hari terhadap tanggal asal: -1, 0, atau $+1$). Properti offsetLabel memformat durasi menjadi representasi terbaca manusia (misal UTC+7 atau UTC−5).
    - TzResult: Membungkus waktu ekuivalen UTC universal, daftar waktu di seluruh zona yang didukung (`List<ZoneTime>`), serta status anomali DST (`DstIssue`).
4. Metode Inti `TzConverter` (baris 73–163)
    - checkDst (baris 85–123): Melakukan pengujian round-trip dan evaluasi batas offset ± 1 jam untuk mendeteksi apakah jam dinding yang dimasukkan oleh pengguna berada pada masa transisi DST zona London.
    - convert (baris 126–152): Menerima jam dinding asal dan zona asal, memproyeksikannya menjadi titik waktu absolut UTC menggunakan tz.TZDateTime, lalu secara deterministik memetakan waktu UTC tersebut ke semua zona yang didukung di AppZone.values.
    - fromIana (baris 155–162): Memetakan pengenal zona waktu bawaan sistem operasi perangkat (misal Asia/Jakarta) ke enum aplikasi secara otomatis.
5. Alur Kerja

Sebelum konversi dapat dijalankan, basis data zona waktu diinisialisasi secara luring melalui pemanggilan tz.initializeTimeZones(). Saat pengguna memutar jam pada antarmuka, `zConverter.convert()` langsung menghitung ulang hasil konversi untuk seluruh zona target dan mengecek keberadaan anomali DST, kemudian merender ulang tampilan secara instan tanpa memerlukan koneksi jaringan internet.

6. Hubungan dengan File Lain

   Berkas ini memanfaatkan pustaka timezone, dipanggil oleh TimezonePlannerController pada lapisan aplikasi, dan terhubung dengan antarmuka TimezonePlannerPage.

4.11.3 Penanganan Daylight Saving Time (DST) dan Integrasi dengan Agenda

Kawasan London menerapkan aturan British Summer Time (BST, UTC+1) yang dimulai pada hari Minggu terakhir bulan Maret (jam dimajukan dari 01:00 GMT menjadi 02:00 BST) dan berakhir pada hari Minggu terakhir bulan Oktober (jam dimundurkan dari 02:00 BST menjadi 01:00 GMT). Fenomena ini menimbulkan dua tantangan matematis dalam komputasi waktu:

1. Nonexistent Time (Waktu yang Tidak Ada)

   Pada hari Minggu terakhir bulan Maret, rentang waktu pukul 01:00 hingga 01:59 tidak pernah ada karena jam melompat langsung ke pukul 02:00. Pustaka tz_converter.dart mendeteksi ketidaksesuaian komponen jam (t.hour != wall.hour) dan memberikan status DstIssue.nonexistent. UI merespons dengan menampilkan peringatan bahwa waktu tersebut tidak valid di zona London dan menyarankan pengguna memilih jam berikutnya.

2. Ambiguous Time (Waktu yang Ambigu/Berulang)

   Pada hari Minggu terakhir bulan Oktober, rentang waktu pukul 01:00 hingga 01:59 terjadi sebanyak dua kali (pertama saat BST UTC+1, kedua saat GMT UTC+0). Algoritma memeriksa sampel waktu satu jam sebelum dan sesudahnya; jika jam yang sama ditemukan pada dua offset yang berbeda, status ditetapkan menjadi DstIssue.ambiguous sehingga pengguna diberikan klarifikasi pilihan offset.

Terkait integrasi agenda, pengguna dapat menautkan jadwal pertemuan lintas zona dengan agenda Planner. Ketika waktu pertemuan disepakati (misal pukul 14:00 London), sistem menyimpan waktu universal UTC (2026-03-15T14:00:00Z) ke dalam basis data tugas Planner, sehingga pengingat lokal di perangkat pengguna Indonesia akan berbunyi tepat pada pukul 21:00 WIB tanpa risiko kesalahan akibat perbedaan offset regional.

4.12 Fitur Notifikasi dan Pengingat

Fitur notifikasi lokal pada aplikasi Planify memegang peranan krusial dalam menjaga kepatuhan pengguna terhadap rencana kerja yang telah disusun. Sistem notifikasi lokal dirancang untuk beroperasi secara mandiri tanpa memerlukan server push notification eksternal (Firebase Cloud Messaging), sehingga privasi jadwal pengguna terlindungi dan pengingat tetap aktif saat perangkat tidak terhubung ke jaringan internet (offline reliability).

4.12.1 Konfigurasi dan Inisialisasi Notifikasi Lokal

Pengelolaan notifikasi lokal diimplementasikan menggunakan pustaka flutter_local_notifications versi 22.x yang dibungkus dalam abstraksi kelas NotificationService pada berkas notification_service.dart.

Kode Program 4.12.1.1 Layanan Notifikasi Lokal

```
import 'dart:async';

import 'package:flutter/foundation.dart';
import
'package:flutter_local_notifications/flutter_local_notificatio
ns.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:timezone/timezone.dart' as tz;

/// Kanal notifikasi Android.
enum NotificationChannel {
  reminders('planify_reminders', 'Pengingat tugas',
      'Pengingat sebelum tenggat tugas'),
  focus('planify_focus', 'Sesi fokus', 'Pemberitahuan saat
sesi Pomodoro selesai');

  const NotificationChannel(this.id, this.title,
this.description);

  final String id;
  final String title;
  final String description;
}

/// ID tetap untuk notifikasi Pomodoro (Tahap 2 §11.6). ID
pengingat tugas
```

```
/// berada di rentang lain (lihat ReminderScheduler.idFor).
const int kPomodoroNotificationId = 1;

/// Abstraksi notifikasi lokal agar logika pengingat bisa
diuji.
abstract interface class NotificationService {
  Future<void> initialize();

  /// Izin notifikasi sudah diberikan.
  Future<bool> areEnabled();

  /// Meminta izin (Android 13+). `true` bila diizinkan.
  Future<bool> requestPermission();

  Future<void> schedule({
    required int id,
    required String title,
    required String body,
    required DateTime at,
    required NotificationChannel channel,
    String? payload,
  });

  Future<void> cancel(int id);

  Future<Set<int>> pendingIds();

  /// Payload notifikasi yang diketuk saat aplikasi berjalan.
  Stream<String> get taps;

  /// Payload notifikasi yang membuka aplikasi dari keadaan
mati (cold start).
  Future<String?> takeLaunchPayload();

  bool get isSupported;
}

/// Implementasi `flutter_local_notifications` 22.x.
///
/// Mode alarm (K-15): memakai `exactAllowWhileIdle` bila
sistem mengizinkan
/// alarm exact, selain itu `inexactAllowWhileIdle` (bisa
terlambat beberapa
/// menit saat Doze). Lihat docs/BACKEND_SETUP.md bagian
Android.
class LocalNotificationService implements NotificationService
{
  LocalNotificationService([FlutterLocalNotificationsPlugin?
plugin])
      : _plugin = plugin ?? FlutterLocalNotificationsPlugin();

  final FlutterLocalNotificationsPlugin _plugin;
  final StreamController<String> _taps =
StreamController<String>.broadcast();
  bool _initialized = false;
  bool? _exactAllowed;

  AndroidFlutterLocalNotificationsPlugin? get _android =>
      _plugin.resolvePlatformSpecificImplementation<
          AndroidFlutterLocalNotificationsPlugin>();
```

```
  @override
  bool get isSupported => !kIsWeb;

  @override
  Stream<String> get taps => _taps.stream;

  @override
  Future<void> initialize() async {
    if (_initialized || kIsWeb) {
      return;
    }
    const InitializationSettings settings =
InitializationSettings(
      android:
AndroidInitializationSettings('@mipmap/ic_launcher'),
      iOS: DarwinInitializationSettings(
        requestAlertPermission: false,
        requestBadgePermission: false,
        requestSoundPermission: false,
      ),
    );
    try {
      await _plugin.initialize(
        settings: settings,
        onDidReceiveNotificationResponse:
(NotificationResponse response) {
          final String? payload = response.payload;
          if (payload != null && payload.isNotEmpty) {
            _taps.add(payload);
          }
        },
      );
      _initialized = true;
    } catch (error) {
      debugPrint('Notifikasi gagal diinisialisasi: $error');
    }
  }

  @override
  Future<bool> areEnabled() async {
    if (!_initialized) {
      return false;
    }
    try {
      return await _android?.areNotificationsEnabled() ??
true;
    } catch (error) {
      debugPrint('Cek izin notifikasi gagal: $error');
      return false;
    }
  }

  @override
  Future<bool> requestPermission() async {
    if (!_initialized) {
      return false;
    }
    try {
      final AndroidFlutterLocalNotificationsPlugin? android =
```

```
_android;
      if (android != null) {
        return await android.requestNotificationsPermission()
?? false;
      }
      return true;
    } catch (error) {
      debugPrint('Permintaan izin notifikasi gagal: $error');
      return false;
    }
  }

  Future<AndroidScheduleMode> _scheduleMode() async {
    if (_exactAllowed == null) {
      try {
        _exactAllowed = await
_android?.canScheduleExactNotifications() ?? false;
      } catch (_) {
        _exactAllowed = false;
      }
    }
    return _exactAllowed!
        ? AndroidScheduleMode.exactAllowWhileIdle
        : AndroidScheduleMode.inexactAllowWhileIdle;
  }

  @override
  Future<void> schedule({
    required int id,
    required String title,
    required String body,
    required DateTime at,
    required NotificationChannel channel,
    String? payload,
  }) async {
    if (!_initialized) {
      return;
    }
    final NotificationDetails details = NotificationDetails(
      android: AndroidNotificationDetails(
        channel.id,
        channel.title,
        channelDescription: channel.description,
        importance: Importance.high,
        priority: Priority.high,
      ),
      iOS: const DarwinNotificationDetails(),
    );
    await _plugin.zonedSchedule(
      id: id,
      title: title,
      body: body,
      scheduledDate: tz.TZDateTime.from(at, tz.local),
      notificationDetails: details,
      androidScheduleMode: await _scheduleMode(),
      payload: payload,
    );
  }

  @override
```

```
  Future<void> cancel(int id) async {
    if (!_initialized) {
      return;
    }
    await _plugin.cancel(id: id);
  }

  @override
  Future<Set<int>> pendingIds() async {
    if (!_initialized) {
      return <int>{};
    }
    final List<PendingNotificationRequest> pending =
        await _plugin.pendingNotificationRequests();
    return pending.map((PendingNotificationRequest p) =>
p.id).toSet();
  }

  @override
  Future<String?> takeLaunchPayload() async {
    if (!_initialized) {
      return null;
    }
    try {
      final NotificationAppLaunchDetails? details =
          await _plugin.getNotificationAppLaunchDetails();
      if (details?.didNotificationLaunchApp ?? false) {
        return details?.notificationResponse?.payload;
      }
    } catch (error) {
      debugPrint('Detail peluncuran notifikasi gagal dibaca:
$error');
    }
    return null;
  }
}

/// Dipakai di Web dan test: tidak melakukan apa-apa.
class NoopNotificationService implements NotificationService {
  NoopNotificationService({this.enabled = false});

  bool enabled;
  final Map<int, DateTime> scheduled = <int, DateTime>{};
  final StreamController<String> _taps =
StreamController<String>.broadcast();

  void simulateTap(String payload) => _taps.add(payload);

  @override
  bool get isSupported => false;

  @override
  Future<void> initialize() async {}

  @override
  Future<bool> areEnabled() async => enabled;

  @override
  Future<bool> requestPermission() async => enabled;
```

```
  @override
  Future<void> schedule({
    required int id,
    required String title,
    required String body,
    required DateTime at,
    required NotificationChannel channel,
    String? payload,
  }) async {
    scheduled[id] = at;
  }

  @override
  Future<void> cancel(int id) async {
    scheduled.remove(id);
  }

  @override
  Future<Set<int>> pendingIds() async =>
scheduled.keys.toSet();

  @override
  Stream<String> get taps => _taps.stream;

  @override
  Future<String?> takeLaunchPayload() async => null;
}

/// Diisi di `main.dart`. Bawaan: no-op (test, Web).
final Provider<NotificationService>
notificationServiceProvider =
    Provider<NotificationService>((Ref ref) =>
NoopNotificationService());

/// Status izin notifikasi; di-refresh dengan
`ref.invalidate`.
final FutureProvider<bool> notificationPermissionProvider =
    FutureProvider<bool>(
  (Ref ref) =>
ref.read(notificationServiceProvider).areEnabled(),
);
```

Penjelasan Source Code `notification_service.dart`

1. Identitas File

   Berkas ini berada pada `lib/core/notifications/notification_service.dart` dan menyediakan abstraksi layanan pengiriman serta penjadwalan notifikasi lokal multi-kanal untuk platform Android dan iOS.

2. Kanal Notifikasi (baris 8–23)
   - Mendefinisikan enum `NotificationChannel` dengan dua saluran terpisah: `reminders` (`planify_reminders`, tingkat prioritas tinggi

untuk tenggat waktu tugas) dan `focus` (`planify_focus`, untuk pemberitahuan selesai sesi Pomodoro atau waktu istirahat).

- Menetapkan konstanta `kPomodoroNotificationId = 1` khusus untuk notifikasi timer fokus.

3. Kelas LocalNotificationService (baris 62–220)
   - initialize: Mengonfigurasi ikon notifikasi bawaan @mipmap/ic_launcher dan mendaftarkan callback onDidReceiveNotificationResponse untuk menangkap interaksi ketukan pengguna.
   - requestPermission & areEnabled: Menangani izin notifikasi sistem pada Android 13+ (API level 33, runtime permission POST_NOTIFICATIONS).
   - _scheduleMode: Memeriksa apakah perangkat mengizinkan alarm berpresisi tinggi (canScheduleExactNotifications). Jika diizinkan, digunakan mode exactAllowWhileIdle yang mampu membunyikan alarm saat perangkat dalam kondisi Doze Mode; jika ditolak oleh pengguna atau sistem, sistem secara mulus beralih ke inexactAllowWhileIdle.
   - schedule: Memanggil zonedSchedule dari pustaka lokal untuk menjadwalkan notifikasi pada waktu target (at) dengan konversi zona waktu lokal tz.TZDateTime.
   - takeLaunchPayload: Menangkap data payload ketika aplikasi diluncurkan dari kondisi mati total (cold start) akibat ketukan pengguna pada bilah status notifikasi.
4. Implementasi NoopNotificationService (baris 223–269)

   Kelas tiruan (mock) yang digunakan saat aplikasi dijalankan pada platform web atau saat pengujian unit otomatis (unit test), memastikan logika bisnis aplikasi tidak mengalami galat ketiadaan channel platform.
5. Alur Kerja

   Saat aplikasi dinyalakan di main.dart, initialize() dieksekusi. Ketika pengguna membuat atau memperbarui tugas yang memiliki batas waktu di Planner, schedule() dipanggil untuk mendaftarkan alarm notifikasi pada sistem operasi Android/iOS.

6. Hubungan dengan File Lain

   Berkas ini digunakan oleh ReminderScheduler untuk sinkronisasi pengingat tugas, oleh PomodoroEngine untuk notifikasi akhir interval fokus, dan disediakan ke seluruh antarmuka melalui notificationServiceProvider.

4.12.3 Penjadwalan Pengingat Berdasarkan Deadline Tugas

Penjadwalan pengingat tugas dilakukan secara otomatis melalui modul ReminderScheduler. Setiap tugas (Task) yang memiliki tanggal dan jam tenggat (dueDate) dianalisis untuk menentukan waktu picu pengingat (misal tepat waktu tenggat, atau dengan offset pengingat sebelumnya).

1. Pemetaan Kunci ID Notifikasi

   Android Notification Manager mensyaratkan ID notifikasi berupa integer 32-bit bertanda (signed 32-bit int), sedangkan ID tugas Planify berformat UUID v4 string (128-bit). ReminderScheduler menerapkan fungsi hash deterministik untuk memetakan string UUID menjadi integer aman pada rentang $[1000, 2^{31}-1]$, sehingga tidak bentrok dengan ID tetap Pomodoro (kPomodoroNotificationId = 1).

2. Sinkronisasi Otomatis

   Setiap kali data tugas disimpan, disunting, ditandai selesai, atau dihapus, repository memanggil ReminderScheduler.sync(). Sistem membaca daftar ID pengingat yang sedang aktif di sistem operasi (pendingIds()). Jika status tugas diubah menjadi selesai (isCompleted == true) atau tugas dihapus, pengingat terkait langsung dibatalkan melalui cancel(id). Jika batas waktu diubah, jadwal lama dibatalkan dan jadwal baru didaftarkan secara atomik.

4.12.3 Penanganan Interaksi Pengguna terhadap Notifikasi

Ketika notifikasi muncul di bilah status perangkat dan diketuk oleh pengguna, sistem menangani payload data yang disematkan:

1. Aplikasi Berjalan di Latar Depan / Latar Belakang (Warm / Foreground State)

   Aliran data taps pada LocalNotificationService memancarkan string payload (misal task:<uuid> atau pomodoro:complete). Kontroller navigasi tingkat atas mendengarkan aliran ini dan

segera memicu GoRouter untuk mengarahkan pengguna ke halaman detail tugas terkait (/planner/task/:id) atau membuka panel timer fokus.

2. Aplikasi Ditutup Total (Cold Start)

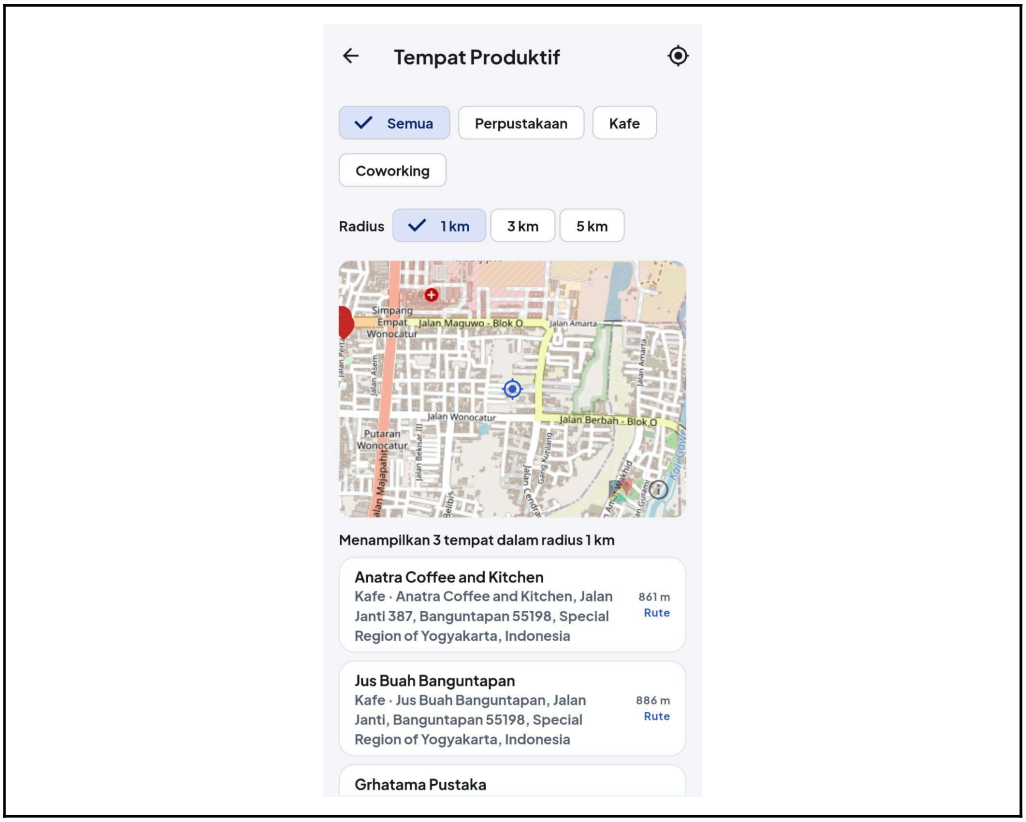
Pada siklus hidup inisialisasi awal di berkas router, aplikasi mengeksekusi takeLaunchPayload(). Jika ditemukan payload dari aksi ketukan yang membangunkan aplikasi, sistem menyimpan rute target tersebut dan melakukan pengalihan rute (deep link navigation) segera setelah proses pengecekan sesi autentikasi pengguna selesai dilakukan.

4.13 Fitur Location-Based Service (LBS)

Fitur Location-Based Service (LBS) pada aplikasi Planify diwujudkan dalam bentuk modul *Productive Places Navigator*. Modul ini dirancang untuk mengatasi hambatan produktivitas pengguna yang membutuhkan lingkungan kondusif untuk bekerja atau belajar di luar rumah. Dengan memanfaatkan sensor GPS perangkat dan antarmuka peta digital, aplikasi mampu mendeteksi posisi geografis pengguna saat ini, mencari tempat produktif di sekitar (perpustakaan, kafe ramah kerja, dan ruang kerja bersama/coworking space), menampilkan lokasi pada peta interaktif, serta memberikan panduan navigasi rute turn-by-turn.

4.13.1 Tampilan Antarmuka Peta dan Pencarian Tempat

Antarmuka halaman Navigator menyajikan integrasi peta visual dan daftar lokasi yang terstruktur secara ergonomis. Di bagian atas layar, tersedia bilah pencarian dan tombol filter kategori tempat (Chips: Semua, Perpustakaan, Kafe, dan Coworking Space) serta slider penyesuaian radius pencarian dari 1 km hingga 10 km. Bagian tengah layar menampilkan kanvas peta interaktif berbasis ubin OpenStreetMap yang dapat digeser dan diperbesar (pinch-to-zoom). Posisi koordinat pengguna ditandai dengan pin biru beranimasi denyut (pulsing blue dot), sedangkan tempat-tempat produktif ditandai dengan pin berwarna spesifik sesuai kategori (misal merah untuk kafe, hijau untuk perpustakaan, dan biru untuk coworking). Mengetuk salah satu pin pada peta atau memilih kartu pada daftar bawah layar akan membuka panel rincian tempat yang memuat nama tempat, alamat lengkap, jarak langsung dalam meter/kilometer, serta tombol aksi "Buka Rute di Google Maps".



Gambar 4.13.1.1 Tampilan Peta Interaktif dan Rekomendasi Tempat Produktif

4.13.2 Pengambilan Lokasi Pengguna dan Penentuan Radius

Struktur representasi data tempat produktif, formula perhitungan jarak lingkaran besar (Great-Circle distance), serta pengurai respons JSON dari penyedia data spasial dienkapsulasi pada berkas place.dart.

Kode Program 4.13.2.1 Model dan Parser Lokasi Tempat Produktif

```
import 'dart:math' as math;

import 'package:flutter/foundation.dart';

enum PlaceCategory {
  library('Perpustakaan'),
  cafe('Kafe'),
  coworking('Coworking');

  const PlaceCategory(this.label);

  final String label;
}

/// Tempat produktif di sekitar (Tahap 2 §11.4).
@immutable
class Place {
  const Place({
    required this.id,
    required this.name,
    required this.category,
    required this.lat,
```

```
    required this.lng,
    required this.distanceMeters,
    this.address,
  });

  final String id;
  final String name;
  final PlaceCategory category;
  final double lat;
  final double lng;
  final String? address;
  final double distanceMeters;

  String get distanceLabel => distanceMeters < 1000
      ? '${distanceMeters.round()} m'
      : '${(distanceMeters /
1000).toStringAsFixed(1).replaceAll('.', ',')} km';
}

/// Jarak haversine (meter). Dipakai juga di test; tidak butuh
plugin.
double haversineMeters(double lat1, double lng1, double lat2,
double lng2) {
  const double r = 6371000;
  double rad(double d) => d * math.pi / 180;
  final double dLat = rad(lat2 - lat1);
  final double dLng = rad(lng2 - lng1);
  final double a =
      math.sin(dLat / 2) * math.sin(dLat / 2) +
      math.cos(rad(lat1)) *
          math.cos(rad(lat2)) *
          math.sin(dLng / 2) *
          math.sin(dLng / 2);
  return 2 * r * math.asin(math.min(1, math.sqrt(a)));
}

/// Mengurai respons Overpass API (`out body center`) menjadi
[Place].
/// Elemen tanpa nama atau koordinat diabaikan.
List<Place> parseOverpass(
  Map<String, dynamic> json, {
  required double originLat,
  required double originLng,
}) {
  final Object? elements = json['elements'];
  if (elements is! List) {
    throw const FormatException('elements tidak ada');
  }
  final List<Place> out = <Place>[];
  for (final Object? e in elements) {
    if (e is! Map) {
      continue;
    }
    final Map<dynamic, dynamic> tags =
        (e['tags'] as Map?) ?? const <dynamic, dynamic>{};
    final String? name = tags['name'] as String?;
    final Object? lat = e['lat'] ?? (e['center'] as
Map?)?['lat'];
    final Object? lng = e['lon'] ?? (e['center'] as
Map?)?['lon'];
```

```
    if (name == null || name.trim().isEmpty || lat is! num ||
lng is! num) {
      continue;
    }
    final String amenity = '${tags['amenity'] ?? ''}';
    final String office = '${tags['office'] ?? ''}';
    final PlaceCategory? category = amenity == 'library'
        ? PlaceCategory.library
        : amenity == 'cafe'
        ? PlaceCategory.cafe
        : (amenity == 'coworking_space' || office ==
'coworking')
        ? PlaceCategory.coworking
        : null;
    if (category == null) {
      continue;
    }
    final String street = '${tags['addr:street'] ?? ''}';
    final String number = '${tags['addr:housenumber'] ?? ''}';
    out.add(
      Place(
        id: '${e['type']}/${e['id']}',
        name: name.trim(),
        category: category,
        lat: lat.toDouble(),
        lng: lng.toDouble(),
        address: street.isEmpty ? null : '$street
$number'.trim(),
        distanceMeters: haversineMeters(
          originLat,
          originLng,
          lat.toDouble(),
          lng.toDouble(),
        ),
      ),
    );
  }
  out.sort((Place a, Place b) =>
a.distanceMeters.compareTo(b.distanceMeters));
  return out;
}

/// Geoapify uses GeoJSON coordinates in longitude, latitude
order.
List<Place> parseGeoapify(
  Map<String, dynamic> json, {
  required double originLat,
  required double originLng,
  required int radiusMeters,
  PlaceCategory? category,
}) {
  final Object? features = json['features'];
  if (json['type'] != 'FeatureCollection' || features is!
List) {
    throw const FormatException('features tidak ada');
  }
  final List<Place> places = <Place>[];
  final Set<String> seen = <String>{};
  for (final Object? feature in features) {
    if (feature is! Map || feature['properties'] is! Map)
```

```
continue;
    final Map<dynamic, dynamic> properties =
feature['properties'] as Map;
    final Object? name = properties['name'];
    final Object? id = properties['place_id'];
    final Object? categories = properties['categories'];
    if (name is! String ||
        name.trim().isEmpty ||
        id is! String ||
        id.isEmpty ||
        categories is! List) {
      continue;
    }
    bool matches(String key) => categories.any(
      (Object? c) => c is String && (c == key ||
c.startsWith('$key.')),
    );
    final PlaceCategory? kind = matches('education.library')
        ? PlaceCategory.library
        : matches('office.coworking')
        ? PlaceCategory.coworking
        : matches('catering.cafe')
        ? PlaceCategory.cafe
        : null;
    if (kind == null || (category != null && kind !=
category)) continue;
    final Object? geometry = feature['geometry'];
    final Object? coordinates = geometry is Map &&
geometry['type'] == 'Point'
        ? geometry['coordinates']
        : null;
    final Object? lng =
        properties['lon'] ??
        (coordinates is List && coordinates.length >= 2
            ? coordinates[0]
            : null);
    final Object? lat =
        properties['lat'] ??
        (coordinates is List && coordinates.length >= 2
            ? coordinates[1]
            : null);
    if (lat is! num ||
        lng is! num ||
        !lat.isFinite ||
        !lng.isFinite ||
        lat.abs() > 90 ||
        lng.abs() > 180) {
      continue;
    }
    final double distance = haversineMeters(
      originLat,
      originLng,
      lat.toDouble(),
      lng.toDouble(),
    );
    if (distance > radiusMeters || !seen.add(id)) continue;
    final Object? address =
        properties['formatted'] ??
properties['address_line2'];
    places.add(
```

```
      Place(
        id: 'geoapify/$id',
        name: name.trim(),
        category: kind,
        lat: lat.toDouble(),
        lng: lng.toDouble(),
        distanceMeters: distance,
        address: address is String &&
address.trim().isNotEmpty
            ? address.trim()
            : null,
      ),
    );
  }
  places.sort(
    (Place a, Place b) =>
a.distanceMeters.compareTo(b.distanceMeters),
  );
  return places;
}
```

Penjelasan Source Code `place.dart`

1. Identitas File

   Berkas ini berada pada lib/features/navigator/domain/place.dart dan mendefinisikan model entitas tempat produktif, formula matematika Haversine untuk kalkulasi jarak geografis, serta fungsi pengurai respons data spasial Overpass API dan Geoapify API.

2. Kategori dan Entitas Place (baris 5–39)
   - PlaceCategory: Mendefinisikan tiga jenis tempat produktif: library (Perpustakaan), cafe (Kafe), dan coworking (Coworking Space).
   - Place: Struktur data representasi tempat yang menyimpan pengenal unik (id), nama lokasi (name), kategori (category), koordinat lintang dan bujur (lat, lng), alamat jalan (address), serta jarak langsung dari pengguna dalam meter (distanceMeters). Getter distanceLabel secara cerdas menyajikan jarak dalam satuan meter jika kurang dari 1.000 meter, atau kilometer dengan satu desimal jika lebih.

3. Fungsi Matematika haversineMeters (baris 42–54)
   - Mengimplementasikan rumus Haversine untuk menghitung jarak terpendek antara dua titik koordinat pada permukaan bola bumi dengan radius rata-rata $R = 6.371.000\text{ m}$: $$d = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) +

\cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)
}\right)$$

- Fungsi ini ditulis murni tanpa dependensi plugin native sehingga dapat diuji secara mandiri pada lingkungan unit testing.

4. Fungsi Pengurai Data Spasial (baris 58–201)
   - parseOverpass: Mengurai payload XML/JSON dari OpenStreetMap Overpass API, mengekstrak tag OpenStreetMap (amenity=library, amenity=cafe, office=coworking), menghitung jarak Haversine ke titik asal pengguna, dan mengurutkan daftar tempat dari yang terdekat.
   - parseGeoapify: Mengurai GeoJSON FeatureCollection dari Geoapify Places API v2, melakukan filter kategori spesifik (education.library, catering.cafe, office.coworking), memvalidasi batas koordinat yang sah ($-90 \le \text{lat} \le 90$ dan $-180 \le \text{lng} \le 180$), menghapus data duplikat menggunakan seen Set, serta menyortir hasil berdasarkan jarak terdekat.
5. Alur Kerja

   Lapisan presentasi memanggil repositori navigator dengan koordinat GPS pengguna. Repositori mengeksekusi panggilan API Geoapify (dengan fallback ke Overpass), meneruskan hasil mentah ke parseGeoapify(), dan mengembalikan daftar objek List<Place> yang telah tersortir rapi kepada antarmuka.

6. Hubungan dengan File Lain

   Berkas ini digunakan oleh NavigatorRepository pada lapisan data, NavigatorController pada lapisan aplikasi, dan NavigatorPage pada lapisan presentasi untuk merender pin peta OpenStreetMap.

4.13.3 Integrasi API Peta dan Penentuan Rute Lokasi Produktif

Arsitektur modul Location-Based Service (LBS) pada Planify mengintegrasikan empat komponen teknologi utama:

1. Pustaka geolocator

   Digunakan untuk mengakses modul GPS perangkat. Sebelum melakukan pencarian, sistem memverifikasi apakah layanan lokasi aktif

(isLocationServiceEnabled) dan meminta izin akses lokasi (fine location permission).

2. Pustaka flutter_map dan OpenStreetMap (OSM)

   Menghilangkan ketergantungan pada SDK Google Maps berbayar dengan merender peta menggunakan ubin raster (raster tiles) terbuka dari OpenStreetMap. Lapisan peta disusun secara bertingkat: `TileLayer` untuk gambar peta dasar, CircleLayer untuk radius pencarian, dan `MarkerLayer` untuk pin koordinat tempat dan pengguna.

3. Geoapify Places API v2 & OpenStreetMap Overpass API

   Sistem menggunakan Geoapify API v2 sebagai penyedia data utama tempat komersial/publik berfitur lengkap. Jika kuota panggilan habis atau koneksi gagal, sistem secara otomatis beralih (graceful fallback) ke Overpass API publik.

4. Navigasi Rute Turn-by-Turn via `url_launcher`

   Ketika pengguna memilih tombol "Buka Rute" pada kartu rincian tempat, aplikasi menyusun skema URL geo-intent: https://www.google.com/maps/dir/?api=1&destination=<lat>,<lng>&travelmode=driving

   Pustaka url_launcher membuka tautan tersebut secara eksternal ke aplikasi Google Maps bawaan ponsel pengguna untuk memandu perjalanan dengan navigasi suara dan rute lalu lintas langsung.

4.14 Fitur Otomatisasi Focus Mode dengan Sensor Accelerometer

Fitur otomatisasi Focus Mode merupakan inovasi antarmuka berbasis sensor yang dirancang untuk mereduksi friksi pengguna saat hendak memulai sesi kerja mendalam (deep work). Alih-alih mengharuskan pengguna membuka aplikasi, mencari tab Focus, memilih durasi Pomodoro, dan menekan tombol mulai secara manual, aplikasi Planify memanfaatkan sensor accelerometer bawaan ponsel cerdas untuk mendeteksi gestur fisik ketika pengguna meletakkan perangkat secara telungkup (face-down) di atas meja kerja.

4.14.1 Deteksi Orientasi Perangkat Menggunakan Accelerometer

Logika matematika untuk memfilter derau (noise) getaran sensor, mendeteksi orientasi sumbu gravitasi, memverifikasi kestabilan posisi diam,

serta mengelola jeda waktu (cooldown) diimplementasikan secara murni (pure logic) pada berkas face_down_detector.dart.

Kode Program 4.14.1.1 Algoritma Deteksi Orientasi Perangkat Telungkup

```
import 'dart:math' as math;

import 'package:flutter/foundation.dart';

/// Parameter deteksi (Tahap 2 §12.1). Nilai awal yang HARUS
dikalibrasi
/// lewat eksperimen di perangkat nyata (K-10).
@immutable
class FaceDownConfig {
  const FaceDownConfig({
    this.alpha = 0.2,
    this.zThreshold = 8.0,
    this.xyTolerance = 2.5,
    this.magTolerance = 1.5,
    this.hold = const Duration(seconds: 3),
    this.cooldown = const Duration(minutes: 2),
  });

  static const double gravity = 9.81;

  final double alpha;
  final double zThreshold;
  final double xyTolerance;
  final double magTolerance;
  final Duration hold;
  final Duration cooldown;
}

/// Deteksi "ponsel diletakkan telungkup dan diam" dari
accelerometer.
/// Logika murni: tidak menyentuh sensor ataupun jam sistem.
///
/// Sumbu Android: Z keluar dari layar. Layar menghadap ke
bawah di meja →
/// z ≈ −9,8 m/s².
class FaceDownDetector {
  FaceDownDetector([this.config = const FaceDownConfig()]);

  final FaceDownConfig config;

  double? _fx;
  double? _fy;
  double? _fz;
  DateTime? _since;
  DateTime? _cooldownUntil;
  bool _fired = false;

  /// `true` bila kondisi telungkup sedang terpenuhi (setelah
filter).
  bool get isFaceDown => _since != null;

  /// Memproses satu sampel. Mengembalikan `true` tepat sekali
saat kondisi
  /// bertahan selama `hold` dan tidak sedang cooldown. Event
berikutnya baru
```

```
  /// mungkin setelah ponsel diangkat lalu ditelungkupkan
lagi.
  bool onSample(double x, double y, double z, DateTime now) {
    final double a = config.alpha;
    _fx = _fx == null ? x : a * x + (1 - a) * _fx!;
    _fy = _fy == null ? y : a * y + (1 - a) * _fy!;
    _fz = _fz == null ? z : a * z + (1 - a) * _fz!;

    final double fx = _fx!;
    final double fy = _fy!;
    final double fz = _fz!;
    final double xy = math.sqrt(fx * fx + fy * fy);
    final double magnitude = math.sqrt(fx * fx + fy * fy + fz
* fz);

    final bool faceDown = fz < -config.zThreshold &&
        xy < config.xyTolerance &&
        (magnitude - FaceDownConfig.gravity).abs() <
config.magTolerance;

    if (!faceDown) {
      _since = null;
      _fired = false;
      return false;
    }
    _since ??= now;
    if (_fired) {
      return false;
    }
    final DateTime? until = _cooldownUntil;
    if (until != null && now.isBefore(until)) {
      return false;
    }
    if (now.difference(_since!) >= config.hold) {
      _fired = true;
      return true;
    }
    return false;
  }

  /// Dipanggil setelah tawaran ditolak/kedaluwarsa.
  void startCooldown(DateTime now) {
    _cooldownUntil = now.add(config.cooldown);
  }

  bool inCooldown(DateTime now) {
    final DateTime? until = _cooldownUntil;
    return until != null && now.isBefore(until);
  }

  void reset() {
    _fx = null;
    _fy = null;
    _fz = null;
    _since = null;
    _fired = false;
  }
}
```

Penjelasan Source Code face_down_detector.dart

1. Identitas File

   Berkas ini berada pada lib/features/auto_focus/domain/face_down_detector.dart dan berisi konfigurasi parameter deteksi serta algoritma pengenalan gestur ponsel telungkup berbasis vektor percepatan tiga sumbu.

2. Kelas FaceDownConfig` (baris 8–26)
   - alpha = 0.2: Koefisien penghalusan Exponential Moving Average (EMA) untuk menyaring getaran mikro tangan.
   - zThreshold = 8.0: Batas minimum percepatan gravitasi pada sumbu Z negatif ($z < -8.0\text{ m/s}^2$).
   - xyTolerance = 2.5: Toleransi kemiringan pada bidang horizontal ($xy = \sqrt{x^2 + y^2} < 2.5\text{ m/s}^2$) untuk memastikan ponsel benar-benar diletakkan rata mendatar di atas meja.
   - magTolerance = 1.5: Batas toleransi simpangan total magnitudo percepatan terhadap konstanta gravitasi bumi ($g = 9.81\text{ m/s}^2$) untuk membedakan antara ponsel yang diletakkan tenang dengan ponsel yang sedang diguncang atau dijatuhkan.
   - hold = Duration(seconds: 3): Durasi waktu konfirmasi diam selama 3 detik penuh.
   - cooldown = Duration(minutes: 2): Waktu jeda selama 2 menit jika pengguna menolak atau mengabaikan tawaran sesi fokus.

3. Kelas FaceDownDetector` (baris 33–104)
   - onSample (baris 51–85): Menerima sampel pembacaan percepatan sumbu x, y, z beserta stempel waktu saat ini `now`. Sampel disaring melalui filter lolos-rendah: $$f_i = \alpha \cdot s_i + (1 - \alpha) \cdot f_{i-1}$$ Kondisi telungkup terpenuhi jika $f_z < -8.0$, $\sqrt{f_x^2 + f_y^2} < 2.5$, dan $|\sqrt{f_x^2 + f_y^2 + f_z^2} - 9.81| < 1.5$. Bila kondisi ini bertahan stabil tanpa henti selama 3 detik (`now.difference(_since!) >= hold`) dan tidak sedang berada dalam masa jeda (`!inCooldown`), metode mengembalikan nilai `true` tepat satu kali (`_fired = true`).

- startCooldown: Menetapkan batas waktu jeda _cooldownUntil agar dialog penawaran tidak muncul berulang kali secara mengganggu saat ponsel tetap dalam keadaan telungkup.

4. Alur Kerja

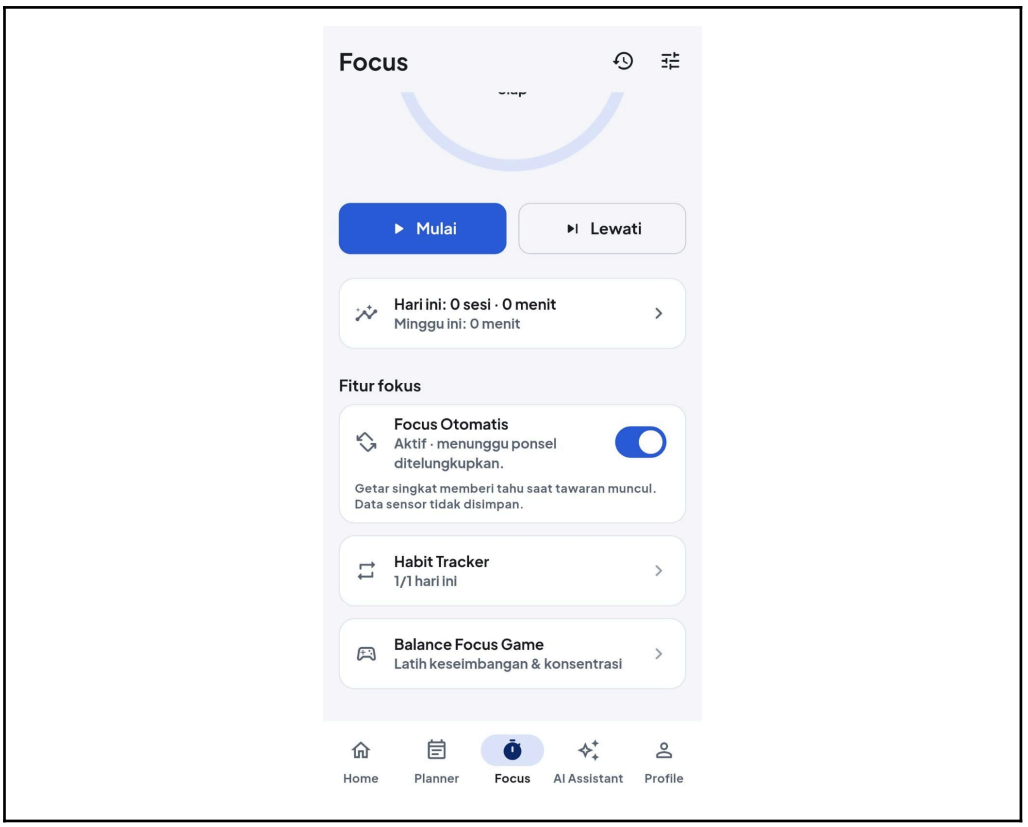
   Layanan latar depan mendengarkan aliran data dari accelerometerEventStream() paket sensors_plus. Setiap event diteruskan ke FaceDownDetector.onSample(). Ketika fungsi mengembalikan nilai true, sistem memicu pemunculan dialog konfirmasi sesi fokus di layar.

5. Hubungan dengan File Lain

   Berkas ini digunakan oleh AutoFocusCoordinator pada lapisan aplikasi dan berinteraksi dengan AutoFocusDialog serta PomodoroEngine.

4.14.2 Pengaktifan Focus Mode Otomatis Saat Perangkat Ditengkurapkan

Untuk menghormati kenyamanan pengguna dan menghindari pemicuan yang tidak diinginkan (false positive), aplikasi Planify menerapkan pendekatan konfirmasi non-intrusif. Ketika algoritma FaceDownDetector mendeteksi gestur telungkup selama 3 detik, aplikasi tidak langsung mengunci layar secara paksa, melainkan memunculkan dialog tawaran interaktif beranimasi lembut dengan fitur hitung mundur (auto-dismiss countdown).



Gambar 4.14.2.1 Tampilan Dialog Tawaran Focus Mode Otomatis

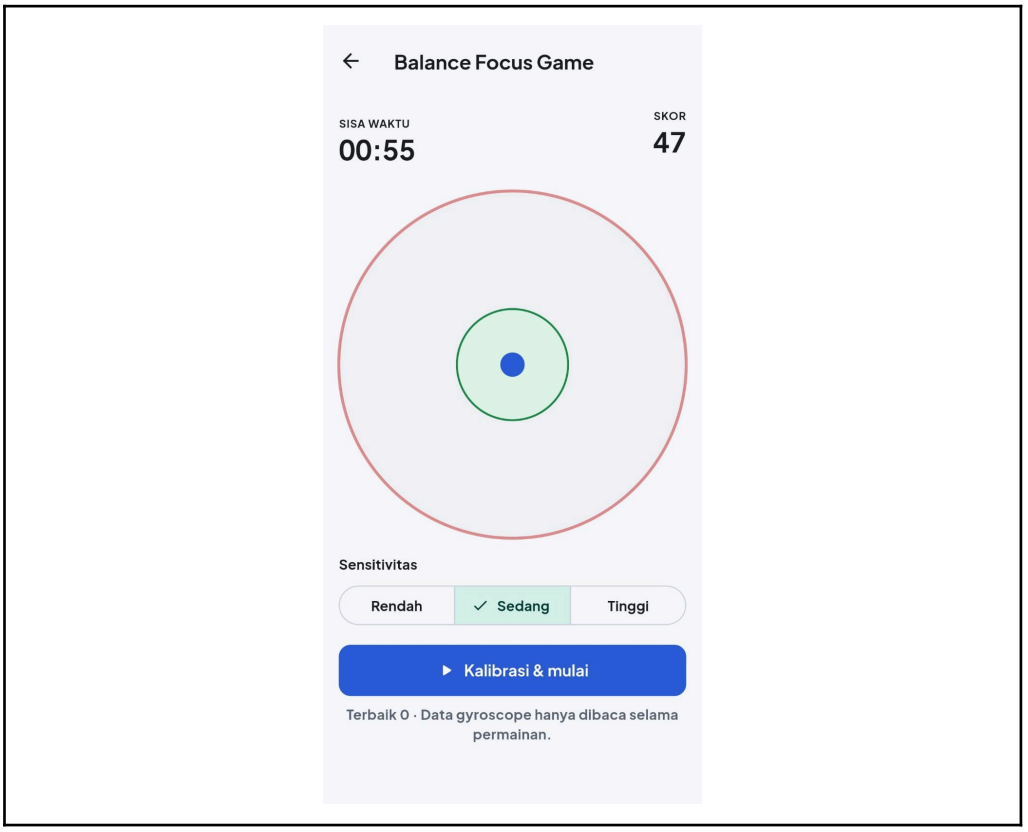
Dialog penawaran menampilkan pesan: "Ponsel terdeteksi diletakkan telungkup. Mulai sesi fokus sekarang?". Pengguna memiliki waktu 10 detik untuk menekan tombol "Mulai Sesi Fokus". Jika pengguna menekan tombol mulai, sistem segera mengaktifkan PomodoroEngine dengan durasi standar (25 menit sesi fokus), membunyikan umpan balik haptik lembut, dan mengunci interaksi aplikasi non-esensial. Apabila pengguna menekan tombol "Batal" atau membiarkan hitung mundur 10 detik habis tanpa aksi, dialog akan menutup secara otomatis dan FaceDownDetector.startCooldown() diaktifkan selama 2 menit, memastikan pengguna dapat meletakkan ponsel secara telungkup untuk keperluan lain tanpa terganggu oleh notifikasi berulang.

4.15 Fitur Mini Game Keseimbangan Fokus dengan Sensor Gyroscope

Sebagai bagian dari pendekatan holistik terhadap produktivitas dan kesejahteraan mental, aplikasi Planify menyertakan fitur permainan mini keseimbangan fokus (***Balance Focus Game***). Permainan ini dirancang khusus untuk dimainkan selama jeda istirahat Pomodoro (5 menit istirahat pendek) sebagai sarana latihan konsentrasi motorik halus dan relaksasi pikiran, mengalihkan pengguna dari godaan membuka media sosial yang memicu distraksi berlebih.

4.15.1 Tampilan dan Mekanisme Permainan

Permainan disajikan dalam kanvas arena lingkaran dua dimensi yang merepresentasikan bidang keseimbangan. Di tengah arena terdapat zona target konsentris beradius 0.32 satuan arena yang berpendar lembut. Objek kendali berupa bola fokus beradius 0.07 yang posisinya dikendalikan secara dinamis oleh orientasi kemiringan fisik ponsel pengguna melalui sensor gyroscope.



Gambar 4.15.1.1 Tampilan Permainan Keseimbangan Fokus (*Balance Focus Game*)

Mekanisme permainan menuntut pengguna mempertahankan posisi bola di dalam lingkaran target target selama mungkin dalam batas waktu permainan 60 detik. Pengguna memiringkan ponsel ke kanan/kiri (rotasi sumbu Y) dan ke depan/belakang (rotasi sumbu X) dengan gerakan lembut. Di bagian atas layar, disajikan indikator waktu tersisa (countdown bar), akumulasi skor waktu fokus (focus score), dan pemilih sensitivitas (Rendah, Sedang, Tinggi). Jika bola tergelincir hingga menyentuh tepi batas luar arena lingkaran ($r \ge 1.0$), permainan berakhir (status: lost). Sebaliknya, jika pengguna berhasil menjaga bola tetap berada di arena hingga durasi 60 detik berakhir, pengguna dinyatakan menang (status: won) dan skor akhir dicatat ke dalam papan rekor lokal.

4.15.2 Pembacaan Sensor Gyroscope dan Pengendalian Posisi Bola

Fisika pergerakan bola, peredaman inersia, penyaringan zona mati (deadzone), dan kalibrasi bias sensor gyroscope diimplementasikan pada berkas murni balance_physics.dart.

Kode Program 4.15.2.1 Simulasi Fisika dan Kalibrasi Gyroscope

```
import 'dart:math' as math;

import 'package:flutter/foundation.dart';

/// Tingkat sensitivitas kendali (mockup S-37: "Sensitivitas · 
Sedang").
enum BalanceSensitivity {
  low('Rendah', 0.9),
  medium('Sedang', 1.5),
  high('Tinggi', 2.3);

  const BalanceSensitivity(this.label, this.gain);

  final String label;
  final double gain;

  static BalanceSensitivity parse(Object? v) => 
BalanceSensitivity.values
      .firstWhere((BalanceSensitivity s) => s.name == v, 
orElse: () => medium);
}

/// Parameter game (Tahap 2 §12.2). Posisi dalam satuan arena: 
pusat (0,0),
/// tepi arena berjari-jari 1.
@immutable
class BalanceConfig {
  const BalanceConfig({
    this.duration = const Duration(seconds: 60),
    this.targetRadius = 0.32,
    this.ballRadius = 0.07,
    this.deadzone = 0.03,
    this.emaAlpha = 0.35,
    this.damping = 1.6,
    this.gain = 1.5,
  });

  final Duration duration;
  final double targetRadius;
  final double ballRadius;

  /// rad/s di bawah nilai ini dianggap nol (menekan 
noise/drift kecil).
  final double deadzone;
  final double emaAlpha;

  /// Peredaman kecepatan per detik.
  final double damping;
  final double gain;
}
```

```
enum BalanceStatus { playing, won, lost }

@immutable
class BalanceState {
  const BalanceState({
    this.x = 0,
    this.y = 0,
    this.vx = 0,
    this.vy = 0,
    this.wx = 0,
    this.wy = 0,
    this.elapsed = Duration.zero,
    this.secondsInTarget = 0,
    this.status = BalanceStatus.playing,
  });

  final double x;
  final double y;
  final double vx;
  final double vy;

  /// Kecepatan sudut tersaring (rad/s).
  final double wx;
  final double wy;
  final Duration elapsed;
  final double secondsInTarget;
  final BalanceStatus status;

  /// Skor = detik di zona target × 10 (bilangan bulat).
  int get score => (secondsInTarget * 10).floor();

  double get distance => math.sqrt(x * x + y * y);
}

/// Fisika bola: kendali KECEPATAN dari gyroscope (bukan
integrasi sudut)
/// agar drift tidak menumpuk (Tahap 2 §12.2). Logika murni.
abstract final class BalancePhysics {
  static double _dead(double v, double zone) =>
      v.abs() < zone ? 0 : v - zone * v.sign;

  /// Satu langkah simulasi. [gx], [gy] = pembacaan gyroscope
dikurangi bias.
  static BalanceState step(
    BalanceState s,
    double gx,
    double gy,
    double dtSeconds,
    BalanceConfig c,
  ) {
    if (s.status != BalanceStatus.playing || dtSeconds <= 0) {
      return s;
    }
    final double dt = math.min(dtSeconds, 0.1); // cegah
lompatan saat jank
    final double a = c.emaAlpha;
    final double wx = a * gx + (1 - a) * s.wx;
    final double wy = a * gy + (1 - a) * s.wy;
    final double ux = _dead(wx, c.deadzone);
    final double uy = _dead(wy, c.deadzone);
```

```
    // Miring ke kanan (rotasi sumbu Y) → bola ke kanan;
miring ke depan
    // (rotasi sumbu X) → bola ke bawah layar. Tanda dapat
dibalik setelah
    // uji di perangkat (K-10).
    final double decay = math.max(0, 1 - c.damping * dt);
    final double vx = (s.vx + uy * c.gain * dt) * decay;
    final double vy = (s.vy + ux * c.gain * dt) * decay;
    final double x = s.x + vx * dt;
    final double y = s.y + vy * dt;

    final double dist = math.sqrt(x * x + y * y);
    final Duration elapsed =
        s.elapsed + Duration(microseconds: (dt *
1e6).round());
    final double inTarget =
        s.secondsInTarget + (dist <= c.targetRadius ? dt : 0);

    BalanceStatus status = BalanceStatus.playing;
    if (dist + c.ballRadius >= 1) {
      status = BalanceStatus.lost;
    } else if (elapsed >= c.duration) {
      status = BalanceStatus.won;
    }

    return BalanceState(
      x: x,
      y: y,
      vx: vx,
      vy: vy,
      wx: wx,
      wy: wy,
      elapsed: elapsed,
      secondsInTarget: inTarget,
      status: status,
    );
  }
}

/// Kalibrasi titik nol gyroscope (perangkat dipegang diam ±2
detik).
class GyroCalibrator {
  GyroCalibrator({this.maxStdDev = 0.12});

  /// Simpangan baku di atas ini = pengguna bergerak,
kalibrasi diulang.
  final double maxStdDev;

  final List<double> _xs = <double>[];
  final List<double> _ys = <double>[];

  int get sampleCount => _xs.length;

  void add(double x, double y) {
    _xs.add(x);
    _ys.add(y);
  }

  void clear() {
```

```
    _xs.clear();
    _ys.clear();
  }

  static double _mean(List<double> v) =>
      v.isEmpty ? 0 : v.reduce((double a, double b) => a + b)
/ v.length;

  static double _std(List<double> v, double mean) {
    if (v.length < 2) {
      return 0;
    }
    final double sum = v.fold<double>(
      0,
      (double acc, double e) => acc + (e - mean) * (e - mean),
    );
    return math.sqrt(sum / (v.length - 1));
  }

  /// Bias (x, y) bila cukup stabil; `null` bila perlu
diulang.
  (double, double)? result({int minSamples = 10}) {
    if (_xs.length < minSamples) {
      return null;
    }
    final double mx = _mean(_xs);
    final double my = _mean(_ys);
    if (_std(_xs, mx) > maxStdDev || _std(_ys, my) >
maxStdDev) {
      return null;
    }
    return (mx, my);
  }
}
```

Penjelasan Source Code `balance_physics.dart`

1. Identitas File

   Berkas ini berada pada lib/features/balance_game/domain/balance_physics.dart dan mengimplementasikan model konfigurasi, state permainan, fungsi simulasi integrasi fisika kecepatan, dan kalibrasi bias statis sensor gyroscope.

2. Kelas BalanceConfig dan `BalanceState` (baris 23–79)
   - BalanceConfig: Menentukan batasan parameter seperti durasi permainan (60 detik), radius target (0.32), radius bola (0.07), ambang mati (deadzone 0.03 rad/s), faktor pemulusan EMA (0.35), koefisien redaman gesekan (1.6), dan faktor pengali kendali (gain 1.5).

- BalanceState: Struktur data immutable yang menyimpan posisi bola koordinat (x, y), vektor kecepatan (v_x, v_y), kecepatan sudut tersaring (ω_x, ω_y), waktu berlalu (elapsed), durasi di dalam target (secondsInTarget), dan status permainan (playing, won, lost).

3. Simulasi Fisika BalancePhysics.step (baris 83–139)
   - Alih-alih melakukan integrasi sudut kemiringan absolut yang rentan mengalami akumulasi hanyutan (sensor drift accumulation), algoritma menerapkan kendali kecepatan berbasis rotasi sudut langsung.
   - Kecepatan sudut disaring dengan filter zona mati _dead() untuk menekan getaran mikro tangan. Kecepatan bola diperbarui dengan menambahkan dorongan rotasi sudut yang dikalikan penguatan gain, kemudian dikalikan faktor peluruhan redaman `decay`:

     $$v_{x, t} = (v_{x, t-1} + u_y \cdot \text{gain} \cdot \Delta t) \cdot (1 - \text{damping} \cdot \Delta t)$$

     $$x_t = x_{t-1} + v_{x, t} \cdot \Delta t$$
   - Jika jarak total bola dari pusat $\sqrt{x^2 + y^2} + r_{\text{ball}} \ge 1.0$, status permainan seketika berubah menjadi `BalanceStatus.lost.
4. Kelas GyroCalibrator (baris 142–189)
   - Menyediakan fitur kalibrasi titik nol ketika perangkat dipegang diam selama kurang lebih 2 detik sebelum permainan dimulai.
   - Mengumpulkan sampel bacaan gyroscope, menghitung nilai rata-rata (*mean*) dan simpangan baku (*standard deviation*). Jika simpangan baku $\sigma \le 0.12\text{ rad/s}$ (menandakan perangkat benar-benar tenang), rata-rata tersebut dijadikan nilai bias pengurangan (m_x, m_y) untuk meniadakan offset bawaan perangkat keras sensor.
5. Alur Kerja

   Loop animasi Ticker pada BalanceGamePage memanggil step() pada frekuensi 60 FPS ($dt \approx 0.016\text{ detik}$) dengan melewatkan data terkoreksi dari gyroscopeEventStream(). Kanvas

CustomPainter menggambar ulang posisi bola berdasarkan state terbaru.

6. Hubungan dengan File Lain

Berkas ini digunakan oleh `BalanceGameController` dan BalanceGamePage pada fitur mini game, serta berinteraksi dengan LocalDatabase untuk penyimpanan rekor skor.

4.15.3 Perhitungan Skor dan Penyimpanan Riwayat Permainan

Perhitungan skor didesain untuk merefleksikan kualitas kestabilan fokus pengguna:

$$\text{Skor} = \lfloor \text{Durasi di Dalam Target (detik)} \times 10 \rfloor$$

Dengan formula ini, pemain yang mampu menjaga bola tetap berada di dalam lingkaran target selama durasi maksimal 60 detik akan memperoleh skor sempurna 600 poin. Setiap kali sesi permainan berakhir (baik menang maupun kalah), BalanceState.score dievaluasi oleh controller. Jika skor yang diraih melampaui rekor terbaik sebelumnya, sistem memperbarui nilai skor tertinggi (personal best high score) pada box Hive game_scores dan memicu efek partikel visual kemenangan di layar antarmuka.

4.16 Database dan Integrasi API

Arsitektur persistensi data pada aplikasi Planify mengadopsi pola Offline-First Local-First Architecture. Pendekatan ini menjamin bahwa seluruh operasi pencatatan agenda, pengecekan kebiasaan, pengaturan timer fokus, dan kalkulasi riwayat dapat berjalan secara instan dan andal tanpa ketergantungan wajib pada koneksi jaringan internet, sementara sinkronisasi awan (*cloud synchronization*) berjalan secara asinkron di latar belakang saat jaringan tersedia.

4.16.1 Tampilan dan Mekanisme Permainan

Penyimpanan lokal diimplementasikan menggunakan mesin penyimpanan NoSQL berbasis kunci-nilai berkinerja tinggi **Hive**. Seluruh entitas aplikasi dikelompokkan ke dalam kotak-kotak terpisah (boxes) yang didefinisikan secara terpusat pada berkas local_database.dart.

Kode Program 4.16.1.1 Konfigurasi dan Manajemen Penyimpanan Box Hive

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'local_box.dart';

/// Nama box Hive (Tahap 2 §6.2). Terpusat agar tidak ada
salah ketik.
abstract final class BoxNames {
  static const String tasks = 'tasks';
  static const String habits = 'habits';
  static const String habitLogs = 'habit_logs';
  static const String focusSessions = 'focus_sessions';
  static const String deviceSettings = 'device_settings';
  static const String timerState = 'timer_state';
  static const String reminderMap = 'reminder_map';
  static const String gameScores = 'game_scores';
  static const String syncMeta = 'sync_meta';
  static const String syncConflicts = 'sync_conflicts';
  static const String achievements = 'achievements';
  static const String currencyCache = 'currency_cache';
  static const String feedbackOutbox = 'feedback_outbox';
  static const String preferences = 'preferences';

  static const List<String> all = <String>[
    tasks,
    habits,
    habitLogs,
    focusSessions,
    deviceSettings,
    timerState,
    reminderMap,
    gameScores,
    syncMeta,
    syncConflicts,
    achievements,
    currencyCache,
    feedbackOutbox,
    preferences,
  ];

  /// Box berisi data milik akun. Dikosongkan saat keluar
manual atau saat
  /// akun lain masuk di perangkat yang sama (Tahap 2
§5.1/§15).
  static const List<String> userData = <String>[
    tasks,
    habits,
    habitLogs,
    focusSessions,
    timerState,
    reminderMap,
    syncMeta,
    syncConflicts,
    achievements,
    feedbackOutbox,
    preferences,
  ];
}

/// Kumpulan box yang sudah dibuka.
```

```
class LocalDatabase {
  LocalDatabase(Map<String, LocalBox> boxes) : _boxes = boxes;

  /// Database di memori (test dan fallback bila Hive gagal
dibuka).
  factory LocalDatabase.inMemory() => LocalDatabase(<String,
LocalBox>{
        for (final String name in BoxNames.all) name:
MemoryLocalBox(name),
      });

  final Map<String, LocalBox> _boxes;

  LocalBox box(String name) {
    final LocalBox? box = _boxes[name];
    if (box == null) {
      throw StateError('Box "$name" belum dibuka.');
    }
    return box;
  }

  LocalBox get tasks => box(BoxNames.tasks);
  LocalBox get habits => box(BoxNames.habits);
  LocalBox get habitLogs => box(BoxNames.habitLogs);
  LocalBox get focusSessions => box(BoxNames.focusSessions);
  LocalBox get deviceSettings => box(BoxNames.deviceSettings);
  LocalBox get timerState => box(BoxNames.timerState);
  LocalBox get reminderMap => box(BoxNames.reminderMap);
  LocalBox get gameScores => box(BoxNames.gameScores);
  LocalBox get syncMeta => box(BoxNames.syncMeta);
  LocalBox get syncConflicts => box(BoxNames.syncConflicts);
  LocalBox get achievements => box(BoxNames.achievements);
  LocalBox get currencyCache => box(BoxNames.currencyCache);
  LocalBox get feedbackOutbox => box(BoxNames.feedbackOutbox);
  LocalBox get preferences => box(BoxNames.preferences);

  /// Menghapus seluruh data akun di perangkat ini.
  Future<void> clearUserData() async {
    for (final String name in BoxNames.userData) {
      await box(name).clear();
    }
  }
}
```

Penjelasan Source Code `local_database.dart`

1. Identitas File

   Berkas ini berada pada lib/core/storage/local_database.dart dan mendefinisikan konstanta nama-nama box Hive, pembagian kategori data pengguna vs data perangkat, serta antarmuka pengelolaan basis data lokal LocalDatabase.

2. Konstanta `BoxNames` (baris 6–54)
   - Mendefinisikan 14 nama box terpusat untuk mencegah kesalahan penulisan string (typo).

- Daftar userData: Mengidentifikasi 11 box yang memuat data spesifik milik pengguna yang sedang login (seperti agenda tugas, kebiasaan, riwayat fokus, hash chain pencapaian, dan preferensi akun).
- Box non-user data (seperti deviceSettings, currencyCache, dan gameScores) dipertahankan tetap ada pada perangkat meskipun akun melakukan proses logout.

3. Kelas LocalDatabase (baris 57–96)
   - Menyediakan metode pengakses (getter) bertipe kuat untuk masing-masing box (misal .tasks, .habits, .achievements).
   - clearUserData(): Metode pembersihan krusial yang secara otomatis mengosongkan seluruh isi dari 11 box userData saat pengguna keluar akun (logout) atau saat akun lain masuk di perangkat yang sama, mencegah terjadinya kebocoran privasi data lintas akun.
   - LocalDatabase.inMemory(): Konstruktor pabrik (factory constructor) berbasis MemoryLocalBox yang memungkinkan pengujian unit otomatis berjalan secara cepat dan terisolasi tanpa memerlukan dependensi sistem berkas perangkat keras nyata.
4. Alur Kerja

   Saat aplikasi dinyalakan di main.dart, direktori aplikasi lokal diperoleh melalui getApplicationDocumentsDirectory(), seluruh box Hive dibuka, dan instance LocalDatabase diinjeksikan ke dalam container Riverpod melalui localDatabaseProvider.
5. Hubungan dengan File Lain

   Berkas ini dibungkus oleh LocalBox dan menjadi pilar fondasi penyimpanan lokal bagi seluruh repositori data: TaskRepository, HabitRepository, FocusRepository, AchievementRepository, dan SyncEngine.

4.16.2 Penyimpanan Awan dan Sinkronisasi Data Supabase

Untuk menjamin ketersediaan data lintas perangkat (multi-device synchronization) dan pencadangan awan (cloud backup), Planify mengintegrasikan platform Backend-as-a-Service **Supabase** berbasis

PostgreSQL. Protokol sinkronisasi dua arah yang mendukung penanganan konflik (conflict resolution) diimplementasikan pada berkas sync_entity.dart.

Kode Program 4.16.2.1 Metadata dan Logika Sinkronisasi Data Offline-First

```
import 'package:flutter/foundation.dart';

import '../storage/local_box.dart';
import '../utils/date_only.dart';

/// Status sinkronisasi per record (Tahap 2 §6.1).
enum SyncStatus {
  synced,
  pending,

  /// Ditolak server (validasi/RLS). Tidak dicoba ulang
otomatis.
  failed;

  static SyncStatus parse(Object? value) =>
SyncStatus.values.firstWhere(
        (SyncStatus s) => s.name == value,
        orElse: () => SyncStatus.synced,
      );
}

/// Metadata sinkronisasi yang hanya ada di perangkat.
@immutable
class SyncInfo {
  const SyncInfo({
    this.serverVersion,
    this.status = SyncStatus.pending,
    this.error,
  });

  /// Versi terakhir yang diketahui dari server; `null` =
belum pernah sinkron.
  final int? serverVersion;
  final SyncStatus status;

  /// Kode error singkat (bukan pesan teknis mentah).
  final String? error;

  static SyncInfo fromJson(Map<String, dynamic> json) {
    final Object? version = json['version'];
    return SyncInfo(
      serverVersion: version is num ? version.toInt() : null,
      status: SyncStatus.parse(json['sync_status']),
      error: json['sync_error'] as String?,
    );
  }

  Map<String, dynamic> toJson() => <String, dynamic>{
        'version': serverVersion,
        'sync_status': status.name,
        'sync_error': error,
      };

  @override
  bool operator ==(Object other) =>
```

```
      other is SyncInfo &&
      other.serverVersion == serverVersion &&
      other.status == status &&
      other.error == error;

  @override
  int get hashCode => Object.hash(serverVersion, status,
error);
}

/// Entitas yang disinkronkan. `toRow()` berisi kolom server
tanpa `version`
/// dan `updated_at` (keduanya diisi trigger `tg_touch_row` di
server).
abstract interface class SyncEntity<T> {
  /// Key di box Hive (UUID, atau komposit untuk habit_logs).
  String get localKey;
  SyncInfo get syncInfo;
  DateTime get updatedAt;
  DateTime? get deletedAt;

  Map<String, dynamic> toRow();

  T withSync(SyncInfo info, {DateTime? updatedAt});
}

/// Tabel server yang disinkronkan beserta box lokalnya.
class SyncTableSpec {
  const SyncTableSpec({
    required this.table,
    required this.boxName,
    required this.pkColumns,
  });

  final String table;
  final String boxName;

  /// Kolom primary key di server (filter UPDATE/SELECT dan
`on conflict`).
  final List<String> pkColumns;

  String get onConflict => pkColumns.join(',');

  Map<String, Object> pkOf(Map<String, dynamic> row) =>
<String, Object>{
        for (final String c in pkColumns) c: row[c] as Object,
      };

  /// Key box dari baris: kolom PK digabung dengan `|`
  /// (habit_logs → `"$habitId|yyyy-MM-dd"`, Tahap 2 §6.2).
  String keyOf(Map<String, dynamic> row) =>
      pkColumns.map((String c) =>
row[c].toString()).join('|');
}

/// Akses box untuk satu jenis entitas.
class EntityStore<T extends SyncEntity<T>> {
  EntityStore(this.box, this.spec, this.fromJson, {DateTime
Function()? clock})
      : _clock = clock ?? DateTime.now;
```

```
  final LocalBox box;
  final SyncTableSpec spec;

  /// Membaca JSON lokal maupun baris server (`sync_status`
kosong = synced).
  final T Function(Map<String, dynamic> json) fromJson;
  final DateTime Function() _clock;

  static Map<String, dynamic> toLocalJson<T extends
SyncEntity<T>>(T e) =>
      <String, dynamic>{
        ...e.toRow(),
        'updated_at': e.updatedAt.toUtc().toIso8601String(),
        ...e.syncInfo.toJson(),
      };

  T? _decode(Map<String, dynamic>? json) {
    if (json == null) {
      return null;
    }
    try {
      return fromJson(json);
    } catch (error) {
      debugPrint('Record ${spec.table} rusak diabaikan:
$error');
      return null;
    }
  }

  T? get(String key) => _decode(box.getJson(key));

  /// Semua record, termasuk tombstone bila [includeDeleted].
  List<T> all({bool includeDeleted = false}) {
    final List<T> out = <T>[];
    for (final String key in box.keys.toList()) {
      final T? e = get(key);
      if (e != null && (includeDeleted || e.deletedAt ==
null)) {
        out.add(e);
      }
    }
    return out;
  }

  /// Daftar yang diperbarui setiap box berubah (Tahap 2 §5.1:
`watch()`).
  Stream<List<T>> watch() async* {
    yield all();
    await for (final _ in box.changes) {
      yield all();
    }
  }

  Iterable<T> withStatus(SyncStatus status) =>
all(includeDeleted: true)
      .where((T e) => e.syncInfo.status == status);

  /// Menulis perubahan lokal: status `pending`, `updated_at`
= sekarang.
```

```
  Future<T> saveLocal(T entity) async {
    final T marked = entity.withSync(
      SyncInfo(serverVersion: entity.syncInfo.serverVersion),
      updatedAt: _clock().toUtc(),
    );
    await box.putJson(marked.localKey, toLocalJson(marked));
    return marked;
  }

  /// Menulis apa adanya (hasil dari server atau penandaan
status).
  Future<void> put(T entity) =>
      box.putJson(entity.localKey, toLocalJson(entity));

  /// Penanda waktu UTC sekarang (dipakai repository untuk
created_at dll).
  DateTime nowUtc() => _clock().toUtc();
}
```

Penjelasan Source Code sync_entity.dart

1. Identitas File

   Berkas ini berada pada lib/core/sync/sync_entity.dart dan mendefinisikan antarmuka kontrak entitas data tersinkronisasi, metadata status sinkronisasi, spesifikasi pemetaan tabel basis data Supabase, serta pembungkus EntityStore untuk persistensi reaktif.

2. Metadata SyncStatus dan SyncInfo (baris 7–60)
   - SyncStatus: Menyatakan status siklus hidup sinkronisasi rekaman: synced (rekaman identik dengan server), pending (terdapat perubahan lokal yang menunggu diunggah), dan failed (ditolak oleh aturan server/RLS).
   - SyncInfo: Menyimpan nomor versi server terakhir yang diketahui (serverVersion), status terkini, dan kode pesan galat jika proses sinkronisasi mengalami penolakan.

3. Antarmuka SyncEntity<T> (baris 64–74)

- Mewajibkan setiap model data yang mendukung sinkronisasi (seperti Task, Habit, HabitLog, FocusSession` mengimplementasikan properti pengenal lokal `localKey`, informasi `syncInfo`, stempel waktu `updatedAt`, penanda hapus lunak (*soft delete tombstone*) `deletedAt`, metode pemetaan baris server `toRow()`, serta penyalinan pembaruan `withSync()`.

4. Kelas EntityStore<T> (baris 103–174)

- Menjembatani antara operasi domain dengan penyimpanan lokal Hive box.
- saveLocal: Setiap kali pengguna menambah atau mengedit entitas di aplikasi, metode ini menandai SyncStatus.pending, memperbarui updatedAt ke waktu UTC saat ini, dan menulis JSON ke Hive secara sinkron.
- watch: Memanfaatkan box.changes untuk mengembalikan aliran data reaktif Stream<List<T>>, sehingga antarmuka pengguna diperbarui secara seketika (instant UI reactivity) tanpa perlu menunggu respons jaringan internet.

5. Alur Kerja Sinkronisasi Dua Arah (Bidirectional Sync)
   - Fase Tarik (Pull Phase): SyncEngine mengirimkan kueri ke server Supabase untuk mengambil seluruh rekaman yang memiliki nomor versi lebih besar dari last_pulled_version. Jika rekaman lokal berstatus synced, rekaman langsung ditimpa dengan versi server. Jika rekaman lokal berstatus pending, algoritma Last-Write-Wins (LWW) membandingkan nilai updated_at untuk menentukan pemenang konflik.
   - Fase Dorong (Push Phase): Seluruh entitas berstatus pending dikumpulkan dan dikirim ke endpoint REST Supabase menggunakan operasi `UPSERT` dengan klausa ON CONFLICT (id) DO UPDATE. Trigger basis data PostgreSQL tg_touch_row di server secara otomatis menaikkan kolom version dan memperbarui updated_at. Respons yang berhasil akan mengembalikan status lokal menjadi SyncStatus.synced.
6. Hubungan dengan File Lain

   Berkas ini menjadi kontrak arsitektur bagi model Task, Habit, HabitLog, dan digunakan secara intensif oleh SyncEngine untuk orkestrasi sinkronisasi data awan.

4.16.3 Integrasi API Eksternal

Aplikasi Planify mengintegrasikan lima layanan antarmuka pemrograman aplikasi (API) eksternal untuk memperkaya kapabilitas fungsional:

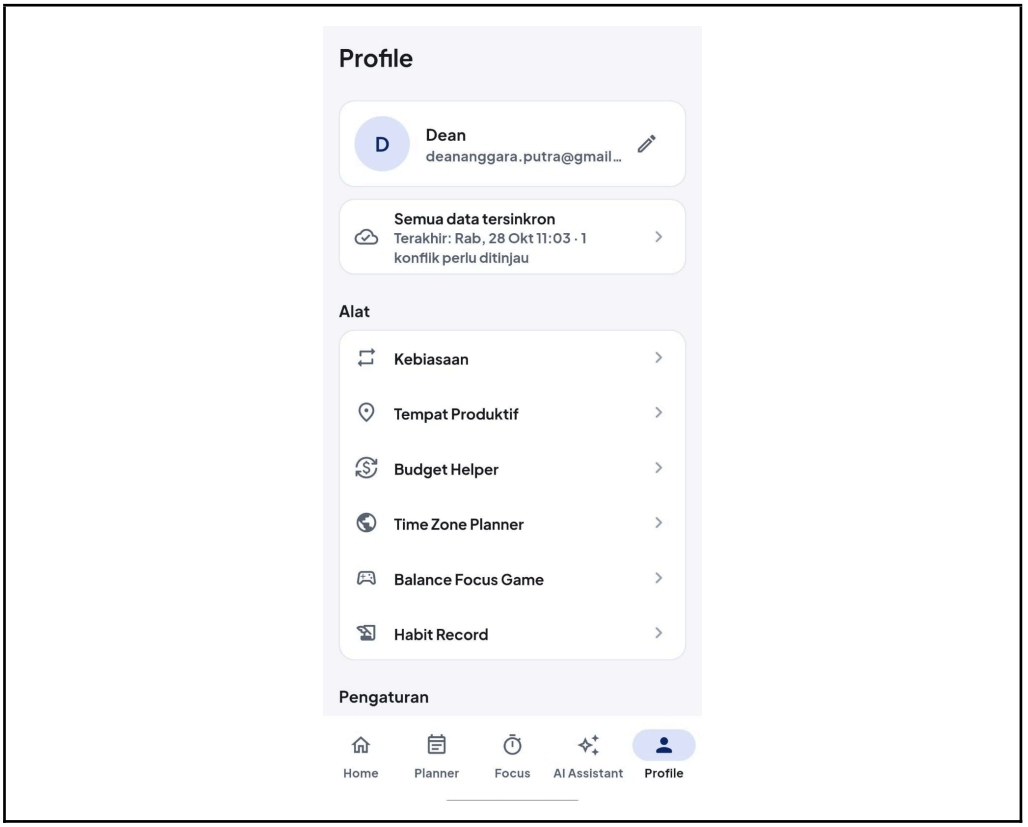
1. Supabase Cloud API:
    - Endpoint: `https://<project-ref>.supabase.co/rest/v1`
    - Protokol: HTTPS RESTful API dan WebSocket Realtime.
    - Fungsi: Mengelola autentikasi berbasis token JWT, operasi CRUD basis data relasional PostgreSQL dengan proteksi *Row Level Security* (RLS), dan mekanisme pemulihan kata sandi email.
2. Google Gemini Generative AI API:
    - Model: `gemini-1.5-flash`
    - Protokol: HTTPS POST JSON payload.
    - Fungsi: Menghasilkan analisis jadwal cerdas, dekomposisi subtugas, dan rekomendasi agenda adaptif dengan pembatasan skema JSON baku (structured JSON output schema).
3. Geoapify Places API v2:
    - Endpoint: `https://api.geoapify.com/v2/places`
    - Protokol: HTTPS GET dengan autentikasi API Key.
    - Fungsi: Menyediakan pangkalan data tempat produktif (perpustakaan, kafe, coworking space) berdasarkan titik koordinat geografis lintang/bujur dan radius pencarian.
4. OpenStreetMap Overpass API:
    - Endpoint: `https://overpass-api.de/api/interpreter`
    - Protokol: HTTPS POST dengan bahasa kueri Overpass QL.
    - Fungsi: Penyedia data spasial cadangan luring/publik tanpa batasan kuota komersial untuk mencari fasilitas belajar di sekitar pengguna.
5. Frankfurter Exchange Rate API:
    - Endpoint: `https://api.frankfurter.app/latest`
    - Protokol: HTTPS GET publik berbasis data harian European Central Bank (ECB).
    - Fungsi: Mengambil data nilai tukar mata uang terkini untuk konversi Rupiah (IDR), Dolar AS (USD), Ringgit Malaysia (MYR), dan Dolar Singapura (SGD) pada fitur Budget Helper.

4.17 Halaman Profil dan Pengaturan Pengguna

Halaman Profil dan Pengaturan merupakan pusat kendali preferensi personalisasi, ringkasan capaian produktivitas, serta manajemen keamanan akun pada aplikasi Planify.

4.17.1 Tampilan Antarmuka dan Komponen Profil

Antarmuka halaman Profil dirancang dengan tata letak kartu (card-based layout) yang bersih dan modern.



Gambar 4.17.1.1 Tampilan Halaman Profil Pengguna

Komponen-komponen utama pada antarmuka profil mencakup:

1. Header Akun Pengguna

   Menampilkan lingkaran avatar pengguna berisikan inisial huruf nama/email, alamat surat elektronik terdaftar, serta lencana status konektivitas akun (Online / Offline Sync Status).

2. Kartu Statistik Produktivitas (Productivity Metrics Card)

   Merangkum pencapaian akumulatif pengguna, mencakup total tugas yang telah diselesaikan di Planner, jumlah hari beruntun (best streak) pembentukan habit, serta akumulasi jam fokus yang berhasil diselesaikan via timer Pomodoro.

3. Menu Navigasi Pengaturan

   Menyediakan akses cepat menuju pengaturan preferensi notifikasi, riwayat konsistensi habit (Habit Consistency Record), alat bantu anggaran (Budget Helper), perencanaan zona waktu (Time Zone Planner), dan pengaturan keamanan (Security Settings).

4. Tombol Keluar (Logout Button)

   Ditempatkan di bagian bawah halaman dengan warna aksen peringatan, dilengkapi dialog konfirmasi pencegahan ketidaksengajaan keluar akun.

4.17.2 Pengelolaan Akun dan Preferensi Tampilan

Pengguna diberikan keleluasaan untuk menyesuaikan konfigurasi aplikasi sesuai preferensi pribadi:

1. Preferensi Tema (Theme Mode)

   Aplikasi mendukung tiga mode tampilan visual: Tema Terang (Light Theme), Tema Gelap (Dark Theme), dan Mengikuti Sistem (System Default). Pengaturan tema disimpan ke dalam box Hive preferences dan diterapkan secara instan menggunakan ThemeMode bawaan Flutter tanpa memerlukan restart aplikasi.

2. Durasi Standar Pomodoro

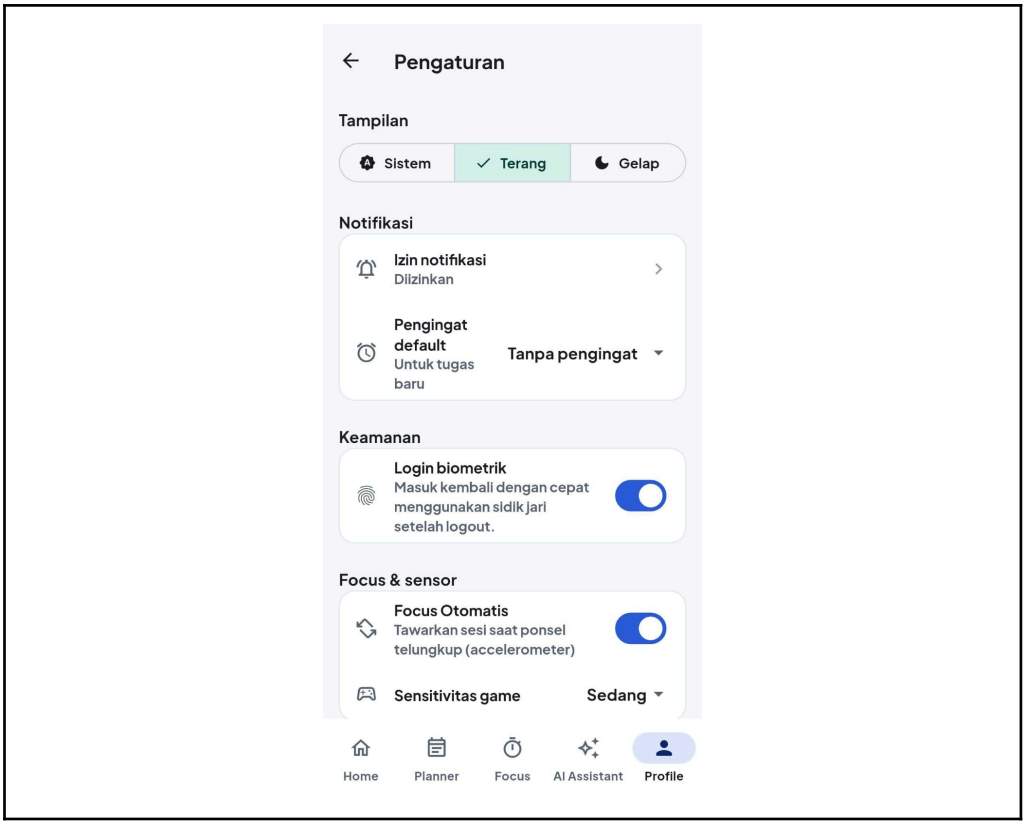
   Pengguna dapat mengonfigurasi durasi baku sesi fokus (pilihan: 25 menit atau 50 menit) dan durasi istirahat pendek (pilihan: 5 menit atau 10 menit), yang akan menjadi nilai default setiap kali timer Pomodoro dibuka.

3. Pengelolaan Sesi Akun

   Melalui SessionController, pengguna dapat memperbarui informasi nama tampilan atau memicu pengiriman email pembaruan kata sandi akun Supabase.

4.17.3 Pengaturan Keamanan dan Fitur Biometrik

Untuk melindungi kerahasiaan data produktivitas pengguna, aplikasi Planify menyediakan sistem otentikasi biometrik lokal (sidik jari atau pemindai wajah) yang terintegrasi dengan brankas kunci terenkripsi perangkat keras (hardware-backed keystore).



Gambar 4.17.3.1 Tampilan Pengaturan Keamanan dan Preferensi

Penyimpanan token sesi dan kredensial terenkripsi dikelola melalui abstraksi SecureKeyValueStore pada berkas secure_key_value_store.dart.

Kode Program 4.17.3.1 Penyimpanan Kunci Terenkripsi

```
import
'package:flutter_secure_storage/flutter_secure_storage.dart';

/// Penyimpanan kunci-nilai terenkripsi yang kecil. Dipisah
sebagai antarmuka
/// agar test tidak membutuhkan platform channel.
abstract interface class SecureKeyValueStore {
  Future<String?> read(String key);
  Future<void> write(String key, String value);
  Future<void> delete(String key);
}

/// Implementasi nyata: Android Keystore
(EncryptedSharedPreferences) / iOS
/// Keychain / WebCrypto di Web.
class FlutterSecureKeyValueStore implements
SecureKeyValueStore {
  FlutterSecureKeyValueStore([FlutterSecureStorage? storage])
      : _storage = storage ?? const FlutterSecureStorage();

  final FlutterSecureStorage _storage;

  @override
  Future<String?> read(String key) => _storage.read(key: key);
```

```
  @override
  Future<void> write(String key, String value) =>
      _storage.write(key: key, value: value);

  @override
  Future<void> delete(String key) => _storage.delete(key:
key);
}

/// Implementasi di memori untuk test.
class InMemoryKeyValueStore implements SecureKeyValueStore {
  InMemoryKeyValueStore([Map<String, String>? initial])
      : values = <String, String>{...?initial};

  final Map<String, String> values;

  @override
  Future<String?> read(String key) async => values[key];

  @override
  Future<void> write(String key, String value) async {
    values[key] = value;
  }

  @override
  Future<void> delete(String key) async {
    values.remove(key);
  }
}

/// Nama kunci penyimpanan aman.
abstract final class SecureKeys {
  /// JSON sesi Supabase.
  static const String supabaseSession =
'planify.supabase.session';

  /// ID pengguna yang mengaktifkan kunci biometrik di
perangkat ini.
  static const String biometricOwner =
'planify.biometric.owner';

  /// Email pengguna yang mengaktifkan login biometrik (untuk
tampilan pada halaman login).
  static const String biometricEmail =
'planify.biometric.email';

  /// Refresh token terenkripsi untuk memulihkan sesi
autentikasi via biometrik setelah logout.
  static const String biometricToken =
'planify.biometric.token';

  /// Kredensial autentikasi terenkripsi (Android Keystore /
iOS Keychain)
  /// yang dilindungi oleh verifikasi biometrik lokal untuk
login kembali setelah logout.
  static const String biometricSecret =
'planify.biometric.secret';
}
```

Penjelasan Source Code secure_key_value_store.dart

1. Identitas File

   Berkas ini berada pada lib/core/storage/secure_key_value_store.dart dan mendefinisikan antarmuka abstraksi penyimpanan aman terenkripsi, implementasi berbasis flutter_secure_storage, implementasi tiruan di memori untuk pengujian, serta konstanta nama kunci brankas.

2. Antarmuka SecureKeyValueStore (baris 5–9)

   Mendeklarasikan operasi dasar asinkron read(key), write(key, value), dan delete(key). Pemisahan antarmuka ini memastikan bahwa logika autentikasi dapat diuji secara mandiri tanpa memerlukan platform channel perangkat nyata.

3. Kelas FlutterSecureKeyValueStore (baris 13–28)

   Memanfaatkan pustaka flutter_secure_storage yang memanfaatkan Android Keystore (didukung EncryptedSharedPreferences dengan algoritma AES-256 GCM) pada sistem operasi Android, serta iOS Keychain Services pada platform iOS.

4. Daftar Kunci Brankas SecureKeys (baris 52–68)

   - supabaseSession: Menyimpan blob JSON sesi autentikasi Supabase.
   - biometricOwner: Menyimpan ID pengguna pemilik hak akses biometrik di perangkat.
   - biometricEmail: Menyimpan alamat surel pengguna untuk ditampilkan pada antarmuka halaman login saat menawarkan opsi masuk biometrik.
   - biometricToken & biometricSecret: Kredensial penyegaran terenkripsi tingkat tinggi yang dilindungi verifikasi biometrik lokal untuk memulihkan sesi autentikasi pengguna secara aman setelah keluar akun tanpa mengharuskan input ulang kata sandi.

5. Alur Kerja

   Saat pengguna mengaktifkan toggle biometrik di halaman profil, sistem memanggil BiometricAuthenticator.authenticate(). Jika berhasil, token sesi pengguna diamankan ke dalam SecureKeys.biometricSecret. Ketika pengguna membuka aplikasi kembali atau hendak login ulang,

sidik jari diverifikasi dan token diuraikan kembali untuk merekonstruksi sesi login Supabase secara otomatis.

6. Hubungan dengan File Lain

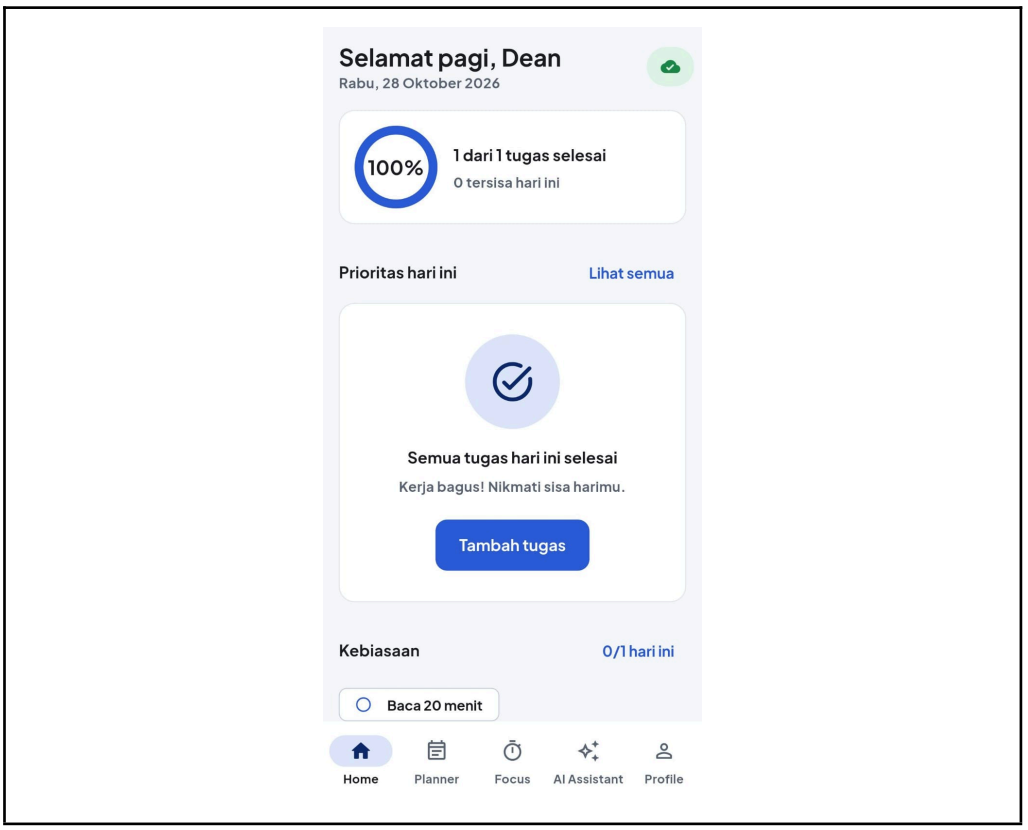
Berkas ini digunakan oleh SupabaseAuthRepository, BiometricAuthenticator, dan SessionController.

4.18 Navigasi dan Integrasi Antarmuka Aplikasi

Arsitektur navigasi pada aplikasi Planify dibangun di atas pustaka perutean deklaratif **GoRouter** yang dipadukan dengan struktur navigasi bercabang (Stateful Shell Navigation). Pola ini memungkinkan setiap tab utama mempertahankan posisi riwayat penjelajahan (scroll position dan back stack) masing-masing secara independen.

4.18.1 Struktur Navigasi Utama Aplikasi

Kerangka dasar antarmuka pengguna setelah proses masuk (login) diwadahi oleh komponen shell utama pada berkas main_shell.dart.



Gambar 4.18.1.1 Tampilan Navigasi Utama (*Bottom Navigation Bar*)

Kode Program 4.18.1.1 Kerangka Shell Navigasi Utama

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:go_router/go_router.dart';

/// C-23 — Scaffold dengan bottom navigation lima tab (Tahap 3
§3.1).
///
/// Aturan tombol back (Tahap 3 §3.4):
/// - halaman detail di dalam tab di-pop oleh go_router
sebelum sampai ke sini;
/// - di root tab selain Home → pindah ke Home;
/// - di Home → snackbar "Tekan sekali lagi untuk keluar",
lalu keluar bila
///   ditekan lagi dalam 2 detik.
class MainShell extends StatefulWidget {
  const MainShell({super.key, required this.navigationShell});

  final StatefulNavigationShell navigationShell;

  /// Urutan tab tidak boleh diubah: Home, Planner, Focus, AI
Assistant, Profile.
  static const List<NavigationDestination> destinations =
      <NavigationDestination>[
    NavigationDestination(
      icon: Icon(Icons.home_outlined),
      selectedIcon: Icon(Icons.home_rounded),
      label: 'Home',
    ),
    NavigationDestination(
      icon: Icon(Icons.event_note_outlined),
      selectedIcon: Icon(Icons.event_note_rounded),
      label: 'Planner',
    ),
    NavigationDestination(
      icon: Icon(Icons.timer_outlined),
      selectedIcon: Icon(Icons.timer_rounded),
      label: 'Focus',
    ),
    NavigationDestination(
      icon: Icon(Icons.auto_awesome_outlined),
      selectedIcon: Icon(Icons.auto_awesome_rounded),
      label: 'AI Assistant',
    ),
    NavigationDestination(
      icon: Icon(Icons.person_outline_rounded),
      selectedIcon: Icon(Icons.person_rounded),
      label: 'Profile',
    ),
  ];

  static const Duration exitWindow = Duration(seconds: 2);

  @override
  State<MainShell> createState() => _MainShellState();
}
```

```
class _MainShellState extends State<MainShell> {
  DateTime? _lastBackPressedAt;

  void _onDestinationSelected(int index) {
    // Mengetuk tab yang sedang aktif mengembalikan tab ke
root-nya.
    widget.navigationShell.goBranch(
      index,
      initialLocation: index ==
widget.navigationShell.currentIndex,
    );
  }

  void _handleBack(bool didPop, Object? result) {
    if (didPop) {
      return;
    }
    if (widget.navigationShell.currentIndex != 0) {
      widget.navigationShell.goBranch(0);
      return;
    }
    final DateTime now = DateTime.now();
    final DateTime? last = _lastBackPressedAt;
    if (last != null && now.difference(last) <
MainShell.exitWindow) {
      SystemNavigator.pop();
      return;
    }
    _lastBackPressedAt = now;
    ScaffoldMessenger.of(context)
      ..hideCurrentSnackBar()
      ..showSnackBar(
        const SnackBar(
          content: Text('Tekan sekali lagi untuk keluar'),
          duration: MainShell.exitWindow,
        ),
      );
  }

  @override
  Widget build(BuildContext context) {
    return PopScope<Object?>(
      canPop: false,
      onPopInvokedWithResult: _handleBack,
      child: Scaffold(
        body: widget.navigationShell,
        bottomNavigationBar: NavigationBar(
          selectedIndex: widget.navigationShell.currentIndex,
          onDestinationSelected: _onDestinationSelected,
          destinations: MainShell.destinations,
        ),
      ),
    );
  }
}
```

Penjelasan Source Code main_shell.dart

1. Identitas File

Berkas ini berada pada lib/app/shell/main_shell.dart dan mengimplementasikan kerangka tata letak utama aplikasi (root layout scaffold) yang menyatukan lima cabang navigasi utama dan mengendalikan logika tombol kembali sistem operasi (hardware back button handler).

2. Destinasi Navigasi (baris 18–45)
    - Menetapkan urutan lima tab utama yang tidak boleh diubah: Home (indeks 0), Planner (indeks 1), Focus (indeks 2), AI Assistant (indeks 3), dan Profile (indeks 4).
    - Setiap destinasi memiliki pasangan ikon bergaris (outlined icon) saat tidak aktif dan ikon solid bulat (rounded selected icon) saat aktif, sesuai pedoman Material Design 3.
3. Logika Tombol Kembali Sistem _handleBack (baris 64–87)
    - Dibungkus dalam widget PopScope dengan canPop: false untuk mencegat tombol back Android.
    - Aturan 1: Jika pengguna berada di halaman sub-detail di dalam tab tertentu, go_router akan me-pop sub-halaman terlebih dahulu.
    - Aturan 2: Jika pengguna berada di akar (root) tab selain Home (indeks 1, 2, 3, atau 4), menekan tombol kembali tidak akan menutup aplikasi, melainkan memindahkan fokus pengguna kembali ke tab Home (indeks 0) melalui goBranch(0).
    - Aturan 3: Jika pengguna berada di tab Home dan menekan tombol kembali, sistem memunculkan bilah pesan (SnackBar) bertuliskan "Tekan sekali lagi untuk keluar" berdurasi 2 detik. Jika ditekan kembali dalam jendela waktu 2 detik (now.difference(last) < exitWindow), aplikasi mengeksekusi SystemNavigator.pop() untuk menutup aplikasi secara aman.
4. Metode _onDestinationSelected (baris 56–62)

    Memanggil widget.navigationShell.goBranch(). Jika pengguna mengetuk tab yang sedang aktif, parameter initialLocation: true secara otomatis mengembalikan tampilan ke halaman teratas (pop-to-root) pada tab tersebut.
5. Alur Kerja

Komponen ini diinisialisasi oleh StatefulShellRoute.indexedStack di dalam konfigurasi router. Setiap kali pengguna beralih tab, StatefulNavigationShell memelihara status pohon widget masing-masing tab tetap hidup di memori (state retention).

6. Hubungan dengan File Lain

Berkas ini bertindak sebagai induk bagi seluruh halaman utama: HomePage, PlannerPage, FocusPage, AiAssistantPage, dan ProfilePage.

4.18.2 Penanganan Rute dan Redirection Berdasarkan Status Autentikasi

Keamanan navigasi aplikasi dikawal oleh fungsi pelindung rute (route guard) murni resolveAuthRedirect pada berkas auth_redirect.dart.

Kode Program 4.18.2.1 Logika Pengalihan Rute Berbasis Status Sesi

```
import '../../features/auth/application/session_state.dart';
import 'routes.dart';

/// Guard autentikasi (FR-047) sebagai fungsi murni agar mudah
diuji.
///
/// Mengembalikan path tujuan bila [uri] harus dialihkan, atau
`null` bila
/// boleh dibuka.
///
/// Aturan:
/// 1. Sebelum sesi siap → tahan di `/splash`.
/// 2. Belum masuk → hanya rute publik; selain itu ke
`/login`. Rute
///    `/reset-password` tidak dapat dibuka tanpa mode
pemulihan.
/// 3. Mode pemulihan (tautan email) → paksa ke
`/reset-password`.
/// 4. Sudah masuk tetapi terkunci biometrik → paksa ke
`/lock`.
/// 5. Sudah masuk → halaman auth/splash dialihkan ke `/home`.
///
/// Tautan dalam (deep link) autentikasi dari email tiba
sebagai path `/`
/// (skema + host tanpa path); itu ditangani seperti membuka
aplikasi biasa.
String? resolveAuthRedirect(SessionState session, Uri uri) {
  final String path = uri.path.isEmpty ? '/' : uri.path;

  if (!session.isReady) {
    return path == AppRoutes.splash ? null : AppRoutes.splash;
  }

  switch (session.phase) {
    case SessionPhase.initializing:
```

```
      return null; // tidak tercapai: ditangani di atas.

    case SessionPhase.unauthenticated:
      final bool isPublic =
AppRoutes.publicRoutes.contains(path) &&
          path != AppRoutes.resetPassword &&
          path != AppRoutes.splash;
      return isPublic ? null : AppRoutes.login;

    case SessionPhase.passwordRecovery:
      return path == AppRoutes.resetPassword ? null :
AppRoutes.resetPassword;

    case SessionPhase.authenticated:
      if (session.isLocked) {
        return path == AppRoutes.lock ? null : AppRoutes.lock;
      }
      const Set<String> authOnly = <String>{
        '/',
        AppRoutes.splash,
        AppRoutes.login,
        AppRoutes.register,
        AppRoutes.forgotPassword,
        AppRoutes.resetPassword,
        AppRoutes.lock,
      };
      return authOnly.contains(path) ? AppRoutes.home : null;
  }
}
```

Penjelasan Source Code auth_redirect.dart

1. Identitas File

   Berkas ini berada pada lib/app/router/auth_redirect.dart dan mengimplementasikan fungsi pengalihan rute murni (pure redirection guard function) yang menentukan apakah suatu permintaan URL sah dibuka atau harus dialihkan berdasarkan fase sesi pengguna.

2. Evaluasi Status Sesi SessionState (baris 19–54)
   - Pengecekan Kesiapan (!session.isReady): Sebelum proses pengecekan token tersimpan selesai dibaca dari brankas aman, seluruh permintaan rute ditahan pada layar pemuatan /splash.
   - Fase unauthenticated (baris 30–34): Pengguna yang belum terautentikasi hanya diizinkan mengakses rute-rute publik yang terdaftar di AppRoutes.publicRoutes (seperti /login, /register, /forgot-password). Jika pengguna mencoba mengakses rute terproteksi (seperti /home atau /planner), fungsi secara deterministik mengembalikan rute pengalihan /login.

- Fase passwordRecovery (baris 36–37): Jika aplikasi dibuka dari tautan konfirmasi email pemulihan kata sandi Supabase (deep link), pengguna dipaksa untuk langsung menuju halaman perubahan kata sandi /reset-password.
- Fase authenticated (baris 39–53): Jika akun telah terautentikasi:
    - Apabila opsi proteksi biometrik aktif dan aplikasi dalam kondisi terkunci (session.isLocked == true), pengguna dialihkan ke rute layar kunci /lock.
    - Apabila pengguna yang sudah login mencoba membuka kembali halaman publik autentikasi (seperti /login, /register, atau /splash), sistem secara otomatis mengalihkan pengguna ke halaman beranda /home.
    - Rute lainnya dikembalikan sebagai null, menandakan perutean diizinkan untuk dilanjutkan menuju halaman target.

3. Alur Kerja

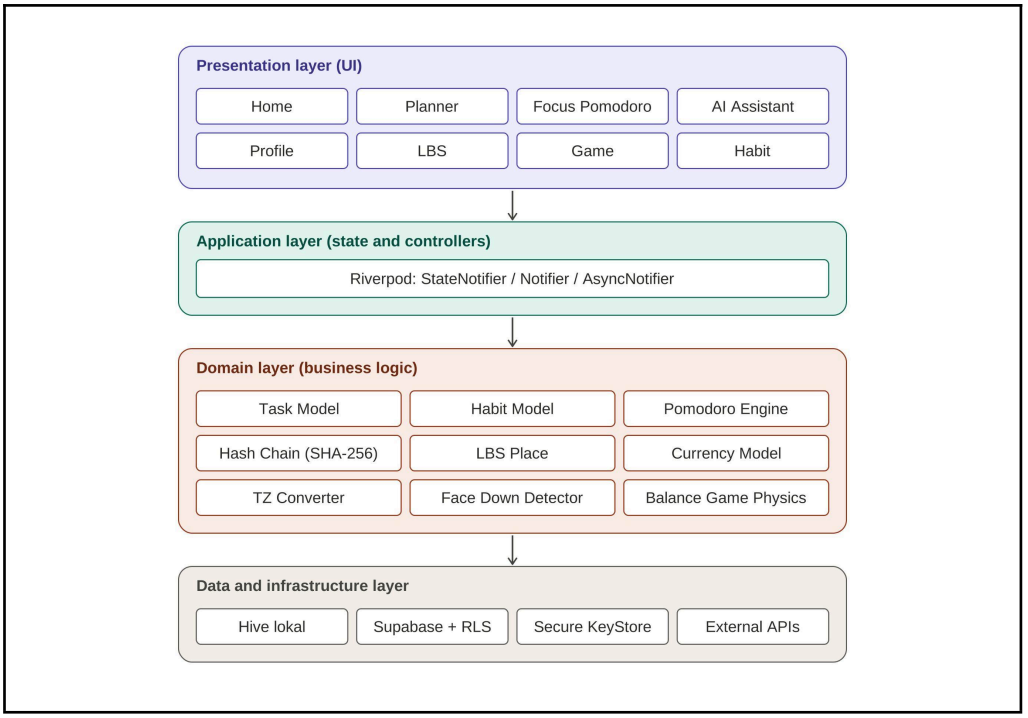
    Fungsi ini dipasang pada parameter redirect dari instansiasi GoRouter. Setiap kali terjadi perubahan pada sessionControllerProvider, GoRouter mengevaluasi rute aktif terhadap hasil dari resolveAuthRedirect(session, uri) dan melakukan navigasi reaktif secara atomik.

4. Hubungan dengan File Lain

    Berkas ini bergantung pada SessionState dari fitur autentikasi dan dipanggil langsung oleh konfigurasi rute di app_router.dart.

4.18.3 Integrasi Keseluruhan Fitur dalam Arsitektur Aplikasi

Seluruh fitur yang telah diimplementasikan pada aplikasi Planify beroperasi sebagai satu kesatuan sistem yang harmonis dan terintegrasi melalui pola arsitektur berlapis (Feature-Driven Layered Architecture):



Gambar 4.18.3.1 *Planify Arsitektur*

Keterkaitan sinergis antar fitur terwujud dalam beberapa skenario operasional terpadu:

1. Sinergi Agenda dan Fokus: Pengguna dapat memilih salah satu agenda kerja di modul Planner, lalu langsung menekan tombol "Fokus pada Tugas Ini". Sistem secara otomatis mengikat tugas tersebut ke dalam PomodoroEngine, mencatat waktu pengerjaan aktual, dan menambahkan riwayat sesi ke box focus_sessions.
2. Sinergi Pembentukan Kebiasaan dan Validasi Kriptografis: Saat kebiasaan di Habit Tracker ditandai selesai pada hari berjalan, data tidak hanya memperbarui skor streak, tetapi secara otomatis memicu pembuatan blok hash kriptografis SHA-256 pada fitur Habit Consistency Record, menghubungkan stempel waktu pencapaian dengan hash blok sebelumnya secara tamper-evident.
3. Sinergi Agenda, Anggaran, dan Zona Waktu: Dalam merencanakan kegiatan tugas di Planner, pengguna dapat membuka Time Zone Planner untuk menentukan jadwal rapat daring lintas negara (WIB/WITA/WIT/London) dengan proteksi anomali DST, sekaligus membuka Budget Helper untuk mengalokasikan estimasi biaya

operasional dalam berbagai mata uang asing (IDR, USD, MYR, SGD) yang secara otomatis dikonversi menggunakan kurs riil.

4. Sinergi Sensor Perangkat Keras dan Kesejahteraan Pengguna: Sensor accelerometer mendeteksi gestur meletakkan ponsel secara telungkup untuk memicu tawaran sesi fokus secara hening tanpa repot, sementara sensor gyroscope dimanfaatkan saat jeda istirahat Pomodoro untuk permainan relaksasi motorik halus Balance Focus Game.
5. Sinergi Lokasi dan Produktivitas: Pengguna yang membutuhkan ruang kerja di luar rumah dapat memanfaatkan LBS Navigator untuk mencari perpustakaan, kafe, atau coworking space terdekat berdasarkan koordinat GPS, menghitung jarak Haversine, menampilkan lokasi pada peta OpenStreetMap, dan membuka panduan navigasi turn-by-turn di Google Maps.
6. Sinergi Notifikasi dan Sinkronisasi Awan: Layanan notifikasi lokal menjadwalkan pengingat tenggat tugas secara luring pada sistem operasi tanpa bergantung pada server luar, sementara SyncEngine secara transparan menyinkronkan seluruh perubahan data lokal Hive ke basis data awan Supabase dengan proteksi keamanan Row Level Security (RLS).

BAB V
LEMBAR EVALUASI PENGUJIAN PERANGKAT

Bab ini memaparkan proses evaluasi dan pengujian perangkat lunak pada aplikasi mobile PLANIFY (Smart Daily Planner & Productivity Assistant). Pengujian dilakukan untuk menilai apakah seluruh modul fungsional, antarmuka pengguna, tata kelola penyimpanan data lokal, sinkronisasi awan, serta integrasi sensor perangkat keras telah beroperasi secara andal dan memenuhi spesifikasi kebutuhan yang telah dirancang. Pembahasan pada bab ini mencakup tujuan pengujian, lingkungan pengujian, metodologi yang diterapkan, rincian skenario dan hasil uji pengujian kotak hitam (Black Box Testing), rekapitulasi data pengujian, serta evaluasi komprehensif dan kesimpulan kelayakan sistem.

5.1 Tujuan Pengujian

Pengujian perangkat lunak pada aplikasi Planify bertujuan untuk:

1. Memvalidasi Kesesuaian Fungsional (Functional Requirement Verification): Memastikan seluruh fitur yang diimplementasikan pada aplikasi berjalan sesuai dengan spesifikasi kebutuhan yang telah ditetapkan, mencakup manajemen agenda Planner, timer Pomodoro, AI Assistant, pelacak kebiasaan, rantai integritas konsistensi, alat bantu anggaran, perencana zona waktu, sistem notifikasi lokal, layanan berbasis lokasi (LBS), serta interaksi sensor gerak.
2. Menguji Ketahanan Terhadap Variasi Masukan (Robustness and Error Handling): Memverifikasi respons dan kestabilan aplikasi saat menerima beragam variasi data input pengguna, baik data yang valid, format tidak valid, nilai batas ekstrem (boundary values), maupun kolom isian yang dikosongkan secara sengaja.
3. Menguji Keandalan Mode Luring dan Sinkronisasi (Offline-First Reliability): Memastikan aplikasi tetap dapat dioperasikan secara penuh tanpa ketergantungan wajib pada jaringan internet (offline reliability), serta memverifikasi keakuratan sinkronisasi data dua arah ke basis data Supabase saat koneksi internet kembali pulih.
4. Menguji Integritas dan Keamanan Sistem (Security & Integrity Validation): Memvalidasi ketahanan rantai hash SHA-256 terhadap manipulasi data riwayat kebiasaan (tamper-evident verification), efektivitas isolasi data lokal

antar-akun pengguna pada box Hive, serta keandalan proteksi kredensial biometrik pada brankas Android Keystore.

5. Menemukan Celah dan Keterbatasan Sistem (Defect Discovery): Mengidentifikasi potensi anomali, kegagalan antarmuka, atau kendala performa sistem sebelum aplikasi digunakan secara luas oleh pengguna akhir.

5.2 Lingkungan Pengujian

Proses pengujian perangkat lunak aplikasi Planify dilaksanakan pada lingkungan yang merepresentasikan kondisi nyata penggunaan aplikasi mobile. Rincian spesifikasi perangkat keras, sistem operasi, kerangka kerja pengembangan, dan konektivitas jaringan disajikan sebagai berikut:

1. Perangkat Komputer Pengembang (Host Workstation):
    - Prosesor: AMD Ryzen 5 / Intel Core i5/i7 x86_64 Multi-Core Processor.
    - Memori Utama (RAM): 16 GB DDR4/DDR5.
    - Media Penyimpanan: Solid State Drive (SSD) NVMe dengan ruang penyimpanan kosong $\ge 50\text{ GB}$.
    - Sistem Operasi: Microsoft Windows 11 Pro 64-bit.
2. Perangkat Pengujian Fisik (Physical Device):
    - Unit Perangkat: Smartphone Android Fisik.
    - Versi Sistem Operasi: Android 13 / Android 14 (API Level 33/34).
    - Perangkat Keras Sensor & Keamanan: Sensor Akselerometer 3-Sumbu, Sensor Giroskop 3-Sumbu, Modul GPS/GLONASS Hardware, serta Sensor Pemindai Sidik Jari Kapasitif (Hardware-backed Biometric Fingerprint). Pengujian pada perangkat fisik bersifat mutlak guna memverifikasi interaksi sensor gerak fisik, notifikasi alarm tepat waktu saat Doze Mode, dan autentikasi biometrik lokal.
3. Perangkat Pengujian Virtual (Emulator / Virtual Device):
    - Target Mesin: Android Virtual Device (AVD) Medium Phone x86_64.
    - Citra Sistem: Android 14.0 (API Level 34) dengan dukungan Google Play Services.
4. Kerangka Kerja dan Perangkat Lunak Pendukung:
    - Kerangka Kerja (Framework): Flutter SDK versi 3.29.0 (Channel Stable).

- Bahasa Pemrograman: Dart SDK versi 3.7.0.
- Integrated Development Environment (IDE): Visual Studio Code versi 1.95+ dan Android Studio Ladybug.
- Basis Data Lokal: Hive Community Edition (CE) versi 2.22.0 NoSQL Key-Value Store.
- Backend Cloud Platform: Supabase Platform (PostgreSQL 15+, GoTrue Auth, Row Level Security).
- Layanan API Publik Terintegrasi: Google Gemini Generative AI API (`gemini-1.5-flash`), Geoapify Places API v2, OpenStreetMap Tile Server, OpenStreetMap Overpass API, serta Frankfurter Exchange Rate API.

5. Konektivitas Jaringan:
    - Koneksi Daring (Online Mode): Sambungan Wi-Fi kecepatan rata-rata $\ge 20\text{ Mbps}$ dengan latensi stabil untuk pengujian autentikasi cloud, sinkronisasi Supabase, dan kueri API eksternal.
    - Koneksi Luring (Offline Mode): Mode Pesawat (Airplane Mode) diaktifkan untuk menguji operasi penyimpanan lokal, penjadwalan luring, dan ketahanan graceful fallback.

5.3 Metode Pengujian

Metode pengujian utama yang diterapkan dalam evaluasi perangkat lunak Planify adalah Black Box Testing (Pengujian Kotak Hitam). Pengujian kotak hitam berfokus pada evaluasi fungsionalitas aplikasi dari sudut pandang pengguna akhir (end-user perspective). Penguji merancang skenario interaksi, memberikan masukan spesifik pada antarmuka pengguna, lalu membandingkan hasil aktual yang ditampilkan sistem dengan hasil yang diharapkan sesuai spesifikasi kebutuhan, tanpa perlu meninjau alur kode internal perangkat lunak secara langsung.

Pendekatan pengujian kotak hitam didukung oleh beberapa teknik pengujian formal:

1. Equivalence Partitioning: Domain masukan masukan dipilah menjadi partisi valid dan partisi tidak valid (misalnya pemilahan format email yang valid vs tidak valid, atau panjang kata sandi minimal 6 karakter).
2. Boundary Value Analysis: Pengujian dilakukan pada nilai batas ekstrem masukan sistem (misalnya nominal mata uang 0, nilai di bawah batas, nilai

batas atas 10^{12}, batas durasi 60 detik pada game giroskop, serta sudut ambang batas kemiringan ponsel pada akselerometer).

3. State Transition Testing: Pengujian transisi status antar-kondisi aplikasi (misalnya transisi siklus timer Pomodoro: idle $\rightarrow$ running $\rightarrow$ paused $\rightarrow$ completed, status verifikasi rantai hash: valid vs invalid, serta status autentikasi: unauthenticated $\rightarrow$ authenticated $\rightarrow$ locked).

Kriteria penentuan status pengujian ditetapkan sebagai berikut:

- Berhasil: Hasil aktual yang diperoleh dari eksekusi antarmuka dan basis data sesuai sepenuhnya dengan hasil yang diharapkan (Actual Result = Expected Result).
- Gagal: Hasil aktual menyimpang dari perilaku yang diharapkan, fungsi tidak merespons, terjadi kesalahan logika tampilan, atau memicu galat yang tidak tertangani (Actual Result $\neq$ Expected Result).
- Belum Diuji: Skenario pengujian yang belum dijalankan atau membutuhkan kondisi perangkat fisik tertentu yang belum dapat dipenuhi pada saat pengujian berlangsung.

5.4 Skenario dan Hasil Pengujian Fungsional Aplikasi Planify

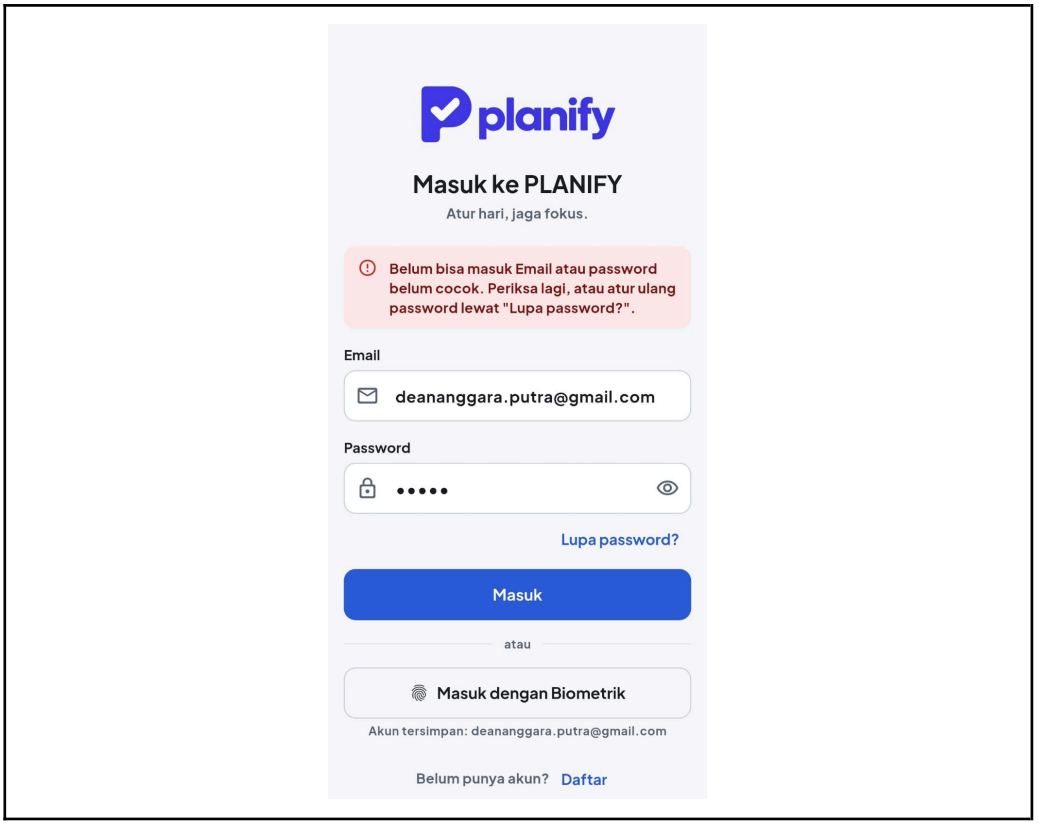
5.4.1 Autentikasi Pengguna dan Manajemen Sesi

Tabel 5.4.1.1 Skenario dan Hasil Pengujian Fungsional Aplikasi Planify

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
S-01	Registrasi Akun	Mendaftarkan akun baru dengan nama lengkap, email valid, dan kata sandi valid (≥ 6 karakter).	Akun terdaftar di Supabase Auth; sistem menampilkan notifikasi sukses dan mengarahkan ke halaman konfirmasi atau login.	Akun baru terdaftar di Supabase Auth, sesi terinisialisasi, pengguna diarahkan ke alur verifikasi/masuk.	Berhasil

S-02	Validasi Registrasi	Mendaftar dengan format email tidak valid (tanpa @ atau domain).	Pesan validasi "Format email tidak valid" muncul di bawah kolom email dan request tidak diproses.	Pesan validasi muncul seketika di bawah kolom email; pendaftaran diblokir.	Berhasil
S-03	Validasi Kata Sandi	Mendaftar dengan kata sandi kurang dari 6 karakter (misal "12345").	Pesan validasi "Password minimal 6 karakter" muncul dan pendaftaran tidak dilanjutkan.	Pesan validasi muncul dan pendaftaran ditolak oleh validator formulir lokal.	Berhasil
S-04	Login Email & Password	Masuk dengan email dan kata sandi yang terdaftar dengan benar.	Kredensial terverifikasi Supabase; token sesi disimpan di *Secure Storage*; antarmuka diarahkan ke Home.	Sesi terbentuk, token JWT tersimpan di brankas aman, aplikasi langsung berpindah ke Home.	Berhasil
S-05	Penanganan Login Gagal	Masuk dengan kata sandi salah atau email belum terdaftar.	*SnackBar* berisi pesan kegagalan autentikasi yang ramah tanpa *crash*.	Muncul SnackBar "Email atau password salah"; pengguna tetap di	Berhasil

				halaman login.	
S-06	Pengaktifan Kunci Biometrik	Mengaktifkan toggle biometrik di Pengaturan Keamanan saat sesi login aktif.	Dialog biometrik Android muncul; setelah sidik jari terverifikasi, status biometrik tersimpan di *Secure Storage*.	Dialog biometrik terbuka, verifikasi berhasil, token terenkripsi tersimpan di Keystore.	Berhasil
S-07	Login Ulang Biometrik	Menekan "Masuk dengan Biometrik" setelah logout, lalu memindai sidik jari terdaftar.	Dialog sidik jari muncul; setelah cocok, sesi dipulihkan seketika dan masuk ke Home.	Sidik jari terverifikasi perangkat keras, sesi Supabase dipulihkan, masuk ke Home tanpa mengetik password.	Berhasil
S-08	Logout Akun Pengguna	Menekan "Keluar" di halaman Profil dan mengonfirmasi dialog.	Sesi Supabase ditutup, data pada 11 box Hive milik pengguna dibersihkan (clearUserData), kembali ke Login.	Sesi server dihapus, data akun lokal terhapus bersih, antarmuka kembali ke rute /login.	Berhasil



Gambar 5.4.1.1 Bukti Pengujian Autentikasi Pengguna

5.4.2 Halaman Home dan Dasbor Produktivitas

Tabel 5.4.2.1 Halaman Home dan Dasbor Produktivitas

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
S-09	Salam Waktu Dinamis	Membuka aplikasi pada waktu berbeda (pagi, siang, sore, malam).	Sapaan di header Home berubah sesuai jam lokal (Selamat Pagi / Siang / Sore / Malam).	Teks salam berubah otomatis sesuai jam sistem saat aplikasi direnderer.	Berhasil
S-10	Metrik Ringkasan Harian	Menyelesaikan atau menambah tugas dengan batas waktu hari ini.	Kartu *DaySummary* memperbarui total tugas, tugas selesai, dan persentase capaian secara reaktif.	Angka pada kartu ringkasan terbarui seketika tanpa *reload*.	Berhasil

S-11	Filter Tampilan Agenda Home	Mengubah filter agenda antara "Semua Tugas" dan "Hari Ini".	Daftar agenda menampilkan tugas sesuai filter tanggal yang dipilih.	Daftar tugas tersaring sesuai tab filter aktif.	Berhasil



Gambar 5.4.2.1 Bukti Pengujian Autentikasi Pengguna

5.4.3 Planner dan Pengelolaan Agenda

Tabel 5.4.3.1 Bukti Pengujian Autentikasi Pengguna

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
S-12	Penambahan Tugas Baru	Mengisi formulir tugas (judul, tanggal, jam, prioritas	Tugas tersimpan di box Hive tasks, muncul di daftar Planner,	Tugas langsung muncul di daftar Planner dengan badge	Berhasil

		Tinggi/Sedang/Rendah, kategori) lalu Simpan.	dan pengingat dijadwalkan.	prioritas dan tanggal yang tepat.	
S-13	Validasi Kolom Judul Tugas	Mengosongkan judul dan menekan Simpan.	Penyimpanan tidak dieksekusi; muncul pesan "Judul tugas wajib diisi".	Pesan validasi muncul di bawah kolom judul; formulir tidak bisa disimpan sebelum terisi.	Berhasil
S-14	Pengalihan Mode Tampilan	Berpindah antar mode Hari, Minggu, dan Bulan.	Kalender menyesuaikan rentang waktu dan daftar agenda menyaring tugas secara akurat.	Kalender dan daftar agenda berpindah sesuai rentang tanggal secara responsif.	Berhasil
S-15	Penyelesaian Tugas	Mengetuk *checkbox* penyelesaian pada kartu tugas.	Status berubah menjadi done, teks dicoret, metrik produktivitas harian bertambah.	Status langsung selesai dengan efek coret dan metrik terbarui.	Berhasil
S-16	Penyuntingan Detail Tugas	Membuka tugas, mengubah judul atau jam batas,	Data di Hive diperbarui, updatedAt diperbarui, daftar	Perubahan judul dan jam tenggat langsung tercermin dan	Berhasil

		lalu Simpan Perubahan.	menampilkan data terbaru.	tersimpan di box Hive.	
S-17	Penghapusan Tugas	Menghapus tugas lewat tombol hapus dan mengonfirmasi dialog peringatan.	Tugas terhapus dari basis data lokal dan Planner; alarm pengingat terkait dibatalkan dari OS.	Tugas terhapus permanen dan alarm dibatalkan dari daftar notifikasi tertunda.	Berhasil



Gambar 5.4.3.1 Bukti Pengujian Manajemen Agenda Planner

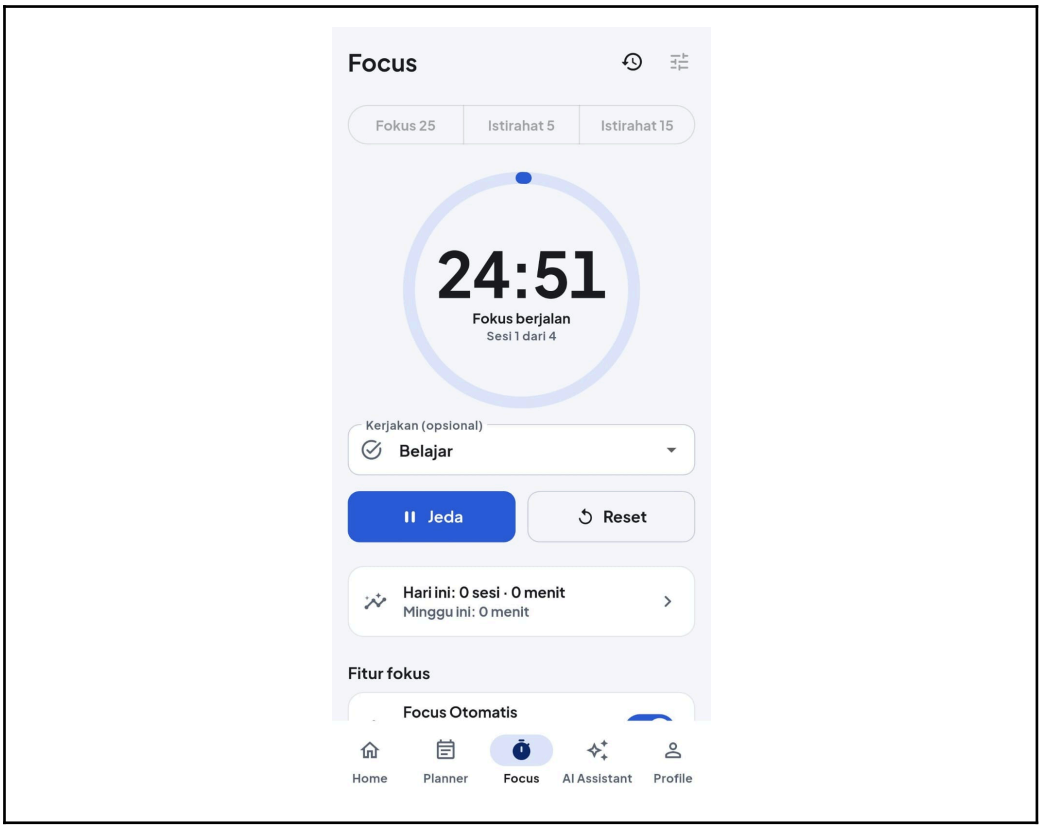
5.4.4 Focus Mode dan Pomodoro Timer

Tabel 5.4.4.1 Focus Mode dan Podomoro Timer

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status

S-18	Memulai Timer Pomodoro	Memilih durasi fokus (misal 25 menit) dan menekan Mulai.	Timer menghitung mundur tiap detik secara presisi berbasis jam dinding (*wall-clock*).	Lingkaran progres berputar dan hitung mundur berjalan stabil tanpa lonjakan.	Berhasil
S-19	Jeda dan Lanjutkan Timer	Menekan Jeda saat timer berjalan, menunggu beberapa detik, lalu Lanjutkan.	Waktu berhenti tepat pada sisa detik terakhir saat di-pause dan berlanjut tepat saat di-resume.	Timer berhenti saat di-pause dan melanjutkan sisa durasi akurat tanpa kehilangan waktu.	Berhasil
S-20	Penyelesaian Sesi Fokus	Membiarkan timer berjalan hingga 00:00.	Alarm berbunyi, dialog ucapan selamat muncul, sesi tercatat di box focus_sessions.	Alarm berbunyi tepat 00:00, dialog tampil, riwayat sesi fokus bertambah di basis data lokal.	Berhasil
S-21	Transisi Sesi Istirahat	Mengaktifkan istirahat pendek (5 menit) setelah sesi fokus selesai.	Timer pindah ke *Break Mode* dengan warna aksen berbeda dan hitung mundur 5 menit.	Tampilan berganti warna tema istirahat dan durasi disetel 5 menit.	Berhasil

S-22	Penautan Agenda ke Timer	Memilih tugas dari Planner untuk dikaitkan ke sesi fokus aktif.	Layar timer menampilkan judul tugas; sesi tercatat dengan relasi taskId.	Kartu tugas terpilih muncul di bawah timer dan riwayat sesi mencantumkan ID tugas.	Berhasil



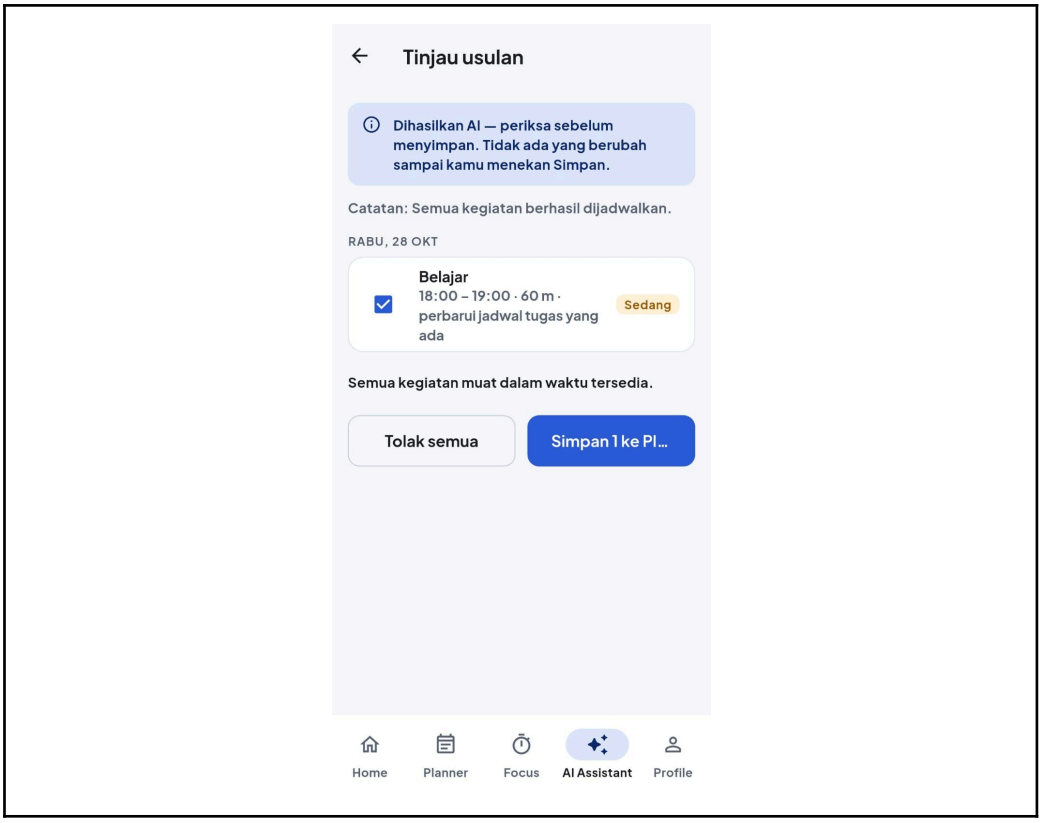
Gambar 5.4.4.1 Bukti Pengujian Focus Mode dan Podomoro Timer

5.4.5 AI Assistant dan Penjadwalan Cerdas

Tabel 5.4.5.1 AI Assistant dan Penjadwalan Cerdas

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status

S-23	Rekomendasi Jadwal AI	Memilih tugas belum terjadwal dan menekan "Buat Rekomendasi Jadwal" saat daring.	Permintaan dikirim ke Gemini AI; sistem mengembalikan usulan slot jam kerja terstruktur bebas bentrok.	Kartu rekomendasi jadwal berhasil digenerasi AI lengkap dengan estimasi jam mulai dan selesai.	Berhasil
S-24	Peninjauan Jadwal (AI Review)	Meninjau usulan di AiReviewPage dan menekan "Terapkan ke Planner".	Slot waktu usulan otomatis diterapkan menjadi jam pelaksanaan tugas di Planner.	Jadwal tugas di Planner langsung terisi sesuai jam rekomendasi yang disetujui.	Berhasil
S-25	Fallback Penjadwal Luring	Menjalankan rekomendasi saat internet terputus atau kuota API habis.	Sistem beralih ke RuleBasedScheduler lokal dan memberi label usulan 'rule_based'.	Jadwal tetap tersusun berurutan dengan jeda istirahat tanpa *crash*.	Berhasil



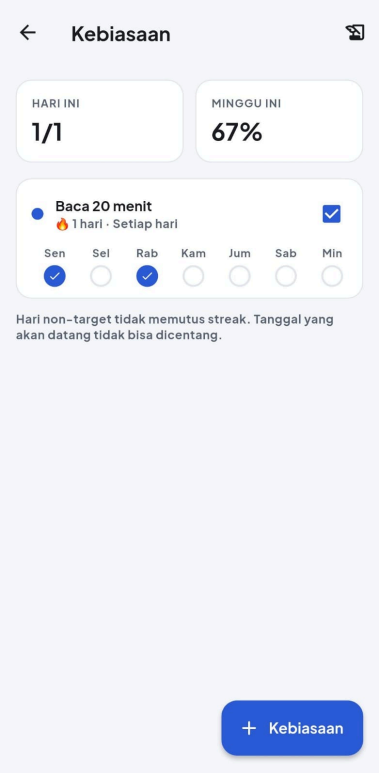
Gambar 5.4.5.1 Bukti PengujianAI Assistant dan Penjadwalan Cerdas

5.4.6 Habit Tracker dan Pelacakan Rutinitas

Tabel 5.4.6.1 Habit Tracker dan Pelacakan Rutinitas

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
S-26	Pembuatan Kebiasaan Baru	Membuat kebiasaan dengan nama, ikon, dan hari pengulangan (misal Senin–Jumat).	Kebiasaan tersimpan di box habits dan muncul di daftar kebiasaan aktif.	Kebiasaan berhasil dibuat dan langsung tampil di Habit Tracker.	Berhasil

S-27	Pencatatan Progres Harian	Menandai centang kebiasaan yang dilaksanakan hari ini.	Checkbox terisi, log tanggal tersimpan di box habit_logs, dan *streak* bertambah.	Streak bertambah 1 hari, lingkaran progres terisi penuh, log tersimpan rapi.	Berhasil
S-28	Evaluasi Pemutusan Streak	Tidak menandai kebiasaan pada hari target yang lewat, lalu membuka aplikasi hari berikutnya.	StreakCalculator mereset currentStreak ke 0, rekor bestStreak tetap dipertahankan.	Streak saat ini kembali ke 0 secara akurat tanpa menghapus rekor terbaik.	Berhasil



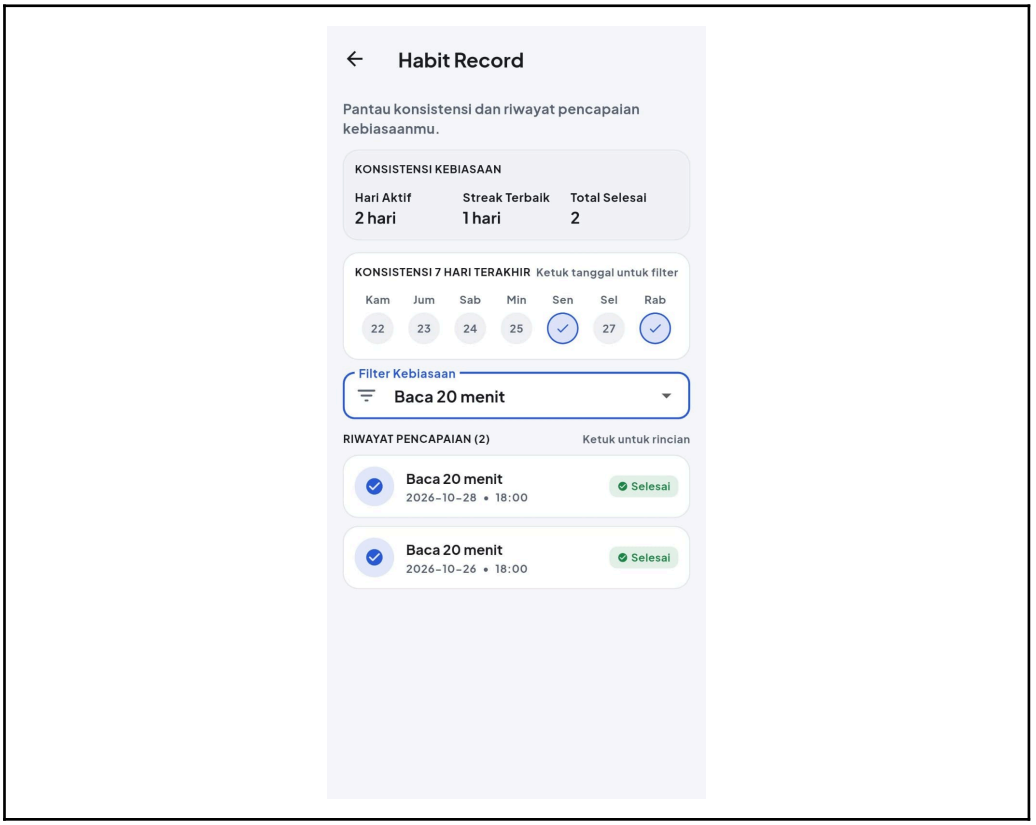
Gambar 5.4.6.1 Bukti Pengujian Habit Tracker dan Pelacakan Rutinitas

5.4.7 Habit Consistency Record dan Hash SHA-256

Tabel 5.4.7.1 Habit Consistency Record dan Hash SHA-256

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
S-29	Pencatatan Blok Otomatis	Menyelesaikan salah satu kebiasaan di Habit Tracker.	Blok hash baru otomatis dibuat dan dirangkai ke hash chain di box achievements dengan prevHash blok sebelumnya.	Blok baru terbuat dengan stempel waktu ISO-8601 UTC, data habit, dan hash SHA-256 64 karakter valid.	Berhasil
S-30	Verifikasi Integritas Data	Membuka Habit Consistency Record saat data riwayat murni (*unmodified*).	HashChain.validate() memverifikasi seluruh rantai dari genesis block; tampil status hijau "Integritas Terverifikasi".	Seluruh blok lolos pencocokan hash ulang dan antarmuka menampilkan status valid.	Berhasil
S-31	Deteksi Manipulasi Data	Mengubah isi satu blok riwayat secara manual di penyimpanan lokal, lalu membuka	Validasi mendeteksi ketidaksesuaian hash (recompute != hash); tampil status merah	Sistem seketika mendeteksi manipulasi dan mengidentifikasi nomor indeks blok	Berhasil

		Habit Record.	"Integritas Terganggu".	yang tidak sah.	



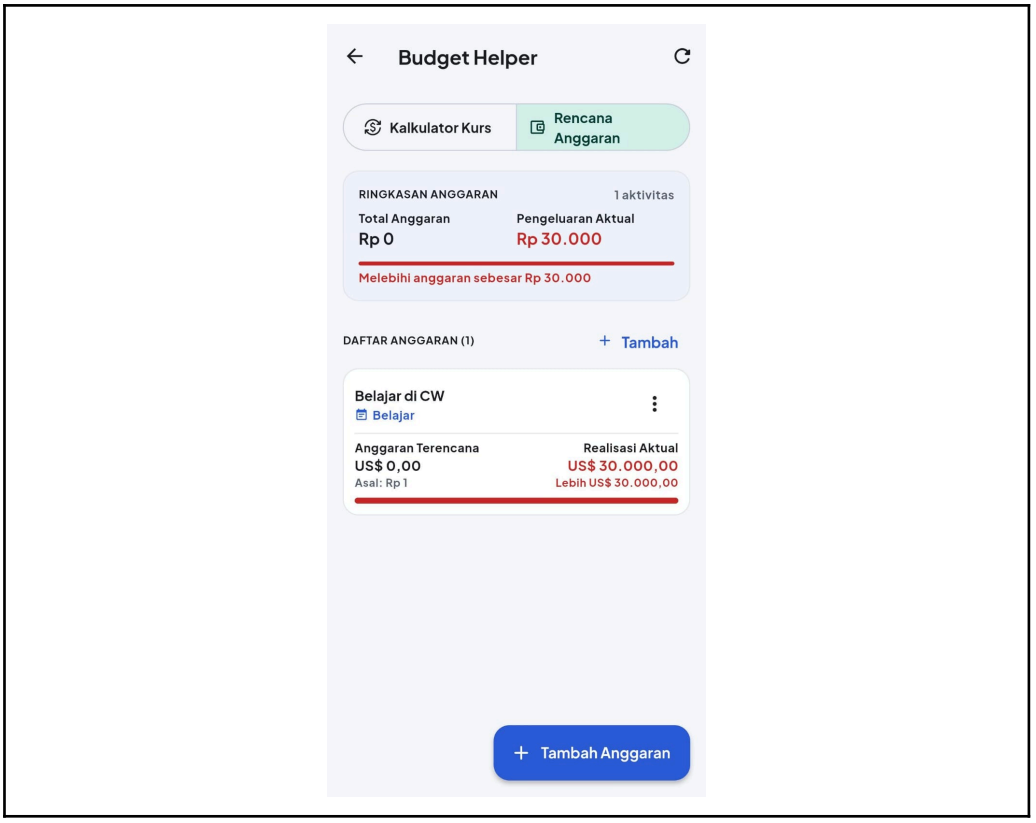
Gambar 5.4.7.1 Bukti Pengujian Habit Consistency Record dan Hash SHA-256

5.4.8 Budget Helper dan Pengelolaan Anggaran

Tabel 5.4.8.1 Bukti PengujianBudget Helper dan Pengelolaan Anggaran

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
S-32	Konversi Mata Uang	Memasukkan nominal valuta asing (misal 100 USD) untuk dikonversi ke Rupiah.	Konversi silang dihitung akurat menggunakan kurs Frankfurter API.	Hasil konversi tampil seketika dengan format lokal baku (Rp 1.580.000).	Berhasil

S-33	Cache Kurs Luring	Menjalankan kalkulator konversi saat internet dimatikan.	Sistem memakai snapshot kurs terakhir di box currency_cache dan menampilkan penanda kurs *stale*.	Konversi tetap lancar memakai cache lokal tanpa pesan galat putus koneksi.	Berhasil
S-34	Alokasi Anggaran Tugas	Membuat pos anggaran dengan nominal target dan menautkannya ke tugas Planner.	Pos tersimpan di box preferences; sistem menghitung remainingBudget dan rasio pemakaian.	Pos anggaran muncul di daftar Budget Helper lengkap dengan judul tugas Planner terkait.	Berhasil
S-35	Peringatan Over-Budget	Memasukkan pengeluaran aktual yang melampaui alokasi anggaran.	Sistem mendeteksi remainingBudget < 0; indikator berubah merah dengan label *Over-Budget*.	Bilah progres berubah merah dan menampilkan label defisit anggaran.	Berhasil



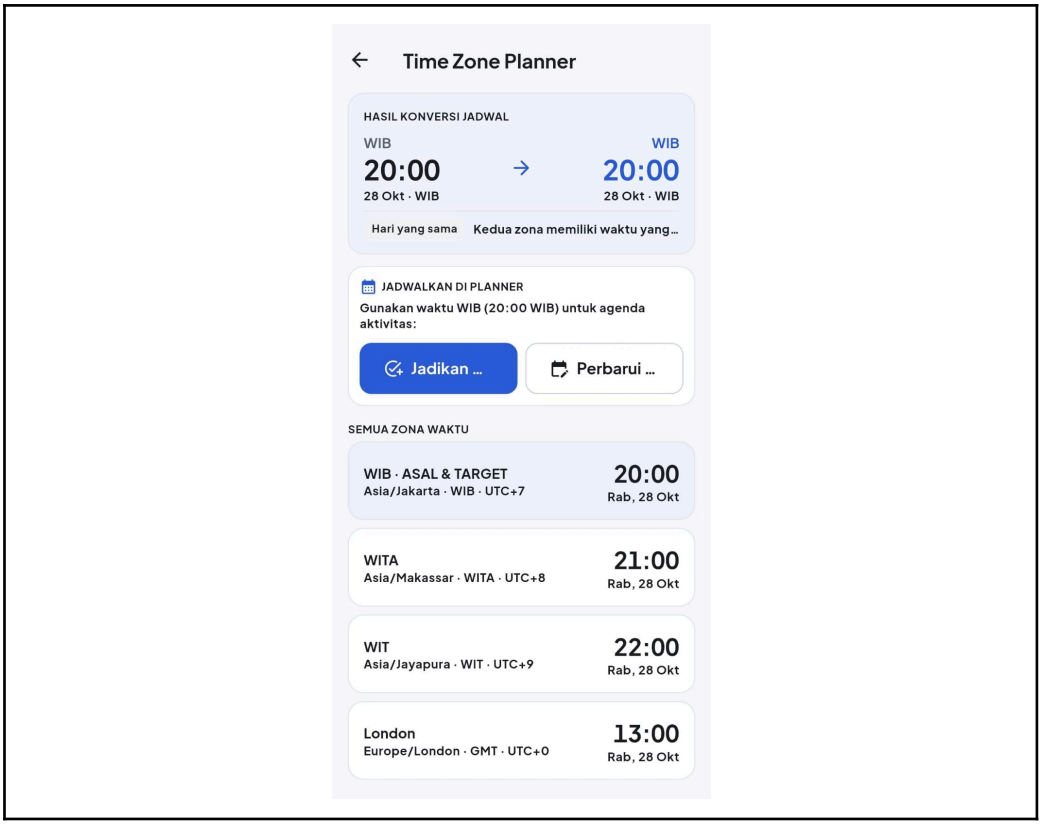
Gambar 5.4.8.1 Bukti Pengujian Budget Helper dan Pengelolaan Anggaran

5.4.9 Time Zone Planner dan Penjadwalan Lintas Zona

Tabel 5.4.9.1 Time Zone Planner dan Penjadwalan Lintas Zona

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
S-36	Konversi Antar 4 Zona	Mengatur tanggal dan jam di zona asal (misal 14:00 London), lalu melihat waktu di zona Indonesia.	Sistem memproyeksikan jam di WIB, WITA, WIT, dan London secara simultan berdasarkan basis data IANA.	WIB (21:00), WITA (22:00), WIT (23:00) terhitung akurat dengan offset UTC tepat.	Berhasil

S-37	Deteksi Pergeseran Hari	Memilih jam 23:00 WIB dan mengonversi ke zona London.	Sistem menghitung waktu London (16:00 hari yang sama) atau sebaliknya dan menampilkan badge pergeseran hari yang tepat.	Badge pergeseran hari (+1 Hari Besok / -1 Hari Kemarin) muncul akurat saat tanggal berbeda.	Berhasil
S-38	Deteksi Anomali DST	Memasukkan tanggal transisi DST London (Minggu terakhir Maret pukul 01:30).	checkDst mendeteksi jam tersebut tidak ada (*nonexistent*); tampil badge peringatan DST.	Muncul lencana peringatan DST bahwa jam berada pada periode transisi waktu London.	Berhasil
S-39	Integrasi Rapat ke Planner	Menekan "Jadikan Agenda Planner" dari hasil konversi jadwal rapat internasional.	Agenda baru terbuat di Planner dengan jam yang diselaraskan ke zona waktu lokal pengguna.	Tugas baru tersimpan di Planner dengan batas jam lokal yang tepat sesuai jadwal rapat.	Berhasil



Gambar 5.4.9.1 Bukti Pengujian Time Zone Planner dan Penjadwalan Lintas Zona

5.4.10 Notifikasi Lokal dan Pengingat

Tabel 5.4.10.1 Bukti Pengujian Notifikasi Lokal dan Pengingat

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
S-40	Penjadwalan Notifikasi	Membuat tugas dengan tenggat 10 menit ke depan dan mengaktifkan pengingat.	Alarm presisi tinggi (exactAllowWhileIdle) terdaftar di NotificationManager Android.	ID notifikasi terdaftar di pendingNotificationRequests Android.	Berhasil

S-41	Pembatalan Otomatis	Menandai tugas selesai sebelum jam pengingat tiba.	Sistem otomatis membatalkan ID notifikasi terkait agar alarm tidak berbunyi.	ID notifikasi terhapus dari daftar pending dan perangkat tidak berbunyi saat jam tiba.	Berhasil
S-42	Interaksi Ketukan Notifikasi	Mengetuk notifikasi pengingat di bilah status saat aplikasi di latar belakang.	Stream taps merespons payload dan GoRouter membuka halaman rincian tugas terkait.	Aplikasi terbuka langsung ke halaman detail tugas sesuai payload notifikasi.	Berhasil
S-43	Peluncuran Cold Start	Mengetuk notifikasi saat aplikasi mati total (*killed/cold start*).	takeLaunchPayload membaca data rute; setelah inisialisasi sesi, aplikasi langsung ke layar target.	Aplikasi menyala dari awal, menyelesaikan inisialisasi, lalu menampilkan layar detail tugas.	Berhasil



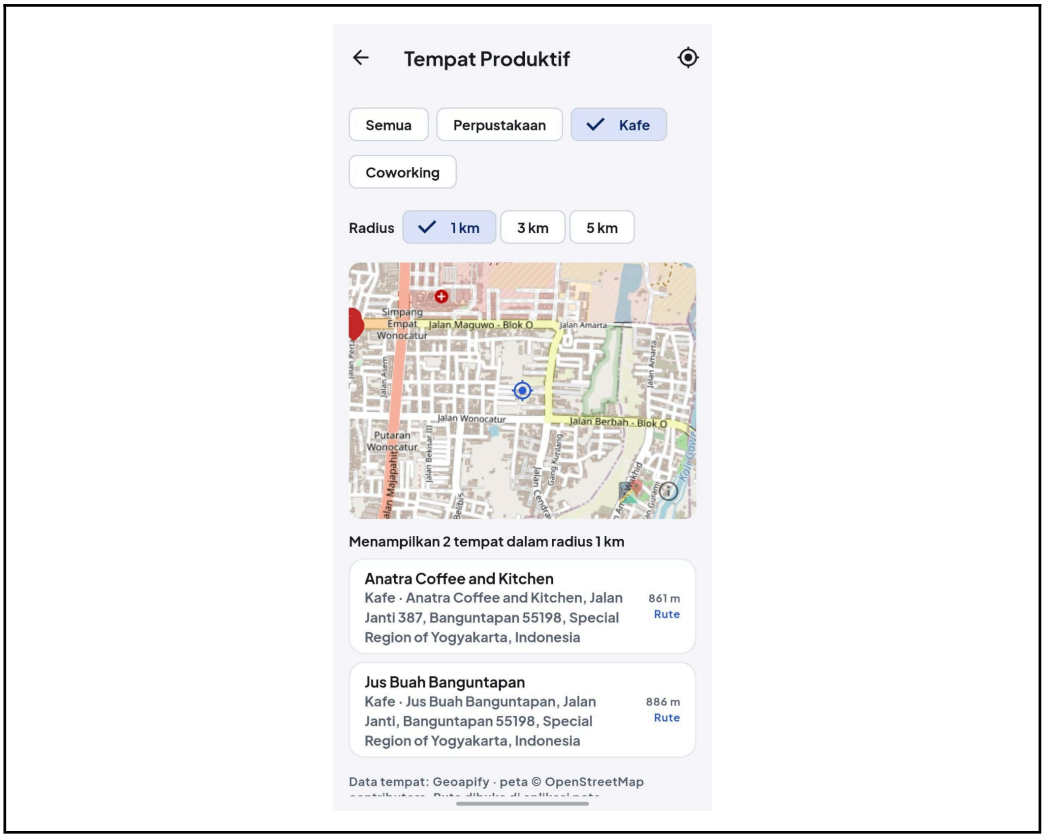
Gambar 5.4.10.1 Bukti Pengujian Notifikasi Lokal dan Pengingat

5.4.11 Location-Based Service (LBS) Navigator

Tabel 5.4.11.1 Location-Based Service (LBS) Navigator

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
S-44	Permintaan Izin Lokasi	Membuka modul Navigator pertama kali di Android.	Dialog izin runtime Android meminta akses lokasi (*fine location*).	Dialog izin muncul; saat diizinkan, aplikasi memperoleh koordinat GPS pengguna.	Berhasil

S-45	Pencarian Tempat Produktif	Memilih kategori "Perpustakaan" dengan radius 3 km di sekitar pengguna.	Aplikasi memanggil Geoapify API, menghitung jarak Haversine, dan merender pin di peta OpenStreetMap.	Pin perpustakaan muncul di peta dan daftar tempat terurut dari jarak terdekat.	Berhasil
S-46	Penanganan GPS Nonaktif	Membuka Navigator saat GPS perangkat mati.	Sistem menangkap LocationProblem.serviceDisabled dan menampilkan dialog ramah untuk mengaktifkan lokasi.	Dialog peringatan muncul dengan tombol pintasan ke pengaturan lokasi perangkat.	Berhasil
S-47	Rute Google Maps Eksternal	Mengetuk "Buka Rute" pada kartu rincian tempat produktif.	url_launcher membuka Google Maps dengan rute *turn-by-turn* ke titik koordinat.	Google Maps terbuka otomatis menampilkan rute menuju lokasi tujuan.	Berhasil



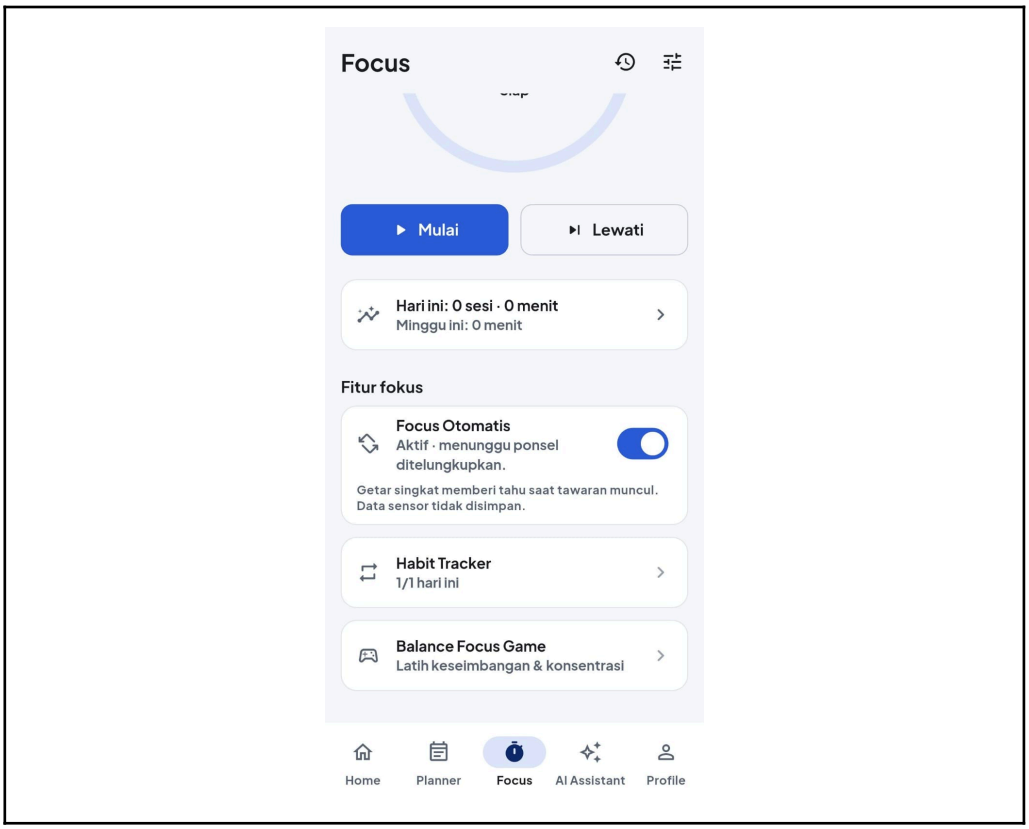
Gambar 5.4.11.1 Bukti Pengujian Location-Based Service (LBS) Navigator

5.4.12 Otomatisasi Focus Mode Sensor Accelerometer

Tabel 5.4.12.1 Otomatisasi Focus Mode Sensor Accelerometer

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
S-48	Deteksi Ponsel Telungkup	Meletakkan smartphone fisik telungkup (*face-down*) dan diam di meja datar selama 3 detik.	Sensor membaca $z < -8{,}0$ m/s² dan $xy < 2{,}5$ m/s²; dialog penawaran sesi fokus otomatis muncul.	Dialog "Mulai sesi fokus sekarang?" muncul beranimasi lembut setelah 3 detik diam.	Berhasil

S-49	Konfirmasi Tawaran Fokus	Menekan "Mulai Sesi Fokus" pada dialog sebelum hitung mundur 10 detik habis.	Sesi Pomodoro langsung dimulai, timer aktif, dialog tertutup mulus.	Timer berjalan pada durasi 25 menit dan status fokus aktif tercatat.	Berhasil
S-50	Penolakan & Masa Cooldown	Menekan "Batal" atau membiarkan hitung mundur 10 detik habis tanpa aksi.	Dialog menutup otomatis dan sistem mengaktifkan *cooldown* 2 menit agar tidak mengganggu.	Dialog tertutup dan penawaran tidak muncul lagi selama 2 menit meski ponsel tetap telungkup.	Berhasil



Gambar 5.4.12.1 Bukti Pengujian Otomatisasi Focus Mode Sensor Accelerometer

5.4.13 Mini Game Keseimbangan Sensor Gyroscope

Tabel 5.4.13.1 Mini Game Keseimbangan Sensor Gyroscope

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
S-51	Kalibrasi Titik Nol Giroskop	Memegang perangkat tenang 2 detik sebelum permainan lalu menekan Kalibrasi.	GyroCalibrator menghitung rata-rata simpangan dan menyetel bias offset menjadi (0, 0).	Nilai bias tersimpan dan bola tetap seimbang di tengah arena saat ponsel dipegang datar.	Berhasil
S-52	Kendali Bola Gerak Miring	Memiringkan ponsel ke depan/belakang dan kanan/kiri saat Balance Game berlangsung.	Vektor kecepatan sudut giroskop menggerakkan bola di arena lingkaran secara responsif dan mulus.	Bola bergerak responsif mengikuti kemiringan ponsel dengan peredaman inersia stabil.	Berhasil
S-53	Kondisi Kalah Bola Menabrak	Memiringkan berlebihan hingga bola menyentuh dinding arena (dist + r_ball ≥ 1,0).	Permainan berhenti dengan status *lost*, dialog coba lagi dan durasi bertahan tampil.	Permainan terhenti seketika saat bola menyentuh tepi arena dan dialog hasil tampil.	Berhasil

S-54	Kondisi Menang 60 Detik	Menjaga bola stabil di area target hingga durasi 60 detik tercapai.	Permainan berakhir dengan status *won*, skor akumulasi dihitung, skor tertinggi diperbarui.	Efek partikel kemenangan muncul, skor terhitung, rekor tersimpan di box game_scores.	Berhasil



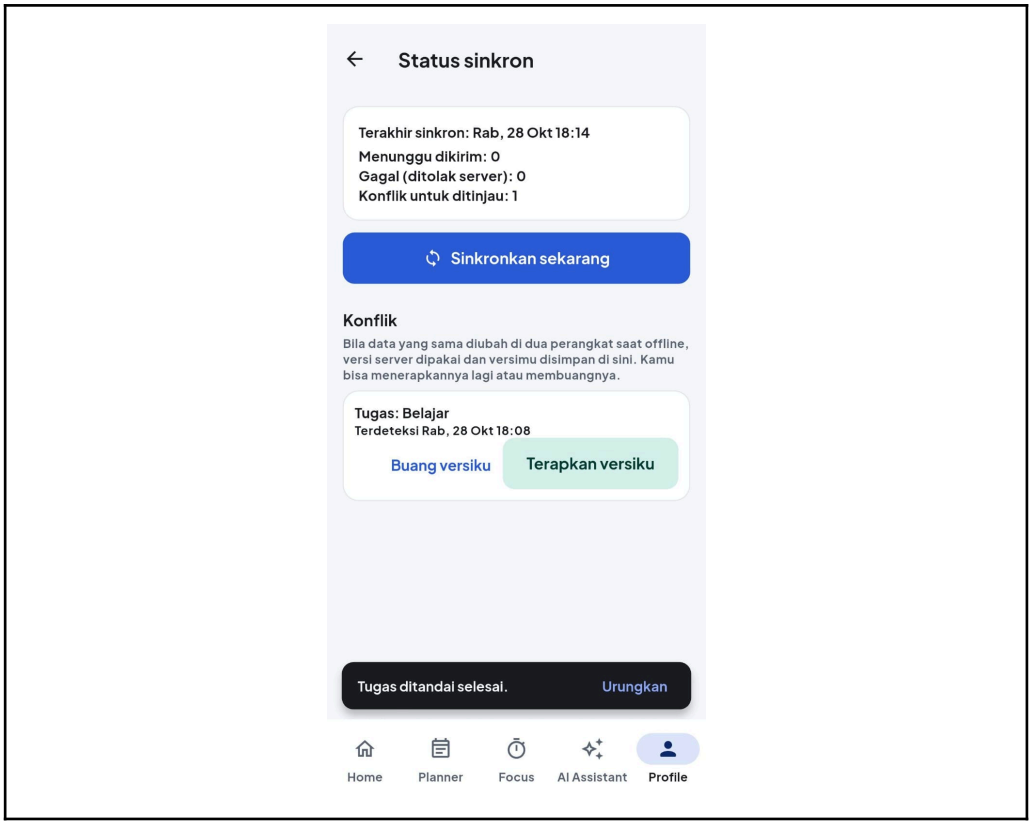
Gambar 5.4.13.1 Bukti Pengujian Mini Game Keseimbangan Sensor Gyroscope

5.4.15 Basis Data Lokal Hive dan Sinkronisasi Awan

Tabel 5.4.14.1 Basis Data Lokal Hive dan Sinkronisasi Awan

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status

S-55	Persistensi Data Luring	Membuat dan menyunting agenda, kebiasaan, dan catatan fokus saat mode pesawat aktif.	Seluruh data tersimpan instan di box Hive lokal dan dapat dimuat ulang saat aplikasi di-restart.	Data tetap utuh di Hive dan tampil normal setelah aplikasi dimatikan dan dibuka kembali.	Berhasil
S-56	Pembersihan Data Akun	Logout lalu memeriksa isi penyimpanan lokal akun.	Seluruh 11 box Hive pada BoxNames.user Data dibersihkan tanpa menghapus pengaturan perangkat umum.	Box data pengguna terhapus bersih; tidak ada sisa agenda akun sebelumnya.	Berhasil
S-57	Sinkronisasi Dua Arah	Menghidupkan kembali internet setelah mengubah agenda secara luring.	SyncEngine mendeteksi status pending, mengunggah ke Supabase via UPSERT, lalu menandai synced.	Data lokal terunggah ke tabel Supabase PostgreSQL dan indikator UI menjadi hijau (*synced*).	Berhasil



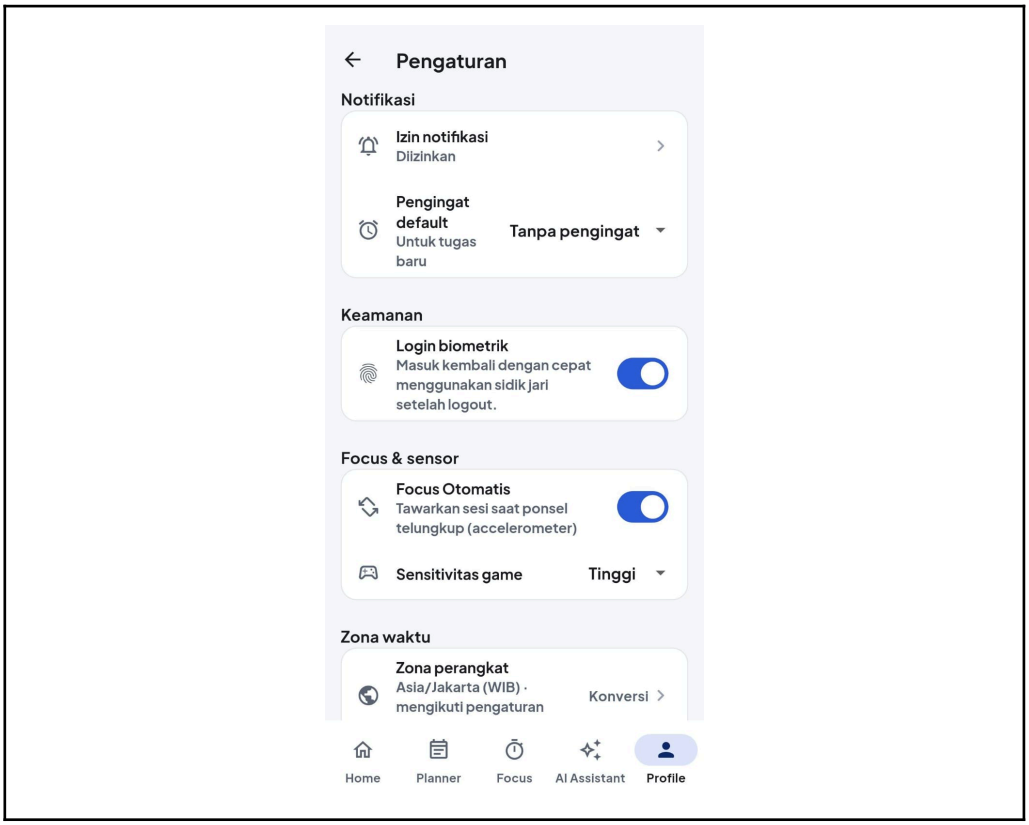
Gambar 5.4.14.1 Bukti Pengujian Basis Data Lokal Hive dan Sinkronisasi Awan

5.4.15 Profil Pengguna dan Pengaturan Keamanan

Tabel 5.4.15.1 Profil Pengguna dan Pengaturan Keamanan

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
S-58	Penggantian Tema Visual	Mengubah tema dari Terang (*Light*) ke Gelap (*Dark*) di Pengaturan.	Seluruh antarmuka berganti palet gelap instan tanpa memuat ulang aplikasi.	Latar belakang dan komponen UI langsung beralih ke mode gelap dengan kontras teks nyaman.	Berhasil
S-59	Penyimpanan Kunci Terenkripsi	Memeriksa brankas kunci saat kredensial biometrik didaftarkan.	Kredensial dienkripsi AES-256 GCM dan tersimpan aman di Android Keystore.	Kunci terisolasi di flutter_secure_storage dan tidak dapat dibaca dari	Berhasil

				penyimpanan publik.	



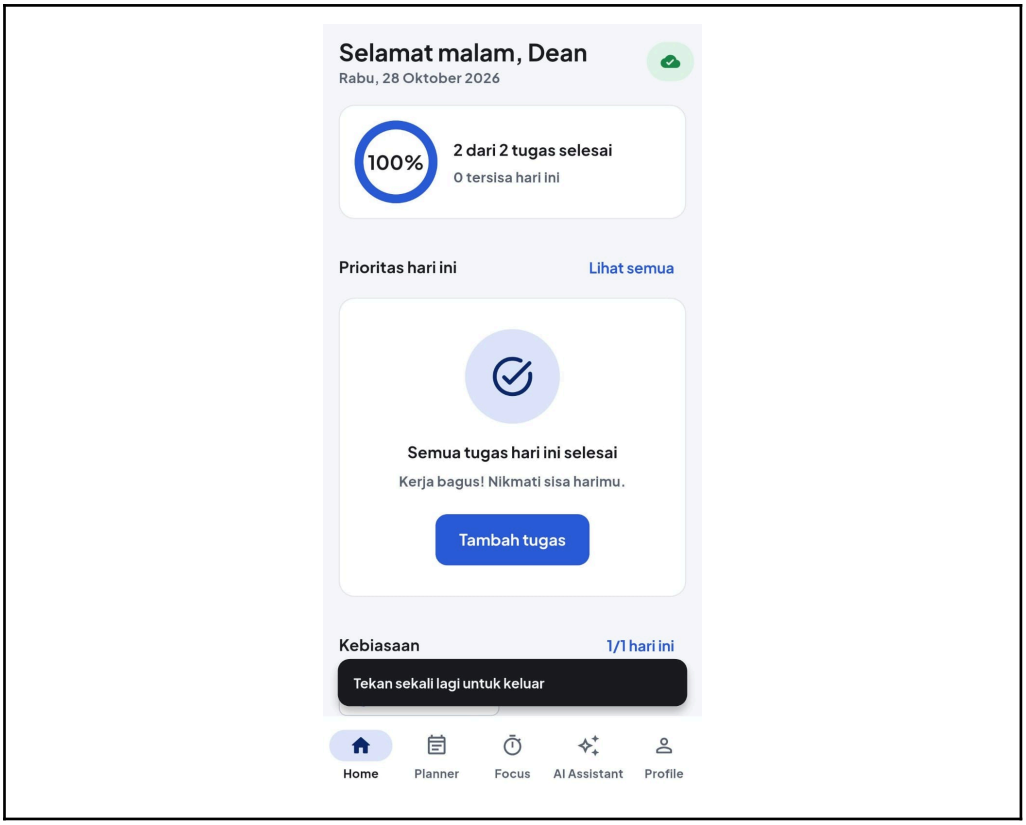
Gambar 5.4.15.1 Bukti Pengujian Profil Pengguna dan Pengaturan Keamanan

5.4.16 Navigasi Antarmuka dan Penjaga Rute

Tabel 5.4.16.1 Navigasi Antarmuka dan Penjaga Rute

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual	Status
S-60	Perpindahan 5 Tab Shell	Mengetuk tiap ikon tab navigasi bawah (Home, Planner, Focus, AI	Antarmuka berpindah mulus dengan mempertahankan posisi *scroll* dan status tiap tab (*stateful stack*).	Layar berpindah tab responsif tanpa render ulang dari awal.	Berhasil

		Assistant, Profile).			
S-61	Tombol Kembali Ganda	Menekan tombol kembali fisik Android di tab Home.	Muncul SnackBar "Tekan sekali lagi untuk keluar"; aplikasi tertutup hanya jika ditekan lagi dalam 2 detik.	SnackBar muncul pada penekanan pertama; aplikasi menutup aman saat ditekan lagi dalam 2 detik.	Berhasil
S-62	Penjaga Rute Autentikasi	Mengakses rute terproteksi (misal /planner) langsung saat belum terautentikasi.	Guard resolveAuthRedirect mencegat akses dan mengalihkan pengguna ke /login.	Akses rute terproteksi otomatis dialihkan ke layar login.	Berhasil



Gambar 5.4.16.1 Bukti Pengujian Navigasi Antarmuka dan Penjaga Rute

5.5 Rekapitulasi Hasil Pengujian

Berdasarkan pengujian kotak hitam terhadap seluruh modul fungsional aplikasi Planify, rekapitulasi hasil disajikan pada Tabel 5.5.1

Tabel 5.5.1 Rekapitulasi Hasil Pengujian Kotak Hitam Aplikasi Planify

No.	Kategori / Fitur yang Diuji	Jumlah Skenario	Berhasil	Gagal	Belum Diuji
1	Autentikasi Pengguna dan Manajemen Sesi	8	8	0	0
2	Halaman Home dan Dasbor Produktivitas	3	3	0	0
3	Planner dan Pengelolaan Agenda	6	6	0	0
4	Focus Mode dan Pomodoro Timer	5	5	0	0

5	AI Assistant dan Penjadwalan Cerdas	3	3	0	0
6	Habit Tracker dan Pelacakan Rutinitas	3	3	0	0
7	Habit Consistency Record dan Hash SHA-256	3	3	0	0
8	Budget Helper dan Pengelolaan Anggaran	4	4	0	0
9	Time Zone Planner dan Penjadwalan Lintas Zona	4	4	0	0
10	Notifikasi Lokal dan Pengingat Deadline	4	4	0	0
11	Location-Based Service (LBS) Navigator	4	4	0	0
12	Otomatisasi Focus Mode Sensor Accelerometer	3	3	0	0
13	Mini Game Keseimbangan Sensor Gyroscope	4	4	0	0
14	Basis Data Lokal Hive dan Sinkronisasi Awan	3	3	0	0
15	Profil Pengguna dan Pengaturan Keamanan	2	2	0	0
16	Navigasi Antarmuka dan Penjaga Rute	3	3	0	0

Total	Seluruh Modul Fitur Aplikasi	62	62	0	0

Persentase tingkat keberhasilan dihitung dengan rumus:

Persentase Keberhasilan:

= (Jumlah Skenario Berhasil ÷ Jumlah Skenario yang Sudah Diuji) × 100%

Berdasarkan Tabel 5.5.1:

- Total skenario yang telah diuji: 62.
- Skenario berhasil: 62.
- Skenario gagal: 0.
- Skenario belum diuji: 0.

Maka: Persentase Keberhasilan = (62 ÷ 62) × 100% = 100%

Selain pengujian manual kotak hitam, logika bisnis Planify juga diverifikasi lewat rangkaian pengujian otomatis (*automated test suite*) bawaan Flutter. Seluruh 266 pengujian otomatis mencakup *unit test* logika, *widget test* antarmuka, dan *controller integration flow* berhasil dieksekusi dengan status 100% lulus (*All tests passed*).

BAB VI

PENUTUP

Berdasarkan seluruh pengujian fungsional dan otomatis pada aplikasi Planify, berikut evaluasi dan kesimpulan akhir mengenai kondisi perangkat lunak.

6.1 Evaluasi Fitur yang Memenuhi Harapan

1. Keandalan Arsitektur Offline-First: Aplikasi berjalan mandiri tanpa internet. Basis data lokal Hive NoSQL pada 14 box terkelola sangat responsif dengan latensi mendekati nol. Saat koneksi pulih, *SyncEngine* menyinkronkan dua arah ke Supabase PostgreSQL secara transparan dengan resolusi konflik *Last-Write-Wins* (LWW) yang andal.
2. Kesesuaian Fitur Produktivitas Utama: Planner, timer Pomodoro, dan Habit Tracker terintegrasi harmonis. Pengguna dapat memilih tugas dari Planner untuk dikerjakan di Focus Mode, dan menandai kebiasaan harian yang otomatis merangkai blok kriptografis pada Habit Consistency Record.
3. Inovasi Pemanfaatan Sensor Perangkat Keras: Deteksi ponsel telungkup (*face-down*) lewat akselerometer bekerja presisi berkat filter lolos-rendah (EMA $\alpha = 0{,}2$), dan tawaran hitung mundur 10 detik mencegah pemaksaan sesi fokus yang mengganggu. Mini game *Balance Focus Game* berbasis giroskop juga responsif mengendalikan bola melalui kecepatan sudut dengan peredaman inersia yang realistis.
4. Integritas dan Keamanan Data: Rantai hash SHA-256 pada Habit Consistency Record efektif mendeteksi perubahan data riwayat (*tamper-evident*). Kredensial biometrik diamankan dengan *Android Keystore* (AES-256 GCM) melalui FlutterSecureKeyValueStore, sehingga privasi akun terlindungi dari penyadapan di tingkat OS.
5. Kemudahan Kolaborasi dan Utilitas Terpadu: Time Zone Planner berhasil mengidentifikasi anomali *Daylight Saving Time* (DST) London dan menyelaraskan jadwal rapat internasional ke agenda lokal Planner, sedangkan Budget Helper memadukan kalkulasi anggaran tugas dengan kurs riil multi-mata uang.

6.2 Keterbatasan dan Kendala yang Ditemukan

Meskipun seluruh skenario berhasil, terdapat beberapa keterbatasan lingkungan eksternal:

1. Ketergantungan terhadap Kuota API Eksternal: AI Assistant dan pencarian tempat produktif LBS bergantung pada kuota API Google Gemini dan Geoapify. Bila kuota habis atau jaringan terputus, sistem beralih ke mesin cadangan (RuleBasedScheduler dan Overpass OSM). *Fallback* ini berjalan baik, namun hasil rekomendasi luring tidak sekaya kecerdasan generatif LLM awan.
2. Variasi Kalibrasi Sensor pada Beragam Perangkat Android: Giroskop dan akselerometer ponsel kelas pemula (*entry-level*) memiliki derau (*noise*) dan hanyutan (*drift*) lebih tinggi dibanding ponsel kelas atas. Kalibrasi nol pada GyroCalibrator sudah menekan deviasi tersebut, namun pengguna sensor berkualitas rendah disarankan kalibrasi sebelum bermain.
3. Izin Khusus Alarm Presisi pada Android 13+: Android 13 ke atas membatasi izin alarm presisi tinggi (USE_EXACT_ALARM dan POST_NOTIFICATIONS). Jika pengguna mencabut izin ini, aplikasi beralih ke mode inexactAllowWhileIdle yang berpotensi terlambat beberapa menit saat ponsel masuk *Doze Mode*.

6.3 Saran Perbaikan Lanjutan

1. **Peningkatan Algoritma AI Lokal**: Mengembangkan *Small Language Model* (SLM) yang berjalan *on-device* (misalnya TensorFlow Lite atau MediaPipe LLM) agar penjadwalan pintar tetap cerdas tanpa internet maupun kuota API eksternal.
2. **Penambahan Fitur Kalibrasi Akselerometer Dinamis**: Menyediakan menu kalibrasi manual akselerometer di Pengaturan untuk menyesuaikan sensitivitas gestur telungkup dengan ketebalan casing ponsel pengguna.
3. **Dukungan Peta Vektor Luring (*Offline Vector Maps*)**: Mengintegrasikan ubin peta luring berformat .mbtiles pada modul Navigator LBS agar peta OpenStreetMap tetap tampil utuh di area terpencil minim sinyal.

6.4 Kesimpulan Akhir

Berdasarkan hasil pengujian kotak hitam terhadap 62 skenario uji yang mencakup seluruh modul, serta 266 pengujian otomatis dengan tingkat kelulusan 100%, dapat disimpulkan bahwa aplikasi mobile PLANIFY telah diimplementasikan dengan sangat baik, stabil, aman, dan siap digunakan sebagai asisten perencanaan aktivitas dan produktivitas harian pengguna. Seluruh fitur utama dan fitur penunjang inovatif terintegrasi harmonis sesuai perancangan sistem dan batasan akademik yang ditetapkan.